

بسم الله الرحمن الرحيم والصلاة والسلام على أشرف المرسلين سيدنا محمد ﷺ
وعلى اهله وصحبه أجمعين

ملخص لكل ما تم دراسته في JS الجزء الثاني

Week 14

What Is BOM ? Lesson #102

Alert, Confirm, Prompt Lesson #103

setTimeout and clearTimeout Lesson #104

setInterval and clearInterval Lesson #105

Window Location Object Lesson #106

Window Open And Close Lesson #107

Window History Object Lesson #108

Scroll, ScrollTo, ScrollBy, Focus, Print, Stop Lesson #109

Scroll To Top Using Scrolly Lesson #110

Assignments From [102] = > [110]

Week 15

Local Storage Lesson #111

Local Storage Color App Practice Lesson #112

Session Storage And Use Cases Lesson #113

Bom Challenge Lesson #114

Assignments From [111] = > [114]

Week 16

Destructuring Arrays Part 1 Lesson #115

Destructuring Arrays Part 2 Lesson #116

Destructuring Arrays Part 3 Swapping Variables Lesson #117

Destructuring Objects Part 1 Lesson #118

Destructuring Objects Part 2 Lesson #119

Destructuring Function parameters Lesson #120

Destructuring Mixed Content Lesson #121

Destructuring Challenge Lesson #122

Assignments From [115] = > [122]

Week 17

Set Data Type And Methods Lesson #123

Set vs WeakSet And Garbage Collector Lesson #124

Map Data Type vs Object Lesson #125

Map Methods Lesson #126

Map vs WeakMap Lesson #127

Array.from Method Lesson #128

Array.copyWithin Method Lesson #129

Array.some Method Lesson #130

Array.every Method Lesson #131

Spread Syntax And Use Cases Lesson #132

Map And Set Challenge Lesson #133

Assignments From [123] = > [133]

Week 18

Intro And What Is Regular Expression Lesson #134

Regular Expression – Modifiers Lesson #135

Regular Expression – Ranges Part 1 Lesson #136

Regular Expression – Ranges Part 2 Lesson #137

Regular Expression – Character Classes Part 1 Lesson #138

Regular Expression – Character Classes Part 2 Lesson #139

Regular Expression – Quantifiers Part 1 Lesson #140

Regular Expression – Quantifiers Part 2 Lesson #141

Regular Expression – Quantifiers Part 3 Lesson #142

Regular Expression – Replace With Pattern Lesson #143

Regular Expression – Form Validation Lesson #144

Test Your Regular Expression Lesson #145

Regular Expression – Challenge Lesson #146

Assignments From [134] = > [146]

Week 19

OOP Introduction Lesson #147

Constructor Function Introduction Lesson #148

Constructor Function New Syntax Lesson #149

Deal With Properties And Methods Lesson #150

Update Properties And Built In Constructors Lesson #151

Class Static Properties And Methods Lesson #152

Class Inheritance Lesson #153

Class Encapsulation Lesson #154

Prototype Introduction Lesson #155

Add To Prototype Chain Lesson #156

Object Meta Data And Descriptor Part 1 Lesson #157

Object Meta Data And Descriptor Part 2 Lesson #158

Assignments From [147] = > [158]

Week 20

Date And Time Introduction Lesson #159

Get Date And Time Lesson #160

Set Date And Time Lesson #161

Formatting Date And Time Lesson #162

Tracking Operations Time Lesson #163

Generator Function Introduction Lesson #164

Delegate Generator Function Lesson #165

Generate Infinite Numbers Lesson #166

Module Import And Export Lesson #167

Named vs Default Export And Import All Lesson#168

Assignments From [159] = > [168]

Week 21

What Is JSON Lesson #169

JSON Syntax And Compare With JS Object Lesson #170

What Is API Lesson #171

Parse And Stringify Lesson #172

Asynchronous vs Synchronous Programming Lesson #173

Call Stack And Web API Lesson #174

Event Loop And Callback Queue Lesson #175

What Is AJAX And Network Informations Lesson #176

Request and Response From Real API Lesson #177

Loop On Data Lesson #178

Assignments From [169] = > [178]

Week 22

Callback Hell Or Pyramid Of Doom Lesson #179

Promise Intro And Syntax Lesson #180

Promise – Then, Catch And Finally Lesson #181

Promise And XHR Lesson #182

Fetch API Lesson #183

Promise – All, All Settled And Race Lesson #184

Async And Trainings Lesson #185

Await And Trainings Lesson #186

Try, Catch, Finally With Fetch Lesson #187

The End And Advices Lesson #188

Assignments From [179] = > [188]

الخاتمة وشويه فضفضة ♥

♥ فقرة إضافية حببت اضيفها ♥

مقدمة بسيطة مهمة وكان لابد منها

سلامٌ من الله عليكم ورحمته وبركاته في الحقيقة انا قبل ما انشر الملخص دا كان عندي إحساس انه للأسف مش على نفس مستوي كل الملخصات اللي فاتت ودا شيء مضايقي جدا ويارب يكون إحساس زائف بس دا كان راجع لسببين

السبب الأول: هو سبب خارج عن ارادتي وهو ان من كتر طلب الجزء الثاني دا من الملخص كنت محطوط تحت ضغط إني عايز انجزه، ولكن صدقني انا اتقيت الله في عملي فيه متقلقش ولكن مستواه مكنش المستوي اللي انا بطمح ليه ويمكن لو كنت خدت وقت زيادة كنت تعمقت اكتر واكثر فيه ودا ينقلنا لتاني سبب

اما السبب الثاني: ففي الحقيقة ان الجزء دا من الملخص ككل انا اعتمدت فيه علي ان اللي هيبدأ يقرأ هو بقي شخص ناضج برمجيا لفهم ابسط الأشياء التي لا داعي للتوغل فيها زي ما كنت بعمل في الجزء الأول

بمعني؟!

الجزء الأول انا كنت مهتم جدا بتفصيل كل حاجة حرفيا لان خدت في الحساب ان الشخص اللي هيقرأه هيكون دا أول احتكاك ليه باللغة دي + ربما يكون اول احتكاك ليه بالبرمجة بشكل عام فكنت مهتم اتعمق في تفاصيل داخل تفاصيل عشان أوضح كل حاجة قدر الامكان لكن في الجزء الثاني انا اعتمدت ان الشخص خلاص بقا عنده قدرة يحلل بعض الحاجات ويستنتج من تلقاء نفسه حاجات انا مش موضحها، بل يمكن صممت ان الشخص لازم يقرأ كويس ويحلل اللي قرأه ويشوف نتيجة المكتوب بناء علي تحليلات من دماغه

بعذر عن المقدمة الطويلة دي ويلا بينا نبداً واتمني من الله ان يكون بمستوي كويس ويرتقي لمستوي اعجاب حضراتكم جميعا والاستفادة منه ولو فعلا مكنش يرتقي لنفس مستوي المخلصات القديقة فأرجو منكم ان تلتمسوا لي العذر

افتكروننا بدعوة طيبة لوجه الله

By: Amr Elsayed

What Is BOM ? Lesson #102

```
/*
BOM [Browser Object Model]
- Introduction
--- Window Object Is The Browser Window
--- Window Contain The Document Object
--- All Global Variables And Objects And Functions Are Members Of Window Object
----- Test Document And Console
- What Can We Do With Window Object ?
--- Open Window
--- Close Window
--- Move Window
--- Resize Window
--- Print Document
--- Run Code After Period Of Time Once Or More
--- Fully Control The URL
--- Save Data Inside Browser To Use Later
*/
```

ال BOM هي اختصار لـ Browser Object Model وهي ببساطة شديدة جدا بتعتبر عن ال Window obj زي ما اتعودنا ديما ان كل Object و أي Method بيكون جواها شوية خصائص خاصة بيها بتعمل وظيفة معينة ال Window object كذلك بداخله خصائص كتير جدا بقدر استخدمها في التعامل مع ال Window او المتصفح بشكل اخر او ال Page الي انت بتتصفحها هي دي ال Window

من امثلة الأشياء الي بيحتوي عليها ال Window obj زي مثلا ال DOM الي يعتبر جزء من ال Window زي ال Console الي كنت ديما بستخدمه .. أي Global Variable او أي Object او Function هي جزء من ال Window

```
window.console.log("Hello From Window");
```

Hello From Window

main.js:20

كذلك ال Document هو جزء من ال Window

```
window.document.title = "Hello JS";
```

1- ممكن نفتح Window جديدة او نقفل ال Window الحالية

2- نعمل Resize لل Window للحجم اللي يناسبك يعني تحكم كامل في ال Window

3- ممكن تعمل Print لل document او تعمل print لجزء معين محدد انت عايزوا

4- ممكن تعمل Run لأي كود في الوقت اللي انت عايزوا زي مثلا لما تدخل موقع تلاقي بعد دقيقة طلع Pop Up هنا هو بيكون محدد Time للوقت اللي كود ال Pop Up دا يشتغل فيه

5- تقدر تتحكم تحكم كامل في ال URL وتفصصه لكل مكوناته

6- ممكن تعمل Save لمجموعة بيانات داخل الموقع

لا تنسونا من دعواتكم

Never Forget That (**Always Search**)

Alert, Confirm, Prompt Lesson #103

```
/*  
  BOM [Browser Object Model]  
  - alert(Message) => Need No Response Only Ok Available  
  - confirm(Message) => Need Response And Return A Boolean  
  - prompt(Message, Default Message) => Collect Data  
*/
```

مبدأيا كدا أي حاجة في الدرس دا هي بس من باب العلم بيها مش اقدر ولا اقل وهتقدر بعد كدا تعمل نفس الحاجات دي بشكل افضل واجمل كثير يعني

ال **Alert** ممكن اكتبه بكذا طريقة وكلهم صح ... ال **Alert** بيطلب مسج هتظهر للشخص مجرد ما يدوس **OK** تختفي خالص

```
window.alert("Test");  
this.alert("Test");  
alert("Test");
```

127.0.0.1:5501 says

Test

OK

أسوء حاجة في ال **Alert** انها بتوقف تحميل الصفحة او البيانات الي فيها لحد ما تدوس **OK** وتقفها ودا بيعطل تحميل الصفحة

Sweetalert

دا بديل كويس جدا لـ **Alert** العادي

ال **Confirm** بقا بيتم استخدامه للتأكد هل انت عايز تنفذ ال **Action** دي ولا لاء علي سبيل المثال لو بتتصفح منتجات في موقع **Amazon** وانت بتضيف منتجات للسلة حبيت تشيل منهم منتج هنا بيطلعك مسج تقولك متأكد انك عايز تشيل المنتج دا النتيجة بتكون **True** لو موافق و **False** لعدم الموافقة والتراجع عن حذف المنتج

```
let confirmMsg = confirm(`Are You Sure?`);
console.log(confirmMsg);

if (confirmMsg === true) {
  console.log("item Deleted");
} else {
  console.log(`Item Not Deleted`);
}
```

ال **Confirm** بيكون فيها **Message** لو ضغط **Ok** يعني **True** ولو ضغط **Cancel** يعني **False** إذا ال **Confirm** دي بترجع **Boolean Value** وعلي أساس ال **Value** دي بيتنفذ ال **Action**

في ال **Code** الي فوق دا انا عملت متغير اسمه **confirmMSg** بخزن فيه ال **Confirm** بال **Message** بتاعتها بعد كدا عملت شرط والله لو ال **confirmMSg** اليوزر ضغط **OK** يعني بقت **True** اطبع رسالة **"Item Deleted"** ولو اليوزر ضغط **Cancel** يعني **False** اطبع رسالة **"Item Not Deleted"**

انا عملت متغير ال **confirmMSg** عشان اقدر استخدم المتغير دا نفسه في اني اعمل بيه الشرط ❤️

127.0.0.1:5501 says
Are You Sure?

OK

Cancel

في حالة ال **OK** يعني **True**

true main.js:14

item Deleted main.js:17

127.0.0.1:5501 says
Are You Sure?

OK

Cancel

في حالة ال **Cancel** يعني **False**

false main.js:14

Item Not Deleted main.js:19

ال Prompt هنا مفيش Ok و Cancel لا هنا المطلوب منك تكتب حاجة

```
let promptMsg = prompt("Good Day To You?", "Write Day With 3 Characters");  
console.log(promptMsg);
```

ال prompt دا بيطلب مني رسالة هتظهر لليوزر وبعدها Default Message تكون زي Placeholder كدا
Write Day With 3 Characters هي دي ال Placeholder او ال Default Message

127.0.0.1:5501 says

Good Day To You ?

Write Day With 3 Characters

OK

Cancel

OK

Cancel

127.0.0.1:5501 says

Good Day To You ?

Sat

OK

Cancel

OK

Cancel

Sat

main.js:14

لا تنسونا من دعواتكم

By: Amr Elsayed

♥ راقّت لي ♥

~~One~~
Day
One

شيل من قاموسك في يوم من الأيام هعمل كذا وكذا وخلي في قاموسك انا انهاردا هعمل 1 و 2 و 3
مفيش بکرا دي انساهها وابدأ دلوقتي حالا بالمتاح

♥ اذكر الله

الزموا ذكر الله ففيه العون على متاعب الحياة ♥
اللَّهُمَّ إِنِّي أَسْأَلُكَ عِلْمًا نَافِعًا، وَرِزْقًا طَيِّبًا، وَعَمَلًا مُتَقَبَّلًا.
اللَّهُمَّ أَعِنِّي عَلَى ذِكْرِكَ، وَشُكْرِكَ، وَحُسْنِ عِبَادَتِكَ.
اللهم صلّ وسلم وزد وبارك على نبينا محمد ﷺ

لا تنسونا من دعواتكم

By: Amr Elsayed

setTimeout and clearTimeout Lesson #104

```
/*  
  BOM [Browser Object Model]  
  - setTimeout(Function, Timeout, Additional Params)  
  - clearTimeout(Identifier)  
*/
```

عن طريق ال **setTimeout Method** هنقدر نشغل **Function** معينه بعد وقت معين انا الي بحدده

ال **setTimeout** دي بتقبل مني أول حاجة ال **Function** الي انا هستدعيها ال **Function** دي اكيد بيكون جواها حاجه محتاج انها تتنفذ

بعد كدا بيطلب مني ال **Timeout** او الوقت الي بيكون بال **MS** الي هي ال **milliseconds** يعني **3000 = 3 ثواني**

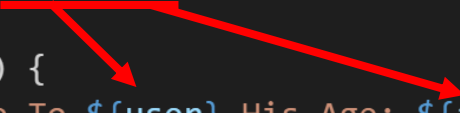
```
setTimeout(function () {  
  console.log("Msg");  
}, 3000);
```

المفروض ان الكود دا معناه انه هيطبع كلمه **"Msg"** بعد **3** ثواني في ال **Console**

ممکن استخدم **Named Function** عادي واعوض باسمها في ال **setTimeout** زي المثال دا وهتنفذ المطلوب

```
setTimeout(sayMsg, 3000);  
  
function sayMsg() {  
  console.log("Iam Message");  
}
```

```
setTimeout(sayMsg, 3000, "Amr", 23);  
  
function sayMsg(user, age) {  
  console.log(`Iam Message To ${user} His Age: ${age}`);  
}
```



في المثال دا لو عندي **Named Function** جواها **Parameters** عايز استخدمها واعوض مكانها بـ **Value** ومع استخدامي لـ **setTimeout** بيكون الحل الصحيح اني احط الـ **Arguments** بتاعت الـ **Parameters** دي بعد الـ **Timeout**

الـ **clearTimeout** هيقف عمل الـ **setTimeout** ... يعني لو عندي **Function** قدامها 5 ثواني وتطبع رسالة معينة الـ **clearTimeout** بيوقف عمل الـ **Function** دي

```
<button>Stop</button>
```

```
let btn = document.querySelector("button");  
  
let counter = setTimeout(sayMsg, 3000);  
  
function sayMsg() {  
  console.log(`Iam Message`);  
}  
  
btn.onclick = function () {  
  clearTimeout(counter);  
};
```

الـ **clearTimeout** بيقبل ميني **ID** أو **Counter**

طيب شرح الكود اللي فات دا ايه

عملت زرار اسمه **Stop** و مسكت الزرار دا وخزنته في المتغير **btn**

عملت **Named Function** باسم **sayMsg** كل وظيفتها انها تطبع رسالة "**Iam Message**"

بعد كدا استخدمت ال **sayMsg** دي في ال **setTimeout** ومعاها ال **time** نفسه

خزنت ال **setTimeout** دي نفسها في متغير اسمه **counter** اللي هستخدمه عشان امره في ال **clearTimeout** عشان يوقف عمل ال **setTimeout**

استخدمت **Event** ال **click** مع الزرار بتاعي عشان يعمل **Function** دورها تشغل ال **clearTimeout** اول ما اضغط ع الزرار دا وبداخل ال **clearTimeout** حطيت المتغير **counter** اللي بيعبر عن ال **setTimeout**

بمجرد ما اضغط ع زرار ال **Stop** ال **Function** بتاعت ال **setTimeout** مش هتشتغل

ممکن اكتب ال **Event** ال **Click** كدا كمان بال **addEventListener**

```
btn.addEventListener("click", function () {
  clearTimeout(counter);
});
```

لا تنسونا من دعواتكم

Never Forget That (**Always Search**)

By: Amr Elsayed

setInterval and clearInterval Lesson #105

```
/*  
  BOM [Browser Object Model]  
  - setInterval(Function, Millseconds, Additional Params)  
  - clearInterval(Identifier)  
*/
```

ببساطة شديدة جدا ال **setInterval** هي بالظبط زي ال **setTimeout** في طريقة الكتابة الفرق الوحيد بينهم ان ال **setTimeout** بتنفذ **Function** مرة واحدة بعد الوقت الي بتحدده اما ال **setInterval** فهي بتنفذ ال **Function** علي **حسب الوقت** الي انت محدده ولكن بدون توقف هتفضل تعيد تنفيذ ال **Function** لحد ما تعملها **clear**

يبقي ال **setTimeout** بتنفذ **Function** مرة واحدة بس وخلص اما ال **setInterval** بتكرر ال **Function** لحد ما تعملها **Clear**

المثال الأول ل **Anonymous Function**

```
setInterval(function () {  
  console.log(`Msg`);  
}, 1000);
```

23 Msg

main.js:8

معني الكود دا ان كلمة **Msg** هتتبع في ال **Console** كل ثانية لمالا نهاية لحد ما تعمل **Clear** لا Code

المثال الثاني ل **Named Function**

```
setInterval(sayMsg, 1000);  
  
function sayMsg() {  
  console.log(`Iam Message`);  
}
```

3 Iam Message

main.js:14

كذلك الحال لو فيه **Arguments** كل حاجه عملنها في ال **setTimeout** هتتكرر عادي بس فيه فرق زي ما قولنا ان ال **setInterval** بيفضل يكرر ال **Function**

```
setInterval(sayMsg, 1000, "Osama", 38);

function sayMsg(user, age) {
  console.log(`Iam Message For ${user} His Age Is: ${age}`);
}
```

5 Iam Message For Osama His Age Is: 38 [main.js:20](#)

<div>5</div>

5

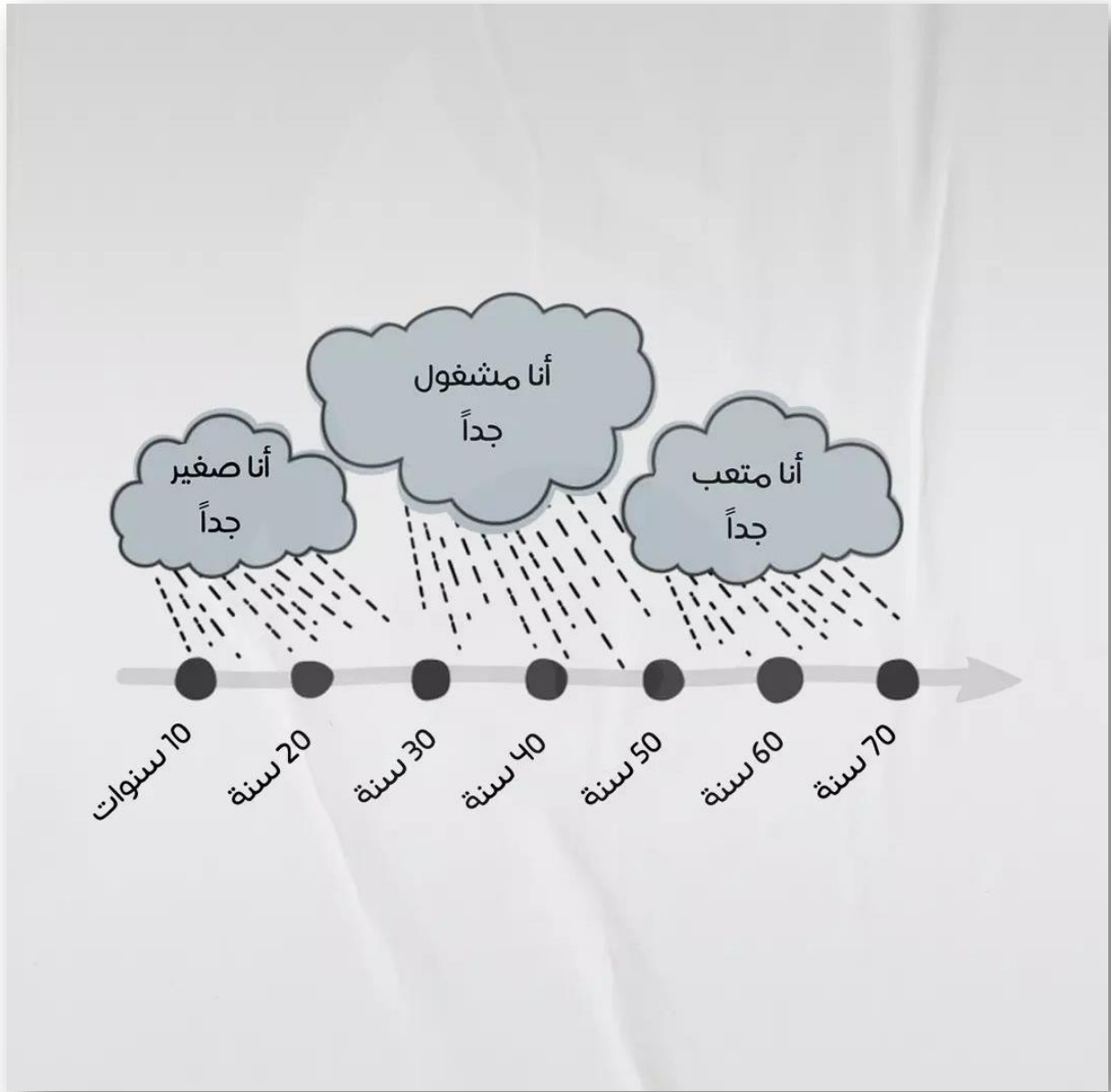
```
let div = document.querySelector("div");

function countdown() {
  div.innerHTML -= 1;
  if (div.innerHTML === "0") {
    clearInterval(counter);
  }
}

let counter = setInterval(countdown, 1000);
```

هنا بقا انا عندي **Counter** بيقلل من الرقم 5 ويفضل ينقص 1 كل ثانيه ممكن استخدم ال **clearInterval** بدون ما يكون في زرار ... ببساطة شديدة جدا هعمل شرط لما الرقم 5 دا يسااوي "0" العداد يقفل او ال **setInterval** تقف ومتكملش العد التنازلي

عبارات تحفيزية



بلاش اعذار وحجج او بلاش تحب نفسك في القاعدة والانتخه قوم اعمل لنفسك جدول بسيط حدد فيه أولوياتك وابدأ تنفيذها واحدة ورا الثانية وكرر دا يومياً

اذكر الله

لا إله الا الله وحده لا شريك له .. له الملك وله الحمد وهو على كل شيء قدير

استغفروا الله العظيم الذي لا إله الا هو الحي القيوم واتوب اليه

اللهم صلّ وسلم وزد وبارك على نبينا محمد ﷺ

لا حول ولا قوة الا بالله العلي العظيم

لا إله الا انت سبحانك إني كنت من الظالمين

لا تنسونا من دعواتكم

Window Location Object Lesson #106

```
/*  
  BOM [Browser Object Model]  
  - location Object  
  --- href Get / Set [URL || Hash || File || Mail]  
  --- host  
  --- hash  
  --- protocol  
  --- reload()  
  --- replace()  
  --- assign()  
*/
```

عن طريق الـ **location object** ومن خلال الـ **methods** الي جواها هنقدر نتعامل مع كل حاجة تخلص الـ **URL** ولو حببت استعلم عن الـ **location** دا هلاقية فيه كتير من الـ **properties** و الـ **methods**

```
console.log(location);
```

```
main.js:12  
Location {ancestorOrigins: DOMStringList, href: 'http://127.0.0.  
▶ 1:5501/index.html', origin: 'http://127.0.0.1:5501', protocol:  
'http:', host: '127.0.0.1:5501', ...}
```

```
console.log(location.href);
```

ممكن استعلم عن الـ **href** بتاعي او بمعنى اصح اعمله **get** وبعدين اغير قيمته لو حابب **set**

```
http://127.0.0.1:5501/index.html main.js:13
```

```
location.href = "https://google.com";
```

ملحوظة في استخدام الـ **href** هتلاحظ ان أي موقع بتتصفح في الـ **Browser** هيكون متسجل يعني أي موقع هتفتحه هيكون متسجل في الـ **history** ليه بقولك كدا عشان في الـ **methods** تانية ممكن استخدمها **بتحذف** أي تصفح بيتم من الـ **history**

HTML

```
<div id="sec01" style="height: 100vh">Section One</div>  
<div id="sec02" style="height: 100vh">Section Two</div>
```

JS

```
location.href = "/#sec02";
```

كمان من خلال ال **href** اقدر اروح لاي **section** موجود عندي في الصفحة او اقدر اروح لاي صفحة ثانية في أي موقع ثاني

بيتم استخدام ال **host** في اني اعرف اسم الموقع بتاعي وكمان ال **port** الي مفتوح منه وال **hostname** زيه ولكن الفرق انه مش بيحيب ال **port** بس مش اكرر

```
console.log(location.host);  
console.log(location.hostname);
```

```
127.0.0.1:5501
```

```
127.0.0.1
```

يعني لو فتحت موقع اكاديمية الزيرو واستعلمت عن ال **host** هلاقي اسم الموقع

```
> location.host  
◁ 'elzero.org'  
> location.hostname  
◁ 'elzero.org'
```

الفكرة كلها ان الموقع مفهوش **port** لذلك ال **Host** وال **hostname** هتلاقيهم زي بعض

عن طريق ال **hostname** لو فاتح موقع وفاتح من خلاله اكر من سيكشن متفرع تقدر تغير ال **hostname** بس وباقي ال **sections** المفتوحة هتفضل موجودة في ال **url** زي ما هي

```
console.log(location.protocol);
```

عن طريق ال **protocol** بقدر استعلم عن الاتفاقية الي بفتح منها الموقع هل هي **http** ولا **https**

```
http:
```

واقدر احول من **https** ل **http** وهكذا

عن طريق الهاش **hash** بقدر استعلم عن الصفحات المتفرعة من الموقع الي انا فاتحه

```
console.log(location.hash);
```

يمكن استخدام ال **hash** فيما بعد عشان لو حابب تعمل **check** علي حاجة معينة لو حد مثلا جاي من صفحة معينة وعالز **action** معين يحصل لما يوصل للموقع بتاعك وهكذا

```
#sec02
```

```
window.location.reload();
```

بيتم استخدامها لعمل **reload** للصفحة لو حابب تستخدم دا

```
// - replace()  
// - assign()
```

```
location.replace();
```

الفرق بينهم ان استخدام ال **replace** مبيخلش الصفحات تتسجل في ال **History** لانه بيعمل استبدال للصفحة المفتوحة بصفحة ثانية فدي بتحل محل دي ومبيكونش ليها وجود في ال **history** لو اتغيرت بعد كدا لصفحة ثانية

```
replace(url: string | URL): void  
  
Removes the current page from the session history and  
navigates to the given URL.  
  
location.replace()
```

```
location.assign();
```

اما ال **assign** بتحتفظ بالصفحات عادي في ال **history** زيها زي ال **href**

لا تنسونا من دعواتكم

Never Forget That (**Always Search**)

By: Amr Elsayed

Window Open And Close Lesson #107

```
/*
  BOM [Browser Object Model]
  - open(URL [Opt], Window Name Or Target Attr [Opt], Win Features [Opt], History Replace [Opt])
  - close()
  - Window Features
    --- left [Num]
    --- top [Num]
    --- width [Num]
    --- height [Num]
    --- menubar [yes || no]

  Search
  - Window.Open Window Features
*/
```

أولاً ال **Close** المتخصصة في قفل ال **Window** مش هتشتغل غير مع **Window** معمولها **Open** من ال **JS**

ال **window.open** بقا بتقبل ميني 4 حاجات وكلهم اختياري وفي حاجات ليها **Default Value**

URL [Opt] الخاص بال **window** اللي هتتفتح

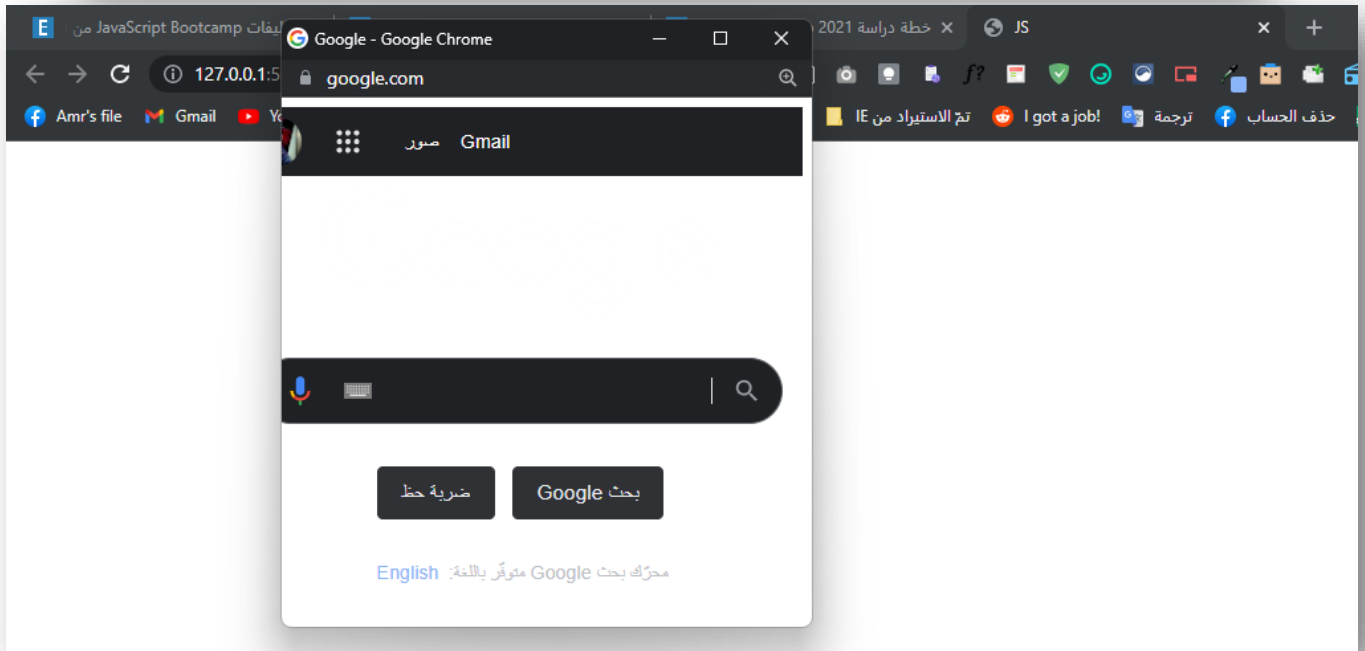
Window Name Or Target Attr [Opt] بتدي **identifier** للصفحة بتاعتك او ممكن تكتب

target attribute زي ال **_blank** اللي بيفتح الصفحة في **New Tab** او تكتب **_self** اللي بيفتح الصفحة مكان المفتوحة

Win Features [Opt] والي من خلالها تقدر تتحكم في المميزات بتاعتها زي طولها وعرضها وموقعها ف الشاشة وهكذا

History Replace [Opt] لو انت عايز تفتح **Window** مكان **window** تانية هل هتسيبها في ال **History** ولا لا وبتقبل **boolean value**

```
setTimeout(function () {  
  window.open(  
    "https://google.com",  
    "_blank",  
    "width=400,height=400,left=200,top=10"  
  );  
}, 2000);
```



اذكر الله ❤️

الزموا ذكر الله ففيه العون على متاعب الحياة ❤️

اللهم صلّ وسلم وزد وبارك على نبينا محمد ﷺ

سبحان الله وبحمده عدد خلقه ورضا نفسه وزنة عرشه ومداد كلماته

لا إله الا انت سبحانك اني كنت من الظالمين

لا حول ولا قوة الا بالله العلي العظيم

أَلَا بِذِكْرِ اللَّهِ تَطْمَئِنُّ الْقُلُوبُ

لا تنسونا من دعواتكم

Never Forget That (**Always Search**)

By: Amr Elsayed

Window History Object Lesson #108

```
/*
  BOM [Browser Object Model]
  - History API
  --- Properties
  ----- length
  --- Methods
  ----- back()
  ----- forward()
  ----- go(Delta) => Position In History

  Search [For Advanced Knowledge]
  - pushState() + replaceState()
*/

console.log(history);
```

```
history.length;
```

```
> history.length
< 8
```

بيجيب ال **Length** او عدد الصفحات الي اتفتحت في ال **History**

```
history.back();
history.forward;
```

من خلالهم بقدر ارجع خطوة للخلف او اروح خطوة للامام

```
history.go()
```

بحط جواها ال **position** او رقم الصفحة الي عايز انتقل ليه
ولو كتبت القيمة **0** هيعمل **Reload** للصفحة بتاعتك

♥ راقِ لي ♥

8 طرق لكيفية التنمية الذاتية :

1. القراءة يومياً
2. التغلب على المخاوف
3. التغذية الراجعة من الناس
4. الإقلاع عن العادات السيئة
5. تجنب الأشخاص السلبيين
6. التعلم من الأخطاء
7. تحديد الأهداف
8. التمرن يومياً
9. التفكير الإيجابي

تابع [@abduhadi_plus](https://www.instagram.com/abduhadi_plus)


ABDULHADI
خبير تسويق إلكتروني و مدرب تنمية ذاتية

♥ اذكر الله ♥

الزموا ذكر الله ففيه العون على متاعب الحياة ♥

اللهم إني أستغفرك واتوب إليك، لا إله الا انت سبحانك إني كنت من الظالمين.

حسبي الله لا إله الا هو عليه توكلت وهو رب العرش العظيم.

أَلَا بِذِكْرِ اللَّهِ تَطْمَئِنُّ الْقُلُوبُ

لا تنسونا من دعواتكم

By: Amr Elsayed

Scroll, ScrollTo, ScrollBy, Focus, Print, Stop Lesson #109

```
/*  
  BOM [Browser Object Model]  
  - stop()  
  - print()  
  - focus()  
  - scrollTo(x, y || Options)  
  - scroll(x, y || Options)  
  - scrollBy(x, y || Options)  
*/
```

```
window.stop()  
window.print()  
window.focus()
```

ال **Stop** يستخدم لعمل **Stop** للصفحة وهي بتعمل **Reload**
ال **print** يستخدم لفتح قائمة الطباعة .. ومستقبلا هتقدر تتحكم
في الجزء المعين اللي حابب انه يطبع من الصفحة
ال **focus** يستخدم في لو عندي صفحة معمولها **open** من خلالي
اني اقدر اعمل **focus** عليها

أي الفرق بين كلمة **scroll** بس و **ScrollTo** الاتنين بيعملوا نفس النتيجة، ولكن في مشكلة في التوافقية لآحدهم
في بعض المتصفحات مش اقدر

```
window.scrollTo(500, 200);
```

ال **ScrollTo** يستخدم عشان لو حابب اعمل **Scroll** لمكان محدد من الصفحة ولو فضلت تكررها هيعيد
ال **scrolling** لنفس الابعاد من جديد مش هيكمل ع القديم يعني

```
window.scrollBy(500, 200);
```

ال **ScrollBy** يستخدم عشان لو حابب اعمل **Scroll** لمكان محدد من الصفحة ولو فضلت تكررها هيكمل ال
scrolling لنفس الابعاد يعني كل مره تكتبها هيعمل **scroll** تاني ويكمل ع القديم عكس ال **ScrollTo**

بمعني أوضح الفرق بين ال **ScrollTo** وال **scrollBy** هو ان ال **ScrollTo** لما تستخدمها وتكتب فيها احداثيات معينة هيوديك للاحداثيات دي ولو كررت كتابة ال **ScrollTo** بنفس الاحداثيات هوديك ليها برضو من جديد

اما ال **scrollBy** بقا لو كتبت احداثيات معينة هتروح ليها ولو عيبتها تاني هتبدأ من مكان ما كنت واقف عند اخر احداثيات وصلتها وتكمل عليها ملوش علاقة انت واقف فين مكان ما تكون واقف هيتحرك تاني مش هيبدأ من الأول زي ال **ScrollTo** ويروح لنفس الاحداثيات

ال **500** دي في كل الأحوال بتعبر عن المحور **X** اللي هو المحور الافقي او ال **scroll** اللي بالعرض
اما ال **200** دي بتعبر عن المحور **Y** اللي هو المحور الرأسى او العمودي او ال **scroll** اللي بالطول

```
window.scrollTo({
  left: 2000,
  top: 3000,
  behavior: "smooth",
});
```

هنا انا بعمل **Scroll** من ع الشمال **2000px** ومن ع اليمين **3000px** وهتحكم في سلوب ال **scroll** بحيث يكون **smooth** او ناعم اثناء عملية ال **scrolling**

لا تنسونا من دعواتكم

Never Forget That (**Always Search**)

By: Amr Elsayed

Scroll To Top Using Scrolly Lesson #110

```
/*  
  BOM [Browser Object Model]  
  - Practice => Scroll To Top  
  - scrollX [Alias => PageXOffset]  
  - scrolly [Alias => PageYOffset]  
*/
```

أولا كذا ال `scrollX === pageXOffset` وكذلك ال `scrollX === pageYOffset` .. يعني ممكن استخدم أي واحد فيهم مع الاخذ بالاعتبار ان ال `PageOffset` لو بتعامل اكر مع متصفحات قديمة

```
console.log(window.scrollX === window.pageXOffset);
```

true

main.js:26

```
<button>Up</button>
```

Up

```
button {  
  background-color: #e91e63;  
  border: none;  
  padding: 8px;  
  color: white;  
  font-size: 18px;  
  border-radius: 10px;  
  position: fixed;  
  right: 20px;  
  bottom: 20px;  
  display: none;  
  cursor: pointer;  
}
```

```

let btn = document.querySelector("button");

window.addEventListener("scroll", () => {
  if (window.scrollY >= 600) {
    btn.style.display = "block";
  } else {
    btn.style.display = "none";
  }
});

btn.addEventListener("click", () => {
  window.scrollTo({
    left: 0,
    top: 0,
    behavior: "smooth",
  });
});

```

طيب تعالي نفهم معني الكود دا بالظبط أول حاجه انا عملت **Target** لـ **button** بتاعي و خزنته في **btn**

بعد كدا عملت **Event** الـ **scroll** من خلال الـ **addEventListener** وعملت **Function** هتأدي وظيفة معينة

عملت بعد كدا شرط بقوله فيه لو الـ **window.scrollY** بقت اكبر من او يساوي **600** هنعمل ظهور لـ **button** المختفي .. يعني بمجرد ما اعمل **scroll** أوصل للقيمة **600** بالطول هيبدأ يظهر الـ **button** في الصفحة والـ **else** الأخيرة بتقول العكس اول ما أكون اقل من الـ **600** دي الزرار دا يرجع يختفي تاني

بعد كدا عملت **Event** كمان خاص بالـ **click** علي الزرار بقول فيه اول ما اضغط ع الزرار دا يعمل الـ **function** اللي منها هستخدم الـ **ScrollTo** اللي سبق واتعلمت منها اني بحدد فيها قيم او احداثيات عشان أتوجه للاحداثيات دي

بمجر ما اضغط ع الزرار هنتقل للاعلي خالص بطريقة **Smooth** وناعمة بس كدا

♥ عبارات تحفيزية ♥



♥ بلاش حجج وكسل ومبررات فاضيه اشتغل ع نفسك اكر

♥ اذكر الله

سبحان الله وبحمده .. سبحان الله العظيم

أستغفر الله العظيم الذي لا اله الا هو الحي القيوم و أتوب إليه

حسبي الله لا اله الا هو عليه توكلت وهو رب العرش العظيم

لا تنسونا من دعواتكم

By: Amr Elsayed

Assignments From [102] => [110]

لينك التكميلات: [\[110\] <= \[102\]](#)

التكليف: 01

التكليف 01

- قم بإستعمال ال Prompt لجلب رسالة من المستخدم
- الرسالة ستكون كالتالي "Print Number From – To"
- الرسالة الافتراضية ستكون كالتالي "Example: 5-20"
- سيقوم الشخص بكتابة رقمين وبينهم علامة –
- يمكن أن يكون الرقم الأول أكبر من الثاني أو العكس
- في كلا الحالتين نقوم بطباعة الأرقام داخل الصفحة من أصغر رقم حتى أكبر رقم. شاهد المثال لتفهم الفكرة

```
// Assignment 1
let promptNum = prompt("Print Numbers From - To", "Example 5 - 20");

let counter = promptNum.split("-").sort();

let count = setInterval(() => {
  let myDiv = document.createElement("div");
  myDiv.innerHTML = counter[0]++;
  if (myDiv.innerHTML === counter[1]) {
    clearInterval(count);
  }
  document.body.appendChild(myDiv);
}, 0);
```

التكليف: 02

التكليف 02

- قم بإنشاء Function تقوم بإنشاء Popup داخل الصفحة كما في الصورة التالية
- ال Popup تحتوي على زرار Close عند الضغط عليه يقوم بحذف ال Popup من الصفحة
- يجب عليك تشغيل ال Function بعد 5 ثواني من تحميل الصفحة

// Assignment 2

```
let div = document.createElement("div");
let heading = document.createElement("h1");
let msg = document.createElement("p");
let button = document.createElement("button");

let headingText = document.createTextNode("Welcome");
heading.appendChild(headingText);

let msgText = document.createTextNode("Welcome To Elzero Web School");
msg.appendChild(msgText);

let buttonSign = document.createTextNode("X");
button.appendChild(buttonSign);

div.appendChild(heading);
div.appendChild(msg);
div.appendChild(button);

// Styling

document.body.style.cssText =
  "font-family: Tohama, Arial; display: flex; justify-content: center;
  align-items: center; text-align: center;";

div.style.cssText =
  "position: relative; transform: translateY(50%); font-size: 15px;
  background-color: #F5F3F4; padding: 20px 100px;
  box-shadow: -1px 3px 28px -7px;";

msg.style.cssText = "font-size: 15px;";

button.style.cssText =
  "position: absolute; right: -3%; top: -9px; border: 2px solid white;
  border-radius: 50%; background-color: red; width: 30px; height: 30px;
  font-size: 17px; color: white; font-weight: bold; cursor: pointer;";

setTimeout(() => {
  document.body.appendChild(div);
}, 5000);
button.addEventListener("click", () => {
  div.remove();
});
```

التكليف 03

■ قم بإنشاء Div داخل الصفحة وبداخله رقم 10 على سبيل المثال

■ قم بعمل Counter يقوم بالعد من الرقم الموجود داخل ال Div وحتى يصل إلى الصفر

■ عند الوصول إلى الصفر قم بإيقاف العد

```
// Assignment 3
let downCount = document.querySelector("div");

let count = setInterval(() => {
  downCount.innerHTML -= 1;
  if (downCount.innerHTML === "0") {
    clearInterval(count);
  }
}, 1000);
```

التكليف 04

■ قم بإستعمال نفس التكليف السابق كما هو بدون اي تغيير

■ عند الوصول للرقم 0 بدلا من إيقاف العداد قم بتحويل الصفحة الحالية لموقع الأكاديمية Elzero.org

```
// Assignment 4
let urlChange = document.querySelector("div");

setInterval(() => {
  urlChange.innerHTML -= 1;
  if (urlChange.innerHTML === "0") {
    window.location.assign("https://elzero.org/");
  }
}, 1000);
```

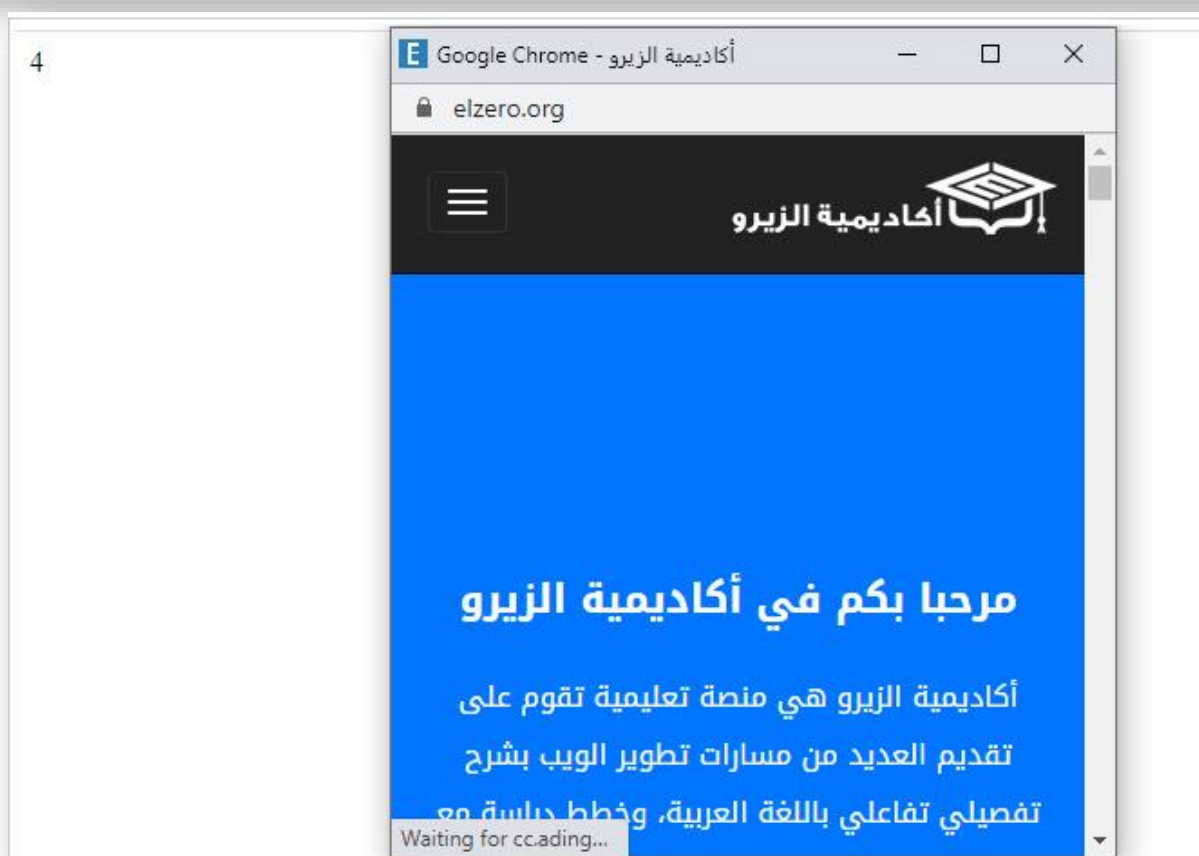
التكليف 05

■ قم بإستعمال نفس التكليف السابق كما هو بدون اي تغيير

■ عند الوصول للرقم 5 قم بفتح صفحة موقع الأكاديمية Elzero.org في Popup

■ شاهد الصورة لتفهم الفكرة

■ عند الوصول للرقم صفر لا تنسى إيقاف العداد



```
// Assignment 5
let openURL = document.querySelector("div");

let count = setInterval(() => {
  openURL.innerHTML -= 1;
  if (openURL.innerHTML === "5") {
    window.open(
      "https://Elzero.org",
      "_blank",
      "width= 400, height= 400, left= 300, top = 100"
    );
  } else if (openURL.innerHTML === "0") {
    clearInterval(count);
  }
}, 1000);
```

♥ راقث لي ♥



اذكر الله ♥

الزموا ذكر الله ففيه العون على متاعب الحياة ♥

لا إله الا الله وحده لا شريك له .. له الملك وله الحمد وهو على كل شيء قدير.

استغفروا الله العظيم الذي لا إله الا هو الحي القيوم واتوب اليه.

اللهم صلّ وسلم وزد وبارك على نبينا محمد ﷺ.

لا إله الا انت سبحانك أني كنت من الظالمين.

أَلَا بِذِكْرِ اللَّهِ تَطْمَئِنُّ الْقُلُوبُ

لا تنسوننا من دعواتكم

By: Amr Elsayed

Local Storage Lesson #111

BOM [Browser Object Model]
Local Storage

- setItem
- getItem
- removeItem
- clear
- key

Info

- No Expiration Time
- HTTP And HTTPS
- Private Tab

ال Local Storge عبارة عن Property في ال window من خلالها بقدر التحكم في ال Storage object
ال Storage object عبارة عن object بقدر أأزن فيه بيانات واستخدمها في الصفحة بتاعتي و احتفظ بيها

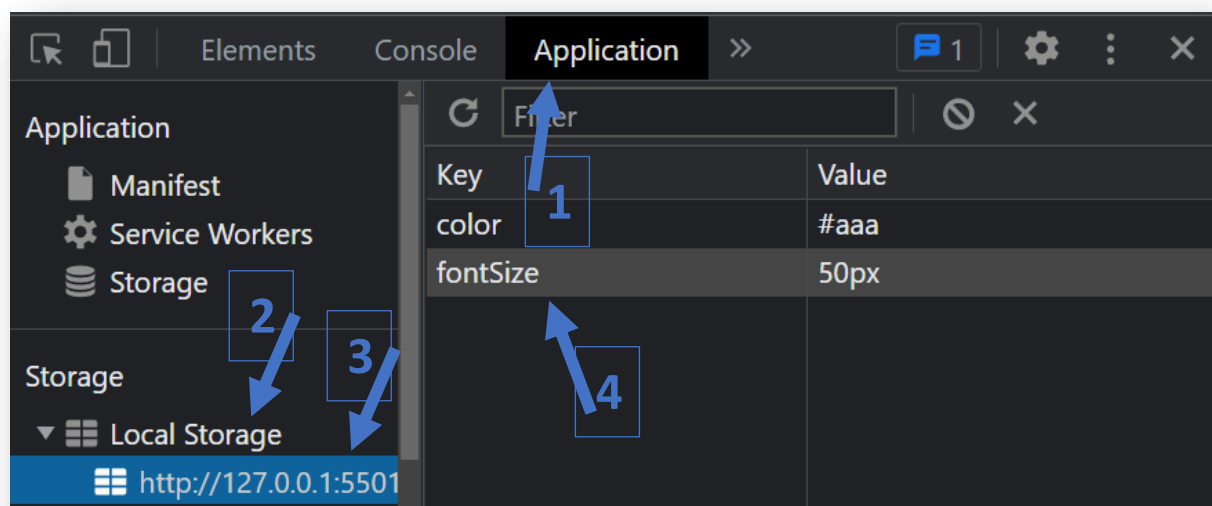
ممكن أأزن في ال Storage بكذا طريقة ولأنه بيعتبر object فقدر اسجل فيه البيانات زي دروس ال object

ال storage object بيكون عبارة عن Key و Value زي ما كنت بكتب ال Object بالظبط

```
console.log(typeof window.localStorage);
```

object

طيب لو حابب اعرف القيم الموجودة في ال local storage دي ممكن اوصلها كالتالي:



```
// Set
window.localStorage.setItem("color", "#F00");
window.localStorage.fontWeight = "bold";
window.localStorage["fontSize"] = "20px";
```

ودي كل الطرق اللي بقدر اضيف فيها بيانات في ال **local storage object** من ضمنهم ال **Dot Notation** و ال **Bracket Notation** وكذلك ال **setItem Method**

Key	Value
fontSize	20px
color	#F00
fontWeight	bold

لو عايز بقا أوصل لل **objects** دي واستخدمها او اطبعها بستخدم نفس الطرق اللي فاتت بتاعت ال **Notation** او باستخدام ال **getItem Method**

```
// Get
console.log(window.localStorage.getItem("color"));
console.log(window.localStorage.fontWeight);
console.log(window.localStorage["fontSize"]);
```

#F00	<u>main.js:30</u>
bold	<u>main.js:31</u>
20px	<u>main.js:32</u>

```
// Set Color In Page
document.body.style.backgroundColor = window.localStorage.getItem("color");
```

وبالطريقة دي انا كدا استخدمتها في الصفحة عندي ومهما اقل الصفحة وافتح هتفضل الصفحة محتفظة بال **Background color** دي

```
// Remove One  
window.localStorage.removeItem("color");
```

لو عايز ازيل عنصر واحد من ال **Storage** بستخدم ال **removeItem** هلاقي ان ال **Color** اتحذف

Key	Value
fontSize	20px
fontWeight	bold

```
// Clear All  
window.localStorage.clear();
```

وبستخدم ال **clear** لو حابب احذف كل ال **Keys** من ال **Local Storage**

Key	Value

```
// Get Key  
console.log(window.localStorage.key(0));
```

ال **Get Key** بديله ال **index** وهو بيحبلي ال **key** او ال **item** علي حسب مكانه في ال **storage**

color main.js:33

في حاجات مهمه عن ال **local Storage** لو فاتح **privet tap** وقفلتها **مش هيتم حفظ** اخر حاجه استخدمتها

استخدامك لل **HTTP** مختلف تماما عن ال **HTTPS** من حيث أسلوب واستخدام ال **local Storage**

مفيش **Expiration Time** او وقت بتنشال فيه البيانات من ال **local Storage**

HTML

```
<ul>  
  <li class="active" data-color="red"></li>  
  <li data-color="green"></li>  
  <li data-color="blue"></li>  
  <li data-color="black"></li>  
</ul>  
<div class="experiment"></div>
```



Js

```
let lis = document.querySelectorAll("ul li");  
let exp = document.querySelector(".experiment");
```

```
lis.forEach((li) => {
  li.addEventListener("click", (e) => {
    // console.log(e.currentTarget.dataset.color);
    // Remove Active Class From all Lis
    lis.forEach((li) => {
      li.classList.remove("active");
    });
    // Add Active Class To Current Element
    e.currentTarget.classList.add("active");
    // Add Current Color To Local Storage
    window.localStorage.setItem("color", e.currentTarget.dataset.color);
    // Change Div Background Color
    exp.style.backgroundColor = e.currentTarget.dataset.color;
  });
});
```

تعالى نشرح الجزء دا قبل ما نبدا نكتب شغل ال **local storage** ونشرحه

أولا انا دلوقتي مخزن كل ال **Lis** الي في ال **HTML** في متغير اسمه **Lis**

مسكت المتغير دا عملت عليه **forEach Loop** عشان امشي على كل **li** موجودة في ال **ul** بعد كدا مسكت كل ال **Lis** عندي عملت عليهم **event** ال **Click** وجواه عملت **loop** ثاني علي كل **li** عندي بحيث لما يحصل **click** علي أي **li** ال **Class** الي اسمه **active** يتشال من علي كل العناصر

يعني أي عنصر **active** عندي بمجرد ما ادوس عليه او علي أي عنصر **li** ثاني غيره **class** ال **active** الي بيميزه هيتشال خالص

بعدها مسكت ال **e** الي مررتة ك **parameter** في ال **addEventListner** عشان استخدمها مع **method** جاهزة عندي اسمها **current target** طيب معناه أي الكلام دا؟! معناه ان أي عنصر **li** ادوس عليه هو دا ال **current target** الي تم الضغط عليه طيب لما ال **current** دا الي تم الضغط عليه يتم فعليا الضغط عليه يحصل ايه؟! يحصل انه يتم إضافة **class** ال **active** ليه عشان يميزه ويظهره هو عن باقي ال **lis** الي جنبه

بعد كدا انا بس ضيفت لون ال **current target** لل **local storage** بحيث كل ما اضغط علي **li** يتخزن اللون بتاع ال **li** دا في ال **local Storage**

بعد كدا مسكت المستطيل الكبير الي عندي .. ربطت ال **background color** بتاعته بلون ال **current target** الي تم الضغط عليه فعليا من خلال ان اغير ال **style** بتاعه ودا حصل من خلال اني استخدمت ال **data set** بتاعت ال **Current target** دا الي من خلالها برضو بعرف اللون الي تم الضغط عليه

```

if (window.localStorage.getItem("color")) {
  // IF There Is Color In Local Storage
  // [1] Change Exp Div Color
  exp.style.backgroundColor = window.localStorage.getItem("color");

  // [2] Remove Active Class From all Lis
  lis.forEach((li) => {
    li.classList.remove("active");
  });

  // [3] Add Active Class To Current Color
  document
    .querySelector(`[data-color = "${window.localStorage.getItem("color")}"]`)
    .classList.add("active");
}

```

بعد كذا انا عايز اثبت ال **current color** في ال **local Storage** عشان لما اعمل **refresh** او اقفل الصفحة وافتحها الالفه محتفظ باللون

هعمل شرط من خلاله انا بتأكد ان كان فيه لون تم اضافته او في لون بالفعل موجود في ال **local storage**؟! تمام طول ما فيه **Color** موجود اول حاجه تتعمل:

1- يتم تغير لون ال **Div** الكبير بنفس لون ال **li** الي دوست عليه

2- ثاني حاجه يشيل ال **class** ال **active** من علي كل ال **li** الي عندي في حالة لو عملت **reload** لل **page** لان دا لو محصلش بمجرد ما اعمل **reload** للصفحة هلاقي ال **Div** الكبير المتسطيل محتفظ بأخر لون دوست عليه ولكن ال **li** الي واخذ ال **class** ال **active** بشكل طبيعي الي هو اول **li** في ال **HTML** هو الي اصبح **active** فبقي ال **div** المستيطل لونه اخر لون انا ضغط عليه وال **li** ال **active** هو العنصر الي كذا كذا واخذ ال **class by default** .. ما عندي في ال **HTML** اول **LI** دا عليه عنصر ال **active** بشكل طبيعي أصلا فاهمني

3- ثالث حاجة انا بقا بضيف ال **class** ال **active** دا للعنصر الي هيتم الضغط عليه فعلا وذلك من خلال اني بقوله اللون الي هدوس عليه دا لو مثلا كان الأحمر طيب هعرف منين انه الأحمر؟! قولتله لما ال **data-color** الي انا حطتها في ال **HTML** تساوي نفس اللون الي موجود في ال **local storage**

الي هيكون موجود فعلا في ال **local storage** عشان انا ضغت عليه بايدي فتم تخزينه في ال **local storage**

اذا انا بقوله شوف بقا اللون الي في ال **local** دا لو بقا بيساوي ال **data-color** الي هو ازرق مثلا في كل الأحوال يبقي اللون دا حط عليه ال **class** ال **active**

♥ عبارات تحفيزية ♥



الفضول وحب الاستطلاع والتعلم هيخطك في Position وأماكن كثير حلوة

♥ اذكر الله ♥

الزموا ذكر الله ففيه العون على متاعب الحياة ♥

سبحان الله وبحمده عدد خلقه ورضا نفسه وزنة عرشه و مداد كلماته

استغفروا الله العظيم الذي لا إله الا هو الحي القيوم واتوب إليه

أَلَا بِذِكْرِ اللَّهِ تَطْمَئِنُّ الْقُلُوبُ

لا تنسونا من دعواتكم

Session Storage And Use Cases Lesson #113

```
/*  
  BOM [Browser Object Model]  
  Session Storage  
  - setItem  
  - getItem  
  - removeItem  
  - clear  
  - key  
  
  Info  
  - New Tab = New Session  
  - Duplicate Tab = Copy Session  
  - New Tab With Same Url = New Session  
*/
```

ال **Session Storage** هو نفس كل حاجه في ال **Local Storage** ولكن الفرق بينهم ان ال **Session** بيحتفظ بالبيانات لجلسة واحدة فقط يعني بمجرد ما تقفل الصفحة وترجع مش هتلاقي البيانات دي ولكن لو عملت **reload** للصفحة هتلاقيه لسه محتفظ بالبيانات عكس ال **Local** الي بيحتفظ بالبيانات حتي لو قفلت المتصفح نهائي وفتحته تاني .. اذا دا الفرق بين الاثنين وزى ما قولنا طريقة الكتابة في الاثنين واحدة

طيب أي الحالات الي ممكن استخدم فيها ال **Session Storage** وتكون مفيدة بالنسبة لي مثلا لو عندي **form** في الصفحة واليوزر عمل **reload** بالغلط بعد ما كتب كلام في ال **input** هنا استخدامي لـ **Session** هيخلي المتصفح يحتفظ بالبيانات الي اكتبته دي مدام لسه في نفس الجلسة

لو قفلت ال **tab** وفتحتها تاني يعتبر **Session** جديدة تماما
لو حابب تفتح **Tab** جديدة وفي نفس الوقت تحتفظ بال **session** يبقي استخدم **duplicate**
لو فتحت **new tab** من نفس ال **URL** بيعتبر كانه **Session** جديدة تماما

في فيديو فيه تطبيق كامل لـ TODO List مع الـ Local Storage هيفيدك اكر لان الكود بتاعي كان معقد شوية ومحتاج اعدل عليه حاجات كتير فمحبتي اضيع وقتك وقولت اديك الفيديو اللي يفيدك افضل

لينك الفيديو



لا تنسونا من دعواتكم

Never Forget That (**Always Search**)

By: Amr Elsayed

Assignments From [111] = > [114]

لينك التكميلات: [\[114\] <= \[111\]](#)

التكليف: 01

HTML

```
<!--Start Fonts-->
<div class="style">
  <label for="Fonts">Choose Font</label>
  <select name="" id="Fonts" class="fonts">
    <option value="Open-Sans">Open Sans</option>
    <option value="Cairo">Cairo</option>
    <option value="Roboto">Roboto</option>
  </select>
</div>
<!--End Fonts-->

<!--Start Colors-->
<div class="style">
  <label for="Colors">Choose Color</label>
  <select name="" id="Colors" class="colors">
    <option value="#000000">Black</option>
    <option value="#ff0000">Red</option>
    <option value="#0000ff">Blue</option>
    <option value="#008080">Teal</option>
    <option value="#FFA500">Orang</option>
    <option value="#A020F0">purple</option>
  </select>
</div>
<!--End Colors-->

<!--Start Size-->
<div class="style">
  <label for="Font-size">Choose Font-Size</label>
  <select name="" id="Font-size" class="font-size">
    <option>16px</option>
    <option>17px</option>
    <option>18px</option>
    <option>19px</option>
    <option>20px</option>
    <option>21px</option>
    <option>22px</option>
    <option>23px</option>
    <option>24px</option>
    <option>25px</option>
    <option>26px</option>
    <option>27px</option>
    <option>28px</option>
    <option>29px</option>
    <option>30px</option>
```

```

    </select>
  </div>
  <!--End Size-->

  <!-- Stat Test -->
  <p>This Paragraph For Test</p>
  <!-- End Test -->

```

CSS

```

body {
  /* Default Settings */
  font-family: 'Open Sans', sans-serif;
  font-size: 16px;
  color: #000;
}
.style {
  margin: 30px;
  display: block;
}
label {
  margin-right: 10px;
}

```

JS

```

let fonts = document.querySelector(".fonts");
let colors = document.querySelector(".colors");
let size = document.querySelector(".font-size");

if (window.localStorage.length > 0) {
  document.body.style.fontFamily =
window.localStorage.getItem("fontFamily");
}
if (window.localStorage.length > 0) {
  document.body.style.color = window.localStorage.getItem("color");
}
if (window.localStorage.length > 0) {
  document.body.style.fontSize =
window.localStorage.getItem("fontSize");
}

fonts.addEventListener("change", () => {
  window.localStorage.setItem("fontFamily", fonts.value);
  document.body.style.fontFamily = fonts.value;
});

colors.addEventListener("change", () => {
  window.localStorage.setItem("color", colors.value);
  document.body.style.color = colors.value;
});

```

```
size.addEventListener("change", () => {  
  window.localStorage.setItem("fontSize", size.value);  
  document.body.style.fontSize = size.value;  
});
```

التكليف: 02

HTML

```
<form>  
  <input type="text" placeholder="Write Your Name" class="name">  
  <input type="email" placeholder="Email" class="email">  
  <input type="tel" placeholder="Your Number" class="number">  
</form>  
  
<div class="style">  
  <label for="Colors">Choose Color</label>  
  <select name="" id="Colors" class="colors">  
    <option value="#000000">Black</option>  
    <option value="#ff0000">Red</option>  
    <option value="#0000ff">Blue</option>  
    <option value="#008080">Teal</option>  
    <option value="#FFA500">Orang</option>  
    <option value="#A020F0">purple</option>  
  </select>  
</div>
```

CSS

```
body {  
  font-family: Arial, Helvetica, sans-serif;  
}  
.style {  
  margin-top: 25px;  
  display: block;  
  text-align: center;  
}  
label {  
  margin-right: 10px;  
}  
form {  
  display: flex;  
  flex-direction: column;  
  align-items: center;  
}  
input {  
  margin-top: 20px;  
  padding: 10px 15px;  
  width: 30%;  
  border: 1px solid #A020F0;  
  outline: none;  
}
```

JS

```
let theName = document.querySelector(".name");
let theEmail = document.querySelector(".email");
let theNumber = document.querySelector(".number");
let colors = document.querySelector(".colors");

if ((window.sessionStorage = window.sessionStorage.getItem("autoSave-Name"))) {
  theName.value = window.sessionStorage.getItem("autoSave-Name", theName.value);
}

if ((window.sessionStorage = window.sessionStorage.getItem("autoSave-Email"))) {
  theEmail.value = window.sessionStorage.getItem(
    "autoSave-Email",
    theEmail.value
  );
}

if (
  (window.sessionStorage = window.sessionStorage.getItem("autoSave-Number"))
) {
  theNumber.value = window.sessionStorage.getItem(
    "autoSave-Number",
    theNumber.value
  );
}

if ((window.sessionStorage = window.sessionStorage.getItem("color"))) {
  colors.value = window.sessionStorage.getItem("color", colors.value);
}

theName.addEventListener("input", () => {
  window.sessionStorage.setItem("autoSave-Name", theName.value);
});

theEmail.addEventListener("input", () => {
  window.sessionStorage.setItem("autoSave-Email", theEmail.value);
});

theNumber.addEventListener("input", () => {
  window.sessionStorage.setItem("autoSave-Number", theNumber.value);
});

colors.onchange = () => {
  window.sessionStorage.setItem("color", colors.value);
};
```

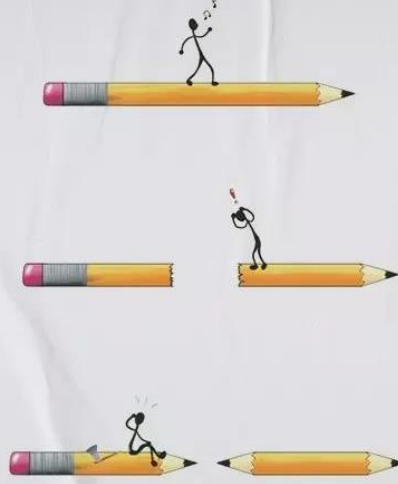
لا تنسونا من دعواتكم

Never Forget That (**Always Search**)

By: Amr Elsayed

♥ عبارات تحفيزية ♥

7- اي مشكلة تخلق لك فرص جديدة:



♥ اذكر الله

الزموا ذكر الله ففيه العون على متاعب الحياة ♥

سبحان الله وبحمده عدد خلقه ورضا نفسه وزنة عرشه ومداد كلماته

اللهم لك الحمد حمدًا طيبًا كثيرًا مباركًا فيه كما ينبغي لجلال وجهك ولعظيم سلطانك يا رب العالمين

اللهم صلّ وسلم وزد وبارك على نبينا محمد ﷺ

أَلَا بِذِكْرِ اللَّهِ تَطْمَئِنُّ الْقُلُوبُ

لا تنسونا من دعواتكم

By: Amr Elsayed

Destructuring Arrays Part 1 Lesson #115

```
/*  
  Destructuring  
  " is a JavaScript expression that allows us to extract data from arrays,  
  objects, and maps and set them into new, distinct variables. "  
  - Destructuring Array  
*/
```

ببساطة شديدة جدا عن طريق ال **Destructuring** هتقدر إنك تستخرج البيانات سواء من ال **arrays** وال **objects** وال **Maps** وتضيفهم في متغيرات وبعدين تبدأ تتعامل مع المتغيرات دي بالكيفية الي تحبها

```
let myFriends = ["Ahmed", "Sayed", "Ali", "Maysa"];
```

```
[a , b, c, d] = myFriends;
```

Destructuring Syntax

```
console.log(a);
```

```
console.log(b);
```

```
console.log(c);
```

```
console.log(d);
```

لو عندي Array فيها بعض الأسماء زي ال **myFriends** بمجرد ما اكتب ال **syntax** بتاع ال **Destructuring** هيبقي عندي القدرة ع استخدام كل عنصر في ال **array** لوحدة

طيب ال **Syntax** دا معناه أي؟! معناه ان كل متغير زي **a,b,c,d** بقا بيحتوي على كل **index** في ال **Array**

Ahmed	main.js:17
Sayed	main.js:18
Ali	main.js:19
Maysa	main.js:20

المثال دا في حالة ان مفيش أي وجود لأي متغيرات وانا بعملها **declare**

طيب في حالة لو عندي أصلا المتغيرات دي موجودة بالفعل وكل متغير ليه قيمة فعلية

```
let a = 1;
let b = 2;
let c = 3;
let d = 4;
```

كل اللي عليا اني اكتب نفس ال **syntax** بتاع ال **Destructuring** ولكن بدون ال **let** اللي المسؤولة عن عملية ال **Declaration** لانك لو حببت تغير قيم المتغيرات دي للقيم الموجودة في ال **array** و احتفظت بكلمة **let** زي اللي كانت موجودة في المثال الأول هيحصل عندك **error** بشكل منطقي لان المتغيرات دي بالفعل تم الإعلان عنها بشكل مسبق

```
let a = 1;
let b = 2;
let c = 3;
let d = 4;

let myFriends = ["Ahmed", "Sayed", "Ali", "Maysa"];

[a , b, c, d] = myFriends;

console.log(a);
console.log(b);
console.log(c);
console.log(d);
```

يبقي استنتج من الكلام دا لو عندي متغيرات معلن عنها بالفعل وتحتوي علي قيم اني اعمل **Update** للقيم اللي فيها دي ودا بيتم عن طريق الاستغناء عن كلمة **let** ... ركز بستغني عن **let** في حالة لو هعمل **Update** للمتغيرات

وفي حالة اني استغنيت عن احد المتغيرات في ال **Syntax** بتاع ال **Destructuring** زي ما هستغني عن ال **d** مثلا

```
[a , b, c] = myFriends;
```

يبقي في الحالة دي هيجيب قيمة ال **d** اللي كانت موجودة وقت الإعلان عنه لما قيمته كانت بـ **4** لان انا يعتبر معملتش أي **Update** للمتغير **d** بأي قيمة جديدة


```
let myFriends = ["Ahmed", "Sayed", "Ali", "Maysa"];

[a , b, c, d, e] = myFriends;

console.log(a);
console.log(b);
console.log(c);
console.log(d);
console.log(e);
```

طيب في حالة لو زودت عدد المتغيرات عندي زي المتغير **e** مثلا وبما ان ال **e** ملوش أي قيمة فعليه يبغي لما اطبعه في ال **console** اللي هيحصل انه هيقولي **Undefined** لان ال **e** دا فعليا ملوش أي قيمة

```
undefined                               main.js:21
```

لكن لو عملت **update** للمتغير **e** دا او أعلنت عن قيمة للمتغير دا هتبقى قيمته موجودة

```
[a , b, c, d, e = "Osama"] = myFriends;
```

```
console.log(e);
```

```
Osama                               main.js:21
```

ولو علمت أي **update** لاي متغير دا ال **syntax** بتاع ال **Destructuring** مش هياخد فيها **وهيتجاهلها** وياخد القيمة الموجودة في ال **array** زي المثال التالي لما احط قيمة للمتغير **a** هلاحظ انه تجاهل القيمة دي

```
let myFriends = ["Ahmed", "Sayed", "Ali", "Maysa"];

[a = "A", b, c, d, e = "Osama"] = myFriends;

console.log(a);
```

```
Ahmed                               main.js:17
```

طيب لو عايز اتجاهل انا أي عنصر في ال **array** ولكن اسم "Ali" مثلا فهسيب مكانه فاضي فقط



```
[a, b, , d] = myFriends;
```

Ahmed

main.js:17

Sayed

main.js:18

Maysa

main.js:20

واي عنصر عايز اتجاهل وجوده في ال **Array** هسيب مكانه فاضي



```
[ , b, , d] = myFriends;
```

Sayed

main.js:18

Maysa

main.js:20

لا تنسونا من دعواتكم

Never Forget That (**Always Search**)

By: Amr Elsayed

Destructuring Arrays Part 2 Lesson #116

```
/*  
  Destructuring  
  - Destructuring Array Advanced Examples  
*/
```

الدرس دا بيتكلم عن لو عندي **Nested Array** في **array** تانيه ازاى اقدر أوصل للعناصر اللي في ال **nested**

```
let myFriends = ["Ahmed", "Sayed", "Ali", ["Shady", "Amr", ["Mohamed", "Gamal"]]];
```

لو مثلا حابب أوصل لـ **"Gamal"** اللي هو في **nested Array** من **nested Array** أصلا كنت بعمل أي؟!

```
console.log(myFriends[3][2][1]);
```

Gamal

main.js:8

طيب لو حابب أوصل لـ **"Shady"** و **"Gamal"** ولكن عن طريق ال **Destructuring**

```
let [, , , [a, , [, b]]] = myFriends;  
console.log(a); // Shady  
console.log(b); // Gamal
```

طيب هنا انا عملت أي؟!

انا هنا **تجاهلت** اول وتاني وتالت اسم اللي هما **"Ahmed"** و **"Sayed"** و **"Ali"** بعدين دخلت علي ال **nested Array** الاولى عوضت عن اسم **"Shady"** بالمتغير **a** بعدين تجاهلت **"Amr"** الاسم اللي جنبه

بعد كدا دخلت ع ال **nested Array** الثانية تجاهلت فيها **"Mohamed"** اللي هو أول اسم وبعدين عوضت عن اسم **"Gamal"** بالمتغير **b**

Shady

main.js:11

Gamal

main.js:12

Destructuring Arrays Part 3 Swapping Variables Lesson #117

```
/*  
  Destructuring  
  - Destructuring Array => Swapping Variables  
*/
```

الغرض من الدرس دا هو معرفة ازاي اقدر اعمل **Swap** او تبديل للمتغيرات بالقيم بتاعتهم وفي طريقتين الطريقة قديمة والطريقة الجديدة الخاصة بال **Destructuring** الطريقة القديمة

```
let book = "Video";  
let video = "Book";  
  
// Save Book Value In Stash  
let stash = book; // Video  
  
// Change Book Value  
book = video; // Book  
  
// Change Video Value  
video = stash; // Video
```

خزنت قيمة المتغير **Book** الي كلمة **"Video"** في متغير اسمه **Stash** اذا المتغير **Stash** دا قيمته الحالية هي كلمة **"Video"**

بعد كدا مسكت قيمة المتغير **Video** الي كلمة **"Book"** خزنتها في المتغير **Book** او بمعنى ادق عملت **Update** للمتغير **Book** بالقيمة **"Video"**

بعد كدا مسكت المتغير **Video** دا عملت لقيمته **Update** بالقيمة المتخزنة في المتغير **Stash**

فالنتيجه النهائية بقت ان المتغير **Book** قيمته أصبحت كلمة **"Book"** والمتغير **Video** قيمته أصبحت كلمة **"Video"**

```
console.log(book);  
console.log(video);
```

Book	<u>main.js:20</u>
------	-------------------

Video	<u>main.js:21</u>
-------	-------------------

الطريقة الجديدة طريقة ال Destructuring

```
let book = "Video";  
let video = "Book";  
  
[book, video] = [video, book];  
  
console.log(book);  
console.log(video);
```

أسندت قيمة المتغير **Book** الي هي كلمة **"Video"** الي المتغير **Video**
وأسندت قيمة المتغير **Video** الي هي كلمة **"Book"** الي المتغير **Book**

Book	<u>main.js:21</u>
Video	<u>main.js:22</u>

لا تنسونا من دعواتكم

Never Forget That (**Always Search**)

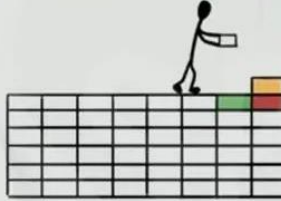
By: Amr Elsayed

♥ راقِ لي ♥

5- الخطوات الصغيرة التي تكررّها يوميًا
هي التي سوف تصل بك:



درجة درجة.



طوبة طوبة.



قطرة قطرة.

♥ اذكر الله ♥

الزموا ذكر الله ففيه العون على متاعب الحياة ♥

لا إله إلا انت سبحانك إني كنت من الظالمين

اللهم صلّ وسلم وزد وبارك على نبينا محمد ﷺ

الحمد لله و الشكر لله

أَلَا بِذِكْرِ اللَّهِ تَطْمَئِنُّ الْقُلُوبُ

لا تنسونا من دعواتكم

By: Amr Elsayed

Destructuring Objects Part 1 Lesson #118

```
/*  
  Destructuring  
  - Destructuring Object  
*/
```

ببساطة شديدة جدا زي ما كنت بستخدم ال **Destructuring** مع ال **Array** دلوقتي هستخدمها مع ال **Object** بيكون في اختلاف طفيف بس في ال **Syntax** بتاع ال **Destructuring** اثناء التعامل مع ال **Object**

```
const user = {  
  theName: "Osama",  
  theAge: 39,  
  theTitle: "Developer",  
  theCountry: "Egypt",  
};
```

دلوقتي لو عندي ال **user** دا وهو **Object** لو كنت حابب اطبع القيم اللي فيه كنت بطبعها عادي زي ما اتعلمت في دروس ال **object** سواء بطريقة ال **dot notation** أو ال **brackets notation**

```
console.log(user.theName);  
console.log(user.theAge);  
console.log(user.theTitle);  
console.log(user.theCountry);
```

```
console.log(user["theName"]);  
console.log(user["theAge"]);  
console.log(user["theTitle"]);  
console.log(user["theCountry"]);
```

Osama	<u>main.js:13</u>
39	<u>main.js:14</u>
Developer	<u>main.js:15</u>
Egypt	<u>main.js:16</u>

طيب دلوقتي انا ممكن اخزن كل قيمة من قيمة ال **user object** داخل متغير يعبر عنها كالتالي:

```
let theName = user.theName;  
let theAge = user.theAge;  
let theTitle = user.theTitle;  
let theCountry = user.theCountry;
```

إذا المتغير **theName** بقي بيحتوي ع القيمة الخاصة بال **property** الي اسمها **theName** داخل ال **user obj** وكذلك باقي ال **Properties**

ولو حابب اطبع ال **Properties** دي بالقيم بتاعتها كل الي عليا إني استخدم اسم المتغير الي اتخزنت فيه فقط

```
console.log(theName);  
console.log(theAge);  
console.log(theTitle);  
console.log(theCountry);
```

Osama	main.js:23
39	main.js:24
Developer	main.js:25
Egypt	main.js:26

كل الي فات دا كان مجرد طرق قديمة كنت بستخدمها عشان أوصل لأي **property** عندي في ال **object**

دلوقتي هستخدم طريقة ال **Destructuring** في اني اعمل **Extract** او استخراج لل **data** الموجودة في ال **Obj** طيب نرجع تاني نسأل نفس السؤال الي اتسأل قبل كدا هل ال **data** الي هستخرجها دي معمولها **Declare** مسبقا ولا انا لسه هعملها عملية **declaration**

انا سبق وعملتها **declare** فوق

إذا لو استخدمت **let** تاني هيحصل عندي **Error** و هو ان ال **Variables** دي **has already been declared**

```
✖ Uncaught SyntaxError:   main.js:29  
Identifier 'theName' has already  
been declared (at main.js:29:9)
```


إذا خلاص انا مش هستخدم ال **let** لان بالفعل المتغيرات دي انا أعلنت عنها مسبقا

```
{ theName, theAge, theTitle, theCountry } = user;
```

بس هنا هلاحظ ان في **Error** حصل عند علامة ال **=** وحله بيكون بسيط جدا

```
Declaration or statement expected. This '=' follows a block of statements, so if you intended to write a destructuring assignment, you might need to wrap the the whole assignment in parentheses. ts(2809)
```

[View Problem](#) No quick fixes available

```
{ theName, theAge, theTitle, theCountry } = user;
```

حل المشكلة دي هي ان احط ال **Destructuring** بين قوسين **()**

```
({ theName, theAge, theTitle, theCountry } = user);
```

```
console.log(theName);  
console.log(theAge);  
console.log(theTitle);  
console.log(theCountry);
```

Osama	main.js:31
39	main.js:32
Developer	main.js:33
Egypt	main.js:34

ولو حابب استبعد أي **Property** مش حابب اطبعها كل المطلوب اني مكتبتهاش وانا بعمل **Destructuring**

كل اللي فات دا في حالة اني عامل **declaration** مسبق للمتغيرات بتاعتي .. لكن لو مش عامل **declaration** في الحالة دي بعمل **declaration** وانا بكتب ال **syntax** بتاع ال **destructuring**

```
const { theName, theAge, theTitle, theCountry } = user;

console.log(theName);
console.log(theAge);
console.log(theTitle);
console.log(theCountry);
```

Osama	<u>main.js:31</u>
39	<u>main.js:32</u>
Developer	<u>main.js:33</u>
Egypt	<u>main.js:34</u>

طيب لو حابب اتجاهل عنصر او **property** مثلا من ال **destructuring** زي **theTitle** مثلا هشيل اسمه بس مش اكثر

```
const { theName, theAge, theCountry } = user;

console.log(theName);
console.log(theAge);
console.log(theCountry);
```

Osama	<u>main.js:31</u>
39	<u>main.js:32</u>
Egypt	<u>main.js:33</u>

لا تنسونا من دعواتكم

Never Forget That (**Always Search**)

By: Amr Elsayed

Destructuring Objects Part 2 Lesson #119

```
/*  
  Destructuring  
  - Destructuring Object  
  --- Naming The Variables  
  --- Add New Property  
  --- Nested Object  
  --- Destructuring The Nested Object Only  
*/
```

المتغيرات التي يتم انشاؤها يمكن تغيير اسمها عادي جدا داخل عملية ال **Destructuring** بحط جنب اسم المتغير المراد تغيير اسمه علامة ال Colon : وبعدين ابدأ في كتابة الاسم الجديد

```
const { theName: n, theAge: a, theCountry } = user;  
  
console.log(n);  
console.log(a);  
console.log(theCountry);
```

أهو بمنتهي البساطة غيرت اسم المتغيرات **theName** و **theAge** لأسماء ثانية وبعدين استخدمتهم

Osama	main.js:24
39	main.js:25
Egypt	main.js:26

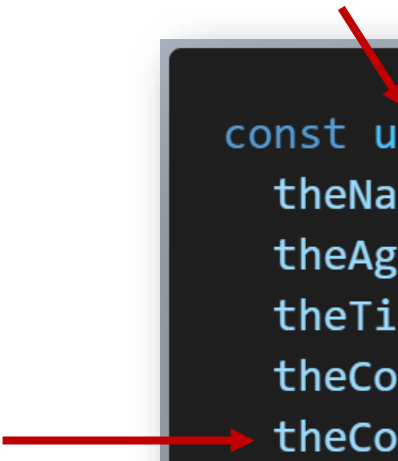
لو حابب تعمل **declare** ل **property** جديدة داخل ال **destructuring** وأبدأ استخدمها .. لا وممكن احطها اسم مختصر زي ما عملت فوق كدا

```
const {  
  theName: n,  
  theAge: a,  
  theCountry,  
  theColor: co = "Red",  
} = user;
```

Red

main.js:33

دا في حالة ال **destructuring** لكن لو انا أصلا ضيفت ال **property** دي من البداية من البداية في ال **user obj** بقيمة مختلفة غير كلمة **red** وليكن **Black** مثلاً



```
const user = {
  theName: "Osama",
  theAge: 39,
  theTitle: "Developer",
  theCountry: "Egypt",
  theColor: "Black",
};
```

في الحالة دي هيتجاهل القيمة الي ضيفتها اثناء عملية ال **Destructuring** وهياخد القيمة الموجودة في ال **user object**

Black

main.js:33

لو عندي **Nested Object** موجودة وعمايز استخدم ال **destructuring**

```
const {
  theName: n,
  theAge: a,
  theCountry,
  theColor: co = "Red",
  skills: { html: h, css },
} = user;
```

```
console.log(`My HTML Skill Progress Is ${h}`);
console.log(`My CSS Skill Progress Is ${css}`);
```

My HTML Skill Progress Is 70

main.js:34

My CSS Skill Progress Is 80

main.js:35

وممكن اعمل destructuring لل nested Object فقط من غير ما استخدم خالص ال User Obj الأساسي



```
const { html: skillOne, css: skillTwo } = user.skills;

console.log(`My HTML Skill Progress Is ${skillOne}`);
console.log(`My CSS Skill Progress Is ${skillTwo}`);
```

```
My HTML Skill Progress Is 70      main.js:39
```

```
My CSS Skill Progress Is 80       main.js:40
```

لا تنسونا من دعواتكم

Never Forget That (**Always Search**)

By: Amr Elsayed

Destructuring Function parameters Lesson #120

```
/*  
  Destructuring  
  - Destructuring Function Parameters  
*/
```

```
const user = {  
  theName: "Osama",  
  theAge: 39,  
  skills: {  
    html: 70,  
    css: 80,  
  },  
};  
  
showDetails(user);  
  
function showDetails(obj) {  
  console.log(`Your Name Is ${obj.theName}`);  
  console.log(`Your Age Is ${obj.theAge}`);  
  console.log(`Your CSS Skill Progress Is ${obj.skills.css}`);  
}
```

في المثال دا انا عملت **Function** ومررت جواها **Parameter** اسمه **obj** بعدين استدعيت ال **Function** بتاعتي وعوضت عن ال **param** دا بـ **argument** اللي هي ال **user object** بال **properties** اللي فيه بعدين بدأت بقا استخدم ال **properties** اللي جوا ال **obj user** من خلال ال **function**

طيب طريقة ال destructuring عبارة عن أي بقا ؟!

```
const user = {
  theName: "Osama",
  theAge: 39,
  skills: {
    html: 70,
    css: 80,
  },
};

showDetails(user);

function showDetails({ theName: n, theAge: a, skills: { css: c } } = user) {
  console.log(`Your Name Is ${n}`);
  console.log(`Your Age Is ${a}`);
  console.log(`Your CSS Skill Progress Is ${c}`);
}
```

الفائدة من استخدام ال **Destructuring** بشكل عام حسب فهمي انا هو الاعتقاد الحالي عندي انها بديل كويس وسريع واقل في كتابة الكود لو عايز تجيب معلومة وتبعد عن الطريقة التقليدية

يعني مثلا لو عندي **User Object** جواها **Nested Object Skills** جوا ال **Skills** دي في عندي **Three Properties** او اكثر من ضمن ال **Skills** دي **CSS** مثلا فعشان توصل لل **CSS** دي بدل ما تكتب الكود كذا <---- **user.skills.css**

في حين انك ممكن بال **Syntax** بتاع ال **Destructuring** تجيب **Css** دي علي طول بشكل مباشر وممكن تحطها حرف مختصر زي **C**

يعني اعتقد الغرض منه هو سرعة وسهولة انك تعمل **Extract** للداتا اللي موجودة عندك لو هي **nested** جوا **nested**

و لو عايز تعمل **Swap** لقيم من متغير لآخر ممكن تبقي ابسط بال **destructuring**

لا تنسونا من دعواتكم

Never Forget That (**Always Search**)

By: Amr Elsayed

عبارات تحفيزية ♥♥



متستعجلش النتائج واصبر اصبر اصبر

الزموا ذكر الله ففيه العون على متاعب الحياة ♥

لا اله الا الله وحده لا شريك له .. له الملك وله الحمد وهو علي كل شيء قدير

سبحان الله وبحمده عدد خلقه ورضا نفسه وزنة عرشه ومداد كلماته

استغفروا الله العظيم الذي لا اله الا هو الحي القيوم واتوب اليه

أَلَا بِذِكْرِ اللَّهِ تَطْمَئِنُّ الْقُلُوبُ

لا تنسونا من دعواتكم

By: Amr Elsayed

Destructuring Mixed Content Lesson #121

```
/*  
  Destructuring  
  - Destructuring Mixed Content  
*/
```

```
const user = {  
  theName: "Osama",  
  theAge: 39,  
  skills: ["HTML", "CSS", "JavaScript"],  
  addresses: {  
    egypt: "Cairo",  
    ksa: "Riyadh",  
  },  
};
```

```
const {  
  theName: n,  
  theAge: a,  
  skills: [, , three],  
  addresses: { egypt: e },  
} = user;  
  
console.log(`Your Name Is: ${n}`);  
console.log(`Your Age Is: ${a}`);  
console.log(`Your Last Skill Is: ${three}`);  
console.log(`Your Live In: ${e}`);
```

ببساطة شديدة جدا انا عندي **Object** وجواه **Nested Array** وجواه **Nested Object** استخدمت ال **Destructuring Syntax** زي ما اتعلمت عوضت عن كل متغير في ال **Object** الأساسي بمتغير بديل ليه زي **theName** عوضت عنه بحرف **n** وكذلك **theAge** عوضت عنه بحرف **a** واستخدمت بدائل المتغيرات دي اللي هي الحروف في اني اطبع بعض الرسائل في ال **Console**

Destructuring Challenge Lesson #122

```
/*  
  Destructuring  
  - Challenge  
*/  
  
let chosen = 3;  
  
let myFriends = [  
  { title: "Osama", age: 39, available: true, skills: ["HTML", "CSS"] },  
  { title: "Ahmed", age: 25, available: false, skills: ["Python", "Django"] },  
  { title: "Sayed", age: 33, available: true, skills: ["PHP", "Laravel"] },  
];
```

لينك ال Challenge

نفس ال Challenge موجود في التكميف 6 في تكليفات الأسبوع دا من الدرس 114 - 122

لا تنسونا من دعواتكم

Never Forget That (**Always Search**)

By: Amr Elsayed

Destructuring Challenge Solution

```
// Solution
```

```
let [obj1, obj2, obj3] = myFriends;

if (chosen === 1) {
  let { title: a, age: b, available: c, skills: [, d], } = obj1;
  console.log(a);
  console.log(b);
  console.log(c ? "Available" : "Not Available");
  console.log(d);
} else if (chosen === 2) {
  let { title: a, age: b, available: c, skills: [, d], } = obj2;
  console.log(a);
  console.log(b);
  console.log(c ? "Available" : "Not Available");
  console.log(d);
} else if (chosen === 3) {
  let { title: a, age: b, available: c, skills: [, d], } = obj3;
  console.log(a);
  console.log(b);
  console.log(c ? "Available" : "Not Available");
  console.log(d);
} else {
  console.log(`Work is Not Available Now`);
}
```

حل ال **Challenge** دا اعتمد ع التالي انت دلوقتي عند **Array** جواها **3 Nested Object** استخدمت ال **Destructuring** مع ال **Array** وعوضت عن كل **object** داخل ال **Array** بمتغير إذا ال **obj 1** دا يحل محل اول **Object** عندك وكذلك ال **obj 2** يحل محل ثاني **Object** وهكذا

بعد كدا عملت الشرط بتاعك لو ال **Chosen = 1** يبقي انا كدا بـ **Target** أول **Object** موجود عندي مسكت كدا انا اول **object** اللي عوضت عنه بـ **obj 1** وبدأت امسك كل **Property** جوا اول **Object** دا

عوضت عن ال **title** بالمتغير **a** , وعن ال **age** بـ **b** , وعن ال **available** بـ **c** , وعن **skills** تجاهلت اول واحدة وعوضت عن ثاني **Skill** بالمتغير **d**

في كل **object** عندي .. عند المتغير **c** حظيت شرط لو ال **Available** بقت **True** يحط في الرسالة اللي هتبتلع **true** لو بقت **False** يحط **Not Available**

لا تنسونا من دعواتكم

Never Forget That (**Always Search**)

By: Amr Elsayed

Assignments From [115] = > [122]

لينك التكميلات: [\[122\] <= \[115\]](#)

التكليف: 01

```
// Assignment 1

let myNumbers = [1, 2, 3, 4, 5];

// Write Your Destructuring Assignment Here
let [a, , , , e] = myNumbers

console.log(a * e); // 5
```

التكليف: 02

```
// Assignment 2

let mySkills = [
  "HTML",
  "CSS",
  "JavaScript",
  ["PHP", "Python", ["Django", "Laravel"]],
];

// Write Your Destructuring Assignment Here
let [a, b, c, [d, e, [f, g]]] = mySkills;

console.log(`My Skills: ${a}, ${b}, ${c}, ${d}, ${e}, ${f}, ${g}`);

// My Skills: HTML, CSS, JavaScript, PHP, Python, Django, Laravel
```

التكليف: 03

```
// Assignment 3

let arr1 = ["Ahmed", "Sameh", "Sayed"];
let arr2 = ["Mohamed", "Gamal", "Amir"];
let arr3 = ["Haytham", "Shady", "Mahmoud"];

// Play With Arrays To Prepare For Destructuring
// Write Your Destructuring Assignment Here
let myFriends = arr1.concat(arr2, arr3)

let [a, , , , , , c, b] = myFriends;

console.log(`My Best Friends: ${a}, ${b}, ${c}`);
// My Best Friends: Shady, Mahmoud, Ahmed
```

```
// Another Solution

let arr1 = ["Ahmed", "Sameh", "Sayed"];
let arr2 = ["Mohamed", "Gamal", "Amir"];
let arr3 = ["Haytham", "Shady", "Mahmoud"];

arr3.push(arr1);

[, a, b, [c, ,]] = arr3;

console.log(`My Best Friends: ${a}, ${b}, ${c}`);

// My Best Friends: Shady, Mahmoud, Ahmed
```

التكليف: 04

```
// Assignment 4
const member = {
  age: 30,
  working: false,
  country: "Egypt",
  hobbies: ["Reading", "Swimming", "Programming"],
};

// Write Your Destructuring Assignment Here
({ age: a, w } = member);

console.log(`My Age Is ${a} And Iam ${w ? "" : "Not"} Working`);
// My Age Is 30 And Iam Not Working
({ country: c } = member);
console.log(`I Live in ${c}`);
// I Live in Egypt
({
  hobbies: [h1, , h3],
} = member);
console.log(`My Hobbies: ${h1} And ${h3}`);
// My Hobbies: Reading And Programming
```

```
// Assignment 5

const game = {
  title: "YS",
  developer: "Falcom",
  releases: {
    "Oath In Felghana": ["USA", "Japan"],
    "Ark Of Napishtim": { US: "20 USD", JAP: "10 USD" },
    Origin: "30 USD",
  },
};

// Write Your Destructuring Assignment/s Here
const { title: t, developer: d } = game;
const [o, a] = Object.keys(game.releases);
const [u, j] = Object.values(game.releases)[0];
const { US: u_price, JAP: j_price } = Object.values(game.releases)[1];
const { Origin: or } = game.releases;

console.log(`My Favourite Games Style Is ${t} Style`);
// My Favourite Games Style Is YS Style

console.log(`And I Love ${d} Games`);
// And I Love Falcom Games

console.log(`My Best Release Is ${o} It Released in ${u} & ${j}`);
// My Best Release Is Oath In Felghana It Released in USA & Japan

console.log(`Although I Love ${a}`);
// Although I Love Ark Of Napishtim

console.log(`${a} Price in USA Is ${u_price}`);
// Ark Of Napishtim Price in USA Is 20 USD

console.log(`${a} Price in Japan Is ${j_price}`);
// Ark Of Napishtim Price in Japan Is 10 USD

console.log(`Origin Price Is ${or}`);
// Origin Price Is 30 USD
```

التكليف ال 6 هو نفس ال Challenge

لا تنسونا من دعواتكم

Never Forget That (**Always Search**)

By: Amr Elsayed

♥ عبارات تحفيزية ♥

ما تحتاجه حقاً:



أنت في حاجة
للمزيد من التركيز.



أنت لست في حاجة
للمزيد من الوقت.

اشتغل بذكاء وتركيز

♥ اذكر الله

الزموا ذكر الله ففيه العون على متاعب الحياة ♥

سبحان الله وبحمده سبحان الله العظيم

أَلَا بِذِكْرِ اللَّهِ تَطْمَئِنُّ الْقُلُوبُ

لا تنسونا من دعواتكم

By: Amr Elsayed

Set Data Type And Methods Lesson #123

```
/*
- Set Data Type
Syntax: new Set(Iterable)
-- Object To Store Unique Values
-- Cannot Access Elements By Index

Properties:
- size

Methods:
- add
- delete
- clear
- has
*/
```

ببساطة شديدة جدا نقدر نقول ان ال **Set Data Type** شبيه جدا بعمل ال **Array** العادية من حيث سرد البيانات ومن حيث التعديل عليهم سواء بالإضافة او الحذف لاي عنصر في البيانات دي الا ان فيه اختلافات كثير بينهم

```
let myData = [1, 1, 1, 2, 3];
let myUniqueData = new Set([1, 1, 1, 2, 3]);
```

يعني مثلا اول اختلاف بينهم ان ال **Array** الي اسمها **myData** دي هتعرض كل البيانات الموجودة دي اما ال **Set Data** يياخد كل العناصر المتكررة ويدمجهم بحيث يبقى عنصر واحد فقط يعني ال 1 المتكرر دا هيكون في الاخر عنصر واحد فقط متمثل في رقم 1

```
console.log(myData);
console.log(myUniqueData);
```

► (5) [1, 1, 1, 2, 3]

main.js:27

► Set(3) {1, 2, 3}

main.js:28

تاني حاجة في حالة ال **Array** كنت بستخدم ال **Length** عشان اعرف عدد العناصر الموجودة

اما في حالة ال **Set Data** بستخدم ال **size**


```
console.log(myUniqueData.size);
```

3

main.js:32

في ال **Set Data Type** مقدersh اعمل **Access** علي العناصر الموجود بال **index** زي ما كنت بعمل في ال **Array** ولو جربت اعمل كذا هلاقي النتيجة **Undefined** علي عكس ال **array** هلاقي ال **index[0]** بيعبر عن القيمة 1

```
console.log(myData[0]);  
console.log(myUniqueData[0]);
```

1

main.js:33

undefined

main.js:34

في اكثر من طريقة لكتابة ال **Syntax** بتاع ال **Set Data**

زي مثلا إني اعوض عن ال **data** الي هتكتب بالأرقام الموجودة في ال **Array** الي انشأتها مسبقا

```
let myUniqueData = new Set(myData);
```

زي مثلا استخدام ال **add** وجواه القيم الي عايز اعملها **Set**

```
let myUniqueData = new Set().add(1).add(1).add(1).add(2).add(3);
```

ممكن كمان اعمل ال **set** فاضي وابدأ جواه اضيف البيانات الي انا عايزها وعلي اكثر من سطر

```
let myUniqueData = new Set();  
  
myUniqueData.add(1).add(1).add(1);  
myUniqueData.add(2).add(3);
```

كل الطرق دي زي بعض وفي النهاية هتأدي نفس الغرض بالظبط

يمكن حذف عنصر من الـ `set data` بطريقة الـ `delete` يعني ع سبيل المثال انا **أحذف** العنصر 2

```
myUniqueData.delete(2);  
console.log(myUniqueData);  
console.log(myUniqueData.size);
```

► `Set(2) {1, 3}`

2

لو كتبت `Code` الـ `delete` دا في الـ `Console` بشرط ان الـ `delete` اللي هعمله يكون لعنصر فعليا موجود عندي زي العنصر رقم 2 لما ابي احذفه هلاقي النتيجة `true`

```
console.log(myUniqueData.delete(2));
```

true

main.js:36

اما لو عملت `delete` لعنصر مش موجود أصلا عندي زي رقم 20 مثلا فهتكون النتيجة `False`

```
console.log(myUniqueData.delete(20));
```

false

main.js:36

طبعا كالعادة لو استخدمت الـ `clear` فدا معناه ان كل حاجه حرفيا هتتحذف

```
myUniqueData.clear();  
  
console.log(myUniqueData);  
console.log(myUniqueData.size);
```

► `Set(0) {size: 0}` main.js:44

0

main.js:45

طيب لو عايز ابحث عن حاجة في ال **data** الي موجودة عندي وليكن عايز ابحث عن عنصر معين يعني كنت في ال **array** بستخدم **method** اسمها **include**

في ال **set data type** بستخدم حاجه اسمها **has**

```
let myUniqueData = new Set();

myUniqueData.add(1).add(1).add(1);
myUniqueData.add(2).add(3).add("A");

console.log(`Is Set Has => A ${myUniqueData.has("A")}`);
```

Is Set Has => A true main.js:26

ممكن أدمج **method** من الي اتعلمتهم مسبقا وانا بستخدم ال **has** زي اني اكتب حرف ال **a** بطريقة ال **small** ولكن ادمج مع الحرف دا ال **toUpperCase()**

```
console.log(`Is Set Has => A ${myUniqueData.has("a".toUpperCase())}`);
```

لاحظ ان حرف ال **A** وهو **capital** كدا لو بحثت عنه وكتبت الحرف **Small** هتكون النتيجة **false**

```
console.log(`Is Set Has => A ${myUniqueData.has("a")}`);
```

Is Set Has => A false main.js:27

من الاستخدامات العظيمة لل **Set Data Type** هو مثلا لو عندي **Array** وجيت قولتك استثني منها كل الحروف والأرقام المتكررة عشان تعمل الحركة دي ممكن تستخدم ال **reduce** او ال **filter** في ال **Higher Order Func** ولكن مع ال **Set Data Type** هتقدر بسطر كود واحد بس تنفذ العملية دي بمنتهي البساطة



Set vs WeakSet And Garbage Collector Lesson #124

```
/*
- Set vs WeakSet
"
  The WeakSet is weak,
  meaning references to objects in a WeakSet are held weakly.
  If no other references to an object stored in the WeakSet exist,
  those objects can be garbage collected.
"
--
Set      => Can Store Any Data Values
WeakSet => Collection Of Objects Only
--
Set      => Have Size Property
WeakSet => Does Not Have Size Property
--
Set      => Have Keys, Values, Entries
WeakSet => Does Not Have clear, Keys, Values And Entries
--
Set      => Can Use forEach
WeakSet => Cannot Use forEach

Usage: Store objects and removes them once they become inaccessible
*/
```

بص يا صديقي عشان ابسطلك الدنيا قدر المستطاع

أولا في مصطلح عندي اسمه **Garbage collection** المصطلح دا أصلا ملوش أي علاقة بال **JavaScript** بمعنى انه المصطلح دا ليه علاقة بال **Memory Management** في أي لغة بتستخدمها .. يعني أي لغة برمجة بتتعلمها بيكون جواها **Engine** في اللغة دي خاص بالتعامل مع ال **Memory** وبيدير ال **Memory**

طيب خليني اديك مثال ع الجافاسكريبت لو عندي بيانات معينة زي **object** او **Function** او أي **primitive data** البيانات دي بعد ما استخدمتها افرض انا مش عايز استخدمها تاني ؟!

اللي بيحصل ان ال **Engine** بتاع اللغة بيروح يعمل **Clean** للبيانات دي من ال **Memory** لانها اكيد بتشغل حيز هو دا بقا مفهوم ال **Garbage collection**

في لغة ال **JS** ال **Memory Management** بيتتم **Automatic** في لغات تانية انت اللي بت **manage** فيها ال **memory**

تعالى دلوقتي بقا نفهم الفرق بين ال **Set** اللي اتعلمتها الدرس اللي فات وال **WeakSet** بتاعت الدرس دا
أولا الفرق بين ال **Set** وال **WeakSet** في حفظ البيانات

1- في ال **Set** بقدر أأخزن أي نوع من البيانات عكس ال **WeakSet** اللي بآخزن فيها **Objects**

```
Set      => Can Store Any Data Values  
WeakSet => Collection Of Objects Only
```

Set

```
// Type Of Data  
  
let mySet = new Set([1, 1, 1, 2, 3, "A", "A"]);  
  
console.log(mySet);
```

```
► Set(4) {1, 2, 3, 'A'}    main.js:29
```

WeakSet

```
// Type Of Data  
  
let myws = new WeakSet([{ A: 1, B: 2 }]);  
  
console.log(myws);
```

```
▼ WeakSet {...} ⓘ    main.js:53  
  ► [[Entries]]  
  ► [[Prototype]]: WeakSet
```

2- في ال Set أقدر استخدم ال Size لكن دا غير متوفر في ال WeakSet

```
Set      => Have Size Property  
WeakSet => Does Not Have Size Property
```

```
// Size  
console.log(`Size Of Elements Inside Set Is: ${mySet.size}`);
```

```
Size Of Elements Inside Set Is: 4    main.js:32
```

3- في ال Set بقدر استخدم ال Keys و ال values و ال Entries وبالمناسبة ال keys تعتبر Alias او بديل لل value

```
Set      => Have Keys, Values, Entries  
WeakSet => Does Not Have clear, Keys, Values And Entries
```

ال Values دي اقدر استخدمها بحيث اني اعمل **Iterator** زي ما ال **for loop** بتعمل بالضبط

```
// Values + Keys [Alias For Values]  
let iterator = mySet.keys();  
  
console.log(iterator.next().value);  
console.log(iterator.next().value);  
console.log(iterator.next().value);  
console.log(iterator.next().value);  
console.log(iterator.next());
```

1	<u>main.js:37</u>
2	<u>main.js:38</u>
3	<u>main.js:39</u>
A	<u>main.js:40</u>

يعني أقدر استخدم ال **key** أو ال **Value** دي في اني أعمل عمل ال **loop** ولكن بإيدي انا ودا عن طريق ال **next()**

```
37 console.log(iterator);
```

```
► SetIterator {1, 2, 3, 'A'}
```

```
main.js:37
```

هستخدَم ال **next** بقا ع ال **iterator** بتاعي دا كدا

```
38 console.log(iterator.next());
```

```
► {value: 1, done: false}
```

```
main.js:38
```

هلاحظ انه جابلي ال **Value** بـ 1 يعني دي القيمة اللي هيتعملها **generate** عشان تطبع
اما ال **done** بالقيمة **False** فدا معناه ان لسه ال **Iteration** بتاعي مخلصش لسه فيه **Values** عندي هتطبع

```
console.log(iterator.next().value);
```

ببدأ استخدم ال **value** بقا عشان اطبع القيمة وبكرر نفس العملية دي عشان أوصل لآخر عنصر عندي

لما بوصل لطباعة آخر **Value** عندي واستخدم ال **next** بلاقي ان ال **done** بقت **true** دا معناه ان كل البيانات
اللي عندي اطبعت كلها

```
console.log(iterator.next());
```

```
► {value: undefined, done: true}
```

```
main.js:44
```

كل دا أقدر اعمله في ال **Set** ولكن في ال **WeakSet** كل دا مش متاح

4- في ال **Set** أقدر استخدم ال **forEach** لكن ال **WeakSet** مقدرش استخدمها

```
Set      => Can Use forEach  
WeakSet => Cannot Use forEach
```

```
// forEach  
  
mySet.forEach((el) => console.log(el));
```

1	<u>main.js:48</u>
2	<u>main.js:48</u>
3	<u>main.js:48</u>
A	<u>main.js:48</u>

```
// Search For WeakSet Use Cases
```

لا تنسونا من دعواتكم

Never Forget That (**Always Search**)

By: Amr Elsayed

Map Data Type vs Object Lesson #125

الدرس دا عبارة عن مقارنة بين ال **Object** الي اتعلمته وبين ال **Map** والفرق بين الاثنين في تخزين واستخدام البيانات

1- أول فرق بين ال **Object** وال **Map** هو ان ال **Object** الفاضي بيكون فيه **default value** لكن ال **map** مفيهوش .. او في ال **Map** تقدر انت تحدد ال **Default Value** دي تكون عبارة عن أي

```
----- Map => Does Not Contain Key By Default  
----- Object => Has Default Keys
```

```
let myObject = {};  
let myMap = new Map();  
  
console.log(myObject);  
console.log(myMap);
```

```
▼ {} ⓘ main.js:29  
  ► [[Prototype]]: Object  
▼ Map(0) {size: 0} ⓘ main.js:30  
  ▼ [[Entries]]  
    No properties  
  size: 0  
  ► [[Prototype]]: Map
```

طيب هل اقدر اعمل **Object** مفيهوش ال **default value** الي بتكلم عليها دي .. ايون كالتالي:

```
let myEmptyObject = Object.create(null);  
console.log(myEmptyObject);
```

```
▼ {} ⓘ main.js:31  
  No properties
```

2- ممكن ال Map يحتوي علي نوع بيانات أي نوع مهما كانت لكن ال Object بيقبل البيانات علي هيئة String

```
----- Map => Key Can Be Anything [Function, Object, Any Primitive Data Types]
----- Object => String Or Symbol
```

```
let myNewObject = {
  10: "Number",
  "10": "String",
};

console.log(myNewObject[10]);
```

لو طلبت اعرف القيمة الأولى في ال **Object** دا وكتبته زي ما هي عشان استعلم عنها هل هتيجي فعلا

String main.js:38

هلاقيه تجاهل أصلا اول **Key** بال **value** بتاعته واخذ الثانية لأنها تعتبر عملت **Override** عليها

لكن بقا لو استخدمت ال **Map** في تخزين نفس البيانات دي وحببت استعلم أي واحدة فيهم

```
let myNewMap = new Map();
myNewMap.set(10, "Number");
myNewMap.set("10", "String");

console.log(myNewMap.get(10));
console.log(myNewMap.get("10"));
```

Number main.js:47
String main.js:48

هلاقيه قدر فعلا يتعرف علي كل واحدة فيهم ويجيب قيمة كل واحدة بدون ما يكون في حاجه اثرت ع الثانية

واقدر كمان احط أي نوع بيانات في ال **Map**

```
myNewMap.set(true, "Boolean");
myNewMap.set({a: 1, b: 2}, "Object");
myNewMap.set(function doSomething() {}, "Function");
```

لاحظ اني بستخدم ال **Set** في وضع البيانات وبستخدم ال **Get** في الحصول علي البيانات دي وكل دا داخل اطار ال **Map**

يعني لو جيت استعلم عن الموجود في ال **Map** وال **Object** هلاحظ التالي:

```
console.log(myNewObject);  
console.log(myNewMap);
```

```
▼ {10: 'String'} ⓘ main.js:52  
  10: "String"  
  ► [[Prototype]]: Object
```

```
main.js:53  
Map(5) {10 => 'Number', '10' => 'String',  
  ► true => 'Boolean', {...} => 'Object', f =>  
    'Function'}
```

هلاحظ ان في ال **Object** مفيش أصلا وجود لأول **Key** انا كتبتة الي هو كان **10** و ال **Value** بتاعته **Number**

لكن في ال **Map** هلاقي ان كل حاجه كتبتة موجودة

3- في ال **Map** بيكون في ترتيب دقيق للبيانات بالشكل الي انا كتبتة عكس ال **Object** مش مضمون في ترتيب البيانات بنسبة 100%

```
----- Map => Ordered By Insertion  
----- Object => Not 100% Till Now
```

-4 في ال **Map** عندي اقدر استخدم ال **Size** لكن في ال **Object** عشان اجيب عدد العناصر الي عندي بحتاج
اعمل دا بشكل يدوي وبكذا خطوة

```
----- Map => Get Items By Size  
----- Object => Need To Do Manually
```

-5 ال **Map** تقدر تعمل **Loop** علي العناصر الي فيه بشكل مباشر عكس ال **Object** لازم عشان تعمل **loop**
تعمل كذا خطوة برضو

```
----- Map => Can Be Directly Iterated  
----- Object => Not Directly And Need To Use Object.keys() And So On
```

-6 ال **Map** افضل بكتير في ال **performance** عكس ال **Object**

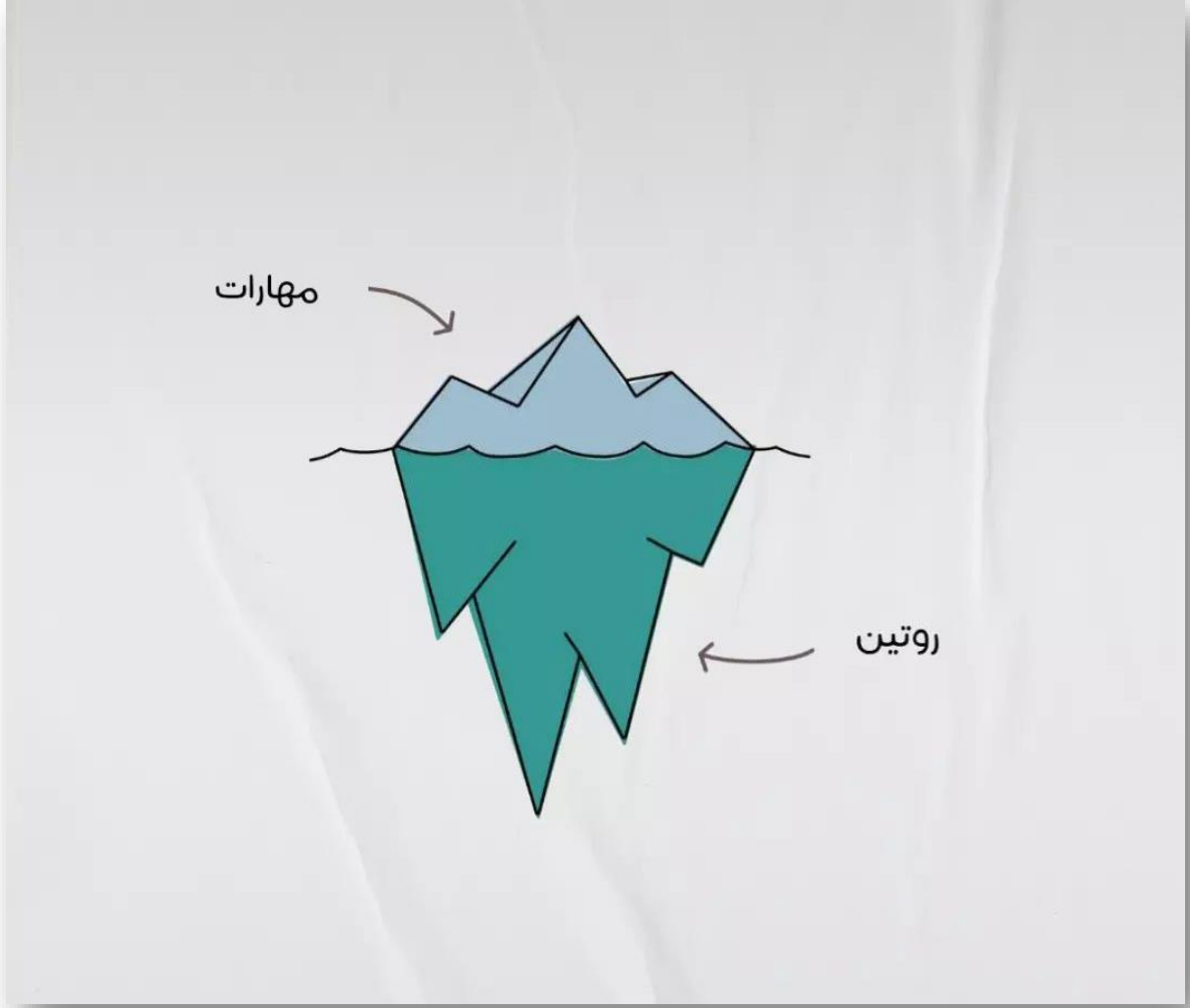
```
----- Map => Better Performance When Add Or Remove Data  
----- Object => Low Performance When Comparing To Map
```

لا تنسونا من دعواتكم

Never Forget That (**Always Search**)

By: Amr Elsayed

❤ عبارات تحفيزية ❤



كل شخص ماهر وشاطر جدا ومميز هو في الأصل عنده روتين بيكره بشكل يومي يساعد يثقل مهاراته الجامدة الي بتشوفها فلو عايز تبقي جامد لازم يبغي روتين بتعمله بشكل متكرر يكبر ويعلي مهارتك

❤ اذكر الله

الزموا ذكر الله ففيه العون على متاعب الحياة ❤

أَلَا بِذِكْرِ اللَّهِ تَطْمَئِنُّ الْقُلُوبُ

اللهم لك الحمد حمداً طيباً كثيراً مباركاً فيه كما ينبغي لجلال وجهك ولعظيم سلطانك يا رب العالمين

لا تنسونا من دعواتكم

Never Forget That (**Always Search**)

By: Amr Elsayed

Map Methods Lesson #126

```
/*
  - Map Data Type
  Methods
  --- set
  --- get
  --- delete
  --- clear
  --- has

  Properties
  --- size
*/
```

أول حاجة عشان اضيف **Key** و **Value** في ال **Map Data Type** عندي طريقتين
الطريقة الأولى اني اضيفهم مباشرة كـ **iterable** جوا ال **new Map Method**
والطريقة الثانية اني اضيفهم من خلال ال **Method** ال **set** فتعالى نشوف الطريقتين
أولا طريقة الإضافة بال **Set**

Key

```
let myMap = new Map();

myMap.set(10, "Number");
myMap.set("Name", "String");

console.log(myMap);
```

Value

```
► Map(2) {10 => 'Number', 'Name' => 'String'} main.js:25
```

وعشان اعمل **Access** عليهم باستخدام ال **get** وجواها بيكون اسم ال **Key**

```
console.log(myMap.get(10));
console.log(myMap.get("Name"));
```

Number main.js:27

String main.js:28

```
let myMap = new Map([
  [10, "Number"],
  ["Name", "String"],
  [false, "Boolean"],
]);

console.log(myMap);
```

وعشان اعمل Access عليهم بستخدم ال get من خلال ال Key

```
console.log(myMap.get(10));
console.log(myMap.get("Name"));
console.log(myMap.get(false));
```

Number	<u>main.js:22</u>
String	<u>main.js:23</u>
Boolean	<u>main.js:24</u>

لما بستخدم ال get وجواه ال Key في ال Console.log فدا معناه اني عايز اطبع ال value بتاعت ال key دا

دلوقتي مع استخدام ال Size هلاقي عدد ال Keys اللي عندي 3

```
console.log(myMap.size);
```

3	<u>main.js:37</u>
---	-------------------

لو عايز احذف عنصر من ال Iterable بتاعي بستخدم ال delete

فبمجرد ما احذف ال "Name" من عندي واستعلم عن ال size تاني هلاقيه بقا 2

```
console.log(myMap.delete("Name"));
console.log(myMap.size);
```

2	<u>main.js:41</u>
---	-------------------

لاحظ ان استخدام ال delete في ال Console لو ال key دا فعلا موجود عندي النتيجة هتكون true

true	<u>main.js:39</u>
------	-------------------

لكن لو جربت استخدم ال `delete` في ال `console` ومررت `Key` مش موجود أصلاً زي `JS` دي مثلاً

```
console.log(myMap.delete("JS"));
```

هلاقي النتيجة في ال `Console` عبارة عن `false`

```
false      main.js:39
```

فمعني ال `true` وال `false` هنا هو تأكيد علي حذف او عدم حذف العنصر لو كانت النتيجة `true` فدا معناه ان العنصر بالفعل تم حذفه ولو كانت `false` فدا معناه ان العنصر لم يتم حذفه لاني أصلاً مررت عنصر غير موجود

استخدام ال `Clear` بيمسح كل حاجه موجودة وكل العناصر

```
myMap.clear();
```

```
console.log(myMap.size);
```

0

main.js:45

بعد كذا اخر حاجه في الدرس عندي هي ال `has` الي بستخدمها عشان اعمل `Check` على `key` هل هو موجود؟

```
console.log(myMap.has(false));
```

```
true
```

main.js:28

لو عملت `Check` علي عنصر هو موجود عندي ولكن كتبت اسمه غلط يعني اول حرف منه بدل ما يكون كبير كتبته انا صغير زي `key` ال `Name` مثلاً اول حرف فيه كبير وانا هكتب اول حرف دا `Small`

```
console.log(myMap.has("name"));
```

```
false
```

main.js:32

هتكون النتيجة `false` ودا معناه اسماء ال `keys` بتكون حساسة لازم تبحث عنها زي ما هي مكتوبة بالضبط

Map vs WeakMap Lesson #127

```
/*
- Map vs WeakMap
"
  WeakMap Allows Garbage Collector To Do Its Task But Not Map.
"
--
Map      => Key Can Be Anything
WeakMap => Key Can Be Object Only
--
*/
```

```
let mapUser = { theName: "Elzero" };

let myMap = new Map();

myMap.set(mapUser, "Object Value");

mapUser = null; // Override The Reference

console.log(myMap);

console.log("#".repeat(20));

let wMapUser = { theName: "Elzero" };

let myWeakMap = new WeakMap();

myWeakMap.set(wMapUser, "Object Value");

wMapUser = null; // Override The Reference

console.log(myWeakMap);
```

Array.from Method Lesson #128

```
Array Methods
- Array.from(Iterable, MapFunc, This)
--- Variable
--- String Numbers
--- Set
--- Using The Map Function
--- Arrow Function
--- Shorten The Method + Use arguments
```

ببساطة شديدة جدا **method** ال **Array.from** الغرض الأساسي منها هي تحويل أي حواجه جواها ل **array** يعني أي **Iterable** هتحة جوا ال **method** دي هيتعمل منه **Array** مباشرة

```
console.log(Array.from("Osama"));
```

شايك ال **Sequence** بتاع "Osama" دا هيتحول كل **Character** فيه ل **Index** موجود في **Array**

```
► (5) ['O', 's', 'a', 'm', 'a'] main.js:12
```

ممكّن داخل ال **Array.from** اعمل **Function** تؤدي وظيفة معينة زي اني اممر في ال **Iterable** شوية ارقام وال **function** بدورها ترجعها ارقام وكمان كل رقم مجموع علي نفسه

```
console.log(
  Array.from("12345", function (n) {
    return +n + +n;
  })
);
```

```
► (5) [2, 4, 6, 8, 10] main.js:13
```

طبعا معني الكود دا اني استخدمت **Function** الغرض منها تبص ع ال **iterable** الموجود في ال **Array.from** ومن ثم استخدمت ال **unary plus** عشان ارجعهم ع شكل ارقام

طيب لو مكنتش استخدمت ال **unary plus** كان ايه اللي هيحصل ؟!

أصلاً في العادي قبل ما استعمل ال **function** عشان تأدي عملية الجمع دي تعالي نستخدم ال **Array.from** علي نفس المثال اللي فات ولكن بدون ال **function**

```
console.log(Array.from("12345"));
```

```
► (5) ['1', '2', '3', '4', '5'] main.js:13
```

النتيجة اللي رجعلي **array** فيها ارقام، ولكن الأرقام دي علي هيئة **string** لذلك انا في نفس المثال دا لما عملت ال **function** عشان تأدي وظيفة اللي هي تجمع كل رقم ع نفسه لو مكنتش استخدمت ال **unary plus** كان هيتعامل مع كل رقم موجود في ال **array** علي انه **String** و معامل الجمع كان هيعتبره علامة **Concatenate** أي ربط **strings** ببعض لذلك لاحظ المثال القادم لما اشي ال **unary plus**

```
console.log(
  Array.from("12345", function (n) {
    return n + n;
  })
);
```

```
► (5) ['11', '22', '33', '44', '55'] main.js:14
```

لاحظ هيبدا يتعامل مع كل رقم علي انه **string** وبدل ما يجمعه علي نفسه لا هيعمل **Concatenate**

ممكن استخدم ال **Arrow function** بدلا من ال **Function** العادية

```
console.log(Array.from("12345", (n) => +n + +n));
```

```
► (5) [2, 4, 6, 8, 10] main.js:25
```

تعالى بقا بعد ما اتعلمنا ال **Set data type** و ال **Map** ندمجهم مع ال **Array.from**

```
let myArray = [1, 1, 1, 2, 3, 4];
```

لنفترض اني عندي ال **array** دي وعاييز ارجع منها الأرقام المميزة بس ... ممكن استخدم ال **Set** صح مش كدا ؟

```
let mySet = new Set(myArray);  
console.log(mySet);
```

```
► Set(4) {1, 2, 3, 4} main.js:32
```

اهو تعاملت مع الرقم المكرر وخليته رقم واحد باستخدام ال **set** ولكن دلوقتي انا عاييز النتيجة دي تكون **array** هنا بقا يجي دور ال **Array.from** لما احط جواه نتيجة ال **set** الي انا عملتها دي بقا

```
console.log(Array.from(mySet));
```

```
► (4) [1, 2, 3, 4] main.js:34
```

بسطر كود واحد بس قدرت احذف تكرار الرقم 1 وكمان ارجع النتيجة علي شكل **Array** ممكن أوصل لنفس الحل بطرق تانية كتير ولكن هتاخد مني وقت ومجهود وسطور كتير لذلك ادي حل مختصر

ممكن استخدم حل اخر من خلال ال **Spread Operator** الي لما استخدمها بقدر ارجع **Array** برضو

```
console.log([...new Set(myArray)]); // Future
```

```
► (4) [1, 2, 3, 4] main.js:36
```

لو عندي **function** اسمها **thisArg** وطلبت جواها اني اعمل **return** لـ **arguments** ومررت الـ **arguments** دي فيما بعد زي المثال التالي

```
function testArg() {  
    return arguments;  
}  
  
console.log(testArg("Osama", "Ahmed", "sayed"));
```

```
main.js:44  
► Arguments(3) ['Osama', 'Ahmed', 'sayed',  
  callee: f, Symbol(Symbol.iterator): f]
```

هلاقي الـ **Arguments** اللي مررتها فعلا رجعتلي

طيب لو عايز اعمل نفس الحركة دي وارجعهم علي شكل **Array** واحط أي قيم انا عايزها هستخدم **Array.from** الـ

```
function af() {  
    return Array.from(arguments);  
}  
  
console.log(af("Osama", "Ahmed", "sayed", 1, 2, 3));
```

```
► (6) ['Osama', 'Ahmed', 'sayed', 1, 2, 3] main.js:53
```

كل دي طرق بقدر استخدم فيها الـ **Array.from** عشان اقدر أوصل لحلول مباشرة وسريعة توصلني لعمل **Array** وبالتالي لما يكون عندي **Array** دا بيديني مساحة اطبق حاجات كتير جدا من خصائص الـ **Array** عشان اقدر اعمل أي حاجه انا عايزها

لا تنسونا من دعواتكم

Never Forget That (**Always Search**)

By: Amr Elsayed

♥ راقّت لي ♥



Abouzaid

@droos_online



لازم تستوعب انك بتبدأ كل يوم بموارد محدودة
وممكن تخلص منك عادي في نص اليوم.

طاقتك، وقتك، تركيزك وانتباهك، كل دول موارد
محدودة فحاول قدر الإمكان استهلاكهم في الحاجات
المهمة أولاً.

تأجيلك للحاجات المهمة لأخر اليوم قرار مش صح
أوي، لان مواردك - في الغالب - هتكون أسْتُنْفِذت
وبتشطب.

26 Jan 2022 • 19:00

♥ اذكر الله

الزموا ذكر الله ففيه العون على متاعب الحياة ♥

اللهم إني أستغفرك واتوب إليك، لا إله الا انت سبحانك إني كنت من الظالمين

أَلَا بِذِكْرِ اللَّهِ تَطْمَئِنُّ الْقُلُوبُ

لا تنسونا من دعواتكم

By: Amr Elsayed

Array.copyWithin Method Lesson #129

Array Methods

```
- Array.copyWithin(Target, Start => Optional, End => Optional)
"Copy Part Of An Array To Another Location in The Same Array"
-- Any Negative Value Will Count From The End
-- Target
---- Index To Copy Part To
---- If At Or Greater Than Array Length Nothing Will Be Copied
-- Start
---- Index To Start Copying From
---- If Ommited = Start From Index 0
-- End
---- Index To End Copying From
---- Not Including End
---- If Ommited = Reach The End
```

الغرض من الدرس دا استخدام **Method** اسمها **Array.copyWithin** .. بيتم استخدامها في اني اخذ جزء من ال **Array** بتاعتي من ال **location** الي هو فيه وانسخه في **location** ثاني في نفس ال **array** دون التأثير علي حجم ال **array** تعالي نشوف

```
let myArray = [10, 20, 30, 40, 50, "A", "B"];
```

اول حاجه بحطها في **Method** ال **Array.copyWithin** هي ال **Target** .. المقصود بيه المكان الي هعمل فيه عملية النسخ دي ... بمعنى ادق المكان الي عايز احط فيه العناصر الي هنسخها المكان دا هو ال **Target**

يعني علي سبيل المثال لو عايز انسخ العنصر "A" من مكانه واحطه مكان العنصر 30 اذا العنصر 30 دا هو ال **Target** بتاعي وال **index** بتاعه هو ال 2

اذا المفروض الشكل بعد نسخ العنصر "A" يبقی كدا

```
[10, 20, "A", 40, 50, "A", "B"];
```

شوية ملاحظات قبل الأمثلة العملية:

1- لو ال **Target** الي كتبتة دا اكبر من عدد العناصر الي في ال **Array** يبقی عملية النسخ مش هتتم

2- انك تكتب في ال **target** او ال **start** او ال **end** قيمة سالبة فدا معناه انك بتبدأ من اخر ال **array** للأول

3- عرفت ال **target** اذا مين بقا ال **start** هو المكان الي هاخذ العنصر المنسوخ من عنده او بداية ال **position** الي هبدأ النسخ من عنده

4- ال **end** هو نهاية ال **position** اللي هنسخ من عنده

5- لو محددتش ال **Start** هبدأ من **index 0** ولو محددتش ال **end** هيكمل لحد اخر **index**

6- لو كتبت ال **Start** و ال **End** افكر القاعدة **Not Including End**

تعالى نبدأ الأمثلة العملية بقا
دي ال **Array** الأصلية

```
let myArray = [10, 20, 30, 40, 50, "A", "B"];
```

```
myArray.copyWithin(3); // [10, 20, 30, 10, 20, 30, 40]
```

دا معناه ايه؟! ... دا معناه ان ال 3 دي هي ال **index** اللي انا عملت **target** عليه اللي هو مكان العنصر 40 ومدام محددتش **Start** ولا **End** يبقى هيبداً من ال **index 0** اللي هو العنصر 10 وهيأخذ معاه العناصر 20 و 30 و 40 مكان العناصر الموجودة في ال **Array**

يعني ال **index 0** اللي هو 10 هيحل محل ال **index 3** اللي هو 40 اللي هو ال **Target** اما العناصر 20 و 30 و 40 لاني محددتش ال **end** لحد فين يبقى العناصر دي هتحل محل العناصر 50 و A و B خلي بالك ان عدد عناصر ال **array** مش هيزيد انت لما بتعمل **target** لمكان معين هتسوخ فيه من غير ما تحدد ال **Start** ولا ال **end** زي ما قولنا هيبداً من ال **index 0** ويكمل لحد النهاية بما يتناسب مع عدد عناصر ال **array**

```
myArray.copyWithin(4, 6); // [10, 20, 30, 40, "B", "A", "B"]
```

في المثال دا ال **target** بتاعي هو ال **index 4** اللي هو في ال **array** الاصلية العنصر 50 وال **start** حددتها اللي هي ال **index 6** اللي هو العنصر "B" يبقى عملية النسخ هتم لعنصر "B" بدل العنصر 50

```
myArray.copyWithin(4, -1); // [10, 20, 30, 40, "B", "A", "B"]
```

ال **Target** هنا ال **index 4** اللي هو العنصر 50 وزي ما قولنا لو ال **Start** بـ -1 هيبداً العد من الاخر يعني "B"


```
myArray.copyWithIn(1, -2); // [10, "A", "B", 40, 50, "A", "B"]
```

ال **target** هو العنصر 20 في ال **index 1** وال **start** هو العنصر "A" من -2 ومدا م محدثش **End** يبقي هيكمل للأخر و هياخذ العنصر "B" كمان

```
myArray.copyWithIn(1, -2, -1); // [10, "A", 30, 40, 50, "A", "B"]
```

هنا ال **target** عندي ال **index 1** اللي هو العنصر 20 وال **start** من ال **index -2** اللي هو العنصر "A" وحددت ال **end** بال **index -1** وافتكر انه **Not Including End** يعني مش هياخذ معاه ال **index -1** دا فالنتيجة النهائية هتكون هي نسخ العنصر "A" من مكانه لمكان العنصر 20

لا تنسونا من دعواتكم

Never Forget That (**Always Search**)

By: Amr Elsayed

Array.some Method Lesson #130

Array Methods

```
- Array.some(CallbackFunc(Element, Index, Array), This Argument)
--- CallbackFunc => Function To Run On Every Element On The Given Array
----- Element => The Current Element To Process
----- Index => Index Of Current Element
----- Array => The Current Array Working With
--- This Argument => Value To Use As This When Executing CallbackFunc
--
Using
- Check if Element Exists In Array
- Check If Number In Range
```

ال **some method** بتطبق **Function** على كل عنصر من العناصر الموجود في ال **Array** بتاعتي وبتحط جواها شرط لو الشرط دا فعلا اتحقق بترجع **true** لو متحققش بترجع **false**

الموضوع مختلف تماما عن ال **filter** لان استخدام ال **filter** كان بيعمل شرط ع العنصر لو انطبق عليه الشرط بياخد العنصر دا معاه او بيضم العنصر دا معاه انما في ال **some** هو بيقولي **true** في حالة لو الشرط اتحقق و **false** في حالة عدم تحقق الشرط

يعني على سبيل المثال عندي مجموعة أسماء فبسأل سؤال هل الأسماء دي فيها اسم "Osama" !!؟ فبيمشي علي عناصر ال **array** دي وبيشوف لو الاسم دا فعلا موجود بيقولي **true** لو مش موجود بيقولي **false**

ببساطة شديدة نقدر نلخص الموضوع في ان استخدام ال **some** بينحصر في عمل فحص لوجود عنصر معين داخل ال **array** لو العنصر دا موجود فعلا يبقى **true** لو مش موجود يبقى **false**

ال syntax

ال **some** بتقبل مني **callback function** والي بيكون وظيفتها انها تمشي علي كل عنصر في ال **array** عشان تشوف الشرط اتحقق ولا لا

وال **this argument** هي ال **this** الخاصة بال **callback function** دي

ال **callback function** دي بتقبل من 3 حاجات

- 1- ال **element** اللي ال **callback function** بتمشي عليه
- 2- وال **index** اللي هو رقم عنصر دا داخل ال **array**
- 3- وال **array** اللي انت بتتعامل معاها

```
let nums = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];

let check = nums.some(function (e) {
  return e > 5;
});
console.log(check);
```

عندي هنا **array** اسمها **nums** جواها ارقام من 1 الي 10 ... عملت متغير عشان اخزن جواه شغل ال **callback** وبعد كذا مسكت ال **array** دي وطبقت عليها **method** ال **some** بال **syntax** بتاعها جوا ال **callback function** دي مررت **parameter** اسمه **e** عشان هو دا العنصر اللي هطبق عليه الشرط او هو دا **العنصر اللي هحطه** في مقارنة مع أي حاجة تانية عشان اتأكد هل الشرط **true** ام **false** الشرط بتاعي كان عبارة عن سؤال .. بقول فيه هل العنصر **e** دي اكبر من 5 هيعمل **loop** علي كل عناصر ال **array** ويشوف الرقم 1 الي هو بيمثل ال **e** دا اكبر من 5؟! النتيجة هتكون **false**

ولحد الرقم 5 النتيجة هتكون **false** اول ما يوصل للرقم 6 هتكون النتيجة **true** لان ال **e** الي هي بتمثل الرقم 6 دلوقتي اكبر فعلا من ال 5

ممکن اطبق نفس الطريقة بال **arrow function**

```
let check = nums.some((e) => e > 5);

console.log(check);
```

true main.js:20

بمجرد ما يوصل للرقم 6 الي هو اكبر من 5 الشرط هيتحقق وتكون النتيجة **true** وبعدها ال **some method** هتوقف ال **loop** ومش هيعمل **check** علي أي عنصر تاني بعد الرقم 6 دا

طيب تعالي نشوف استخدام ال **this**

some method ال Syntax

```
Array.some(CallbackFunc(Element, Index, Array), This Argument)
```

```
let nums = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];  
  
let myNumber = 10;  
  
let check = nums.some(function (e) {  
    return e > this;  
}, myNumber);  
  
console.log(check);
```



عملت متغير اسمه **myNumber** قيمته تساوي 10 ... بعد كدا عوضت عنه مكان ال **this** في ال **syntax** بتاع ال **some method**

إذا انا بقارن عناصر ال **element** بال **this** اللي هو بيعوض عن المتغير **myNumber** بالقيمة 10
فالسؤال هيكون كالتالي **=>** هل في عنصر اكبر من 10؟! ... النتيجة هتكون **false**

```
false      main.js:32
```

تعالى ناخد مثال تاني يكون **more advanced** شوية

```
let nums = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
```

```
function checkValues(arr, val) {  
    return arr.some(function (e) {  
        return e === val;  
    });  
}
```

هنا انا عملت **function** ممرت جواها **parameters 2** الي هما **arr** و **val** بعد كذا جواها عملت الشرط بتاعي قولتله **arr.some** هل ال **e** الي هو بيمثل كل عنصر في ال **Array** بتاعي بيساوي ال **val**؟! طيب مين ال **arr** ومين ال **val**

فاكر دروس ال **function** زمان لما كنت بمرر **parameters** واعوض عنها بـ **arguments** دا بالظبط الي هعمله



```
console.log(checkValues(nums, 20));  
console.log(checkValues(nums, 5));
```

ال **nums** الي عليه السهم دا هو ال **argument** الي هيحل محل أول **parameter** موجود في ال **function** الي هو اسمه **arr** ال **arr** دا هو الي استخدمت عليه ال **some method**

يبقي استنتج من الكلام دا ان انا شلت ال **arr.some** وعوضت عنها بـ **nums.some** ... فاهمني؟!
ومكان تاني **parameter** الي هو **val** عوضت عنه بال **argument** مرة 20 ومرة 5

فبقي السؤال كالتالي هل ال **val** دا لو قيمته 20 بيساوي ال **e** الي هو أي قيمة تنحصر ما بين 1 و 10 الي هما عناصر ال **nums array**؟! **.... <=** النتيجة هتكون **false** في اول سؤال

```
false      main.js:41
```

تاني سؤال هل ال **val** دي بالقيمة 5 تساوي ال **e** الي هو أي قيمة موجودة ما بين 1 و 10 عناصر ال **array**؟!

```
true       main.js:42
```

تعالى ناخد مثال تاني يكون **more advanced** برضو عبارة عن فحص للعنصر هل هو موجود في **ring** معين؟!

```
let nums = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
```

```
let range = {  
  min: 10,  
  max: 20,  
};
```

```
let checkNumberInRange = nums.some(function (e) {
  return e >= this.min && e <= this.max;
}, range);

console.log(checkNumberInRange);
```

عملت **object** اسمه **ring** فيه **minimum** بيساوي 10 و **maximum** بيساوي 20
 بعد كذا عملت متغير اسمه **checkNumberInRange** هخزن جواه النتيجة اللي هتحصل من شغل ال **some**
 استخدمت ال **some** ع ال **array** الموجودة عندي بال **syntax** بتاعها وعوضت عن ال **this** بال **range obj**
 الشرط بتاعي بيقول هل ال **e** اللي هو بيمثل أي عنصر من عناصر ال **nums array** دا اكبر من او يساوي
 ال **this.min** اللي هي قيمتها 10 وباقى الشرط هل ال **e** دي اصغر من ال **this.max** اللي هي قيمتها 20
 علما بان ال **this** دي تعوض ع ال **range**

طبعا موجود الرقم 10 في ال **array** بتاعتي الشرطين هيتحققوا مع بعض ان ال **e** لما تكون قيمتها بـ 10
 هتكون فعلا اكبر من او تساوي ال **min** الموجود في ال **Range** وكمان هي اصغر من اوي تساوي ال **max**

```
true      main.js:53
```

طيب لو روجت شيلت الرقم 10 من ال **array**

```
let nums = [1, 2, 3, 4, 5, 6, 7, 8, 9];
```

هيبقي ال **e** لما يوصل لآخر رقم اللي هو 9 هيسأل نفس السؤال هل هي اكبر من او تساوي ال **min** اللي قيمته
 بتساوي 10 في ال **range**؟! <= هتكون النتيجة **false** الشرط متحققش

```
false      main.js:53
```

لا تنسونا من دعواتكم

Never Forget That (**Always Search**)

By: Amr Elsayed

Array.every Method Lesson #131

Array Methods

```
- Array.every(CallbackFunc(Element, Index, Array), This Argument)
--- CallbackFunc => Function To Run On Every Element On The Given Array
----- Element => The Current Element To Process
----- Index => Index Of Current Element
----- Array => The Current Array Working With
--- This Argument => Value To Use As This When Executing CallbackFunc
```

ال **every** زي ال **some** في ال **Syntax** الفرق بينهم ان ال **some** بترجع **true** لو عنصر من عناصر ال **Array** حقق الشرط فعلا لكن ال **every** عشان ترجع **true** لازم كل العناصر تحقق الشرط لو عنصر واحد بس محقق الشرط ال **every** هتكون نتيجتها **false**

```
const locations = {
  20: "Place 1",
  30: "Place 2",
  50: "Place 3",
  40: "Place 4",
};

let mainLocation = 15;
```

أول حاجة عشان احول ال **object** دا ل **array** عشان اقدر اعمل **loop** علي كل عناصره واناكد من الشرط بتاعي هستخدم **method** اسمها **object.keys** وجواها هحط اسم ال **object** بتاعي

```
let locationsArray = Object.keys(locations);

console.log(locationsArray);
```

```
► (4) ['20', '30', '40', '50'] main.js:22
```

بعد كذا هحتاج اني احول عناصر ال **array** بتاعي دي من **string** ل **numbers**

```
let locationArrayNumbers = locationsArray.map((n) => +n);

console.log(locationArrayNumbers);
```

```
► (4) [10, 20, 30, 40] main.js:26
```

بعد كذا هعمل المتغير اللي هخزن فيه شغل ال **every** وارجع استدعيه عشان اشوف النتيجة

```
let check = locationArrayNumbers.every(function (e) {  
  return e > this;  
}, mainLocation);  
  
console.log(check);
```

عوضت عن ال **this** بال **mainLocation** اللي قيمته فووووق كانت بـ 15 عشان استخدمه في الشرط بتاعي

ودا نفس الحل بالظبط بشكل مختصر جدا جدا عن اللي فات

```
console.log(locationsArray = Object.keys(locations).map((n) => +n).every(function(e) {  
  return e > this  
},mainLocation));
```

true main.js:64

حولت اخلي الحل مختصر اكثر بالشكل دا بس تقريبا في حاجة غلط
مش قادر اوظف ال **this** مع ال **arrow function** وكانت النتيجة بتطلع **false** مع ان المفروض تكون **true**

```
console.log(locationsArray = Object.keys(locations).map((n) => +n).every((e) => e > this, mainLocation));
```

false main.js:69

لكن اول ما شلت ال **this** وعوضت بالمتغير اللي اسمه **mainLocation** مباشرة الكود اشتغل عادي

```
console.log(locationsArray = Object.keys(locations).map((n) => +n).every((e) => e > mainLocation));
```

true main.js:70

وبكدا يكون الحل تم في سطر واحد فقط

♥ راق لي ♥



أي خوف جواك هتواجهه وتشتغل عليه هتحقق انجاز واي خوف هيخوفك وهتكسل او تخاف تواجهه هيتحول لنقطة سودة في حياتك وندم ملازمك واجه خوفك .. مش عارف حاجه اتعلمها اوعي تستسلم يا صاحبي

♥ اذكر الله ♥

لا إله الا الله وحده لا شريك له .. له الملك وله الحمد وهو على كل شيء قدير.

استغفروا الله العظيم الذي لا إله الا هو الحي القيوم واتوب اليه.

اللهم صلّ وسلم وزد وبارك على نبينا محمد ﷺ.

لا إله الا انت سبحانك إني كنت من الظالمين.

Never Forget That (**Always Search**)

By: Amr Elsayed

Spread Syntax And Use Cases Lesson #132

```
/*  
  Spread Operator => ...Iterable  
  "Allow Iterable To Expand In Place"  
*/
```

كلمة **spread** معناه انتشار وييتم كتابة **spread** ال **syntax** عن طريق كتابة " 3 نقط ... " قبل ال **iterable** الغرض من استخدام ال **spread** هو اني اعمل **extract** أو **expand** للعناصر المستهدفة

```
// Spread With String => Expand String
```

```
console.log("Osama");  
console.log(..."Osama");  
console.log([... "Osama"]);
```

Osama	main.js:8
O s a m a	main.js:9
► (5) ['O', 's', 'a', 'm', 'a']	main.js:10

أول حاجة طبعت اسمه "Osama" عادي جدا ... ثاني حاجة عملت **Spread** للاسم فعمل انتشار للحروف
ثالث حاجة عملت **Spread** للاسم داخل اقواس ال **array** فحول كل حرف ل **index** في **Array**

```
// Concatenate Arrays
```

```
let myArray1 = [1, 2, 3];  
let myArray2 = [4, 5, 6];  
  
let allArrays = [...myArray1, ...myArray2];  
console.log(allArrays);
```

```
► (6) [1, 2, 3, 4, 5, 6]    main.js:18
```

لو عندي اكثر من **Array** ممكن استخدم ال **spread** في عمل **concatenate** ليهم فيتم دمجهم

```
// Copy Array
```

```
let copiedArray = [...myArray1];  
console.log(copiedArray);
```

```
► (3) [1, 2, 3]    main.js:23
```

أقدر من خلال ال **Spread** إني أخزن **array** جوا متغير او بمعني ثاني اعمل منها نسخة

```
// Push Inside Array
```

```
let allFriends = ["Osama", "Ahmed", "Sayed"];  
let thisYearFriends = ["Sameh", "Mahmoud"];  
  
allFriends.push(...thisYearFriends);  
  
console.log(allFriends);
```

```
► (5) ['Osama', 'Ahmed', 'Sayed', 'Sameh', 'Mahmoud']    main.js:32
```

من خلال ال **spread** اقدر اضيف عناصر **Array** داخل عناصر **array** ثانية كبديل لل **push Method**

```
// Use With Math Object
```

```
let myNums = [10, 20, -100, 100, 1000, 500];  
console.log(Math.max(...myNums));
```

```
1000    main.js:37
```

عن طريق ال **Spread** أقدر طبق خواص ال **Math Method** علي **array** موجودة فعلا عندي بدل ما كنت بحط ارقام بداخل ال **Math Method** عشان أوصل للنتيجة

```
// Spread With Objects => Merge Objects
```

```
let objOne = {  
  a: 1,  
  b: 2,  
};  
let objTwo = {  
  c: 3,  
  d: 4,  
};  
  
console.log({ ...objOne, ...objTwo, e: 5 });
```



```
► {a: 1, b: 2, c: 3, d: 4, e: 5} main.js:50
```

عن طريق ال **spread** أقدر ادمج اكثر من **object** مع بعض ولكن عشان العملية دي تتم لازم احط العملية دي داخل اقواس ال **object {}**

كذلك في عمليات دمج ال **Array** او أي حاجة تخص التعامل مع ال **Array** لازم احطها داخل قوسين ال **array []**

Map And Set Challenge Lesson #133

```
/*
  Map And Set + What You Learn => Challenge
  Requirements
  - You Cant Use Numbers Or True Or False
  - Don't Use Array Indexes
  - You Cant Use Loop
  - You Cant Use Any Higher Order Function
  - Only One Line Solution Inside Console
  - If You Use Length => Then Only Time Only
  Hints
  - You Can Use * Operator Only In Calculation
  - Set
  - Spread Operator
  - Math Object Methods
*/

let n1 = [10, 30, 10, 20];
let n2 = [30, 20, 10];
```

Map And Set Challenge Solution

```
console.log([...n1,...n2].length * Math.max(...n2) ); // 210
console.log([...n1,...n2].length * new Set(n2).size * Math.min(...n1) ); // 210
```

لا تنسونا من دعواتكم

Never Forget That (**Always Search**)

By: Amr Elsayed

Assignments From [123] = > [133]

لينك التكميلات: [\[133\] <= \[123\]](#)

التكليف: 01

```
// Assignment 1
let setOfNumbers = new Set([10]);
setOfNumbers.add(20).add(setOfNumbers.size);
console.log(setOfNumbers);
console.log([...setOfNumbers][2]);
```

التكليف: 02

```
// Assignment 2
let myFriends = ["Osama", "Ahmed", "Sayed", "Sayed", "Mahmoud", "Osama"];

console.log(Array.from(new Set(myFriends)).sort());
console.log([...new Set(myFriends)].sort());

// Needed Output
// (4) ['Ahmed', 'Mahmoud', 'Osama', 'Sayed']
```

التكليف: 03

```
// Assignment 3
let myInfo = {
  username: "Osama",
  role: "Admin",
  country: "Egypt",
};

let myMap = new Map(Object.entries(myInfo));
console.log(myMap);
console.log(myMap.size);
console.log(myMap.has("role"));

// Needed Output
// Map(3) {'username' => 'Osama', 'role' => 'Admin', 'country' => 'Egypt'}
// 3
// true
```

```
// Assignment 4
let theNumber = 100020003000;

console.log([...new Set(theNumber.toString())].sort().slice(true).join(""));

console.log(
  +[...new Set(Array.from(theNumber.toString(), (e) => (e > 0 ? e : "")))].join(
    ""
  )
);
// Needed Output
// 123
```

```
// Assignment 5
let theName = "Elzero";

// [1]
console.log(theName.split(""));

// [2]
console.log(Array.from(theName));

// [3]
console.log([...theName]);

// [4]
let emptyArray = [];
for (let i = 0; i < theName.length; i++) {
  emptyArray.push(theName[i]);
}
console.log(emptyArray);

// [5]
console.log([...new Set(theName)]);

// [6]
console.log(Object.assign([], theName))
// Needed Output
// ['E', 'l', 'z', 'e', 'r', 'o']
```

```
// Assignment 6

// [1]
let charsOne = ["A", "B", "C", "D", "E", 10, 15, 6];

console.log(charsOne.sort().copyWithin(0,3,6))

// Needed Output
// ['A', 'B', 'C', 'A', 'B', 'C', 'D', 'E']

// [2]
let charsTwo = ["A", "B", "C", 20, "D", "E", 10, 15, 6];

console.log(charsTwo.sort().copyWithin(0, 4, 8))

// Needed Output
// ['A', 'B', 'C', 'D', 'A', 'B', 'C', 'D', 'E']

// [3]
let charsThree = ["Z", "Y", "A", "D", "E", 10, 1];

console.log(charsThree.copyWithin(2))

// Needed Output
// ["Z", "Y", "Z", "Y", "A", "D", "E"]
```

```
// Assignment 7
let numsOne = [1, 2, 3];
let numsTwo = [4, 5, 6];

// [1]
console.log([...numsOne,...numsTwo])

// [2]
console.log(numsOne.concat(numsTwo))

// [3]
console.log([...Array.from(numsOne), ...Array.from(numsTwo)]);

// [4]
numsOne.push(...numsTwo);
console.log(numsOne)

// Needed Output
// [1, 2, 3, 4, 5, 6]
```


❤ عبارات تحفيزية ❤



❤ اذكر الله

الزموا ذكر الله ففيه العون على متاعب الحياة ❤

لا إله إلا الله وحده لا شريك له .. له الملك و له الحمد وهو على كل شيء قدير

لا إله إلا أنت سبحانك إني كنت من الظالمين

سبحان الله وبحمده سبحان الله العظيم

أَلَا بِذِكْرِ اللَّهِ تَطْمَئِنُّ الْقُلُوبُ

لا تنسونا من دعواتكم

Never Forget That (**Always Search**)

By: Amr Elsayed

Intro And What Is Regular Expression Lesson #134

```
/*  
  Regular Expression  
  - Email  
  - IP  
  - Phone  
  - URL  
*/
```

ببساطة شديدة جدا ال **Regular Expression** موجود في معظم لغات البرمجة مش مقتصر ع ال **JS** بس طيب أي الفائدة منه باختصار ...

```
let str1 = '10 20 100 1000 5000';  
let str2 = '0s1 0s12 0s123 0s1230s 0s123120s123';
```

لو اديتك **str1** دا وقولتلك انا عايزك تجبلي أي رقم اصغر من 20 هتقولي بسيطة هحواله ال **string** دا ل **array** وبعدين هعمل **condition** ومن خلاله هقدر أوصل للرقم الأصغر من 20 او هستخدم ال **Filter** لو اديتك **str2** منك ترجعلي أي **String** موجود قبله ارقام وبعده ارقام وبعد الأرقام دي فيه **String** كمان وبعد ال **string** دا في ارقام كمان !!

هتلاقي ان الموضوع بقا صعب جدا عشان هتحتاج تعمل كمية **conditions** كتيرة وهتخش في مليون سيناريو عشان تحقق المطلوب

هنا بقا يجي دور ال **Regular Expression** اللي ببساطة من خلالها بقدر اعمل شرط بسيط الشرط دا بيكون فيه **Pattern** معين ثابت متعارف عليه ومتعارف انه ال **pattern** دا بيكون **Valid** او صحيح يعني مثلا دا **pattern** لميل **Not valid** او غير صحيح

```
let invalidEmail = 'Osama@@@gmail....com';
```

ولكن الميل دا **valid** وال **pattern** بتاعه او النمط المكتوب بيه سليم وصحيح

```
let validEmail = 'o@nn.sa';
```

والمتعارف عليه في أي ميل **valid** او **صحيح** ان يكون مثلاً اوله اسم الشخص او حرف من اسمه وأرقام بعد كذا بتيجي **علامة** ال **@** بعد كذا بيجي اسم ال **Email provider** او المكان الي بيقدملك الميل دا زي **Gmail** او **Hotmail** وغيرهم ... بعد كذا بيدور علي **النقطة** . الي بعد ال **Email provider** بعد كذا بيجي الدور علي ال **Top level Domain** الي هو ممكن يكون **com** او **org** او **net** او **info** او حاجات تانية كتير مختلفة كل الحاجات دي لو موجودة في **ايميل بشكل معين** او بـ **pattern** او **نمط معين** بتكون صحيحة

لو انت استخدمت ال **if condition** العادي هيبقي الموضوع صعب جداً و**هتحتاج ملايين السيناريوهات** عشان تتطابق كل الشروط الصحيحة لأي ميل **valid** او أي **URL** لموقع مكتوب بشكل صحيح وهنا بقا يجي دور ال **Regular Expressions** وبساطة تعاملها في المواقف الي زي دي

ملحوظة خد بالك منها ان هنا مفيش أي مقارنة بين ال **IF Conditions** و ال **Regular Expressions** هنا انا مش بفضل حاجة ع التانية انت المفروض اتعلمت ان كل **Tool** عندي او **Method** بيكون ليها استخدام **غير التانية** في موقف غير الثاني

```
let ip = '192.168.2.1'; // IPv4
```

علي سبيل المثال ال **IP** دا **valid** وصحيح ولكن عشان تتأكد من صحة كل مقطع فيه لو استخدمت ال **IF cond** هتواجه صعوبة شديدة ودا يعتبر **IP** بسيط مابالك بقا لو **IPv6** مثلاً او ال **Mac Address**

وعلي سبيل المثال برضو كل ال **URL** المكتوبة هنا او كل **pattern** او **نمط** من دول مكتوب بيه ال **URL** هو **نمط صحيح وسليم** فانت بتستخدم ال **Regular Expression** عشان تعمل **Matching** او **مطابقة** لكل مقطع موجود في ال **URL** وتتأكد ان كل مقطع من دول لو اكتب بالشكل الي في ال **screen** هو سليم وصحيح فتقوله لو ال **URL** مكتوب **http** بس سليم لو **https** سليم لو في الاخر فيه **Com** سليم لو **net** سليم او **org** سليم

```
let url = 'elzero.org';
let url = 'elzero.org/';
let url = 'http://elzero.org/';
let url = 'http://www.elzero.org/';
let url = 'https://elzero.org/';
let url = 'https://www.elzero.org/';
let url = 'https://www.elzero.org/?facebookid=asdadasd';
```

Regular Expression – Modifiers Lesson #135

Regular Expression

Syntax

```
/pattern/modifier(s);  
new RegExp("pattern", "modifier(s)")
```

Modifiers => Flags

```
i => case-insensitive  
g => global  
m => Multilines
```

Search Methods

- match(Pattern)

Match

- Matches A String Against a Regular Expression Pattern
- Returns An Array With The Matches
- Returns null If No Match Is Found.

بعد ما فهمنا وظيفة ال **Regular Expression** دلوقتي هنتعرف ع ال **Syntax** بتاعها وطرق الكتابة

```
let myString = "Hello Elzero Web School I Love elzero hello";
```

دلوقتي انا عندي ال **string** دا موجود فيها **Elzero** الاول حرف **E** كبير .. و **elzero** التانيه اول حرف **e** صغير
دلوقتي هنعمل **Pattern** معين من تصميمنا ونبدأ نقارن الكلام الموجود في ال **string** مع ال **pattern** اللي احنا
عملناه ونشوف النتيجة

```
let regex = /Elzero/;  
  
console.log(myString.match(regex));
```

بعد ما عملت ال **Pattern** هبدأ استخدم **method** اسمها **match** همرر جواها المتغير **regex** اللي انا مخزن
جواه ال **Pattern** بتاعي دا .. بالظبط كاني دخلت مكان معين ومعايا مواصفات لشخص معين عايز ادور عليه

```
main.js:27  
▶ ['Elzero', index: 6, input: 'Hello Elzero Web School I Love elzero hello', groups: undefined]
```

قالي ان **Elzero** اول حرف فيها **Capital** موجود عند ال **index 6**

```
let regex = /elzero/;  
  
console.log(myString.match(regex));
```

لو حطيت في ال **Pattern** بتاعي اول حرف في **elzero** صغير

```
main.js:27  
▶ ['elzero', index: 31, input: 'Hello Elzero Web School I Love elzero hello', groups: undefined]
```

هيقولي موجود اهو عند ال **index 31**

إذا دا ال **syntax** بتاعي بيكون فيه اول حاجه ال **pattern** بعد كدا عندي حاجة اسمها **Modifier**

```
Syntax  
/pattern/modifier(s);
```

بيتم استخدام ال **modifier** دا عشان اعدل في طريقة البحث

```
Modifiers => Flags  
i => case-insensitive  
g => global  
m => Multilines
```

يعني لو كتبت جنب ال **Pattern** دا أي حاجه من ال **modifiers** دي وليكن ال **i** دا وكأني بقوله تجاهل حساسية الحرف الأول سوا انا كتبتة **Small** او **Capital**

```
let regex = /elzero/i;  
  
console.log(myString.match(regex));
```

لاحظ ان ال **Pattern** بتاعي **elzero** اول حرف **Small** وبعده ال **modifier** بتاعي ال **i**

```
main.js:27  
▶ ['Elzero', index: 6, input: 'Hello Elzero Web School I Love elzero hello', groups: undefined]
```

فبعد ما لغيت حساسية الحروف وبدا يتعامل مع اول حرف ال **e** سوا **capital** او **small** علي انه مجرد حرف النتيجة بقت **Elzero** اول كلمة في ال **string** اللي الحرف بتاعها **Capital** لانه تجاهل حساسية الحروف

والنتيجة كانت **Elzero** اللي اول حرف فيها **Capital** لانه اثناء البحث بيمشي بالترتيب وبيجيب اول نتيجة

```
let regex = /elzero/ig;  
  
console.log(myString.match(regex));
```

تاني **flag** عندي اللي هو ال **g** يعني **Global** يعني هيدور علي اسم **elzero** سواء اول حرف فيه **Small** او **capital**

```
► (2) ['Elzero', 'elzero'] main.js:27
```

في الطريقة دي هو بيدور علي كل أسماء ال **elzero** ويرجعها لي في شكل **Array**
لو كتبت **Pattern** معين مش موجود في ال **String** بتاعك هيرجعلك **null**

Match

- Matches A String Against a Regular Expression Pattern
- Returns An Array With The Matches
- Returns null If No Match Is Found.

تالت **flag** اللي هو ال **m** يعني **Multilines** .. لو عندي اكر من سطر كود عايز اعمل **match** لحاجة جوا السطور دي مع ال **pattern** بتاعي

طيب بالمناسبة دا شكل ال **syntax** التاني اللي بيتعمل بال **constructor**

```
new RegExp("pattern", "modifier(s)")
```

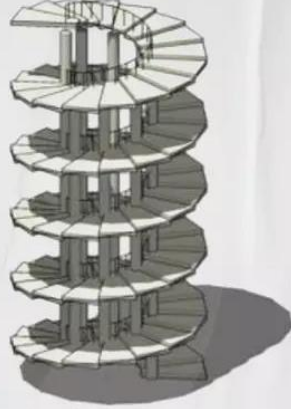
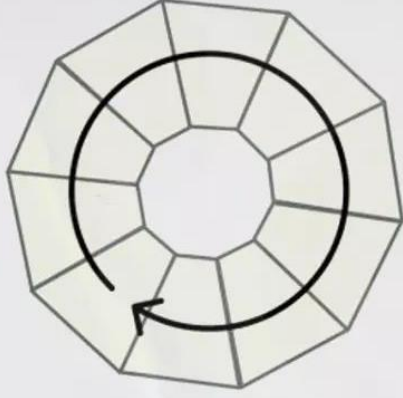
```
let regex = new RegExp("elzero", "ig");  
console.log(myString.match(regex));
```

```
► (2) ['Elzero', 'elzero'] main.js:30
```

هي هي نفس النتيجة، ولكن يُفضل استخدام اول طريقة

♥ راقِ لي ♥

أنت لا تسير في دوائر ...



تم الترجمة من قبل @ihamadfahad7

... أنت فقط بحاجة الى النظر
بشكل مختلف

♥ كمل سعي

♥ اذكر الله

الزموا ذكر الله ففيه العون على متاعب الحياة ♥

سبحان الله و الحمد لله و لا اله الا الله و الله أكبر و لا حول ولا قوة الا بالله
اللهم صلّ وسلم وزد وبارك على نبينا محمد ﷺ

اللهم إني أستغفرك واتوب إليك، لا إله الا انت سبحانك إني كنت من الظالمين

ألا بِذِكْرِ اللَّهِ تَطْمَئِنُّ الْقُلُوبُ

By: Amr Elsayed

Regular Expression – Ranges Part 1 Lesson #136

Ranges

- Part 1

(X|Y) => X Or Y

[0-9] => 0 To 9

[^0-9] => Any Character Not 0 To 9

Practice

ال **ranges** مميز جدا في ال **Regular Expression** كمان هيساعدك تحل مسائل بطريقة أسهل كثير عن الأول

```
let tld = "Com Net Org Info Code Io";

let tldRe = /(info|org|io)/i;
console.log(tld.match(tldRe));
```

في المثال دا استخدمت علامة ال **|** **OR** عشان أقوله يرجعلي أي واحده من ال **3** حاجات الي موجودين في ال **pattern** بتاعي فهيمشي علي العناصر الموجودة في المتغير **tld** اول حاجه هيلاقبها بتطابق الكلام الموجود في ال **pattern** هتكون هي كلمة **org** ولاحظ اني استخدمت ال **Flag i** عشان يتجاهل حساسية الحروف

```
main.js:45
(2) ['Org', 'Org', index: 8, input: 'Com Net Org Info Code Io', groups: undefined]
```

```
let tldRe = /(info|org|io)/ig;
```

ولو حطيت ال **flags** **i** و **g** هيجيب كل العناصر الي بت **match** مع ال **Pattern** الي انا حطيته

```
► (3) ['Org', 'Info', 'Io']
```

```
let nums = "12345678910";
let numsRe = /[0-9]/g;
console.log(nums.match(numsRe));
```

هنا مثلا عندي متغير اسمه **nums** موجود فيه مجموعة ارقام بعد كدا متغير اسمه **numsRe** بحط فيه ال **Pattern** بتاعي بقوله فهيه انا عايز الأرقام من **1-9** وكتبت ال **flag g** عشان يجيب كل الأرقام

```
► (11) ['1', '2', '3', '4', '5', '6', '7', '8', '9', '1', '0']
```



```
let notNums = "12345678910";
let notNsRe = /^[^0-2]/g
console.log(notNums.match(notNsRe));
```

في المثال دا استخدمت ال **not ^** اللي بتعبر عن اني بقوله هاتلي كل الأرقام ماعدا الموجودة في ال **ring** من 0 ل 2

```
► (7) ['3', '4', '5', '6', '7', '8', '9']
```

```
let specialNums = "1!2@3#4$5g8910" ;
let specialNumsRe = /^[^0-9]/g
console.log(specialNums.match(specialNumsRe));
```

في المثال دا بقوله هاتلي كل العناصر الموجودة عندك ما عدا الأرقام الموجود في ال **رينج** من 0 الي 9 فتهيطلع كل ال **characters** اللي موجودة بين الأرقام

```
► (5) ['!', '@', '#', '$', '%']
```

طبعا شايف الطريقة دي سهلة جدا مقارنة بال **condition** و ال **loop** و ال **filter** والحاجات دي

```
let practice = "Os1 Os10s Os2 Os8 Os80s";
let practiceRe = /Os[5-8]Os/g;
console.log(practice.match(practiceRe));
```

في المثال دا انا بقوله ال **pattern** بتاعي بيبدأ بكلمة **Os** وحتيت **رينج** ارقام من 5 الي 8 وبعدين كلمة **Os** ومعاها ال **Global Flag**

```
► ['Os80s']
```

Regular Expression – Ranges Part 2 Lesson #137

Ranges

- Part 2

[a-z] -- [^a-z]

[A-Z] -- [^A-Z]

[abc] -- [^abc]

```
let myString = "AaBbcdefG123!234%^&*";
```

عندك متغير **myString** عبارة عن **Capital & small Letters** و **Numbers** و **Characters**

في أول مثال انا الشرط بتاعي كان اني احصل علي الحرف من **a-z** ال **small**

```
let atozSmall = /[a-z]/g;  
console.log(myString.match(atozSmall));
```

```
► (6) ['a', 'b', 'c', 'd', 'e', 'f']
```

وفي المثال التالي هستخدم ال **^** الي بتدل علي كلمة **Not** او **النفي** وبالتالي انا بقوله عايز كل الموجود في متغير **myString** دا عدا .. عدا الحروف من **a-z** ال **small** فهيجيب كل حاجة الا من **a-z**

```
let NotAtozSmall = /^[a-z]/g;  
console.log(myString.match(NotAtozSmall));
```

```
► (14) ['A', 'B', 'G', '1', '2', '3', '!',  
      '2', '3', '4', '%', '^', '&', '*']
```

```
let atozCapital = /[A-Z]/g;
console.log(myString.match(atozCapital));

let NotAtozCapital = /^[^A-Z]/g;
console.log(myString.match(NotAtozCapital));
```

```
► (3) ['A', 'B', 'G']
```

```
(17) ['a', 'b', 'c', 'd', 'e', 'f',
► '1', '2', '3', '!', '2', '3', '4',
  '%', '^', '&', '*']
```

نفس المثال الي فوق، ولكن المرادي مع **الحروف** ال **Capital** مرة انا عايزهم ومرة عايز كل حاجه الا هما

```
let aAndcAnde = /[ace]/g;
console.log(myString.match(aAndcAnde));

let NotaAndcAnde = /^[^ace]/g;
console.log(myString.match(NotaAndcAnde));
```

```
► (3) ['a', 'c', 'e']
```

```
(17) ['A', 'B', 'b', 'd', 'f', 'G',
► '1', '2', '3', '!', '2', '3', '4',
  '%', '^', '&', '*']
```

نفس الأمثلة الي فوق، ولكن ع الحروف **abe** مرة انا عايزهم ومرة ثانيه باستخدام ال **^** عايز كل حاجه الا هما

```
let lettersCapsAndSmall = /[a-zA-Z]/g;  
console.log(myString.match(lettersCapsAndSmall));
```

```
► (9) ['A', 'a', 'B', 'b', 'c', 'd', 'e', 'f', 'G']
```

في المثال دا انا بحدد اني عايز ال **letters** كلهم سواء **Capital** او **small** فقط بدون **Numbers** او **Characters**

```
let numsAndSpecials = /^[^a-zA-Z]/g;  
console.log(myString.match(numsAndSpecials));
```

```
► (11) ['1', '2', '3', '!', '2', '3', '4', '%', '^', '&', '*']
```

في المثال دا مع استخدام ال **Not** [^] انا بقوله عايز كل حاجه سوا **Numbers** او **Characters** ما عدا ال **letters** كلها سواء **Capital** او **small**

```
let specials = /^[^a-zA-Z0-9]/g;  
console.log(myString.match(specials));
```

```
► (5) ['!', '%', '^', '&', '*']
```

وفي المثال دا مع استخدام ال **Not** [^] انا حددت اني عايز بس ال **Special Characters** فقط ما دون ذلك سواء **Numbers** او **Letters** مش محتاجهم

لا تنسونا من دعواتكم

Never Forget That (**Always Search**)

By: Amr Elsayed

Regular Expression – Character Classes Part 1 Lesson #138

Regular Expression

Character Classes

```
. => matches any character, except newline or other line terminators.  
\w => matches word characters. [a-z, A-Z, 0-9 And Underscore]  
\W => matches Non word characters  
\d => matches digits from 0 to 9.  
\D => matches non-digit characters.  
\s => matches whitespace character.  
\S => matches non whitespace character.
```

ال **character classes** هتساعدك **تحلل وتفكك** الاكواد المعقدة او من خلالها هتقدر تعمل **match** بسهولة لأي داتا بحيث تنتقي ال **Validation data** بس

```
let email = "O@@@g...com O@g.com O@g.net A@Y.com O-g.com o@s.org 1@1.com";
```

اول **character** هو ال **.** **dot** ودا بي **match** كل ال **Characters** حتى المسافات ما عدا ال **new lines** او ال **line terminators**

```
let dot = ./g;  
console.log(email.match(dot));
```

```
(59) ['O', '@', '@', '@', 'g', '.', '.', '.', 'c', 'o', 'm', ' ', 'O', '@',  
'g', '.', 'c', 'o', 'm', ' ', 'O', '@', 'g', '.', 'n', 'e', 't', ' ', 'A',  
'@', 'Y', '.', 'c', 'o', 'm', ' ', 'O', '-', 'g', '.', 'c', 'o', 'm', ' ',  
'o', '@', 's', '.', 'o', 'r', 'g', ' ', '1', '@', '1', '.', 'c', 'o', 'm']
```

بعد كدا عندي ال **\w** ولاحظ ان ال **w** مكتوبة **small** ودا بيستخدم عشان ي **match** او يستخرج ال **word characters** زي الحروف من **a-z** و **A-Z** و الأرقام وما الي ذلك و ال **underscore_**

```
let Word = /\w/g;  
console.log(email.match(Word));
```

```
(35) ['O', 'g', 'c', 'o', 'm', 'O', 'g', 'c', 'o', 'm', 'O', 'g', 'n', 'e',  
't', 'A', 'Y', 'c', 'o', 'm', 'O', 'g', 'c', 'o', 'm', 'o', 's', 'o', 'r',  
'g', '1', '1', 'c', 'o', 'm']
```

المسافات و ال . dot

► (24) ['@', '@', '@', '.', '.', '.', '.', '@', '.', ' ', '@', '.', ' ', '@',
 '.', ' ', '-', '.', ' ', '@', '.', ' ', '@', '.']

Diagram illustrating the generation of email addresses from a template. The template is divided into segments: \w, @, \w, ., com, |, net. Arrows indicate the mapping of these segments to the generated email addresses: ['0@g.com', '0@g.net', 'A@Y.com', '1@1.com'].

لا تنسونا من دعواتكم

Never Forget That (Always Search)

By: Amr Elsayed

Regular Expression – Character Classes Part 2 Lesson #139

```
Regular Expression
Character Classes
\b => matches at the beginning or end of a word.
\B => matches NOT at the beginning/end of a word.

Test Method
pattern.test(input)
```

هناك مع بعض ال **Character classes** وكمان هندرس حاجه اسمها **Test Method**

لو عندي موقع وفيه اشخاص هتسجل فيه فطبيعي هيسجل ناس بأسماء عادية اللي هي أسمائهم لكن في اشخاص تانيه بيكون **Spammers** فانا عايز استخدم ال **regular expression** عشان اعرف الأشخاص دي واتجنبهم مثلاً

```
let names = "Sayed 1Spam 2Spam 3Spam Spam4 Spam5 Osama Ahmed Aspamo";
```

```
let re = /(\bspam|spam\b)/ig;
console.log(names.match(re));
```

استخدمت ال **\b** وبعدها كلمها **Spam** وبعدين كتبت **|** وبعدها كلمة **Spam** وبعدها **\b**
انا هنا بقوله أي اسم بيبدأ بكلمة **Spam** او بينتهي بكلمة **Spam** هو دا اللي انا عايزوا

طيب فيما تستخدم ال **Test Method** تعالى نشوف

```
console.log(re.test(names));
```

true

استخدمت ال **test** مع ال **pattern** المكتوب في متغير ال **re** فالنتيجة كانت **true** لأن ال **pattern** الموجود في ال **re** بينطبق على خمس أسماء موجودين في متغير ال **names** الرئيسي اللي هي كلمة **Spam** سواء في الأول او في الآخر

```
console.log(/(\bspam|spam\b)/ig.test("Osama"));
```

false

دلوقتي لما استخدمها مع اسم "**Osama**" فأنا هنا بقوله هل الاسم دا بينطبق عليه ال **Pattern** اللي انا كاتبه؟! النتيجة هتكون **False** لان اسم "**Osama**" مفهوش أصلاً كلمة **Spam**
لو نفس المثال السابق بدل "**Osama**" كتبت مثلاً "**spam1**" أو "**1spam**" هتكون النتيجة **true** لان الاسمين دول انطبق عليه ال **Pattern** اللي انا كاتبه!

♥ عبارات تحفيزية ♥

عدد الاخفاقات :



شخص ناجح



شخص عادي

♥ كل ما زادت محاولتك زادت إخفاقات زادت مهارتك زدت نجاحًا ♥

♥ اذكر الله ♥

الزموا ذكر الله ففيه العون على متاعب الحياة ♥

لا إله الا الله وحده لا شريك له .. له الملك و له الحمد وهو على كل شيء قدير

أَلَا بِذِكْرِ اللَّهِ تَطْمَئِنُّ الْقُلُوبُ

لا تنسونا من دعواتكم

Never Forget That (**Always Search**)

By: Amr Elsayed

Regular Expression – Quantifiers Part 1 Lesson #140

Regular Expression

Quantifiers

n+	=> One Or More
n*	=> zero or more
n?	=> zero or one

ال **Quantifiers** هي ال **characters** التي بتساعد على تحديد الكمية تعالي نشوف بالأمثلة

```
let mails = "o@nn.sa osama@gmail.com elzero@gmail.net osama@mail.ru"; // All Emails
```

انا عايز اعمل **Pattern** اطابق فيه كل ال **mails** دي

```
let mailsRe = /\w+@\w+.\w+/ig;  
console.log(mails.match(mailsRe));
```

```
► (4) ['o@nn.sa', 'osama@gmail.com', 'elzero@gmail.net', 'osama@mail.ru']
```

اول **Quantifier** معنا هو ال **+** باستخدمه عشان اعمل **pattern** هيكون فيه حرف او اكثر من حرف جنب بعض

يعني ال **\w** زي ما اتعلمنا مسبقا بتستخدم عشان اجيب كل ال **Characters** من **a-z** مع استخدام ال **+** فانا بقوله ال **character** دا ممكن يكون حرف واحد ما بين ال **a-z** او اكثر من حرف جنب بعض

بعد كدا استخدمت ال **@** عشان سواء هتيجي بقا بعد حرف واحد او اكثر من حرف

بعد ال **@** بيجي ال **email provider** الي هو سواء **Gmail** أو **Hotmail** او **Mail** فاستخدمت ال **\w+** عشان أقوله انه بعد ال **@** ممكن يكون ال **email provider** مكون من حرف او اكثر

بعد كدا كتبت ال **.** او النقطة الي بتيجي بعد ال **email provider**

بعد كدا استخدمت ال **\w+** ثاني عشان أقوله ان بعد ال **.** او النقطة ممكن يكون حرف او اكثر زي **ru** و **com** وهكذا

تاني **Quantifier** معناها هو ال ***** ودا بستخدمه في التعبير عن احتمال يكون فيه **character** واحد او **number** واحد بين مجموعة ارقام او مفيش نهائي

```
let nums = "0110 10 150 05120 0560 350 00"; // 0 Numbers Or No 0
```

```
let numsRe = /\d*0/ig;  
console.log(nums.match(numsRe));
```

```
► (4) ['0110', '05120', '0560', '00']
```

يعني في ال **Pattern** دا انا بقوله هاتي مجموعة الأرقام بشرط الي بتبدا بـ **0**

وبعدين استخدمت **\d** الي بقوله فيها بعد ال **0** دا في **digits** او ارقام تانية ولكن مع استخدام ال ***** فانا بقوله بعد اول **0** ممكن يكون فيه رقم واحد او ارقام او مفيش ولا رقم خالص

بعد كدا قولتله **0** تاني

يعني باختصار بقوله انا عايز مجموعة الأرقام الي بتبدأ بـ **0** وتنتهي كمان بـ **0** وبين الصفرين دول ممكن يكون فيه رقم او ارقام او مفيش أي رقم نهائي بين الصفرين

تالت Quantifier معنا هو ال ؟ ودا بيستخدم في حالة لو عايز character واحد يا موجود مرة واحدة يا مش موجود نهائي

```
let urls = "https://google.com http://www.website.net web.com"; // http + https
```

```
let urlsRe = /(https?:\/\/)?(www.)?\w+.\w+/ig;  
console.log(urls.match(urlsRe));
```

```
► (3) ['https://google.com', 'http://www.website.net', 'web.com']
```

بعد حرف ال s في كلمة **https** كتبت ال ؟ عشان أقوله ان ال s دي ممكن تكون موجودة مرة واحدة زي **https** او مش موجودة ولا مرة زي **http**

بعد كدا حطيت ال **//**: ولكن قبل ال **//** حطيت علامة ال **escape** اللي دي \ اللي كنت بستخدمها عشان اعرف لغة البرمجة ان ال **character** دا تبقي انا مش تبعك انتي فيتم تجاهله من اللغة

بعدها حطيت علامة ال ؟ بعد ال **(https?:\/\/)** عشان أقوله ممكن ال **pattern** دا يكون موجود في الموقع مرة او ممكن مش موجود ولا مرة زي اخر موقع كدا في ال **urls** الي فوق (وبص على اخر ملحوظة هتفهم اكر)

بعدها برضو كتبت ال **(www.)** وبعدها علامة ال ؟ عشان أقوله هي ممكن تكون موجودة مرة او مش موجودة ولا مرة زي اخر موقع كدا في ال **urls** الي فوق برضو لاحظ ان ال **(www.)** هي جروب كدا كله علي بعض فيكون موجود كله مرة واحدة او مش موجود ولا مرة عشان كدا بعد الجروب دا حطيت ال ؟

بعدها كتبت **\w+** عشان أقوله بعد ال **www.** لو موجودة او مش موجودة خالص هيجي كلمة او اكر زي **google** و **website** دا في حالة وجود ال **www.** اما في حالة عدم وجود ال **www.** زي اخر موقع هلاقي باستخدام ال **\w+** انه كتب كلمة **web** الموجودة في اخر موقع

بعد كدا كتبت ال . او النقطة الي بتيجي قبل ال **Top level Domain** زي **.com** و **.net**. وهكذا

بعد ال . استخدمت ال **\w+** عشان أقوله ان بعد ال . هيجي حرف او اكر زي **com** و **net** و **ur** و **org** حرف او اتنين او اكر

ملحوظة: أي حاجة بين قوسين () بعد القوسين دول علامة ؟ يعني انا بقوله الجزء دا كدا كله علي بعضه يا موجود مرة واحدة يا مش موجود نهائي

Regular Expression – Quantifiers Part 2 Lesson #141

Regular Expression

Quantifiers

`n{x}` => Number of

`n{x,y}` => Range

`n{x,}` => At Least x

```
let serials = "S100S S3000S S50000S S950000S";
```

هنكمل كلام عن ال **Quantifiers** والمرادي هنتعرف علي حاجات جديدة

في حالة لو عايزك تكتب **Pattern** ي **match** أي حاجة بتبدأ بحرف ال **S** وبتنتهي بحرف ال **S** وبين ال **S** وال **S** يكون فيه عدد معين من ال **digits** وليكن **3 أرقام** فقط... فهتكون دي الطريقة

```
console.log(serials.match(/s\d{3}s/ig));
```

▶ `['S100S']`

طيب في حالة لو عايزك تكتب **Pattern** ي **match** أي حاجة بتبدأ بحرف ال **S** وبتنتهي بحرف ال **S** وبين ال **S** وال **S** يكون فيه عدد معين من ال **digits** ولكن في نطاق من **4** ل **5** أرقام فقط... فهتكون دي الطريقة

```
console.log(serials.match(/s\d{4,5}s/ig));
```

`['S3000S', 'S50000S']`

طيب في حالة لو عايزك تكتب **Pattern** ي **match** أي حاجة بتبدأ بحرف ال **S** وبتنتهي بحرف ال **S** وبين ال **S** وال **S** يكون فيه عدد معين من ال **digits** ولكن في نطاق من **4** ل **مالا نهاية** من الأرقام... فهتكون دي الطريقة

```
console.log(serials.match(/s\d{4,}s/ig));
```

`['S3000S', 'S50000S', 'S950000S']`

Regular Expression – Quantifiers Part 3 Lesson #142

Regular Expression

Quantifiers

\$ => End With Something

^ => Start With Something

?= => Followed By Something

?! => Not Followed By Something

```
let myString = "We Love Programming";  
let names = "1OsamaZ 2AhmedZ 3Mohammed 4MoustafaZ 5GamalZ";
```

أول **Quantifier** عندي وهو الـ **\$** بستخدمه عشان اعمل **Test** هل الـ **string** بتاعي بيتنهي بحروف معينة؟!؟

```
console.log(/ing$/ig.test(myString));
```

true

استخدمت الـ **\$** وسألته ع المتغير الـ **myString** هل بيتنهي بالـ **Pattern** اللي انا كتبتة دا؟!؟
وفعلا اخر كلمة فيه هي **Programming** بتننتهي بنفس الـ **Pattern** اللي كتبتة

```
console.log(/lz$/gi.test(names));
```

true

وكذلك لو طبقت دا علي تاني **string** عندي بتاع الـ **names** فعلا اخر حاجه بيه بتننتهي بالـ **LZ**

ثاني **Quantifier** عندي وهو ال **^** بستخدمه عشان اعمل **Test** هل ال **string** بتاعي بيبدأ بحروف معينة؟!

```
console.log(/^we/ig.test(myString));
```

true

```
console.log(/^d/gi.test(names));
```

true

في اخر مثال دا انا بقوله في المتغير **names** دا هل بيبدأ بأي **digits** من خلال استخدام ال **d** **** والاجابة **true** لان اول اسم بيه بيبدأ ب **10**

ثالث **Quantifier** عندي وهو ال **=?** بستخدمه عشان اعمل **matching** لأي حاجة متبوعة بال **character** اللي انا هحطه دا

```
console.log(names.match(/\d\w{5}(?=Z)/ig));
```

['10sama', '2Ahmed', '5GamaL']

انا هنا بقوله انا عايز العنصر اللي بيبدأ ب **digits** **\d** بعدها كلمة مكونه من **5** حروف **\w{5}** متبوعة بحرف ال **Z** (**?=Z**)

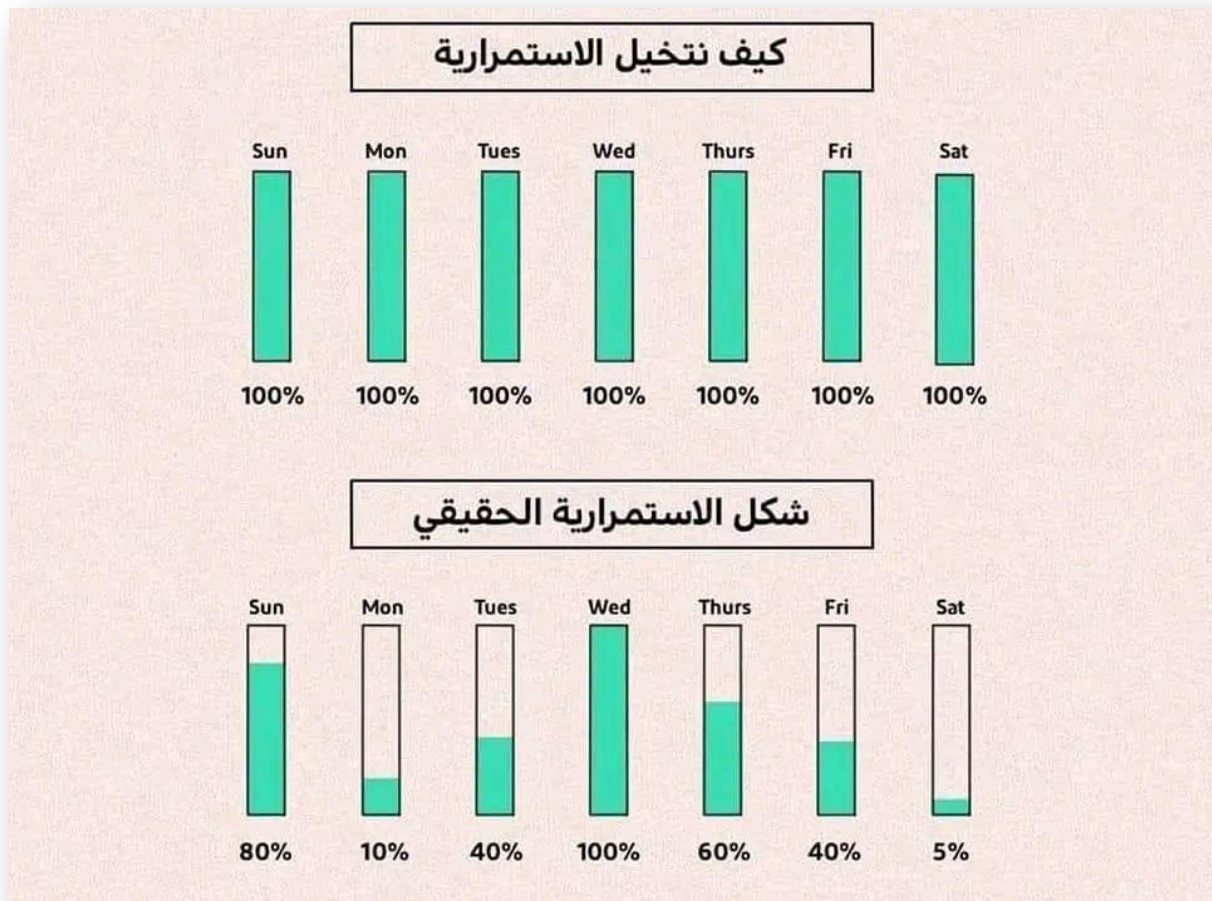
رابع **Quantifier** عندي وهو ال **?!?** بستخدمه عشان اعمل **matching** لأي حاجة ليست متبوعة بال **character** اللي انا هحطه دا

```
console.log(names.match(/\d\w{8}(?!Z)/ig));
```

► **['3Mohammed']**

انا هنا بقوله انا عايز العنصر اللي بيبدأ ب **digits** **\d** بعدها كلمة مكونه من **8** حروف **\w{8}** وليست متبوعة بحرف ال **Z** (**?!Z**)

♥ راقب لي ♥



لا تقلق انت كويس جدا يا صديقي وهو دا مفهوم الاستمرارية الصحيح

♥ اذكر الله

لا إله الا الله وحده لا شريك له .. له الملك وله الحمد وهو على كل شيء قدير

استغفروا الله العظيم الذي لا إله الا هو الحي القيوم واتوب اليه

اللهم صلّ وسلم وزد وبارك على نبينا محمد ﷺ

لا حول ولا قوة الا بالله العلي العظيم

لا إله الا انت سبحانك اني كنت من الظالمين

لا تنسونا من دعواتكم

Never Forget That (**Always Search**)

By: Amr Elsayed

Regular Expression – Replace With Pattern Lesson #143

Regular Expression

- replace
- replaceAll

```
let txt = "We Love Programming And @ Because @ Is Amazing";
```

```
console.log(txt.replace("@", "JavaScript"));
```

We Love Programming And JavaScript Because @ Is Amazing

استخدمت ال **replace method** عشان أبديل أول @ في الجملة الموجودة في المتغير **txt** بكلمة **JavaScript**

```
console.log(txt.replaceAll("@", "JavaScript"));
```

We Love Programming And JavaScript Because JavaScript Is Amazing

استخدمت ال **replaceAll** عشان أبديل كل @ في الجملة الموجودة في المتغير **txt** بكلمة **JavaScript**

```
let re = /@/ig;  
console.log(txt.replaceAll(re, "JavaScript"));  
console.log(txt.replaceAll(/@/ig, "JavaScript"));
```

We Love Programming And JavaScript Because JavaScript Is Amazing

يمكن كمان استخدم ال **regular expression** في اني احط **pattern** معين واستخدم عليه ال **replace** ممكن اعمل ال **pattern** دا في متغير زي **re** وممكن اعمل ال **pattern** دا في ال **console** مباشرة

Regular Expression – Form Validation Lesson #144

Regular Expression - Input Form Validation Practice

```
<form id="register" action="" method="get">
  <input type="text" id="phone" name="the-phone" maxlength="15" />
  <input type="submit" value="Register" />
</form>
```

```
document.getElementById("register").onsubmit = function () {
  let phoneInput = document.getElementById("phone").value;
  let phoneRe = /\(\d{4}\)\s\d{3}-\d{4}/; // (1234) 567-8910
  let validationResult = phoneRe.test(phoneInput);
  if (validationResult === false) {
    return false;
  }
  return true;
};
```

عملت هنا event ال **onsubmit** ع ال **form** بتاعي

عملت متغير اسمه **phoneInput** وظيفته تسجيل ال **value** اللي ال **user** هيكتبها في ال **input**

بعدا كدا عملت ال **pattern** بتاعي اللي هو عبارة عن **4 digits** بين قوسين وعملت طبعا **escape** للاقواس

بعده **\s** تعمل **space** بعدين **digits3** بعدين - بعدين **digits4**

بعد كدا عملت متغير اسمه **validationResult** بعمل فيه **test** لل **Value** اللي ال **user** هيكتبها مع ال **pattern** اللي انا حطيته

وبقول في شرط لو ال **validationResult** دا كان **false** يعني الشخص كتب **values** غير متطابقة مع ال **pattern** اللي انا حطيته يبقي مفيش أي بيانات هتتبع

ولو كانت **true** يبقي الشخص كتب **values** بنفس ال **pattern** اللي انا حطيته يبقي البيانات تتبعت عادي

```
let register = document.getElementById("register");
let phoneInput = document.getElementById("phone");

register.addEventListener("submit", (e) => {
  phoneRe = /\(\d{4}\)\s\d{3}-\d{4}/; // (1234) 567-8910;
  if (phoneRe.test(phoneInput.value) === false) {
    e.preventDefault();
  } else {
    return true;
  }
});
```

انا هنا حببت اعملها انا بشكل مختلف بنفسي وطلعت نفس النتيجة بالظبط

اذكر الله ❤️

الزموا ذكر الله ففيه العون على متاعب الحياة ❤️

اللهم صلّ وسلم وزد وبارك على نبينا محمد ﷺ

سبحان الله وبحمده عدد خلقه ورضا نفسه وزنة عرشه ومداد كلماته

لا إله الا انت سبحانك اني كنت من الظالمين

اللهم لك الحمد حمدًا طيبًا كثيرًا مباركًا فيه كما ينبغي لجلال وجهك ولعظيم سلطانك يا رب العالمين

لا حول ولا قوة الا بالله العلي العظيم

أَلَا بِذِكْرِ اللَّهِ تَطْمَئِنُّ الْقُلُوبُ

لا تنسونا من دعواتكم

Never Forget That (**Always Search**)

By: Amr Elsayed

المواقع المذكورة في الدرس

regexr.com

regextester.com

regex101.com

[لينك الدرس نفسه](#)

بالنسبة للـ Challenge فالتكليف 6 هو الـ Challenge

اذكر الله ❤️

الزموا ذكر الله ففيه العون على متاعب الحياة ❤️

اللهم إني أستغفرك واتوب إليك، لا إله إلا انت سبحانك إني كنت من الظالمين.

حسبي الله لا إله إلا هو عليه توكلت وهو رب العرش العظيم.

أَلَا بِذِكْرِ اللَّهِ تَطْمَئِنُّ الْقُلُوبُ

لا تنسونا من دعواتكم

Never Forget That (**Always Search**)

Assignments From [134] = > [146]

لينك التكميلات: [\[146\] <= \[134\]](#)

في بعض التكميلات هتلاقيني عامل ليها اكر من حلل بس مغير في شكل ال Variables

التكليف: 01

```
// Assignment One
let ip = "2001:db8:3333:4444:5555:6666:7777:8888";

let ipRe = /\d{4}:\w+:\d{4}:\d{4}:\d{4}:\d{4}:\d{4}:\d{4}/gi;
console.log(ip.match(ipRe));

// Another Solution
let ipRee = /(\w+|\D+)+/gi;
console.log(ip.match(ipRee));
```

التكليف: 02

```
// Assignment Two

let specialNames = "Os100 Os0 Os1000 Osa1000 Os1000 Os100m";

let specialNamesRe = /Os(\d+|)O/gi;
let specialNamesRee = /Os\d*O/gi;

console.log(specialNames.match(specialNamesRe));
console.log(specialNames.match(specialNamesRee));
```

التكليف: 03

```
// Assignment Three
let phone = "+(995)-123 (4567)";

let phoneRe = /\D(\d{3})\D\d{3}s(\d{4})/gi;
let phoneRee = /\D*\d+\D*\d+s*\D*\d+\D*/g;
let phoneReee = /\+(\d{3})-\d{3}s(\d{4})/gi;

console.log(phone.match(phoneRe));
console.log(phone.match(phoneRee));
console.log(phone.match(phoneReee));
console.log(phoneRe.test(phone), phoneRee.test(phone), phoneReee.test(phone));
```

التكليف: 04

محتاج كلام كثير مش لازم مش لازم 😊

```
// Assignment Five
let date1 = "25/10/1982";
let date2 = "25 - 10 - 1982";
let date3 = "25 10 1982";
let date4 = "25 10 82";

// Write Pattern Here
// let dateRe = /\d{2}\s*-\s*\d{2}\s*-\s*\d{2,}/g;

// Another Solution
// let dateRe = /\d+(\.|\s-\s|\s)\d+(\.|\s-\s|\s)\d+/gi;

//Another Solution
let dateRe = /\d{2}(\s|\.)(-)?(\s)?\d+(\s|\.)(-)?(\s)?\d+/gi;

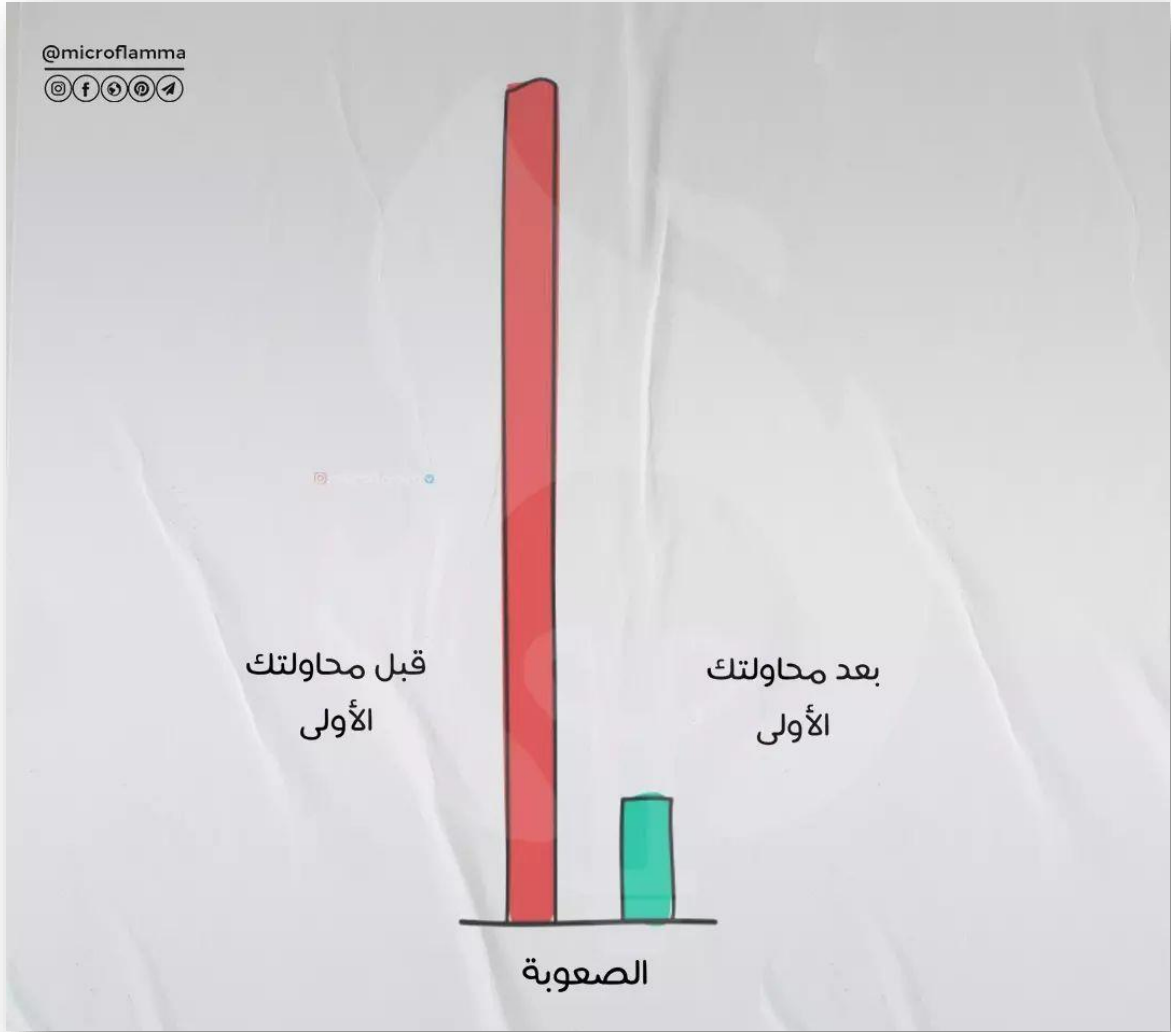
console.log(date1.match(dateRe)); // "25/10/1982"
console.log(date2.match(dateRe)); // "25 - 10 - 1982"
console.log(date3.match(dateRe)); // "25 10 1982"
console.log(date4.match(dateRe)); // "25 10 82"
```

```
// Assignment Six
let url1 = "elzero.org";
let url2 = "http://elzero.org";
let url3 = "https://elzero.org";
let url4 = "https://www.elzero.org";
let url5 = "https://www.elzero.org:8080/articles.php?id=100&cat=topics";

let urlRe = /((http?)s?:\/\/)?(www.)?\w+.\w+(:\w+\/.\w+.\w+\D+\w+\D+)?/g;

// Another Solution
// let urlRe =
// /((http?)s?:\/\/)?(\w+.)?\w+.\w+(:\d+\/\w+.\w+\D\w+\D\d+\D\w+\D\w+)?/ig;
console.log(url1.match(urlRe));
console.log(url2.match(urlRe));
console.log(url3.match(urlRe));
console.log(url4.match(urlRe));
console.log(url5.match(urlRe));
```

♥ راقِ لي ♥



الصعوبة ديمًا بتقل بعد أول محاولة وانت اكيد مقتنع ان بعد أول محاولة الشيء الصعب بتكتشف انه بقا سهل

♥ اذكر الله

الزموا ذكر الله ففيه العون على متاعب الحياة ♥

سبحان الله وبحمده سبحان الله العظيم
سبحان الله والحمد لله ولا إله إلا الله والله أكبر ولا حول ولا قوة إلا بالله العلي العظيم
أَلَا بِذِكْرِ اللَّهِ تَطْمَئِنُّ الْقُلُوبُ

لا تنسونا من دعواتكم

Never Forget That (**Always Search**)

By: Amr Elsayed

OOP Introduction Lesson #147

[ال Slides الي كان بيتشرح منها الجزء النظري](#)

- ال OOP اختصار ال Object Oriented Programming

- ال OOP هي مجرد Style مختلف شوية في كتابة ال JS وبيعتمد في الكتابة علي شكل كتابة ال object

- ال OOP مش حاجة جديدة بالعكس دا موجود من الستينات وفي بعض اللغات بتدعم ال OOP ولغات تانية لا ولغات تانية بتدعم حاجات من ال OOP وحاجات تانية لاء

- ال Object نفسه زي ما اتعلمنا عبارة عن حقبة كدا او Package بيكون جواها Properties و ال Methods وجود الاتنين دول مع بعض بيعمل اتحاد عشان يعمل App بـ Style معين

علي سبيل المثال <--- مثال العربية .. العربية هنا عبارة عن Object

- أي معلومات عن العربية زي لونها نوعها اسمها الموديل بتاعها كل دي تعتبر ال Properties

- اما سلوك العربية نفسه زي انها تتحرك توقف تمشي تغير سرعات كل دي ال Functions او Methods

ليه استخدم ال OOP وهل ليه مساوي وعيوب ؟!

- ايون ال OOP ليه بعض العيوب ولكن انت هتفهم شرح ال OOP وبعدين وانت شغال هل هو بطريقته بكل حاجة تخصه مناسب للتطبيق او الفكرة الي بتعملها ولا لاء ؟!

- لكن مميزات انك تستخدم ال OOP هو انك من خلاله هتقدر تكتب Large and Complex Software يعني أي تطبيق كبير ومعقد جدا ال OOP هيساعدك تفكك التعقيد دا ببساطة وتشتغل

- هتقدر من خلال ال OOP انك تعمل Encapsulation او إخفاء لاجزاء من ال code

- هتقدر تعمل Reusable objects تقدر تستخدمها في اكر من مكان + ال inheritance او التوريث

لا تنسونا من دعواتكم

Never Forget That (Always Search)

Constructor Function Introduction Lesson #148

عن طريق ال **Constructor Function** هنقدر اننا ننشئ **Object**

أي الحكمة من استخدام ال **Constructor Function** في انشاء ال **object**؟!

تخيل انا لو معايا **compound** سكني مليون عمارات وشقق فطبيعي جدا يكون عندي **مخطط** او حاجة بيطلق عليها **blueprint** من خلالها ببدأ أنشئ المساكن والعمائر دي بشكل منظم وبحسب كل عمارة هتأخذ مساحة قد أي وهتكون مكونة من كام طابق وكام شقه في الطابق كدا دا من خلال **مخطط** عندي

الحكمة من وجود **المخطط** دا هو اني يكون عندي **نظام ثابت** ماشي عليه بشكل منظم ووظيفته اني لما ابص عليه من فوق زي نظام ال **Bird view** كدا اللي هو نظرة عامة من الاعلي ابقى قادر افهم هو بيعبر عن أي!

عالم ال **OOP** هو عبارة عن حاجة بتخليني اعمل **المخطط** دا بقا وكمان اخذ أجزاء من **المخطط** دا اطبقها في أماكن تانية

```
const userOne = {  
  id: 100,  
  username: "Elzero",  
  salary: 5000,  
};
```

```
const userTwo = {  
  id: 101,  
  username: "Hassan",  
  salary: 6000,  
};
```

```
const userThree = {  
  id: 102,  
  username: "Sayed",  
  salary: 7000,  
};
```

دا شكل ال **Objects** اللي عندي قبل ما اعمل **مخطط**
تخيل لو حابب ازود ال **salary** بتاع كل موظف عندي
فعشان دا يحصل هتضطر تمشي علي كل **object**
موجود عندك وتعديل ال **salary** فيه

فتخيل لو عندك 1000 موظف هل منطقي؟!
انك تمشي عليهم بإيدك كلهم واحد واحد وتعديل عليهم؟

تعالى نشوف لو عندنا ال **Constructor Function** هنقدر ببساطة نعدل ازاي البيانات؟!؟

```
function User(id, username, salary) {  
  this.i = id;  
  this.u = username;  
  this.s = salary;  
}
```

بعمل **function** عادية جدا وبمرر فيها **Parameters** الي هرجع اعوض عنها بـ **arguments** فيما بعد اسم ال **function** يفضل يكون اول حرف فيه **Capital** زي **User** كدا كل الي حصل دا عملية اسمها **instantiate** اما ال **this** دي بتعود علي ال **object** نفسه الي هيتم انشائه جديد داخل ال **constructor** دا

طيب معني الكلام دا أي؟!؟
لما احي أنشئ **object** جديد ال **id** الي الشخص هيكتبه هيكون هو ال **argument** الي هيتمرر مكان ال **parameter id** في ال **function** الي عملتها فوق وكذلك باقي البيانات

```
User(id: any, username: any, salary: any): User  
let userOne = new User();
```

لما احي اعمل **user** جديد من خلال ال **keyword** الي اسمها **new** دي وجنبها اسم ال **constructor** بتاعي الي هو أسم ال **function** <--- **User**
هكتب أي بقا بداخل ال **user** الجديد دا؟!؟ ... هكتب ال **id** و ال **username** و ال **salary**

```
let userOne = new User(100, "elzero", 5000);  
let userTwo = new User(102, "Hassan", 6000);  
let userThree = new User(103, "Sayed", 7000);
```

اهو عملت **userOne** و **userTwo** و **userThree**
وعوضت عن ال **id** و ال **username** و ال **salary** بالقيم الخاصة بكل **user** جديد انا عملته
هما فوق كانو **Parameters** انا وانا بعمل **user** جديد عوضت عنهم بال **arguments** بقا

طيب عايز اعمل Access علي أي user عندي جديد عملته هعمل أي؟! ... تعالى نشوف

```
console.log(userOne.i);  
console.log(userOne.u);  
console.log(userOne.s);  
  
console.log(userTwo.i);  
console.log(userTwo.u);  
console.log(userTwo.s);  
  
console.log(userThree.i);  
console.log(userThree.u);  
console.log(userThree.s);
```

100	main.js:29
elzero	main.js:30
5000	main.js:31
102	main.js:33
Hassan	main.js:34
6000	main.js:35
103	main.js:37
Sayed	main.js:38
7000	main.js:39

قولته ال **userOne.i** وال **i** دي بتعبر عن ال **id** اللي انا أصلا فوق قولته **this.i** ال **this** دي بتعود علي **user** بتاعي الجديد اللي انا عملته من خلال ال **new** سواء **one** او **two** او **three** وال **i** بتعود علي ال **id** وال **u** بتعود علي ال **username** وال **s** بتعود علي ال **salary**

فانا قولته مكان ال **this** هكتب **userOne** ومكان ال **i** هتكون ال **id** وهكذا

روحت انا بقا عايز ازيد ال Salary بـ 1000 لكل موظف او كل user عندي هعمل أي؟!

```
function User(id, username, salary) {  
  this.i = id;  
  this.u = username;  
  this.s = salary + 1000;  
}
```

100	<u>main.js:29</u>
elzero	<u>main.js:30</u>
6000	<u>main.js:31</u>
102	<u>main.js:33</u>
Hassan	<u>main.js:34</u>
7000	<u>main.js:35</u>
103	<u>main.js:37</u>
Sayed	<u>main.js:38</u>
8000	<u>main.js:39</u>

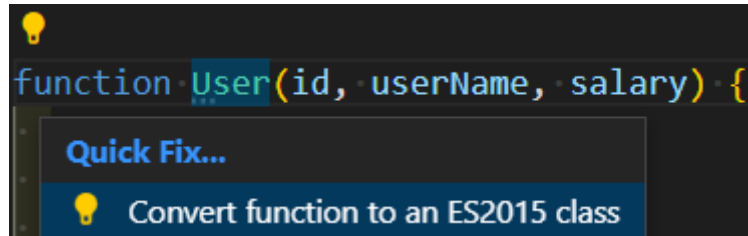
كل اللي فات انا بعمل Constructor بال Syntax العادي في الدروس الجاية هيكون في syntax مختلف وافضل وتابع الـ ES6

لا تنسونا من دعواتكم

Never Forget That (**Always Search**)

By: Amr Elsayed

Constructor Function New Syntax Lesson #149



هاجي علي ال **Constructor Function** بتاعي القديم واضغط <---- **ctrl + .** واعمل **convert** ل **syntax** ال **ES6**

دا ال **ES6 Syntax** في كتابة ال **Constructor Function** بتاعي

```
class User {  
  constructor(id, userName, salary) {  
    this.i = id;  
    this.u = userName;  
    this.s = salary;  
  }  
}
```

دلوقتي لو حابب استعلم عن ال **instance** اللي تم منه ال **user** الجديد دا اعمل أي؟! او عايز اعمل المخطط اللي تم من خلاله انشاء ال **userOne** دا فهسال من اني اكتب اسم ال **new Object** بتاعي دا وأقوله هل انت **instanceof** من ال **User Constructor** دا؟!

```
console.log(userOne instanceof User);
```

true

ممکن اسأل برضو هل ال **Constructor** بتاع ال **userOne** بيساوي **User**؟!

```
console.log(userOne.constructor === User);
```

true

وبما انه في كل الأحوال ال **userOne** دا **instance** من ال **User Constructor** هتكون النتيجة **Ture**

♥ عبارات تحفيزية ♥



♥ متستعجلش النتائج فقط اصبر واستعن بالله ♥

♥ اذكر الله ♥

♥ الزموا ذكر الله ففيه العون على متاعب الحياة ♥

اللهم إني أستغفرك واتوب إليك، لا إله إلا أنت سبحانك إني كنت من الظالمين.

حسبي الله لا إله إلا هو عليه توكلت وهو رب العرش العظيم.

أَلَا بِذِكْرِ اللَّهِ تَطْمَئِنُّ الْقُلُوبُ

لا تنسونا من دعواتكم

Never Forget That (**Always Search**)

Deal With Properties And Methods Lesson #150

Constructor Function

- Deal With Properties And Methods

```
class User {
  constructor(id, userName, salary) {
    // Properties
    this.i = id;
    this.u = userName || "Unknown";
    this.s = salary < 6000 ? salary + 500 : salary;
    this.msg = function () {
      return `Hello ${this.u}, Your Salary is ${this.s}`;
    };
  }

  // Methods
  writeMsg() {
    return `Hello ${this.u}, Your Salary is ${this.s}`;
  }
}
```

في ال **this.u** عملت **condition** في حالة لو ال **user** مكتبش اسمه فهتكتب مكان الاسم كلمة **Unknown** اما في ال **this.msg** فالرسالة الي هتتطبّع كتبت **this.u** بدلا من **username** عشان لو انا كتب **username** وهو بيطبّع الرسالة في حالة لو ال **user** مكتبش اسمه الرسالة الي هتتطبّع دي هتكون مكان الاسم فاضي تماما فالرسالة هتتكتب **hello** من غير أي اسم هتكون فراغ **empty string**

اما في حالة لو كتبت **salary** بس بدلا من **this.s** هتكتب ال **salary** الي ال **user** عوض عنه ك **argument** تحت يعني لما هو كتب **5000** فهي الي هتتطبّع ف الرسالة لكن انا أصلا عامل شرط انه لو اقل من **5000** فيزود **500** إضافي لذلك انا كتبت ال **this.s** عشان يقرأ الشرط الي حصل في ال **this.u** أصلا ويعوض في الرسالة عن ال **salary** بـ **5500**

فلو كتبت salary و userName بدلا من this.i و this.u

```
this.msg = function () {  
    return `Hello ${userName}, Your Salary is ${salary}`;  
};
```

هتكون دي النتيجة

في حالة اول user ال 5000 بتاعت ال salary اتكتب زي ما هي ولكن فين ال 500 الي زودتهم انا لو ال salary اقل من 5000 ؟! ... مش موجودة

```
Hello Elzero, Your Salary is 5000
```

```
Hello , Your Salary is 6000
```

في حالة ثاني user فين اسمه ؟! ولو مكتبش اسمه فين Unknown الي انا قولتله عوض عنها في حالة عدم وجود اسم لل user ؟!

لذلك كان لابد من كتابة ال this.s وال this.u

كل الي فاتوا دول عبارة عن Properties اما الي جاي هو شغل ال Methods

```
// Methods  
writeMsg() {  
    return `Hello ${this.u}, Your Salary is ${this.s}`;  
}
```

```
console.log(userOne.writeMsg());
```

```
Hello Elzero, Your Salary is 5500
```

Update Properties And Built In Constructors Lesson #151

Constructor Function

- Update Properties
- Built In Constructors

بما انه في آخر الدرس دا قالك انا عايزك تفهم قبل ما تتعمق مش تتعمق الأول بعدين تفهم فتعالى نشرح بطريقة نفهم بيها كل اللي أتذكر في الدرس دا

أول حاجه لو عايز اعمل **Update** لل **username** يبقى انا محتاج اعمل **Method** او **function** تنفذ المطلوب دا

```
class User {  
  constructor(id, username, salary) {  
    this.i = id;  
    this.u = username;  
    this.s = salary;  
  }  
  updateName(newName) {  
    this.u = newName;  
  }  
}
```

```
let userOne = new User(100, "Elzero", 5000);
```

عملت **Method** اسمها **updateName** ومررت فيها **Param** اسمه **newName** هيثم التعويض عنه بالاسم الجديد اللي انا هحطه بعدين قولتله ال **this.u = newname** لو تفتكر ال **this** دي بتعود علي ال **object** اللي انا هعمله انشاء بنفسى

```
console.log(userOne.u);  
userOne.updateName("Osama");  
console.log(userOne.u);
```

Elzero

Osama

زي ما انت شايف كان الاسم الأول **Elzero** بعدين استخدمت ال **method** اللي عملتها وعملت **Update** للاسم ب **Osama**


```
let strOne = "Elzero";
let strTwo = new String("Elzero");
```

لو قولتلك دلوقتي اني عندي **strOne** بيعمل **string** بيساوي كلمة ال **"Elzero"** وعندي **strTwo** برضو بيعمل **string** بساوي كلمة **"Elzero"** برضو

ياتري أي الفرق بين الاثنين مع ان الاثنين تم فيهم انشاء **string** أصلا يعني كل واحد في دول وظيفته انشاء **string** اسمه **"Elzero"**

تعالى الأول نشوف نوع بيانات كل واحد في الطريقتين اللي تم انشاء ال **string** بيهم

```
console.log(typeof strOne);
console.log(typeof strTwo);
```

string
object

اول واحد هو عبارة عن **String** اما الثاني **object** ودا حصل لاني استخدمت ال **string Constructor** فيها في الحالتين ولكن الحالة الأولي مش متماثلة او مش **instance** من ال **String Constructor** أصلا

تعالى بص بقا ع دي عشان هتفهم الفرق في اللي حصل فوق

```
console.log(strOne instanceof String);
console.log(strTwo instanceof String);
```

false
true

هنا انت بتقوله هل ال **strOne** دا متماثل من ال **string constructor**؟! هتكون النتيجة لا **false** طيب هل ال **strTwo** دا متماثل او مأخوذ من ال **string constructor**؟! هتكون النتيجة اه **true** دا حصل فعلا

طيب هو أي اللي حصل في الحالتين في اول حالة انت فعلا في حالة ال **strOne** بتستخدم ال **string constructor** فعلا ولكنه مش متماثل من ال **string constructor** لانه معمول بطريقة ثانية

اما ال **strTwo** متماثل فعلا من ال **string constructor** لذلك هتلاقي **strOne** نتيجته **false** اما ال **strTwo** نتيجته **true**

طيب تيجي نتأكد بقا ان الاتنين دول معمولين من ال **String Constructor** ولكن كل واحد معمول بطريقة غير الثانية؟!

```
console.log(strOne.constructor === String);  
console.log(strTwo.constructor === String);
```

true

true

انا بقوله هنا هل ال **strOne** معمول من ال **String Constructor**؟!
و بقوله هل ال **strTwo** معمول من ال **String Constructor** برضو؟!

هلاقي النتيجة في الحالتين **true** ولكن زي ما قولنا الطريقة الأولى معموله خلال ال **String Constructor** ولكنها مش **instance** منه او مش متماثله منه او مش من عيلته يعني اما الثانية معموله من خلال ال **String Constructor** فعلا و هي **instance** منه لذلك كان نوع كل واحد مختلف واحده **String** وواحد **Object**

لا تنسونا من دعواتكم

Never Forget That (**Always Search**)

By: Amr Elsayed

Class Static Properties And Methods Lesson #152

Class
- Static Properties And Methods

ال **static** هي خواص تخص ال **Class** نفسه وملهاش أي علاقة بال **objects** الي بنشائها من ال **Class** دا يعني لو عندي **Class** اسمه **User** أي خاصية بكتبها مسبوقة بكلمة **Static** فالخاصية دي اصبحت تخص ال **Class** دا نفسه واي **Object** بنشئه من ال **Class** دا ملوش أي علاقة بالخاصية دي لانها تخص ال **Class**

انا مثلا عندي ال **Class** الي اسمه **User** دا وجواه خاصية اسمها **count** قيمتها 0

```
class User {  
  count = 0;  
  constructor(id, username, salary) {  
    this.i = id;  
    this.u = username;  
    this.s = salary;  
  }  
}
```

جيت من ال **User Class** دا عملت ال **2 Objects** دول

```
let userOne = new User(100, "Elzero", 5000);  
let userTwo = new User(101, "Ahmed", 5000);
```

لما أجي اعمل **Access** على ال **count Property** دي من ال **object** نفسه هلاقي النتيجة دي:

```
console.log(userOne.count);
```

0

إذا ال **Object** الي هو **userOne** يقدر ي **Access** علي أي حاجة في ال **User Class**

طب لو انا مسكت ال **Class** نفسه الي اسمه **User** دا وكتبت التالي

```
console.log(User.count);
```

```
undefined
```

هلاحظ ان ال **User Class** دا مقدرش ي **Access** علي ال **count Property** الموجودة جواه

هنا يجي بقا موضوع ال **static** الي بنتكلم عنها

```
class User {  
  // Static Property  
  static count = 0;  
  constructor(id, username, salary) {  
    this.i = id;  
    this.u = username;  
    this.s = salary;  
  }  
}
```

هنا بقا ال **User Class** هو هيقدر ي **Access** علي ال **count** دي وال **object** بتاع ال **userOne** مش هيقدر يوصلها يبقي الي حصل بعد كلمة **static** قبل خاصية ال **count** جعل الوضع السابق معكوس فاهمني؟! اذا زي ما قولنا في أول سطر فوق ال **static** بتخص ال **Class** نفسه وال **objects** الي بيتم انشائها منه ملهاش علاقة بأي **Static Property**

```
20 console.log(userOne.count);  
21 console.log(User.count);
```

```
undefined    main.js:20
```

```
0           main.js:21
```

بص شوفت الي حصل الموضوع انعكس بعد استخدام ال **static**

وكذلك الوضع مع أي **Method** داخل ال **Class** نفسه لو سبقه كلمة **Static** يبقي هي تخص ال **Class** نفسه واي **object** بيتم انشائه من ال **Class** دا ميقدريش ي **Access** علي ال **Method** دي

طيب تعالي ناخذ مثال **Advanced** شوية ونطبق فيه فكرة ال **static** و ان ال **Property** و ال **Methods** تكون بتخص ال **Class** نفسه

```
class User {  
  // Static Property  
  static count = 0;  
  
  constructor(id, username, salary) {  
    this.i = id;  
    this.u = username;  
    this.s = salary;  
    User.count++;  
  }  
  
  // Static Methods  
  static countMembers() {  
    return `${this.count} Members Created`;  
  }  
}
```

```
console.log(User.countMembers());
```

طبعا انت خلاص فهمت ان ال **countMembers** دي بقت تخص ال **User** نفسه بقا لذلك ال **User** نفسه هو الي قادر انه ي **Access** عليها

طيب معني ال **code** أي ؟ أو انا عملت أي؟!

ببساطة انا عملت **static Property** اسمها **count** قيمتها **0**

وفي ال **method** بتاعت ال **countMembers** انا قولتله **this.count** ال **this** دي بتعود علي ال **Class** نفسه يعني انا ممكن اشيل ال **this** دي واحط **User.count** ولكن انا بستخدم ال **this** عشان لو انا غيرت اسم ال **Class** من **User** لأي اسم ثاني مبقاش مضطر انزل اغير اسمه مرة كمان تحت فاهم انا ليه استخدمت ال **this** المهم زي ما قولنا ال **this** دي بتعود علي ال **User Class** نفسه

داخل ال **Constructor** نفسه بقا انا قولتله ال **User.count++** عشان ال **count** دي تخص ال **User** فهو الوحيد القادر علي انه يقرأها ويتعامل معاها فاهم

معني الكلام دا انه بقوله ال **count** الي بـ **0** دا مع كل انشاء لـ **Object** جديد من ال **User Class** دا ال **Count** دي تزيد بمقدار عدد ال **Objects** الي تم انشاؤها فعليا وبما اني في المثال عندي **2 Objects** بس يبقي هما دول قيمة ال **count** الجديدة **2** بس او **2 Members Created** ومع كل زيادة تزيد قيمة ال **count**

2 Members Created

Class Inheritance Lesson #153

Class - Inheritance

التوريث من اهم أعمدة ال OOP

لو عندي ال **User Class** وجواه بعض ال **Properties** و ال **Method** وبستخدم ال **Class** دا في انشاء **users** كل **user** بيكون ليه **ID** و **userName** و **Password** مثلا

ونفترض اني عندي **class** ثاني اسمه **Admins** بستخدمه عشان أنشئ حساب لل **Admin** و نفس ال **Class** دا كل **admin** فيه بيكون ليه **id** برضو و **userName** و **password** وكمان بيكون ليه مميزات ثانيه إضافية

هنا يجي دور ال **inheritance** او التوريث فبدل ما تكون بت **repeat your self** أو بتكرر نفسك او بمعني اخر بتعيد انشاء العجلة من أول وجديد وكمان قتلت مبدأ ال **reusability** او إعادة الاستخدام

فانت ممكن تورث بعض الخصائص الموجودة في ال **User Class** دا لل **Admin Class** الجديد وبس كدا هي دي كل الفكرة

واللي جاي بعد كدا مجرد بس أسلوب لكتابة نقل التوريث دا من مكان لآخر فقط لا غير

فمثلا دا ال **User Class**

```
// Parent Class
class User {
  constructor(id, username) {
    this.i = id;
    this.u = username;
  }
  sayHello() {
    return `Hello ${this.u}`;
  }
}
```

و دا ال **Admin Class**

```
// Derived Class
class Admin extends User {
  constructor(id, username, permissions) {
    super(id, username);
    this.p = permissions;
  }
}
```

انا بقوله جنب ال **Admin Class** دا كلمة **extends** من ال **User Class** يعني انا بقوله انا عايز شوية حاجات من ال **User Class** دا استخدمها في ال **Admin Class**

وبعدين قولتله **Super** وجواها الحاجات اللي انا هستمدها من ال **User Class** دا زي ال **id** و ال **userName**

```
let userOne = new User(100, "Elzero");
let adminOne = new Admin(110, "Mahmoud", 1);

console.log(userOne.u);
console.log(userOne.sayHello());
console.log("####");
console.log(adminOne.i);
console.log(adminOne.u);
console.log(adminOne.p);
console.log(adminOne.sayHello());
```

Elzero	main.js:35
Hello Elzero	main.js:36
####	main.js:37
110	main.js:38
Mahmoud	main.js:39
1	main.js:40
Hello Mahmoud	main.js:41

وممكن تكرر العملية دي وتورث من **Class** ل **Class** ل **Class** وهكذا

لا تنسونا من دعواتكم

Never Forget That (**Always Search**)

♥ راقِ لي ♥



لما تحس بحماس لفكرة معينة نفذها دلوقي وحماسك عالي لان كل ما تتأخر حماسك بيقل

♥ اذكر الله

الزموا ذكر الله ففيه العون على متاعب الحياة ♥

سبحان الله و الحمد لله و لا اله الا الله و الله أكبر و لا حول ولا قوة الا بالله

اللهم صلّ وسلم وزد وبارك على نبينا محمد ﷺ

اللهم إني أستغفرك واتوب إليك، لا إله الا انت سبحانك إني كنت من الظالمين

أَلَا بِذِكْرِ اللَّهِ تَطْمَئِنُّ الْقُلُوبُ

لا تنسونا من دعواتكم

Never Forget That (**Always Search**)

Class Encapsulation Lesson #154

Encapsulation

- Class Fields Are Public By Default
- Guards The Data Against Illegal Access.
- Helps To Achieve The Target Without Revealing Its Complex Details.
- Will Reduce Human Errors.
- Make The App More Flexible And Manageable.
- Simplifies The App.

ال Encapsulation او تغليف وحماية بعض البيانات

لما يكون عند ال User Class وجواه بعض الخصائص وتيجي تنشئ Object من ال Class دا
فكل الخصائص الموجودة في ال User Class بتكون Public وتقدر ت Access عليها كلها بسهولة

في بعض الحالات هتكون محتاج ان يكون عند Property بحيث تكون privet ومش بسهولة يتم الوصول ليها
او حتي يكون ممنوع الوصول ليها وللقيمة الموجودة فيها

ال eSalary دا عشان اخليه encrypted او مشفر هحط قبل الاختصار بتاع هاش # وللازم اعمل Declaration

```
class User {  
    // Private Property  
    #e;  
    constructor(id, username, eSalary) {  
        this.i = id;  
        this.u = username;  
        this.#e = eSalary;  
    }  
}
```

```
let userOne = new User(100, "Elzero", 5000);
```

```
console.log(userOne.u);  
console.log(userOne.e * 0.3);
```

Elzero

NaN

لو بعد تشفير ال eSalary دا حبيت Access عليه مش هتلاقي أصلا ليه أي قيمة !
ولاحظ اني كاتب 5000 رقم مش string ولا حاجة ومع ذلك جابلي NaN ومجبلش أي ارقام

```
console.log(userOne.e);
```

```
undefined
```

حتي لو لغيت العملية الحسابية خالص وحببت **Access** ع ال **eSalary** دا مش هلاقيه أصلا بالرغم من وجوده وكل دا حصل لاني عامله تشفير

طيب في حالة لو ال **eSalary** اتكتب بالشكل دا "**5000 Gneh**" هنا انا محتاج احلل البيانات دي واطلع منها الرقم بعدين اعمل العملية الحسابية وكل دا في ايطار تشفير البيانات برضو

```
let userOne = new User(100, "Elzero", "5000 Gneh");
```

طيب مين الي يقدر يتعامل مع ال **privet Property** دي؟! قالك ياسيدي الوحيد الي يقدر يتعامل معاها هما ال **family** الي معاها في ال **Class** يقصد باقي ال **properties** التانية و ال **Methods**

معني الكلام دا ان كل العناصر الي بداخل ال **User Class** هي الي تقدر تشوف ال **privet property** دا و تتعامل معاها طيب نطبق الفكرة دي ازاى؟! تعالي نشوف

```
class User {  
  // Private Property  
  #e;  
  constructor(id, username, eSalary) {  
    this.i = id;  
    this.u = username;  
    this.#e = eSalary;  
  }  
  getSalary() {  
    return parseInt(this.#e);  
  }  
}
```

عملت ال **getSalary()** الي هي **method** وتعتبر من ضمن العائلة بتاعت ال **class** الي هتقدر تشوف ال **privet property** عادي جدا وطلبت منها انها تحلل البيانات وتجيلي الأرقام لو مكتبة جنب **string**

```
console.log(userOne.getSalary() * 0.3);
```

```
1500
```

Prototype Introduction Lesson #155

Prototype

- Introduction
- Prototypes are the mechanism by which JavaScript objects inherit features from one another.

ال **prototype** معناها نموذج او مخطط وهي ببساطة شديدة جدا الألية اللي بيتم فيها استنساخ أي خواص موجودة داخل أي **Class** او **Built-in Constructors** و استخدمها داخل أي **Class** هيرث من ال **Class** الأب نقدر نقول بشكل مختصر اكثر انها أليه عن طريقها ال **Objects** بتورث الخواص من بعض!

```
class User {  
  constructor(id, username) {  
    this.i = id;  
    this.u = username;  
  }  
  sayHello() {  
    return `Hello ${this.u}`;  
  }  
}
```

يعني في حالة وجود ال **User Class** دا عند انشائه بيكون فيه **Prototype** مخصص ليه جواه الخواص والمميزات بتاعته

لما اجي بقا استخدم ال **new** عشان اعمل **Object** جديد من ال **User Class** دا أقدر اني اورث كل المميزات اللي هي موجودة داخل ال **Class** او المخطط بتاع ال **Class** دا

```
console.log(User.prototype);
```

```
▼ {constructor: f, sayHello: f} ⓘ  
  ► constructor: class User  
  ► sayHello: f sayHello()  
  ► [[Prototype]]: Object
```

زي ما قولنا أي **Object** هيتم انشاءه من ال **Class** دا هiestفيد من الخواص او المخطط الموجود بداخل ال **Class** دا !

لو عملت متغير جديد اسمه **strOne** وانشأت ليه القيمة "**Elzero**" العملية دا تمت ازاي ؟!

```
let strOne = "Elzero";
```

تم انشاء ال **string** دا من خلال ال **String Constructor** ال **constructor** دا يا زميلي عبارة عن **built-in Function** موجود في اللغة وظيفته انشاء ال **strings**

وال **built-in Function** الي هو ال **String Constructor** دا ليه هو كمان **Prototype** او مخطط المخطط دا بيكون جواه مميزات وخصائص مين بقا الي بيستفيد بأي مميزات داخل المخطط دا ؟!

اكيد ال **string** الي تم انشاءه من خلال ال **built-in function** دي

```
console.log(String.prototype);
```

```
main.js:25  
► String {'', constructor: f, anchor: f, at: f, big: f, ...}
```

إذاً كل الحاجات الموجودة داخل ال **String Constructor** دا ال **strOne** الي انا عملته يقدر يتمتع بيها ويستخدمها

لا تنسونا من دعواتكم

Never Forget That (**Always Search**)

By: Amr Elsayed

Add To Prototype Chain Lesson #156

Prototype

- Add To Prototype Chain
- Extend Built In Constructors Features

زي ما قولنا سابقا أي **Function** او **Class** بيتم انشائه او حتي أي **Built-in Function** بيكون ليها **Prototype** أو **مخطط** كدا بيكون جواه مميزات وبيتم توريث المميزات دي

يعني لما يكون عندي ال **User Class** دا موجود وليه خصائص أي **Object** بقا هيتم انشائه من ال **User Class** دا يتمتع بكل خواص ال **User Class** دا وكل الحاجات الموجودة داخل **مخطط** ال **User Class** دا

انا دلوقتي عندي ال **User Class** دا وأنشأت منه ال **userOne** اللي هو ال **object** اللي يورث الحاجات الموجودة داخل **مخطط** ال **User Class** دا

```
class User {
  constructor(id, username) {
    this.i = id;
    this.u = username;
  }
  sayHello() {
    return `Hello ${this.u}`;
  }
}

let userOne = new User(100, "Elzero");
```

```
console.log(User.prototype);
console.log(userOne);
```

```
▶ {constructor: f, sayHello: f}
▶ User {i: 100, u: 'Elzero'}
```

```
▼ User {i: 100, u: 'Elzero'} ⓘ
  i: 100
  u: "Elzero"
  ▼ [[Prototype]]: Object
    ▶ constructor: class User
    ▶ sayHello: f sayHello()
    ▶ [[Prototype]]: Object
```

بص بقا ع ال **userOne** دا هتلاقي جواه البيانات بتاعتنا اللي هي ال **username** و ال **id** جواه بقا برضو هتلاقي **Prototype** اللي هو ال **object** هو مش ال **userOne** دا أصلا **Object**؟! يبقى هستخدم خواص او مخطط ال **Object Constructor** ... كمان ال **Prototype** الموجود بتاع ال **User class**

يعني يا باشا ببساطة كدا ال **userOne** دا عشان هو في الأصل معمول من ال **User Class** فهيورث الخصائص اللي فيه ... وكمان عشان هو أصلا كدا كدا نوعه **Object** فهيورث الخصائص بتاعت ال **Object Constructor** كمان .. فاهمني؟!

ال **Prototype** لأي عنصر دا انا ممكن اضيف فيه أي حاجة انا عايزه يعني مثلا ال **User Class** دا انا هضيف جوا ال **Prototype** بتاعه خاصية معينة من خلالي فأني **object** بقا هيورث أصلا خصائص ال **User Class** دي ال **Default** أصلا بالإضافة للحاجة اللي انا هعملها بإيدي!

```
User.prototype.sayWelcome = function () {  
    return `Welcome ${this.u}`;  
};
```

User Class

```
▼ User {i: 100, u: 'Elzero'} ⓘ  
  i: 100  
  u: "Elzero"  
  ▼ [[Prototype]]: Object  
    ► sayWelcome: f ()  
    ► constructor: class User  
    ► sayHello: f sayHello()  
    ► [[Prototype]]: Object
```

زي ما انت شايف كدا ال **User Class** دا انا أصلا ضيفت **function** جوا ال **Prototype** بتاعه اقدر استخدمها عادي ضيفتها بإيدي انا

كدا مخطط ال **User** انا ضيفت جواه **Function** وظيفتها تقول **Welcome** لل **userName**

ال userOne دا عبارة عن object مش كدا؟! ال object دا ليه prototype جواه شوية خصائص مش كدا؟! انا برضو ممكن اضيف خصائص تانيه بايدي غير ال default الموجودة دي أصلا واستخدمها عادي

```
Object.prototype.love = "Elzero Web School";
```

```
▼ User {i: 100, u: 'Elzero'} ⓘ  
  i: 100  
  u: "Elzero"  
  ▼ [[Prototype]]: Object  
    ► sayWelcome: f ()  
    ► constructor: class User  
    ► sayHello: f sayHello()  
  ▼ [[Prototype]]: Object  
    love: "Elzero Web School"  
    ► constructor: f Object()
```

```
> userOne.love  
◀ 'Elzero Web School'
```

يا عم طيب سيبك من كل القصص الي فاتت دي انا عندي ال **String Constructor** دا اتفقنا ان ليه **Prototype** او مخطط فأي **string** بيتم انشائه في العالم بيورث كل حاجة من مخطط ال **String Constructor** مش كدا؟! متفقين؟!

انا بقا بداخل ال **String Constructor** دا هضيف خاصية او **function** بايدي انا تتحط داخل المخطط بتاعه واي **string** هيتم انشائه في شغلي يقدر ي **Access** علي الخاصية او ال **function** الي عملتها بايدي دي!

```
String.prototype.addDotBeforeAndAfter = function (val) {  
    return `.${this}`;  
};
```

```
let myString = "Elzero";
```

```
> myString.addDotBeforeAndAfter();  
◀ '.Elzero.'
```

مسكت ال **String Constructor** دا ضيفت في ال **prototype** بتاعه **method** او **Function** يعني ومررت ليها **Parameter** هيتشال ويحل محله قيمة أي **String** هيتكتب وظيفة ال **function** دي بقا انها تمسك ال **val** او ال **string** الي هيتكتب دا تحط قبله نقطة وبعده نقطة

وبس كدا ال **myString** بتاعي الي قيمته "Elzero" انا هضيف عليه ال **method** الي انا علمتها بايدي دي وأخلي ال **string** الموجود الي هو "Elzero" يحل محل ال **parameter** الي فوق ويطبق عليه ال **Function**

لا تنسونا من دعواتكم

Never Forget That (**Always Search**)

By: Amr Elsayed

♥ راقِ لي ♥



Mahmoud Samir Fayed

8 minutes ago · 🌐



بعد مرور اول سنة وانا شغال كده مع نفسي في تطوير لغة الـ رينج Ring ومكنتش لسه حتي حددت اسمها ... كنت بعمل شوية تجارب وكان في Bugs موقفة الدنيا ... فقعدت كده افكر مع نفسي وقلت انا بقالي سنة كل اللي عملته في اللغة Hello World و دعم الـ Functions والـ Loops وكام دالة بسيطة وكل اللغة علي بعضها 5 الاف سطر بالـ C ... فقلت انا لو فضلت ماشي بالشكل ده انا محتاج ١٠٠ سنة عشان يبقي عندي لغة اعرف اشتغل بيها ... وخطر في بالي ارمي اللغة اللي مالهاش اسم دي وخطوة الي الخلف واشتغل بالـ Python او بالـ Ruby وشكرا علي كده.

لكن كان جوايا حاجه بتقولي ماينفعش اقف رغم اني لو عملت كده ولا كان حد هيعرف اي حاجه لكني فعلا كنت عايز لغة ابرمج بيها زي ما انا حابب واستخدمها في شغلي ... كملت كمان سنة ونص لحد اما نشرت اول اصدار ووقتها كان المشروع ١٠٠ الف سطر ... وبعد كده مع الوقت عدي الـ ٥٠٠ الف من كذا سنة ... يعني ١٠٠ ضعف الحجم اللي لما كنت عنده فكرت اقف وما اكملش ومع فارق طبعا ان كل يوم ب ابدأ يومي وانا بفكر هعمل ايه النهارده عشان اخلي الـ رينج Ring احسن وتنجح اكتر.

الشاهد من كلامي او الرسالة اللي عايز اوصلهاك ... في رحلتك هتواجه لحظات ضعف وهتفكر تقف تماما وترمي كل حاجه خصوصا اول اما تكتشف انك طالع جبل عالي ... اياك تقف اياك تقف عشان تعبت ... او عشان خايف ... او عشان لسه فاضل ١٠٠ ضعف اللي انت عملته ... كل المطلوب منك انك تستريح شوية وتكمل ... تكمل و تشتغل اكتر وتفكر في طرق تخليك توصل اسرع ... تكمل وما تقفش لحد اما توصل.

♥ اذكر الله

الزموا ذكر الله ففيه العون على متاعب الحياة ♥

لا اله الا الله وحده لا شريك له .. له الملك و له الحمد وهو علي كل شيء قدير

أَلَا بِذِكْرِ اللَّهِ تَطْمَئِنُّ الْقُلُوبُ

By: Amr Elsayed

Object Meta Data And Descriptor Part 1 Lesson #157

```
Object Meta Data And Descriptor
- writable
- enumerable
- configurable [Cannot Delete Or Reconfigure]
```

```
const myObject = {
  a: 1,
  b: 2,
};
```

لنفترض اني عندي ال **myObject** دا وعايز اضيف جواه **Property** جديدة وليكن **c: 3** مثلا
فعشان اعمل دا بشكل احترافي ومميز هعمله من خلال ال **syntax** التالي واللي فيما بعد هنفهم كل حاجة فيه

```
Object.defineProperty(myObject, "c", {
  writable: true,
  enumerable: true,
  configurable: true,
  value: 3,
});
```

```
console.log(myObject);
```

```
► {a: 1, b: 2, c: 3}
```

أول حاجه في ال **syntax** اللي فات دا هو اني بحدد ال **object** اللي هضيف جواه **Property** جديدة
تاني حاجة بحط اسم ال **property** اللي هي "c" وبعدين بيجي الجزء الخاص بال **descriptor {...}** اللي بيكون
جواه بعض العناصر لو مكتبتهاش ال **Default** بتاعها بيكون **False**
بعد كدا بتيجي ال **value** اللي انا بضيفها بقا

في حالة لو حابب اعدل ال **Value** بتاعت ال **property** اللي اسمها "c"

```
myObject.c = 100;
```

```
► {a: 1, b: 2, c: 100}
```

التعديل هيتم عادي جدا بدون أي مشاكل وكل دا حصل عشان في جزء ال **descriptor** دا في قيمة عندي اسمها **writable** قيمتها ب **true**

لو انا بقا عدلت القيمة دي ب **false** في الحالة دي لو حبيت اعدل ال Value بتاعت ال "c" فمش هينفع!!

لو حبيت اعمل **Looping** علي عناصر ال **Object** اللي عندي دا

```
for (let prop in myObject) {  
  console.log(prop, myObject[prop]);  
}
```

a 1

b 2

c 3

لكن لو جيت في جزء ال **descriptor** دا عند القيمة اللي اسمها **enumerable** وعدلت قيمتها ب **False**

a 1

b 2

هنا هيستثني ال "c" من ال **loop** ومش هيعمل حسابها لاني بكل بساطه منعته من الاشتراك في ال **Loop** ولكن مش بتت حذف من ال **Object** نفسه هي بتفضل موجودة ولكن بتتمنع من الاشتراك في أي **loop** هيتم

في حالة لو عايز احذف ال **property** اللي اسمها "c" بال **value** بتاعتها فعندي أصلا **method** اسمها **delete** من خلالها بقدر اتمم عملية الحذف دي

```
console.log(delete myObject.c);
```

true

▶ {a: 1, b: 2}

أهو تم حذف ال **property** من ال **object**

لكن لو جيت في جزء ال **descriptor** دا عند القيمة اللي اسمها **configurable** وعدلت قيمتها ب **False**

ساعتها لو جيت احذف ال "c" من ال **object** بتاعي دا مش هيحصل

يبقي ال **configurable** لو بقت قيمته بتساوي **false** مش هتقدر تحذف ال **property** ولا تعدل عليها ولا تعمل **redefined** للقيمة دي في أي مكان تاني

Object Meta Data And Descriptor Part 2 Lesson #158

```
Object Meta Data And Descriptor  
- Define Multiple Properties  
- Check Descriptors
```

في الدرس اللي فات عرفنا ازاى ممكن نضيف **Property** واحدة لل **object** اللي عندي والتعديل في ال **descriptor** الخاص بال **property** دي من خلال **method** اسمها **defineProperty**

في الدرس دا هنشوف ازاى نقدر نضيف اكر من **property** بال **descriptor** بتاعها ولكن ال **method** المرادي هتكون **defineProperties** بالجمع للدلالة على وجود اكر من **Property**

```
Object.defineProperties(myObject, {  
  c: {  
    configurable: true,  
    value: 3,  
  },  
  d: {  
    configurable: true,  
    value: 4,  
  },  
  e: {  
    configurable: true,  
    value: 5,  
  },  
});
```

```
console.log(myObject);
```

```
► {a: 1, b: 2, c: 3, d: 4, e: 5}
```

طيب لو عندي **property** معينة انا حابب ابحث في ال **descriptor** بتاعها
بكتب **Object.getOwnPropertyDescriptor** ولاحظ ان **descriptor** مفرد

```
console.log(Object.getOwnPropertyDescriptor(myObject, "d"));
```

```
▼ {value: 4, writable: false, enumerable: false, configurable: true} ⓘ  
  configurable: true  
  enumerable: false  
  value: 4  
  writable: false  
  ► [[Prototype]]: Object
```

لما بحثت عن ال "d" وعن ال **Descriptor** بتاعتها تحديدا لقيت فعلا ان ال **Configurable** اللي انا كنت عامله **true** موجود باقي القيم التانيه بـ **false** لان دا ال **default** بتاعها لو انا معدلتش عليها

طيب لو حابب ابحث عن كل الخصائص اللي في ال **object** نفسه كلها بقا مش هقتصر ع خاصية واحدة
فهكتب **Object.getOwnPropertyDescriptors** ولاحظ ان **descriptors** جمع

```
console.log(Object.getOwnPropertyDescriptors(myObject));
```

```
▼ {a: {...}, b: {...}, c: {...}, d: {...}, e: {...}} ⓘ  
  ► a: {value: 1, writable: true, enumerable: true, configurable: true}  
  ► b: {value: 2, writable: true, enumerable: true, configurable: true}  
  ► c: {value: 3, writable: false, enumerable: false, configurable: true}  
  ► d: {value: 4, writable: false, enumerable: false, configurable: true}  
  ► e: {value: 5, writable: false, enumerable: false, configurable: true}  
  ► [[Prototype]]: Object
```

لا تنسونا من دعواتكم

Never Forget That (**Always Search**)

By: Amr Elsayed

Assignments From [147] = > [158]

لينك التكميلات: [\[158\] <= \[147\]](#)

التكليف: 01

```
// Assignment One

// function Car(name, model, price) {
//   this.n = name;
//   this.m = model;
//   this.p = price;
// }

class Car {
  constructor(name, model, price) {
    this.n = name;
    this.m = model;
    this.p = price;
  }
  run() {
    return `Car Is Running Now`;
  }
  stop() {
    return `Car Is Stopped Now`;
  }
}

// Needed Output

let carOne = new Car("MG", 2022, 420000);
let carTwo = new Car("Tesla", 2023, 1000000);
let carThree = new Car("BMW", 2021, 850000);

console.log(
  `Car One Name Is ${carOne.n} And Model Is ${carOne.m} And Price Is ${carOne.p}`
);
console.log(
  `Car One Name Is ${carTwo.n} And Model Is ${carTwo.m} And Price Is ${carTwo.p}`
);
console.log(
  `Car One Name Is ${carThree.n} And Model Is ${carThree.m} And Price Is
  ${carThree.p}`
);

console.log(carOne.run());
console.log(carOne.stop());
// "Car One Name Is MG And Model Is 2022 And Price Is 420000"
// "Car Is Running Now"
```

```
// Assignment Two

class Phone {
  constructor(name, serial, price) {
    this.name = name;
    this.serial = serial;
    this.price = price;
  }
}

// Write Tablet Class Here

class Tablet extends Phone {
  constructor(name, serial, price, size) {
    super(name, serial, price);
    this.size = size || "Unknown";
  }
  fullDetails() {
    return `${this.name} Serial is ${this.serial} And Size Is ${this.size}`;
  }
}

let TabletOne = new Tablet("iPad", 100200300, 1500, 12.9);
let TabletTwo = new Tablet("Nokia", 350450650, 800, 10.5);
let TabletThree = new Tablet("LG", 250450650, 650);

console.log(`${TabletOne.fullDetails()}`);
// iPad Serial is 100200300 And Size Is 12.9

console.log(`${TabletTwo.fullDetails()}`);
// Nokia Serial is 350450650 And Size Is 10.5

console.log(`${TabletThree.fullDetails()}`);
// LG Serial is 250450650 And Size Is Unknown
```



```
// Assignment Three

// Edit The Class
class User {
  #c;
  constructor(username, card) {
    this.u = username;
    this.#c = card;
  }
  creditCard() {
    return this.#c.toString().match(/\d{4}/gi).join("-");
  }
  get showData() {
    return `Hello ${this.u} Your Credit Card Number Is ${this.creditCard()} `;
  }
}

// Do Not Edit Anything Below

let userOne = new User("Osama", "1234-5678-1234-5678");
let userTwo = new User("Ahmed", "1234567812345678");
let userThree = new User("Ghareeb", 1234567812345678);

console.log(userOne.showData);
// Hello Osama Your Credit Card Number Is 1234-5678-1234-5678

console.log(userTwo.showData);
// Hello Ahmed Your Credit Card Number Is 1234-5678-1234-5678

console.log(userThree.showData);
// Hello Ghareeb Your Credit Card Number Is 1234-5678-1234-5678

console.log(userOne.c); // Prevent Accessing To Card Property Here
// Undefined
```

```
// Assignment Four

// Write Your Code Here
String.prototype.addLove = function (val) {
  return `I love ${this} Web School`;
};

// Do Not Edit Below
let myStr = "Elzero";
console.log(myStr.addLove()); // I Love Elzero Web School
```



```
// Assignment Five

const myObj = {
  username: "Elzero",
  id: 100,
  score: 1000,
  country: "Egypt",
};

// Write Your Code Here
Object.defineProperty(myObj, {
  id: {
    enumerable: false,
  },
  score: {
    writable: false,
  },
});

myObj.score = 500;

delete myObj.country;

for (let prop in myObj) {
  console.log(`${prop} => ${myObj[prop]}`);
}

console.log(myObj);

// Needed Output

// "username => Elzero"
// "score => 1000"
// {username: 'Elzero', score: 1000, id: 100}
```

لا تنسونا من دعواتكم

Never Forget That (**Always Search**)

By: Amr Elsayed

♡ عبارات تحفيزية ♡



اذكر الله ♡

الزموا ذكر الله ففيه العون على متاعب الحياة ♡

اللهم إني أستغفرك واتوب إليك، لا إله إلا أنت سبحانك إني كنت من الظالمين

أَلَا بِذِكْرِ اللَّهِ تَطْمَئِنُّ الْقُلُوبُ

لا تنسونا من دعواتكم

Never Forget That (Always Search)

Date And Time Introduction Lesson #159

Date And Time
- Date Constructor

Static Methods
- Date.now()

- To Track Time You Need Starting Point
- Epoch Time Or Unix Time In Computer Science Is The Number of Seconds Since January 1, 1970.
- Why 1970 [829 Days To 136 Years]

Search For
- Year 2038 Problem in Computer Science.

```
let dateNow = new Date();  
console.log(dateNow);
```

Sun Oct 23 2022 01:17:51 GMT+0200 (Eastern European Standard Time)

دلوقتي انا عندي ال **Date object** بستعلم منه عن الوقت الحالي بالظبط بالتاريخ

```
console.log(Date.now()); // 1000 mill-seconds = 1 second
```

1666481241054

الوقت دا مكتوب بالملي ثانيه من اول تاريخ 1970 لحد اللحظة اللي بكتب فيها الكلام دا !

```
console.log(Date.now()); // 1000 Mill = 1 Second  
  
let seconds = Date.now() / 1000; // Number Of Seconds  
console.log(`Seconds ${seconds}`);  
  
let minutes = seconds / 60; // Number Of Minutes  
console.log(`Minutes ${minutes}`);  
  
let hours = minutes / 60; // Number Of Hours  
console.log(`Hours ${hours}`);  
  
let days = hours / 24; // Number Of Days  
console.log(`Days ${days}`);  
  
let years = days / 365; // Number Of Years  
console.log(`Years ${years}`);
```

Get Date And Time Lesson #160

Date And Time

- getTime() => Number Of Milliseconds
- getDate() => Day Of The Month
- getFullYear()
- getMonth() => Zero Based
- getDay() => Day Of The Week
- getHours()
- getMinutes()
- getSeconds()

في الدرس دا هنتعرف على بعض ال **methods** اللي بستخدمها مع ال **Date Constructor**

```
let dateNow = new Date();
let birthday = new Date("nov 9, 98");

let dateDiff = dateNow - birthday;
console.log(dateDiff / 1000 / 60 / 60 / 24 / 365);
```

هنا انا حددت تاريخ معين هيبدأ من عنده الي هو 9 نوفمبر 1998 وقولتله انا عايز الفرق بين التاريخ الحالي و التاريخ الي انا حددته دا والنتيجة هتكون بالمللي ثانية

فعشان احول الفرق بين التاريخين دا من المللي ثانية للسنين عملت المعادلة الي اتعلمت في اول درس

وهي اني بقسم الرقم الي طلع بالمللي ثانية دا علي 1000 عشان اجيب الثواني
وبعدين اقسم الثواني دي علي 60 عشان اجيب الدقائق ما هو كل دقيقة فيها 60 ثانية
وبعدين اقسم الدقائق دي علي 60 عشان اجيب الساعات ما هو كل ساعة فيها 60 دقيقة
وبعدين قسمت الساعات دي علي 24 ساعة عشان اجيب الأيام ما هو كل يوم فيه 24 ساعة
وبعدين قسمت الأيام دي علي 365 يوم عشان اجيب السنين ما هو كل سنة فيها 365 يوم تقريبا

23.970067621987567

فكانت دي النتيجة فاضلي أيام ويجي عيد ميلادي فعلا: D"
وكل عام وانا طيب وحلو وبسمسم كدا ان شاء الله: D"

```
console.log(dateNow);  
console.log(dateNow.getTime());  
console.log(dateNow.getDate());  
console.log(dateNow.getFullYear());  
console.log(dateNow.getMonth());  
console.log(dateNow.getDay());  
console.log(dateNow.getHours());  
console.log(dateNow.getMinutes());  
console.log(dateNow.getSeconds());
```

Sun Oct 23 2022 02:15:29 GMT+0200 (Eastern European Standard Time)

Numbers Of Mill Seconds: 1666484129108

Current Day: 23

Current Year: 2022

Index Of The Current Month "October": 9

Index Of The Current Day "Sunday": 0

Current Hour: 2

Current Minute: 15

Current Second: 29

لا تنسونا من دعواتكم

Never Forget That (**Always Search**)

By: Amr Elsayed

Set Date And Time Lesson #161

Date And Time

- setTime(Milliseconds)
- setDate() => Day Of The Month [Negative And Positive]
- setFullYear(year, month => Optional [0-11], day => Optional [1-31])
- setMonth(Month [0-11], Day => Optional [1-31]) [Negative And Positive]
- setHours(Hours [0-23], Minutes => Optional [0-59], Seconds => Optional [0-59], MS => Optional [0-999])
- setMinutes(Minutes [0-59], Seconds => Optional [0-59], MS => Optional [0-999])
- setSeconds(Seconds => [0-59], MS => Optional [0-999])

```
let dateNow = new Date();  
console.log(dateNow);
```

Sun Oct 23 2022 11:27:33 GMT+0200 (Eastern European Standard Time)

بعمل **reset** للوقت والتاريخ وبمجرد ما اخلي ال **setTime(0)** فدا معناه ان بقوله مفيش ولا ملي ثانية عدت من بداية ال **Unix Time**

```
dateNow.setTime(0);  
console.log(dateNow);
```

Thu Jan 01 1970 02:00:00 GMT+0200 (Eastern European Standard Time)

ال **setDate** هنا بالقيمة واحد كأني بقوله اليوم **1** في الشهر الحالي يعني انا دلوقتي يوم **23** في الشهر انا عدلتها بحيث تبقي **1** في الشهر ودا حصل من خلال ال **setDate**

```
dateNow.setDate(1);  
console.log(dateNow);
```

Sat Oct 01 2022 11:45:26

في **trick** هنا وهي اني لو عملت تاريخ اليوم بـ **0** ما هو مفيش يوم في الشهر رقمه **0** فدا معناه انه هيرجع يوم لورا في الشهر اللي قبل الحالي ولو عملت بالسالب هيرجع بالايام للشهر الماضي اما لو عملت اليوم بأكبر من **30** او **31** يوم بحد اقصى وليكن **35** مثلا هيروح بالايام والتاريخ للشهر الجديد

```
dateNow.setDate(35);  
console.log(dateNow);
```

Fri Nov 04 2022

```
dateNow.setDate(-1);  
console.log(dateNow);
```

Thu Sep 29 2022

انا هنا حدد السنة نفسها وخليتها 2020 من خلال ال `setFullYear` وممكن تكتب الشهر اختياري

```
dateNow.setFullYear(2020);  
console.log(dateNow);
```

Fri Oct 23 2020 11:51:46 GMT+0200 (Eastern European Standard Time)

انا هنا بحدد الشهر بال `setMonth` فمثلا ال 0 دا بداية لأول شهر اللي هو يناير من حيث ال `index` اما 11 فهو ال `index` لشهر ديسمبر

```
dateNow.setMonth(0);  
console.log(dateNow);
```

Sun Jan 23 2022

```
dateNow.setMonth(11);  
console.log(dateNow);
```

Fri Dec 23 2022

وطبعا مش محتاج اقولك لو خليتها بالسالب هيرج تاني بالسالب لشهر ديسمبر اللي وراه ولكن السنة بدل ما هتكون 2022 هتكون 2021 لان رجعت لديسمبر السنة اللي فاتت وبرضو لو عدت ال 11 `index` وخليتها 12 فهترج ليناير السنة اللي بعدها فبدل ما هتكون 2022 هتبقى 2023

```
dateNow.setHours(1);  
dateNow.setMinutes(22);  
dateNow.setSeconds(44);
```

Sun Oct 23 2022 01:22:44 GMT+0200

طبعاً نفس الأفكار تنطبق ع الساعات والدقائق والثواني بالظبط بتحدددهم والسالب يرجع لورا
وبيكون في ترحيل للوقت للخلف ولو بزيادة عن الطبيعي بيقول فيه ترحيل للوقت للامام وانت فاهمني بقا: D"

كل حاجة كبيرة بتكتبها بيقول ليك حرية انك تعدل في الأقل منها بشكل مباشر
يعني لو انا كتبت سنة معينة ولكن 2020 من خلال ال `setFullYear` فمممكن احدد بعده الشهر واليوم

ولو كتبت الشهر من خلال ال `setMonth` فمسموحلي اكتب اليوم ولكن مقدرش اكتب سنة

كذلك الامر في الساعة لو كتبت الساعة من خلال ال `setHours` فاقدر اكتب دقائق وثواني معاها
ولو كتبت الدقائق من خلال ال `setMinutes` فاقدر بعدها احدد الثواني ولكن مقدرش طبعاً احدد الساعة قبلها

فزي ما قولتلك كل ما تبدأ من مكان اكبر تقدر تتحكم في الحاجات الي بتكون اصغر منه وتابعة ليه

لا تنسونا من دعواتكم

Never Forget That (**Always Search**)

By: Amr Elsayed

Formatting Date And Time Lesson #162

Date And Time

```
new Date(timestamp)
new Date(Date String)
new Date(Numeric Values)
```

Format

- "Oct 25 1982"
- "10/25/1982"
- "1982-10-25" => ISO International Standard
- "1982 10"
- "1982"
- "82"
- 1982, 9, 25, 2, 10, 0
- 1982, 9, 25
- "1982-10-25T06:10:00Z"

```
Date.parse("String") // Read Date From A String
```

عملت **reset** لـ **Date** فبدأ من اول 1 يناير 1970

```
let date1 = new Date(0);
console.log(date1);
```

```
Thu Jan 01 1970 02:00:00 GMT+0200 (Eastern European Standard Time)
```

أول حاجة عندي **Method** بتعمل تحليل لـ **date** لو مكتوب **String** وترجعلي النتيجة بالمللي ثانيه
فانا خدت النتيجة الي بالمللي ثانيه دي بعد ما عملت تحليل للبيانات وحطيتها في الـ **Date** بالتالي هيجيب
تاريخ ميلادي

```
console.log(Date.parse("nov 9 98"));
let date2 = new Date(910562400000);
console.log(date2);
```

```
910562400000
```

```
Mon Nov 09 1998 00:00:00 GMT+0200 (Eastern European Standard Time)
```

```

29 let date3 = new Date("11 9 98");
30 console.log(date3);
31
32 let date4 = new Date("1998-11-9");
33 console.log(date4);
34
35 let date5 = new Date("1998-11");
36 console.log(date5);
37
38 let date6 = new Date("1998");
39 console.log(date6);
40
41 let date7 = new Date("98");
42 console.log(date7);

```

Mon Nov 09 1998 00:00:00 GMT+0200 (Eastern European Standard Time)	main.js:30
Mon Nov 09 1998 00:00:00 GMT+0200 (Eastern European Standard Time)	main.js:33
Sun Nov 01 1998 02:00:00 GMT+0200 (Eastern European Standard Time)	main.js:36
Thu Jan 01 1998 02:00:00 GMT+0200 (Eastern European Standard Time)	main.js:39
Thu Jan 01 1998 00:00:00 GMT+0200 (Eastern European Standard Time)	main.js:42

في الطرق دي بيقولك تقدر تكتب ال **date** بكل الطرق دي برحتك وممكن التاريخ المكتوب بالشكل دا **1998-11-9** تكتبه كدا **1998/11/9** او تبدل القيم وتسبب بينهم مسافات او تكتبه كدا **1998@11@9** كل دا هيتقرأ عادي ممكن **تكتب السنة والشهر بس زي كدا 1998-11** ممكت تكتب السنة بس سواء **1998** او **98** بس وهو بيكمل باقي التاريخ بـ **1** يناير في حالة لو انت محددتش الشهر واليوم

```
44 let date8 = new Date(1998, 10, 9, 4, 20, 10);
45 console.log(date8);
46
47 let date9 = new Date(1998, 10, 9);
48 console.log(date9);
49
50 let date10 = new Date("1982-10-25T06:10:00Z");
51 console.log(date10);
```

Mon Nov 09 1998 04:20:10 GMT+0200 (Eastern European Standard Time)	main.js:45
Mon Oct 25 1982 00:00:00 GMT+0200 (Eastern European Standard Time)	main.js:48
Mon Oct 25 1982 08:10:00 GMT+0200 (Eastern European Standard Time)	main.js:51

في السطر 44 انا كتبت التاريخ والوقت بالكامل

وفي السطر 47 انا كتبت برضو التاريخ ولكن في الحالة دي عند الشهر انت لما تكون عايز شهر 11 بتكتب ال index بتاع الشهر دا الي هو هيكون 10 لان شهر 1 ال index بتاعه 0

في السطر ال 50 كتبت التاريخ بال time zone

فمثلا التاريخ بتاعم 1982-10-25 بعد حرف t عشان يفصل التاريخ عن الوقت الي هتكتبه بعد كدا اما حرف ال z دا بيعبر عن ال time zone وبما اني كاتب الوقت الساعة 6 و 10 دقائق فال Time Zone هيزود ساعتين الي هما الفرق بين توقيت بلدي وتوقيت جرينتش غالبا: "D"

♥ عبارات تحفيزية ♥



مش معني كدا إنك تتجاهل دراستك لكن مهم جدا إنك تهتم بتعليم نفسك بنفسك

♥ اذكر الله ♥

♥ الزموا ذكر الله ففيه العون على متاعب الحياة ♥

اللهم لك الحمد حمدًا طيبًا كثيرًا مباركًا فيه كما ينبغي لجلال وجهك ولعظيم سلطانك يا رب العالمين

لا تنسونا من دعواتكم

Never Forget That (Always Search)

By: Amr Elsayed

Tracking Operations Time Lesson #163

Date And Time
- Track Operations Time

Search
- performance.now()
- performance.mark()

```
// Start Time
let start = new Date();

// Operation
for (let i = 0; i < 100000; i++) {
  // document.write(`

${i}</div>`);
  let div = document.createElement("div");
  div.appendChild(document.createTextNode(i));
  document.body.appendChild(div);
}

// Time End
let end = new Date();

// Operation Duration
let duration = end - start;

console.log(duration);


```

عملت متغير **start** فيه ال **Date** الحالي وعملت متغير **end** فيه برضو ال **Date** الحالي

وبين ال **Start** و ال **End** عملت **Operation** انه يعمل **div 100000**

بعدين قولتله ال **duration** هيكون الفرق بين الوقت الحالي لـ **start** و الوقت الحالي لـ **end** والي بينهم بيتم **operation** فيها **div 100000** كل دا عشان احسب الوقت الي هتتم فيه العملية دي بالملي ثانية

249

وبالمناسبة الطريقة بتاعت ال **createElement** اسرع بكثير من ال **document.write()**

Generator Function Introduction Lesson #164

Generators

- Generator Function Run Its Code When Required.
- Generator Function Return Special Object [Generator Object]
- Generators Are Iterable

ال **Generators** او **المولدات** بتكون عبارة عن **Function** ولكن بشكل مختلف شوية عن شكل ال **function** المعتاد وال **code** بتاعها مش بيشتغل الا لما انت بتحتاجه وبتكون عبارة عن **object** وكمان **Iterable** تقدر تعمل **Loop** عليه عادي جدا

مثال علي ال **Generators** وهو مثلا لما تدخل موقع وتلاقي شخص حاطط بعض الاعمال او المنتجات وبعدين كاتب في الاخر **Show More** ال **show more** دي بيتم فيها عمل **Generation** او توليد **لباقى البيانات** عشان تبدأ تظهر في الصفحة

ولو تاخد بالك انت هنا هتعمل **action** وهو انك تدوس علي **show more** ودا اللي قولنا عاليه فوق ان **code** ال **Generator** مش بيشتغل الا لما تحتاجه ... طيب امته بحتاجه؟! .. بحتاجه لما ال **user** يدوس على **Show More** ... أيش هذا والله نفس المعني **D"**

```
function* generateNumbers() {  
  yield 1;  
  yield 2;  
  yield 3;  
  yield 4;  
}
```

ودا بيكون شكل ال **Generator Function** و ال **keyword** الي اسمها **yield** هي المسئولة عن عملية التوليد

```
let generator = generateNumbers();  
console.log(generator);  
console.log(typeof generator);
```

► *generateNumbers {<suspended>}*

object


```
function* generateNumbers() {
  yield 1;
  alert("Hello After Yield 1");
  yield 2;
  yield 3;
  yield 4;
}
```

ال **Generator function** دي لو هتعمل **Iteration** ليها باستخدام ال **next()** فهتلاحظ ان كل **yield** مش بتيجيلك غير لو انت فعليا استدعيته او طلبتها يعني علي سبيل المثال

```
console.log(generator.next());
console.log(generator.next());
```

```
► {value: 1, done: false}
```

ال **next()** الأولي هتلاقيه طبع اول **yield** الي هي ال **Value** بتاعته بـ **1** و ال **done** دي بـ **false** عشان لسه فيه بيانات ممكن يتعملها **Generate** من ال **function** وتاني **next()** دي مش هتشتغل غير لما تتخلص من ال **alert** الي انت حاطه قبلها

127.0.0.1:5501 says

Hello After Yield 1

OK

```
► {value: 1, done: false}
```

```
► {value: 2, done: false}
```

بعد اول **yield** بيتم استدعائه البيانات وال **code** بيوقف تماما الا لو انت عملته استدعاء لذلك بعد أول **next()** هتلاحظ ان ال **alert** نفسها مشغلتهش الا لما انت استخدمت تاني **next()** عشان تستدعي تاني **yield**

```
function* generateNumbers() {
  yield 1;
  console.log("Hello After Yield 1");
  yield 2;
  yield 3;
  yield 4;
}
```

```
for (let value of generateNumbers()) {
  console.log(value);
}
```

1

Hello After Yield 1

2

3

4

```
for (let value of generateNumbers()) {
  console.log(value);
}

for (let value of generator) {
  console.log(value);
}
```

ليه في الحالة الأولى ال **loop** تم واشتغل وفي الحالة الثانية مشتغلش؟!
 عشان ببساطه في الحالة الثانية حالة ال **generator** انت **already** عملتها **yield** قبل كدا اهو:

```
console.log(generator.next());
console.log(generator.next());
console.log(generator.next());
console.log(generator.next());
console.log(generator.next());
```

لذلك هي مشتغلش في ال **loop** الثاني لانك بالفعل استخرجت كل البيانات ومبقاش فيه حاجة ثاني تستخرجها
 ولو جربت توقف ال **next()** وتشغل ثاني **loop** هتلاقيه اشتغل عادي

Delegate Generator Function Lesson #165

Generators
- Delegate Generator

مصطلح ال Delegate يعني نائب طيب اقدر استخدمه او اوظفه ازاي ؟!

عندي Function بت Generate ارقام من 1 الي 3

```
function* generateNums() {  
  yield 1;  
  yield 2;  
  yield 3;  
}
```

و عندي Function بت Generate الحروف من A الي C

```
function* generateLetters() {  
  yield "A";  
  yield "B";  
  yield "C";  
}
```

بعملت Function تالته بت generate كل ال functions الي فاتت وكمان عملت جواها yield ل Array

```
function* generateAll() {  
  yield* generateNums();  
  yield* generateLetters();  
  yield* [4, 5, 6];  
}
```

لاحظ ان اخر **function** الي اسمها **generateAll()** دي ال **yield** الي جوها بحط بعده * ودا عشان
اقدر اعمل **Access** علي كل العناصر الي جوا كل **function** من الي عملتهم مسبقا

يعني لو مفيش ال * دي عند ال **function** الي بت **generate** الأرقام في الحالة دي هيجيب ال **function** نفسها
لكن انا عايز العناصر الي جوا ال **function** دي لذلك استخدمت ال *

خزنت ال **function** الأخيرة الي بتنوب عن كل ال **functions** الثانية في متغير اسمه **generator**

```
let generator = generateAll();
```

وبدأت استخدم ال **next()** عشان اعمل **iteration** يجيب كل العناصر الموجودة في كل ال **functions** الي
عندي

```
console.log(generator.next());  
console.log(generator.next());  
console.log(generator.next());  
console.log(generator.next());  
console.log(generator.next());  
console.log(generator.next());  
console.log(generator.next());  
console.log(generator.next());  
console.log(generator.next());  
console.log(generator.next());
```

```
▶ {value: 1, done: false}  
▶ {value: 2, done: false}  
▶ {value: 3, done: false}  
▶ {value: 'A', done: false}  
▶ {value: 'B', done: false}  
▶ {value: 'C', done: false}  
▶ {value: 4, done: false}  
▶ {value: 5, done: false}  
▶ {value: 6, done: false}  
▶ {value: undefined, done: true}
```

آخر حاجة لو جيت في النص استخدمت ال **return()** فاستخدمها بيعمل **stop** لل **iteration** ومبتكملش من
بعدها أي حاجة

```
console.log(generator.next());  
console.log(generator.return("Z"));  
console.log(generator.next());
```

```
▶ {value: 'C', done: false}  
▶ {value: 'Z', done: true}  
▶ {value: undefined, done: true}
```

لذلك لو تلاحظ بعد ال **return("Z")** هتلاقيه وقف ال **Iteration** تماما ومجبش ارقام ال **array** الي كنت
عاملها فوق واكتفي بانه يظهر ما قبل ال **return()** دي الي هما اول **Functions 2** عملتهم فوق بتاعت الأرقام
والحروف

Generate Infinite Numbers Lesson #166

Generators

- Generate Infinite Numbers
- Use Return Inside Generators

وظيفة ال **return** في ال **function** العادية انه بيحزن الوظيفة اللي عملتها ال **function** دي ولما احتاج اللي حصل جوا ال **Function** دي استخدم ال **return** في اني اخذ الوظيفة المتخزنة جوا ال **function** وطبعاً زي ما احنا عارفين ان ال **return** بتوقف عمل أي حاجه بعدها او بتلغي أي حاجة بتيجي بعدها !
لذلك في ال **generator functions** انا مش محتاج استخدم ال **return** دي خالص لان زي ما قولتك بتوقف عمل أي حاجة هتيجي بعدها + اني ال **generator functions** بستخدمها عشان انتج اكتر من قيمة فمش محتاج ال **return** انها تخزن حاجة وترجعها لي!

طيب لو عايز اعمل **Generate** لاي عدد ممكن

```
function* generateNumbers() {  
  let index = 0;  
  
  while (true) {  
    yield index++;  
  }  
}  
  
let generator = generateNumbers();
```

فانا هعمل **While Loop** بتبدأ من ال 0 اللي هي قيمة المتغير **index** وهعمل **yield** لـ **index++** بحيث انه كل مرة استدعي فيها رقم من خلال ال **next()** هيعمله **generate**

```
console.log(generator.next());  
console.log(generator.next());  
console.log(generator.next());  
console.log(generator.next());
```

```
▶ {value: 0, done: false}  
▶ {value: 1, done: false}  
▶ {value: 2, done: false}  
▶ {value: 3, done: false}
```

```
console.log(generator.next());
```

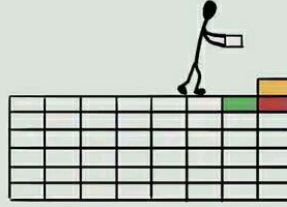
```
▶ {value: 4, done: false}
```

لاحظ ان ال **while loop** موجودة داخل ال **Function**

♥ عبارات تحفيزية ♥



STEP
BY
STEP



BRICK
BY
BRICK



DROP
BY
DROP

قليل دائم خير من كثير منقطع .. محاولة ومحاولة استمرارية ورا استمرارية
هتلاقي نفسك حققت كل اللي نفسك فيه

♥ اذكر الله

♥ الزموا ذكر الله ففيه العون على متاعب الحياة

لا إله الا الله وحده لا شريك له .. له الملك و له الحمد وهو على كل شيء قدير

اللهم صلّ وسلم وزد وبارك علي نبينا محمد ﷺ

أَلَا بِذِكْرِ اللَّهِ تَطْمَئِنُّ الْقُلُوبُ

لا تنسونا من دعواتكم

Never Forget That (**Always Search**)

By: Amr Elsayed

Module Import And Export Lesson #167

Modules
- Import And Export

جميع الأكواد التي تم ذكرها في الفيديو هنا

ببساطة شديدة جدا ال **import** وال **export** أي الاستيراد والتصدير

لما تكون شغال ب framework معين بيكون متقسم لأكثر من **modules** وال **modules** دا تقدر تقول عليه انه وحدة برمجية بتنفذ وظيفة معينة

علي سبيل المثال بيكون عندي **module** بتنفذ وظيفة معينة زي انه بيتعامل مع الصور فال **module** هنا بيكون عبارة عن ملف فيه مجموعة من ال **functions** بيتعامل مع الصورة فيكون عندي **function** بتعمل **resize** للصور و **function** تانية بتعمل **crop** للصورة وهكذا اكثر من **function**

اول حاجة عندي في ملف ال **html** بكتب كل الملفات بتاعتي الي هعمل منها **import** والي هعمل فيها **export** وبكتب **attribute** اسمه **type** قيمته بتكون كلمة "**module**"

HTML

```
<script src="main.js" type="module"></script>
<script src="app.js" type="module"></script>
```

Main.js

```
let a = 10;
let arr = [1, 2, 3, 4];

function saySomething() {
  return `Something`;
}

export { a, arr, saySomething };
```

عندي في ال `main.js` دا بعد متغير اسمه `a` قيمته بـ `10` وعندي `array` قيمتها بـ `[1, 2, 3, 4]`
وعندي `function` اسمها `saySomething()` بتـ `return` كلمة `'Something'`

عملت `export` او تصدير لكل الحاجات دي زي ما انت شايف في اخر سطر في الكود

app.js

```
import { a, arr, saySomething as s } from "./main.js";
```

```
console.log(a);  
console.log(arr);  
console.log(s());
```

10

► (4) [1, 2, 3, 4]

Something

روحت لملف ال `app.js` عملت `import` او استيراد لكل الحاجات الي عملتلها `export` او تصدير من ملف ال `main.js` وكتبت المسار بتاعه بالـ `syntax` الي مكتوب في أول سطر دا

وممكن اسم ال `function` الي هو `saySomething()` اغير اسمه باني أقوله بعد اسمه `as` والاسم الي انا عايزه زي المثال في الكود الي فوق كدا وممكن اغير اسم أي حاجه بعملها استيراد عادي من خلال اني اديها اسم مستعار او بيطلق ع العلمية دي `alias`

بعدين بدأت استخدم كل الحاجات الي عملتلها `import` في ملف ال `app.js` من ملف ال `main.js`

Named vs Default Export And Import All Lesson#168

Modules

- Export Alias
- Named Export
- Default Export
- Import All

زي ما ممكن مع ال **Import** تحط **alias** لبعض الأسماء زي كذا

```
import { a, arr, saySomething as s } from "./main.js";
```

فانت ممكن كمان تعمل ال **alias** دا اثناء ال **export** نفسه ولكن هتحتط ال **alias** بقا اثناء ال **import**

```
export { a as myNumber, arr, saySomething };
```

```
import { myNumber, arr, saySomething as s } from "./main.js";  
  
console.log(myNumber);  
console.log(arr);  
console.log(s());
```

كل اللي فات دا اسمه **Named export**

اما ال **default export** كل **module** موجود بيكون ليه **one default export** وتقدر تستدعيها من أي مكان حتي بدون استخدام اسمها دا وليها طريقة خاصة في الاستدعاء تعالي نشوف

```
export default function sayHello() {  
  return `Hello`;  
}
```

```
import sayHello, { myNumber, arr, saySomething as s } from "./main.js";  
console.log(sayHello());
```

Hello

وممكن زي ما قولتلك تستدعيها بأي اسم انت عايزو عادي جدا يعني تخيل لو ال **function** بتاعت **sayHello** دي هي أصلا **anonymous function** !؟

ودا لانها ال **default export** عادي تسميها باي اسم وانت بتعملها **import**

```
import elzero, { myNumber, arr, saySomething as s } from "./main.js";  
console.log(elzero());
```

طيب أي رائيك لو قولتلك اني عايز اعمل **import** لكل ال **functions modules** اللي عندي ومش عايز انا او مش حابب اقعد اكتب أسمائهم كذا زي الطريقة اللي فاتت !؟

```
import * as all from "./main.js";  
console.log(all);
```

```
▼ Module {Symbol(Symbol.toStringTag): 'Module'} ⓘ  
  ► arr: Array(4)  
  ► default: f sayHello()  
    myNumber: 10  
  ► saySomething: f saySomething()
```


في الطريقة دي من خلال ال `*` انا عملت `import` لكل ال `functions` اللي عندي و خزنتها `alias` اسمه `all` ممكن تختار أي اسم يعجبك اسم `all` دا مش `standard` يعني

طيب لو حابب استخدم `Function` معينة مثلا اعمل أي؟!
هكتب ال `alias` بتاعي وبعده نقطة . وهيبي بعد كدا كل ال `functions` اللي عندي

```
console.log(all.);  
arr: number[] × [arr  
  default  
  myNumber  
  saySomething
```

```
console.log(all.arr);  
console.log(all.default);  
console.log(all.myNumber);  
console.log(all.saySomething());
```

```
► (4) [1, 2, 3, 4]
```

```
Hello
```

```
10
```

```
Something
```

لا تنسونا من دعواتكم

Never Forget That (**Always Search**)

By: Amr Elsayed

Assignments From [159] = > [168]

لينك التكميلات: [\[168\] <= \[159\]](#)

التكليف: 01

```
// Assignment One

let dateNow = new Date();
let birthday = new Date("nov 9 98");
let diffNow = dateNow - birthday;
console.log(diffNow);
let seconds = diffNow / 1000;
console.log(`${Math.trunc(seconds)} Seconds`);

let minutes = seconds / 60;
console.log(`${Math.trunc(minutes)} Minutes`);

let hours = minutes / 60;
console.log(`${Math.trunc(hours)} Hours`);

let days = hours / 24;
console.log(`${Math.trunc(days)} Days`);

let years = days / 365;
console.log(`${Math.trunc(years)} Years`);

console.log(`#`.repeat(20));
```

التكليف: 02

```
// Assignment Two
let starterDate = new Date(1980, 0, 1, 0, 0, 1);
// starterDate.setTime();
console.log(starterDate);
// Needed Output

// "Tue Jan 01 1980 00:00:01 GMT+0200 (Eastern European Standard Time)"
```

التكليف: 03

```
// Assignment Three

// Needed Output
// "Sat Apr 30 2022 18:13:20 GMT+0200 (Eastern European Standard Time)"
// "Previous Month Is April And Last Day Is 30"

// The Result Will Be Change Because The Difference Between Time

let date = new Date();
date.setDate(0);
```

```

let months = [
  "Jan",
  "Feb",
  "Mar",
  "Apr",
  "May",
  "Jun",
  "Jul",
  "Aug",
  "Sep",
  "Oct",
  "Nov",
  "Dec",
];

console.log(`Previous Month is ${months[8]} And Last Day Is ${date.getDate()}`);

```

التكليف: 04

```

// Assignment Four

let birthdayOne = new Date(1998, 10, 9);
console.log(birthdayOne);

let birthdayTwo = new Date("nov, 9, 98");
console.log(birthdayOne);

let birthdayThree = new Date(Date.parse("nov, 9, 98"));
console.log(birthdayThree);

let birthdayFour = new Date("1998-11-09T00:00:00");
console.log(birthdayFour);

```

التكليف: 05

```

// Assignment Five

// Start
let start = performance.now();

// Operation
for (let i = 0; i < 100000; i++) {
  console.log(i);
}

// End
let end = performance.now();

// Duration
let duration = end - start;
console.log(`The Duration is ${Math.trunc(duration)} Milliseconds.`);
// Needed Output "Loop Took 1921 Milliseconds."

```

```
// Assignment Six

function* gen() {
  let index = 14;
  let num = 140;
  let sum = num + index;
  yield index;
  while (true) {
    yield sum;
    sum += num += 200;
  }
}

let generator = gen();
console.log(generator.next()); // {value: 14, done: false}
console.log(generator.next()); // {value: 154, done: false}
console.log(generator.next()); // {value: 494, done: false}
console.log(generator.next()); // {value: 1034, done: false}
console.log(generator.next()); // {value: 1774, done: false}
console.log(generator.next()); // {value: 2714, done: false}
console.log(generator.next()); // {value: 3854, done: false}
console.log(generator.next()); // {value: 5194, done: false}
console.log(generator.next()); // {value: 6734, done: false}
```

```
// Assignment Seven

function* genNumbers() {
  yield* [1, 2, 2, 2, 3, 4, 5];
}

function* genLetters() {
  yield* ["A", "B", "B", "B", "C", "D"];
}

// Write Your Generator Function Here
function* genAll() {
  yield* new Set(genNumbers());
  yield* new Set(genLetters());
}

let generator = genAll();
console.log(generator.next()); // {value: 1, done: false}
console.log(generator.next()); // {value: 2, done: false}
console.log(generator.next()); // {value: 3, done: false}
console.log(generator.next()); // {value: 4, done: false}
console.log(generator.next()); // {value: 5, done: false}
console.log(generator.next()); // {value: "A", done: false}
console.log(generator.next()); // {value: "B", done: false}
console.log(generator.next()); // {value: "C", done: false}
console.log(generator.next()); // {value: "D", done: false}
```

HTML Inside body tag

```
<script src="main.js" type="module"></script>  
<script src="mod-one.js" type="module"></script>  
<script src="mod-two.js" type="module"></script>
```

main.js

```
import calc from "./mod-one.js";  
import * as modOne from "./mod-two.js";  
console.log(calc(modOne.numOne, modOne.numTwo, modOne.numThree)); // 60
```

mod-one.js

```
export default function (numOne, numTwo, numThree) {  
  return numOne + numTwo + numThree;  
}
```

mod-two.js

```
import calc from "./mod-one.js";  
  
// mod-two.js File  
let a = 10; // Do Not Edit Names  
let b = 20; // Do Not Edit Names  
let c = 30; // Do Not Edit Names  
  
export { a as numOne, b as numTwo, c as numThree };
```

لا تنسونا من دعواتكم

Never Forget That (**Always Search**)

By: Amr Elsayed

عبارات تحفيزية



اذكر الله

الزموا ذكر الله ففيه العون على متاعب الحياة

اللهم صلّ وسلم وزد وبارك على نبينا محمد ﷺ
سبحان الله وبحمده عدد خلقه ورضا نفسه وزنة عرشه ومداد كلماته
لا إله الا انت سبحانك اني كنت من الظالمين
أَلَا بِذِكْرِ اللَّهِ تَطْمَئِنُّ الْقُلُوبُ

لا تنسونا من دعواتكم

Never Forget That (**Always Search**)

By: Amr Elsayed

What Is JSON Lesson #169

What Is JSON ?

- JavaScript Object Notation
- Format For Sharing Data Between Server And Client
- JSON Derived From JavaScript
- Alternative To XML
- File Extension Is .json

Why JSON ?

- Easy To Use And Read
- Used By Most Programming Languages And Its Frameworks
- You Can Convert JSON Object To JS Object And Vice Versa

JSON vs XML

=====		
= Text Based Format	= Markup Language	=
= Lightweight	= Heavier	=
= Does Not Use Tags	= Using Tags	=
= Shorter	= Not Short	=
= Can Use Arrays	= Cannot Use Arrays	=
= Not Support Comments	= Support Comments	=
=====		

- كلمة **JSON** اختصار لـ **JavaScript Object Notation**

- الـ **JSON** عبارة عن **Format معين** يتم من خلاله مشاركة البيانات بين الـ **Server Side** و الـ **Client Side** المشاركة دي بتكون من خلال ولنفترض انك شغال علي تطوير موقع وفيه قسم لد **Articles** فالـ **Back-End** بيبعتلك ملف **JSON** فيه **كل المقالات** اللي هيتم نشرها في الموقع انت كـ **Front-End** بتستلم الملف دا وتهندله في **صفحة المقالات بالتنسيقات المظبوطة** بحيث ان صفحة المقالات تقرأ الـ **Data** الموجودة في ملف الـ **JSON**

- الـ **JSON** مستمد من الـ **JS** وهو بديل للـ **XML** وامتداد الملف بيكون **.json**.

ليه بيتم استخدام **JSON** ؟!

- **عشان سهولة في القراءة وسهولة في التعامل معاها لأنها شبه الـ Object** اللي اتعلمناه في قبل كدا
- كمان الـ **JSON** مدعوم في اكثر من لغة برمجة
- تقدر تحول الـ **JSON Object** دا لـ **JS Object** والعكس

JSON	XML
شبة ال JS Object في الكتابة	شبة ال HTML في الكتابة
خفيف وسهل في الكتابة	ثقل وصعب نسبيا في الكتابة عن ال JSON
مش بيستخدم Tags	بيستخدم ال Tags زي ال HTML كدا
بيكون قصير والكود بيكون سهل تقرأه	مش قصير في شكله يعني الكود بيكون شكله طويل
بتقدر تستخدم في ال JSON ال Array عادي جدا	مش بتقدر تستخدم فيه ال Array
ال JSON لا يدعم ال Comments	بيدعم ال Comments

في اللينك دا هتلاقي شكل ملف ال JSON و شكل لملف ال XML

اذكر الله ❤️

الزموا ذكر الله ففيه العون على متاعب الحياة ❤️

لا إله الا انت سبحانك إني كنت من الظالمين

أَلَا بِذِكْرِ اللَّهِ تَطْمَئِنُّ الْقُلُوبُ

لا تنسونا من دعواتكم

Never Forget That (**Always Search**)

By: Amr Elsayed

JSON Syntax And Compare With JS Object Lesson #170

JSON Syntax

- Data Added Inside Curly Braces { }
- Data Added With Key : Value
- Key Should Be String Wrapped In Double Quotes
- Data Separated By Comma
- Square Brackets [] For Arrays
- Curly Braces {} For Objects

Available Data Types

- String
- Number
- Object
- Array
- Boolean Values
- null

قبل ما نتكلم عن ال **Syntax** خليني اقولك ان ملف ال **JSON** بيدعم بعض أنواع البيانات زي ال **Strings** وال **Numbers** وال **Objects** وال **Arrays** وال **Boolean Values** وال **Null** كمان هتلاحظ ان مفيش بقا **Function** ولا **Calculation** والحاجات دي لان الملف دا مخصص لمشاركة البيانات

ال Syntax

ال **Syntax** بتاع ال **JSON** بيكون داخل القوسين دول {...} زي ال **JS Object** بالظبط بيكون مكون من **key** و **value** ولكن ال **key** لازم يكون محاط بـ **double quotes** بالشكل التالي دا كدا

```
{
  "string": "Elzero",
  "number": 100,
  "object": {
    "EG": "Cairo",
    "KSA": "Riyadh"
  },
  "array": ["HTML", "CSS", "JS"],
  "boolean": true,
  "null": null
}
```

وهو دا كدا ال **Syntax** بتاع ال **JSON**

What Is API Lesson #171

JSON

- API Overview
- Tools To Test API
- Preview Github API

ال API اختصار لـ **Application Programming Interface** تقدر تقول عليها كدا بالعربي **الواجهة البرمجية** ال API هو الوسيط في عملية مشاركة البيانات وخليني اديك مثال حسب فهمي للموضوع:

لما تدخل مطعم فشكل المطعم والوانه والديكورات و شكل الكراسي هو دا شغل ال **front-end** سليم؟ سليم دي الحاجات اللي بتشوفها بعينك وتقدر تتعامل معاها

اما المطبخ جوا اللي بيتم تحضير الوجبات فيه وتجهيز الأكل دا هنشبهه بال **back-end**

هتقولي اومال فين ال API في الموضوع واستخدامه ؟

هقولك ان ال API دا هو ال **waiter** او **مصطلح الجرسون** اللي بيقبل منك الطلبات هو دا بقا ال API هو دا الشخص اللي بياخد الأوردر من ال **front-end** ويوصله للمطبخ او ال **back-end** والعملية دي بيكون اسمها **request** فيقوم المطبخ يستلم الطلب دا بعدين يجهز الطلب ويديه للجرسون او ال API فيوصله لل **front-end** والعملية دي بيكون اسمها **response** او الرد علي الطلب اللي اتبعت

كل اللي فات دا هو عملية مشاركة البيانات بين ال **front** و ال **back** و اللي بيساعد في المشاركة دي هو ال **API**

بشكل عام بقا لو عندي موقع انا عايز اعرض فيه كل يوم أسعار العملات والاختلاف بينهم .. هل منطقي إني كل يوم افتح ال **code editor** بتاعي واعدل أسعار العملات دي يوميا كدا بإيدي ؟! اللي هو يا فتاح يا عليم يا رزاق يا كريم اصطبحننا واصططح الملك لله سعر الدولار بقا عامل كام انهارد ؟! D" طبعا دي دُعاة مفيش الكلام دا ولكن الحل الأفضل اني اعتمد علي API في الموقع بتاعي وهو يحدث الأسعار دي يوميا من تلقاء نفسه وانا بس اللي كل اللي عليا ك **front-end** اني اضبط الشكل العام والتنسيقات اللي هيتم عرض البيانات دي فيها دا بالإضافة للتنسيقات اني اقدر طبعا أوصل للبيانات دي وتجيبي واتعامل معاها

ال API فيه منه نوعين **Public API** و **Private API**

طبعا انت لازم هتشوف الفيديو عشان تفهم كل التفاصيل الكثير اللي اتقالت

لينك الدرس فيه اللينكات الي تم عرضها في الفيديو

مثال مضحك على فكرة عمل ال **API** وربطها بين ال **Front-End** وال **Back-End**



لا تنسونا من دعواتكم

Never Forget That (**Always Search**)

By: Amr Elsayed

Parse And Stringify Lesson #172

JSON

- JSON.parse => Convert Text Data To JS Object
- JSON.stringify => Convert JS Object To JSON

احنا في الدرس دا هنتعلم ازاى نحول من **JSON Object** الي **JavaScript Object** والعكس
بص يا صاحبي ال **JSON** دا بيكون **Text Format** زي ما عرفنا مسبقا يعني نوعه لازم هيكون **String**
خلي بقا المعلومة دي دلوقتي معاك هنرجعها ال **JSON** نوعه **string** بأكد تاني أهو

يعني علي سبيل المثال دا **JSON Object** نوعه **String** لانه بيكون **Text Format**

```
// Get From Server
const myJsonObjectFromServer = '{"Username": "Osama", "Age": 39 }';
console.log(typeof myJsonObjectFromServer);
console.log(myJsonObjectFromServer);
```

string

```
{"Username": "Osama", "Age": 39 }
```

عايز بقا احول ال **JSON** دا لـ **JS Object** وعشان التحويل دا يتم بستخدم **Method** اسمها **JSON.parse**
وجواها بحد اسم ال **JSON Object**

```
// Convert JSON To JS Object
const myJsObject = JSON.parse(myJsonObjectFromServer);
console.log(typeof myJsObject);
console.log(myJsObject);
```

object

```
► {Username: 'Osama', Age: 39}
```

طيب ليه احوله أصلا من **JSON** لـ **JS**؟! عشان من خلال التحويل دا اعدل البيانات او اعملها **Update** وهي نوعها **JS Object** عادي بسهولة وبعدين احولها تاني من **JS Object** لـ **JSON Object** بعد التعديل ما يتم

التعديل دا ممكن اعمله سواء بستخدم الـ **Dot Notation** زي كدا

```
myJsonObject.Username = "Elzero";  
myJsonObject.Age = 40;
```

أو بستخدم الـ **Bracket Notation** زي كدا

```
myJsonObject["Username"] = "Elzero";  
myJsonObject["Age"] = 40;
```

أهو في كلا الحالتين البيانات حصلها **Update** واتعدلت

```
console.log(myJsonObject);
```

```
► {Username: 'Elzero', Age: 40}
```

بعد التعديل هحول البيانات دي تاني من **JS Object** الي **JSON Object** ودا بيتم من خلال **Method** أسمها **JSON.stringify**

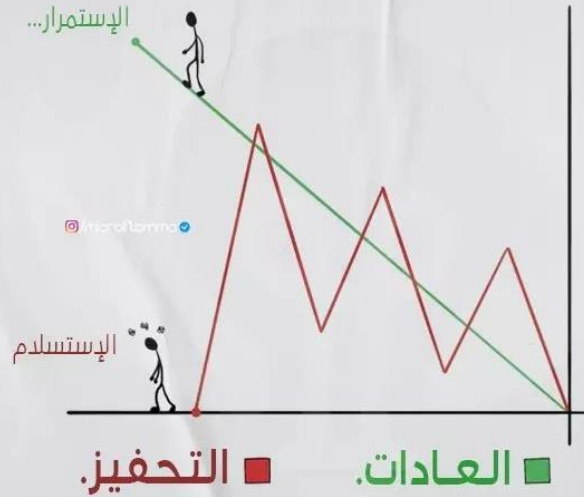
```
// Send To Server  
const myJsonObjectToServer = JSON.stringify(myJsonObject);  
console.log(typeof myJsonObjectToServer);  
console.log(myJsonObjectToServer);
```

```
string
```

```
{"Username": "Elzero", "Age": 40}
```


♥ عبارات تحفيزية ♥

4-التحفيز يجعلك تبدأ لكن العادات تجعلك تستمر:



بناء عادات يومية والاستمرار عليك هينجحك بالعافية 😊

♥ اذكر الله

الزموا ذكر الله ففيه العون على متاعب الحياة ♥

حَسْبِيَ اللَّهُ لَا إِلَهَ إِلَّا هُوَ عَلَيْهِ تَوَكَّلْتُ وَهُوَ رَبُّ الْعَرْشِ الْعَظِيمِ

يا حي يا قيوم برحمتك استغيث أصلح لي شأني كله ولا تكلني الي نفسي طرفة عين
الحمد لله بلا سبب، الحمد لله بلا طلب، الحمد لله بلا عدد، الحمد لله حتى يرضى.

أَلَا بِذِكْرِ اللَّهِ تَطْمَئِنُّ الْقُلُوبُ

لا تنسونا من دعواتكم

Never Forget That (**Always Search**)

By: Amr Elsayed

Asynchronous vs Synchronous Programming Lesson #173

To Understand Ajax, Fetch, Promises

Asynchronous vs Synchronous Programming

- Meaning

Synchronous

- Operations Runs in Sequence
- Each Operation Must Wait For The Previous One To Complete
- Story From Real Life

Asynchronous

- Operations Runs In Parallel
- This Means That An Operation Can Occur while Another One Is Still Being Processed
- Story From Real Life

- Facebook As Example

- Simulation

Search

- JavaScript Is A Single-Threaded
- Multi Threaded Languages

في الدرس دا انا عندي مفاهيمين لابد من اني افهمهم عشان وانا داخل اتعلم ال AJAX فيما بعد وال Fetch
أكون فاهم الدنيا اكتر واكثر

المفهومين دول هما (ال **Synchronous** متزامن -- وال **Asynchronous** غير متزامن)

ال **Synchronous** المتزامن فهو عبارة عن:

1- مجموعة من ال **Operations** او العمليات بتتم بتسلسل او ترتيب

2- كل عملية هتتم لسه لازم تنتظر العملية اللي قبلها لحد ما تخلص للأخر ومينفعش عملياتين يشتغلوا سوا

مثال علي ال **Synchronous**:

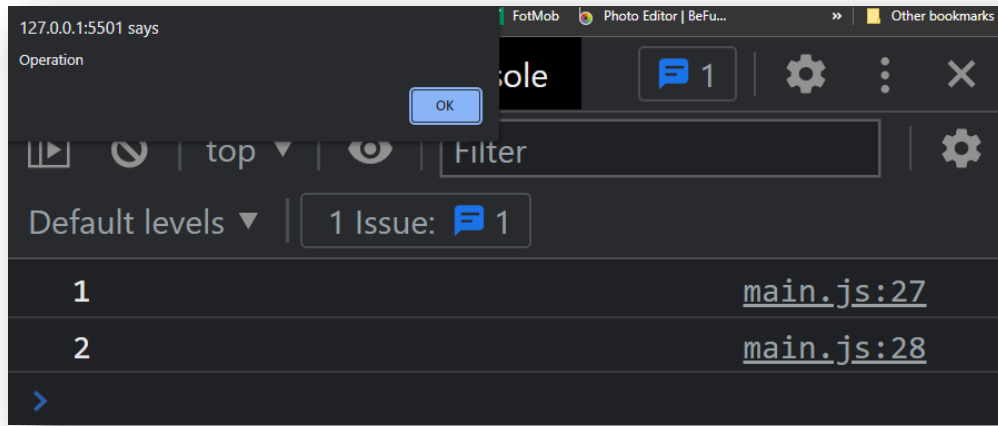
لو حضرتك دخلت مترو الانفاق فلقيت صف طويل جدا للناس اللي عايضة تقطع تذاكر للقطر فحضرتك بتقف
في الصف تنتظر دورك وماشي في تسلسل واحد مع الناس اللي قبلك والناس اللي بعدك واكيد مينفعش تسبب
دورك او تقطع تذكرة قبل ما الناس اللي قبلك تخلص هو دا مفهوم ال **Synchronous** انه متزامن مع بعضه
ومفيش عملية ينفع تسبق التانيه او تحصل قبل ما اللي قدامها تخلص خالص

مثال علي ال **Synchronous** المتزامن بال Code

```
// Synchronous
```

```
console.log("1");  
console.log("2");  
window.alert("Operation");  
console.log("3");
```

العمليات دي هتتم بالترتيب والعملية الي هتطبع الرقم 3 هتقف تماما لحد ما ال **Alert** يشتغل ويخلص للأخر



اما ال **Asynchronous** غير المتزامن فهو عبارة عن:

- 1- مجموعة من ال **Operations** او العمليات بتتم بشكل **In Parallel** او بالتوازي مع بعض
- 2- مش شرط ان كل عملية تستني الي قبلها عشان تبدأ .. لا ممكن اكثر من عملية تتم في نفس الوقت عادي

مثال علي **Asynchronous**:

لو انا دخلت مطعم ولقيت فيه تراييزات كتير جدا و اكثر من **Waiter** فالعادي ان كل **Waiter** بيروح لكل تراييزة يشوف أي طلبتها وانا بطلب الأكل ممكن التراييزة الي ورايا تطلب في نفس الوقت والي قدامي والي جنبي كلنا في وقت واحد ممكن نطلب أكل سوا عادي

مثال علي ال **Asynchronous** غير متزامن بال Code

```
// Asynchronous

console.log("1");
console.log("2");
setTimeout(() => console.log("Operation"), 3000);
console.log("3");
```

1

2

3

Operation

الي حصل انه طبع الأرقام عادي وال **setTimeout** كانت شغاله في نفس الوقت وطبعت كلمة "**Operation**" بعد 3 ثواني يعني العملية دي تمت عادي واشتغلت اثناء ما كل العمليات الثانية كانت شغالة ولسه هتفهم أكثر

Call Stack And Web API Lesson #174

To Understand Ajax, Fetch, Promises

Call Stack || Stack Trace

- JavaScript Engine Uses A Call Stack To Manage Execution Contexts
- Mechanism To Make The Interpreter Track Your Calls
- When Function Called It Added To The Stack
- When Function Executed It Removed From The Stack
- After Function Is Finished Executing The Interpreter Continue From The Last Point
- Work Using LIFO Principle => Last In First Out
- Code Execution Is Synchronous.
- Call Stack Detect Web API Methods And Leave It To The Browser To Handle It

Web API

- Methods Available From The Environment => Browser

بص يا صديقي **عشان أبسط الموضوع** دا قدر الإمكان ال **JS** ليها **Engine** او محرك بيشغل ال **Code** بتاعها ال **Engine** دا بيكون **Single Threaded** يعني أي؟!

يعني لما بييجي يشغل **Code** معين بيشغله سطر ورا سطر سطر ورا سطر بطريقة **synchronous** متزامنة يعني كل عملية لازم تستني الي قبلها تتم الأول قبل ما تبدأ تشتغل

لحد كذا الأمور واضحة وتمام !

طيب جميل العمليات الي بتتم ورا بعض دي بتتخط كذا في حاجة اسمها **Call Stack** يعني اعتبره صندوق كذا كبير دخل جواه كل العملية بالترتيب ورا بعض

طيب لو عندي عملية هتاخذ وقت كبير جدا زي إني مثلا أقوله **SetTimeout** لحاجة هتاخذ 10 ثواني مثلا ؟ عشان تتطبع ومحطوبة وسط كذا عملية **Console.log** أي الي بيحصل هنا ؟!

الي بيحصل ان ال **JS** بتاخذ العملية بتاعت ال **SetTimeout** دي تخرجها من ال **Call Stack** بتاعها خالص وتوديتها مكان ثاني اسمه **Web API** يقوم ال **Web API** دا يهندل العملية دي مع نفسه وبعدين يروح يقول لل **JS** يا **JS** انا خلصت العملية الي انتي بعتهاالي ها تستلمها مني ؟!

ال **JS** تقوله استني اما ابص على ال **Call Stack** او الصندوق بتاعي اشوفه فاضي وخلص كل ال **Console.log** الي كانت موجودة مع ال **setTimeout** ولا لا !

تروح ال **JS** تبص تلاقي ان ال **call stack** بقي فاضي فعلا وكل العمليات الي كانت مع ال **setTimeout** تمت خلاص فتروح ال **JS** تقول لل **Web API** ابعت ال **setTimeout** عندي اشغلها بقا والعلمية دي كلها على بعضها كذا تندرج تحت مصطلح ال **JS Runtime**

من ضمن الحاجات الي ال **Web API** بيساعد بيها ال **JS** هي ال **DOM** و ال **setTimeout** وغيرهم حاجات ثانية كتير ال **Web API** بيقدّمها لل **JS**

آخر حاجة ال **Web API** دا يا صديقي حاجه خاصة بالمتصفحات نفسها مش بلغة برمجة معينة بس كذا

كل دا اعتبره مقدمة للدرس نفسه اللي بيتشرح في الفيديو!

طيب ال **Call Stack** نفسه دا بيشتغل ازاى او أي هي المنهجية اللي بيتبعها في شغله في ترتيب الكود وتشغيله؟!

ال **Call Stack** دا ياسيدي بيتبع منهجية اسمها **LIFO** او **Last In First Out**

طيب دا معناه أي؟! معناه ان اخر حاجة بتدخل الصندوق دي واللي هي بالمنطق كدا هتكون اول حاجة فوق لما تفتح الصندوق مع انها اخر حاجة محطوبة ولكن هي بتكون برضو أول حاجة تطلع من الصندوق دا هو دا مفهوم ال **LIFO** برضو مفهيمش دا بيتم ازاى في ال **JS** خليني اديلك مثال

```
function one() {
  console.log("One");
}

function two() {
  one();
  console.log("Two");
}

function three() {
  two();
  console.log("Three");
}

three();
```

أي اللي هيحصل هنا؟! تعالي نفتح **الصندوق** او ال **Call Stack** ونشوف مين هيخش الأول ومين هيطلع الأول

دلوقتي انا عندي **3 Functions**

ال **one()** <= function بتطبع كلمة "One"

ال **two()** <= function أولا بتشغل ال **one()** وبعدين بتطبع كلمة "Two"

ال **three()** <= function أولا بتشغل ال **two()** وبعدين بتطبع كلمة "Three"

بعدين بشغل **three()** <= function آخر سطر في المثال اللي فوق

أي اللي هيحصل أو ال **Code** هيشغل ازاى؟!

اول حاجة هنطبق مبدأ **LIFO** اللي هو **Last In First Out** يبقى هنشوف أي آخر حاجة دخلت عشان تطلع هي أول حاجة

لما نشغل ال `three()` <= function من خلال الكود `three()` آخر سطر في المثال اللي فوق

ال **JS** هتعمل تشغيل لل `three()` هتلاقي أول حاجة جواه انه يعمل تشغيل ل `function two()` وبعدين يطبع كلمة "Three"

يبقى ال `three()` دي هي أول حاجة اشتغلت و أول حاجة دخلت ال `stack` بتاعي

هتشتغل بقا ال `two()` بناءا علي امر من ال `three()` لما تشتغل ال `two()` هتلاقي أول حاجة مطلوبة فيها انها تشتغل ال `one()` بعددين تطبع "Two"

يبقى ال `two()` دي ثاني حاجة اشتغلت هتبقى ثاني حاجة تخش الصندوق او ال `stack` يعني

هتشتغل بقا ال `one()` بناءا علي امر من ال `two()` لما تشتغل ال `one()` هتلاقي أول حاجة مطلوبة فيها انها تطبع كلمة "one"

يبقى ال `one()` دي ثالث حاجة اشتغلت اذا هتبقى ثالث حاجة تخش الصندوق او ال `stack`

أي اللي هيشغل آخر حاجة هي ال `console.log("One")` يبقى ال `console` دي اخر حاجة هتخس ال `stack` لانها اخر حاجة هتشتغل بعد ما ال `one()` أصلا نفسها تشتغل لان ال `one()` كدا كدا هي اللي هتخلي ال `console.log` اللي موجود جواها يشتغل وبكدا هيكون دا اخر حاجة اشتغلت في ال `code` واخر حاجة هتخس ال `stack` لانها اخر عملية في الاخر خالص تمت واخر عمليه دي هتخس الصندوق هتبقى هي اللي علي وش الصندوق

النتيجة هتطلع من ال `stack` بقا بمنهج ال **LIFO** أي اخر حاجة دخلت ؟! لسه قايل فوق من سطرين اخر حاجة دخلت أي ؟! ال `console.log("One")` يبقى هي أول حاجة تطلع من ال `stack` وتشتغل

طيب أي اللي تحتها اللي تحتها كان ال `one()` ال `one()` بالفعل اشتغل وعمل ال `console.log("One")` يبقى يطلع هو كمان من ال `stack`

طيب أي اللي كان تحتها كانت ال `two()` جواها امر اول حاجة انها تشتغل ال `one()` وبما ان ال `one()` اشتغل بالفعل وعمل اللي عليه يبقى يتشال بعد كدا هتلاقي ال `console.log("Two")` فدا اللي يطلع برا ال `stack` ويشغل

طيب أي اللي تحت ال `two()` كان ال `three()` وجوها أمر اول حاجة تشتغل ال `two()` خلاص ال `two()` اشتغلت وعملت اللي عليها خس اللي بعده اللي بعدها في ال `three()` انها تطبع ال `console.log("Three")`

يبقى دا آخر حاجة هتخرج

يبقى الترتيب كدا :

`console.log("One") => One`

`console.log("Two") => Two`

`console.log("Three") => Three`

One

Two

Three

طيب بص ثاني عشان حاسس انك ممكن تكون **confused** شوية أو مرتبك يعني خليني أوضح أكثر وأكثر

هي **function one()** دي عشان تشتغل محتاجة أي؟! محتاجة انه يتعملها **Call** او استدعاء

طيب امته تم استدعاؤها .. **تم استدعاؤها** في ال **function two()** طيب أصلا **function two()** دي عشان تشتغل محتاجة أي؟! محتاجة انه يتعملها **Call** او استدعاء هي كمان

طيب امته تم استدعاؤها .. **تم استدعاؤها** في ال **function three()** طيب أصلا **function three()** دي عشان تشتغل محتاجة أي؟! محتاجة انه يتعملها **Call** او استدعاء هي كمان

جيت اخر سطر عملت لل **function three()** استدعاء

يبقي مين أول حاجة اشتغلت وشغلت الباقي؟!

function three() مش كدا يبغي **function three()** دي اول واحدة دخلت ال **stack** او الصندوق لما دخلت ال **stack** الي حصل انها شغلت **function two()** وبعدين مطلوب منها تطبيع كلمة "Three"

بعدها حصل أي؟!

اشتغلت **function two()** ودي كانت ثاني حاجة تخش ال **stack** لما دخلت ال **stack** الي حصل انها شغلت **function one()** وبعدين مطلوب منها تطبيع كلمة "Two"

بعدها حصل أي؟!

اشتغلت **function one()** ودي كانت ثالث حاجة تخش ال **stack** لما دخلت ال **stack** الي حصل انها لقت المطلوب منها تطبع كلمة "one"

يبقي طباعة كلمة "one" أو **console.log("one")** بمعنى ادق زي ما الكود مكتوب هي رابع حاجة تخش ال **stack** او بمعنى ادق هي كدا اخر حاجة تخش ال **Stack**

تعالى نطبق مبدأ ال **LIFO <== Last In First Out** يعني اخر حاجة تدخل الصندوق تكون أول حاجة تخرج منه

يبقي اخر حاجة دخلت هي ال **console.log("one")** يبغي هي أول واحدة تطلع من ال **stack** وكلمة **One** تتطبع

بعدين دخل قبلها **function one()** كدا كدا **function one()** اشتغلت خلاص وطبعت كلمة "One" يبغي ولا كأنها موجودة مع السلامة انتي كدا كدا عملي شغلك وطبعتي كلمة "One"

قبل ال **function one()** دخلت ال **function two()** كان مطلوب منها تشغل **function one()** خلاص يا **برنس** اشتغلت والمصحف يبغي طريق السلامة أي الي باقي من **function two()** انها تطبع كلمة "Two"

قبل ال **function two()** دخلت ال **function three()** وكانت دي اول واحدة تدخل ال **stack** يبغي دلوقتي زي مانت شايف كدا هتكون اخر واحدة تطلع من ال **stack** المهم كان مطلوب منها تشغل **function two()** خلاص يا **برنس** اشتغلت وعملت الي عليها يبغي مع السلامة من ال **stack** أي الي باقي من **function three()** انها تطبع كلمة "three"

يبقي بالترتيب اطبعت كلمة "One" بعدها ثاني سطر كلمة "Two" بعدها ثالث سطر اطبعت كلمة "Three"

وبس كدا شكرا ويبقي ال **Stack** طبق مبدأ ال **LIFO <== Last In First Out**

طيب فافكر أول صفحة في الدرس فوق لما شرحنا شوية حاجات عن ال **Web API** تيجي ناخذ مثال ونشره ؟!

```
setTimeout(() => {  
  console.log("Web API");  
}, 0);  
  
function one() {  
  console.log("One");  
}  
function two() {  
  one();  
  console.log("Two");  
}  
function three() {  
  two();  
  console.log("Three");  
}  
  
three();
```

دلوقتي ال **code** دا نفس اللي فات بس انا زوت **setTimeout** في الأول هتاخذ **0** من **الثواني** عشان تنفذ تفكر كدا الرسالة اللي جواها دي هتتبع فين ؟!

One
Two
Three
Web API

اطبعت في الاخر مع انها مكتوبه في الأول ... طيب دا حصل ليه أو أي اللي حصل ؟!

اللي حصل ان ال **JS** خلصت الحاجات بتاعتها اللي هما ال **Functions** الموجودة ودخلتهم ال **stack** زي ما اتشرح فوق

اما ال **setTimeout** دي ال **JS** بعتهها لل **Web API** يتعامل معاها

لما خلصها بعتهها تاني لل **JS** قامت ال **JS** بصت في الصندوق بتعها لقت ان ال **Functions** اللي كانوا عندها

خلاص طلعوها من الصندوق واطبعوا فردت علي ال **Web API** ابعت اللي عندك

ال **Web API** بعت بقا ال **setTimeout** فدخلت ال **stack** بعد ما كل ال **Functions** خرجت بعدين اطبعت تحتهم في الآخر

Event Loop And Callback Queue Lesson #175

To Understand Ajax, Fetch, Promises

Event Loop + Callback Queue

Story

- JavaScript Is A Single Threaded Language "All Operations Executed in Single Thread"
- Call Stack Track All Calls
- Every Function Is Done Its Popped Out
- When You Call Asynchronous Function It Sent To Browser API
- Asynchronous Function Like Settimeout Start Its Own Thread
- Browser API Act As A Second Thread
- API Finish Waiting And Send Back The Function For Processing
- Browser API Add The Callback To Callback Queue
- Event Loop Wait For Call Stack To Be Empty
- Event Loop Get Callback From Callback Queue And Add It To Call Stack
- Callback Queue Follow FIFO "First In First Out" Rule

بص ياعم الحج الموضوع **بسيط** وهتشوف دا بنفسك دلوقتي وانا بشرحك تاني بعد ما تشوف فيديو الدرس

دلوقتي انا عندي **Call Stack** شبهته بالصندوق كان ماشي بمنهج **LIFO** يعني **Last In First Out** يعني اخر حاجة تخش الصندوق دا هتبقى هي أول حاجة تطلع م الصندوق دا حلوكدا؟! حلو أوي

واتفقنا مسبقا اني عندي نوعين مزامنة (نوع **Asynchronous** غير متزامن -- ونوع **synchronous** متزامن) وعرفنا الفرق بينهم قبل كدا

واتفقنا برضو ان النوع ال **Asynchronous** غير متزامن بيتم من خلال ال **Web API** لأن ال **JS** بتكون **Single threaded** وبشتغل بشكل **synchronous** متزامن

اتفقنا كمان ان ال **JS** وهي بتشغل الكود بتدي الحاجات ال **Asynchronous** دي لل **Web API** يتعامل معاها هو وبعدين يبتعها تاني لل **JS** ف ال **JS** بتقوله استني يا **Web API** اشوف الصندوق بتاعي خلص اللي جواه ولا لسه في حاجات شغاله؟! تبص في الصندوق تلاقيها لسه مخلصتش تقوم تقول لل **Web API** استني شوية

هنا هيجي السؤال بتاع الدرس العمليات اللي تمت من خلال ال **Web API** دي لما بتكون منتظرة دورها عشان تخش الصندوق او ال **Call Stack** وبعدين تبدأ تطلع منه بتنتظر فين وبتخش ازاى ال **Stack** واي المنهج اللي بتخش ال **Stack** بيه؟ هو دا سؤال حلقتنا انهاردا: "D تعالي اقولك بقا

ال **Web API** دا يا صديقي بيكون ليه صندوق خاص بيه هو كمان بينفذ فيه العمليات بتاعته بعيدا عن ال **stack** او الصندوق بتاع ال **JS**

بعد ما بيخلص ال **Web API** الشغل بتاعه بيقوم بعته لقائمة الانتظار وهنتكلم عن قائمة الانتظار دي دلوقتي متقلقش

أي بقا قصة قائمة الانتظار دي ؟!

بص يا سيدي ال **Web API** بعد ما بتخلص شغلها بتبعته لقائمة انتظار اسمها **Callback Queue** بتبقي برضو زي صندوق كدا مجمع كل الشغل الي تم من ال **Web API** بالترتيب ... الي دخل صندوق ال **Web API** وخلص بسرعة وطلع منها الأول ... يروح لقائمة الانتظار او ال **Callback Queue** ال **Callback Queue** دي ليها منهج او قاعدة هي كمان اسمها **FIFO** او **First in First Out**

أي بقا المنهج او القاعدة دي ؟! ... ال **Callback Queue** بيقولك اول حاجة هتخش عندي او أول حاجة هتجيلي من صندوق ال **Web API** هتكون هي أول حاجة تطلع برضو من عندي وتروح لل **Call Stack** بتاع ال **JS** عشان يظهرها ... اذًا يبقى **FIFO** دا هو منهج ال **Callback Queue**

تقدر تقول ان ال **Callback Queue** ماشي بمبدأ طابور العيش كدا تروح بدري والأول تاخد العيش بدري والأول

طيب أي هو ال **Event Loop**

بص يا معلم ال **Callback Queue** دي بيشغلها حاجه اسمها **Event Loop** تقدر تقول علي ال **Event Loop** دا هو القفل بتاع الصندوق بيفتح اول ما ال **Stack** بتاع ال **JS** يخلص شغله وبيفضل مقفول طول ما ال **stack** لسه مليون يعني من الاخر ال **Event Loop** دا هو الي بينظم مرور البيانات من قائمة الانتظار الي هي ال **Callback Queue** لل **Call Stack** بتاع ال **JS**

تعالى ناخد مثال خارج سياق الفيديو

```
console.log("Hi");

setTimeout(() => {
  console.log("Log This After 20ms");
}, 20);

setTimeout(() => {
  console.log("Log This After 0ms");
}, 0);

setTimeout(() => {
  console.log("Log This After 10ms");
}, 10);

console.log("Bye");
```

دلو قتي ال **Code** دا مكون من **console.log 2** واحدة تطبع "**Hi**" والثانية تطبع "**Bye**"
وبينهم فيه **3 setTimeout** بتوقيات مختلفة **الأولي** هتطبع رسالة بعد **20ms** و **الثانية** بعد **0ms**
و **الثالثة** بعد **10ms** تيجي نشوف النتيجة ؟!

```
Hi
Bye
Log This After 0ms
Log This After 10ms
Log This After 20ms
```

أي الي حصل ؟! الي حصل ان ال **JS** خلصت الحاجات بتاعتها الي هما ال **Console** **اليتين** الي موجودين
لان **دول** اول **حاجة** دخلو ال **Call Stack** وخرجوا منه
اما ال **setTimeout 3** دي ال **JS** بعتهها لل **Web API** يتعامل معاها
فدخلو ال **3 setTimeout** عند ال **Web API** بص عليهم لقا الي هي **0ms** خلصت الأول طلعتها الأول وحطها في
ال **Callback Queue**

بعدين لقي الي هي **10ms** خلصت **الثانية** طلعتها **تاني حاجة** وحطها في ال **Callback Queue**
بعدين لقي الي هي **20ms** خلصت **الثالثة** طلعتها **تالت حاجة** وحطها في ال **Callback Queue**

بعدين جت ال **Event Loop** القفل الي موجود على صندوق ال **Callback Queue**
بص على ال **Call Stack** بتاع ال **JS** لقاها طبع كلمة **Hi** وطبع كمان كلمة **Bye** قام فاتح القفل
وشاف مين اول حاجه كانت في **الطابور** او ال **Callback Queue** قام مطبق مبدأ **FIFO => First In First Out**
شاف مين دخل الأول **قام مطلعه هو الأول** ودخله لل **Call Stack** بعدين دخل الي وراها بالترتيب الي كانوا بيه
في الطابور

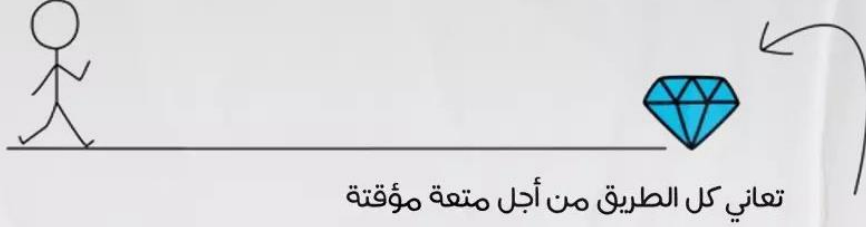
والي دخل الأول كانت ال **0ms** و **الثانية** كانت **10ms** و **الثالثة** **20ms**

مع ان ترتيبهم في ال **code** بتاع المثال مش بالترتيب ولكن كلهم راحو لل **web API** ال **0ms** خلصت الأول
والي بعدها والي بعدها ودخلو طابور الانتظار كذا بالترتيب الي خلص الأول دخل الأول

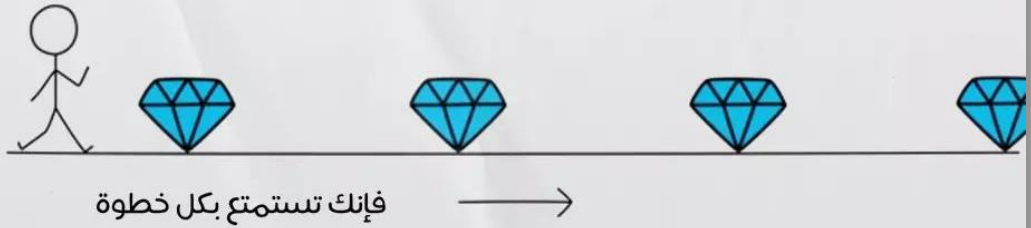
طابور الانتظار بقا او **Callback Queue** موجود عليه **قفل** اسمه **Event Loop** بص علي **الطابور لقي ال 0ms**
دخل اول واحد قام مطلعه لل **Call Stack** بتاع ال **JS** **أول واحد** وهكذا بالترتيب

♥ راقب لي ♥

عندما تقوم بالأشياء من أجل حافز



عندما تقوم بالأشياء من أجل هذه الأشياء نفسها



♥ حب الي بتعمله عشان تستمتع بيه حب الرحلة الي انت ماشي فيها عشان متملش ♥

اذكر الله ♥

سبحان الله و الحمد لله و لا اله الا الله و الله اكبر و لا حول ولا قوة الا بالله
اللهم صلّ وسلم وزد وبارك علي نبينا محمد ﷺ

ألا بِذِكْرِ اللَّهِ تَطْمَئِنُّ الْقُلُوبُ

لا تنسونا من دعواتكم

Never Forget That (Always Search)

By: Amr Elsayed

What Is AJAX And Network Informations Lesson #176

AJAX

- Asynchronous JavaScript And XML
- Approach To Use Many Technologies Together [HTML, CSS, Js, DOM]
- It Use "XMLHttpRequest" Object To Interact With The Server
- You Can Fetch Data Or Send Data Without Page Refresh
- Examples
 - Youtube Studio
 - Google Drive
 - Upload Article Photo
 - Form Check Name

```
Test new XMLHttpRequest();  
Request And Response  
Status Code
```

انهاردا هنتكلم عن ال **AJAX** وهو اختصار ل **Asynchronous JavaScript And XML** ال **AJAX** مش **Framework** ولا **لغة برمجة** ولا **تقنية** هو مجرد وصلة كدا **بتتيحلك** استخدام اكر من تقنية مع بعض زي ال **HTML, CSS, JS, DOM**

ال **AJAX** دا ليه **Object** خاص بيه كدا اسمه **XMLHttpRequest()** عشان يتفاعل مع ال **Server** عشان تقدر تبعت بيانات **للسيرفر** و **تستقبل بيانات من السيرفر** من غير ما تحتاج تعمل **Refresh**

خليني اديك مثال بسيط علي النقطة بتاعت انك مش محتاج تعمل **Refresh** دي

- تخيل معايا لو انت فاتح ال **Google drive** مثلا وعندك **Document** بتكتب فيه شوية كلام ف وانت بتكتب بيطلعك كل فترة رسالة كدا بتقولك ان اللي انت كتبتة اتعمله **Save** لكن مش منطقي يعمل **Refresh** لان لو دا حصل الكلام الي انت كتابته كله هيضيع ويطيير

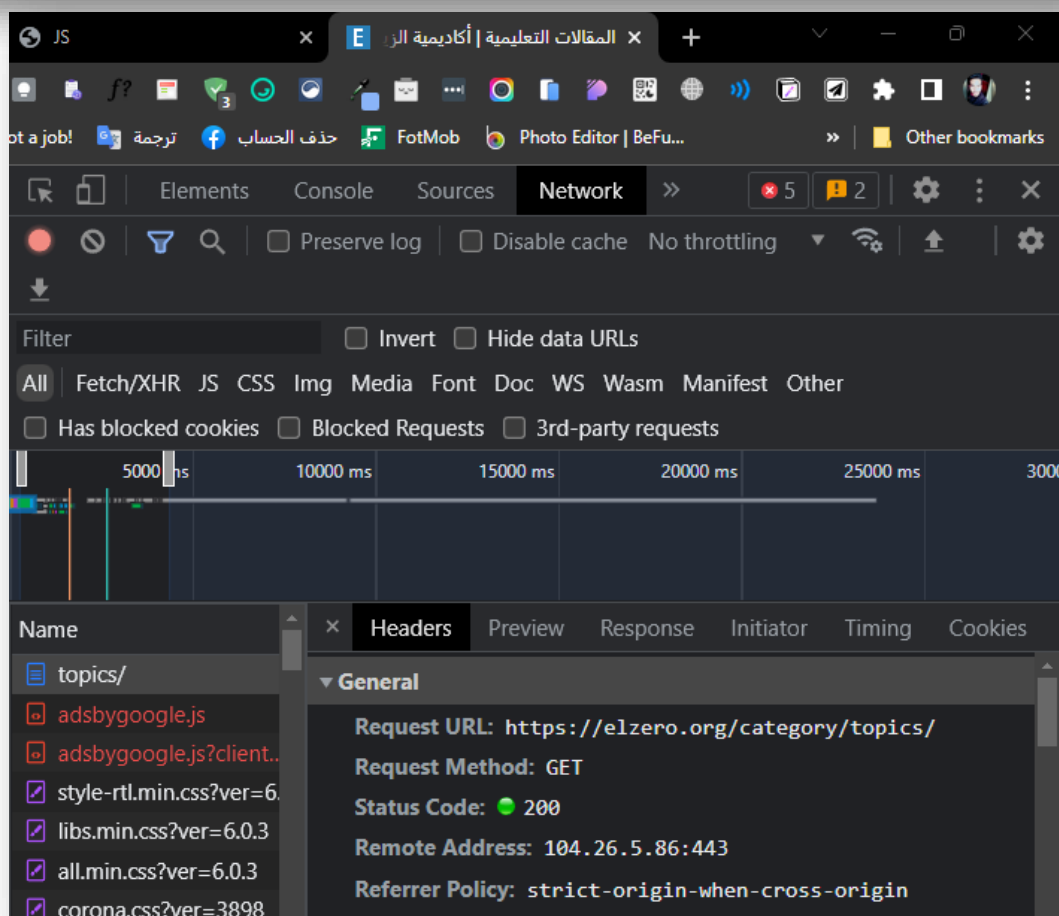
- نفس المثال لو بتكتب **مقال** في **موقع معين** وعازي **ترفع صورة** وانت بتكتب المقال **مش منطقي ان الصورة** بعد ما تترفع ان الموقع دا يعمل **Refresh** لانه ممكن يطير الي انت بتكتبه وكل دا وانت بتكتب عادي زي مانت

- لو انت في موقع معين وفيه **Form** وانت بتملي ال **Form** دا مكان ال **user Input** دا هو مش مستني انك تعمل **submit** لا هو بياخد ال **data** الي انت كتبتة دي يروح يبص عليها بصة في ال **Server** يشوفها لو في حد تاني ليه نفس ال **user name** بقولك ان ال **name** دا مش متاح فانت تغيره فيروح يكرر نفس العملية ويرجعك بالرد فمش منطقي انك مرة يروح يبص ع الداتا ويرجع يحصل **Refresh** وتضيع باقي الكلام الي اتكتب في ال **Form**

لما تحب تفتح موقع بيحصل حاجه اسمها **Request** او **طلب** بتطلب بيانات الصفحة دي وبعد كدا بيرجعك من ال **Request** دا **استجابة** او **Response** فاحنا بقا هنشوف بعض الاكواد الي هتعرفنا ال **response** دا وصلنا سليم ونوعه ايه ؟!

```
let req = new XMLHttpRequest();
console.log(req);
```

```
main.js:20
XMLHttpRequest {onreadystatechange: null, readyState: 0, timeout: 0, withCredentials: false, upload: XMLHttpRequestUpload, ...} i
  onabort: null
  onerror: null
  onload: null
  onloadend: null
  onloadstart: null
  onprogress: null
  onreadystatechange: null
  ontimeout: null
  readyState: 0
  response: ""
  responseText: ""
  responseType: ""
  responseURL: ""
  responseXML: null
  status: 0
  statusText: ""
  timeout: 0
  ▶ upload: XMLHttpRequestUpload {onloadstart: null, onprogress: null, onabort:
    withCredentials: false
  ▶ [[Prototype]]: XMLHttpRequest
```



انت بس كل اللي محتاج تفهمه حاليا ان في حاجة اسمها **Status Code** وانه أي موقع بتطلبه من خلال **Request** بيكون فيه **Response** بيرجع

Request and Response From Real API Lesson #177

Ajax

- Ready State => Status Of The Request
 - [0] Request Not Initialized
 - [1] Server Connection Established
 - [2] Request Received
 - [3] Processing Request
 - [4] Request Is Finished And Response Is Ready
- Status
 - [200] Response Is Successful
 - [404] Not Found

```
let myRequest = new XMLHttpRequest();
myRequest.open("GET", "https://api.github.com/users/elzeroweb school/repos");
myRequest.send();
console.log(myRequest);
```

عملت Request علي API من ال Github

استخدمت ال `myRequest.open()` الي بيكون جواها نوع ال **Request Method** يعني **GET** او **POST** وال **URL** بتاع ال **API** الي هسحب منه البيانات وبعد كدا هل ال **Call** دا **Synchronous** ولا **Asynchronous** ولو فيه **Authentication** علي ال **API** فمحتاجين نكتبه ولو فيه **Username** و **Password**

بعد كدا استخدمت `myRequest.send()` عشان يبعثلي البيانات بتاعت ال **API** او يبعثلي **Response**

```
main.js:17
XMLHttpRequest {onreadystatechange: null, readyState: 1, timeout: 0, withCredentials: false,
  upload: XMLHttpRequestUpload, ...} ⓘ
  abort: null
  onerror: null
  onload: null
  onloadend: null
  onloadstart: null
  onprogress: null
  ▶ onreadystatechange: f ()
  ontimeout: null
  ▶ readyState: 4
  response: "[\n {\n   \"id\": 222275692,\n   \"node_id\": Show more (204 kB) Copy
  responseText: "[\n {\n   \"id\": 222275692,\n   \"node_id\": Show more (204 kB) Copy
  responseType: ""
  responseURL: "https://api.github.com/users/elzeroweb school/repos"
  responseXML: null
  ▶ status: 200
  statusText: ""
  timeout: 0
  ▶ upload: XMLHttpRequestUpload {onloadstart: null, onprogress: null, onabort: null, onerror: r
    withCredentials: false
  ▶ [[Prototype]]: XMLHttpRequest
```

لقيت في ال **Response** حاجتين دلوقتي انا محتاج اعرفهم ومهمين جدا زي ال **readyState** قيمته بـ 4 وال **status code** قيمتها 200

اما ال **readyState** دي بالقيمة 4 فهي بتشير الي الأرقام من 0 الي 4 حيث ان :

- 0- الطلب لسه مبدأش او متفعلش
- 1- فيه اتصال حصل مع ال **Server**
- 2- الطلب تم استلامه من ال **Server**
- 3- بيتم معالجة الطلب
- 4- الطلب انتهى خلاص والرد او **Response** جاهز

وال **status code** بالقيمة 200 فدي معناه ان ال **Response** نجح ورجعلي ببيانات اقدر استخدمها

خلي بالك ان ال **readyState** بالقيمة 4 مش دليل علي ان ال **status code** نجح فعليا انه يرجع بيانات يعني خلي بالك عشان متخلطش بين الاثنين فمش معني ان ال **readyState** بالقيمة 4 انه كمان الرد جاب بيانات ابدأ بس القيمة 4 دليل اني فيه **response** جالي بغض النظر ان الرد دا فيه بيانات بقا او لا

ليه لازم اعمل **check** واتمم الحاجتين دول تحديدا لانهم بيكملوا بعض وبيكونو دليل علي ان ال **Request** تمام وال **Response** كمان تمام وابدأ اكمل شغلي لان لو فيهم واحد مش تمام كل الشغل اللي جاي بعد كدا هيبقي مش تمام

بعمل بقا عملية ال **Check** دي ازاي ؟!

هتلاقي ال **screen** فوق اخر **Screen** من ال **Console** نزلتها فيها ايفنت اسمه **onreadystatechange** هو دا الايفنت اللي بتستخدمه عشان اعمل **Check** علي ال **readyState** و ال **status code**

```
myRequest.onreadystatechange = function () {  
    console.log(myRequest.readyState);  
    console.log(myRequest.status);  
};
```

```
2  
200  
3  
200  
4  
200
```

```
myRequest.onreadystatechange = function () {  
    // console.log(myRequest.readyState);  
    // console.log(myRequest.status);  
    if (this.readyState === 4 && this.status === 200) {  
        console.log(this.responseText);  
    }  
};
```

فانا عملت **Check** وبأكد عليه في ال **IF** ان الاثنين لازم القيم بتاعتهم تبقي كدا ولو هي كدا فعلا هتعمل أي ؟! ابعثلي ال **responseText** اللي هو بيكون عبارة عن ملف **JSON** كدا كبير

Loop On Data Lesson #178

Ajax
Loop On Data

Search
- Cross Origin API [CORS]
- API Authentication

```
let myRequest = new XMLHttpRequest();
myRequest.open("GET", "https://api.github.com/users/elzerowebSchool/repos");
myRequest.send();
myRequest.onreadystatechange = function () {
  if (this.readyState === 4 && this.status === 200) {
    // console.log(this.responseText);
    let jsData = JSON.parse(this.responseText);
    // console.log(jsData);
    for (let i = 0; i < jsData.length; i++) {
      let div = document.createElement("div");
      let repoName = document.createTextNode(jsData[i].full_name);
      div.appendChild(repoName);
      document.body.appendChild(div);
    }
  }
};
```

انا هنا عملت أي انا دلوقتي كان عندي **JSON Object** فيه كل ال **Repos** الي كانت في اللينك بتاع ال **API** فمسكت ال **JSON Object** دا حولته لـ **JS Object** من خلال ال **JSON.parse()**

خلاص رجعت الداتا بشكل ال **JS Object** بدأت اعمل **Loop** علي الداتا دي كلها بال **length** او عددهم وداخل ال **Looping** انا عملت **div** وعملت **textNode** حطيت فيه ال **JS Object** بتاعي دا وعملت **appendChild** لل **textNode** داخل ال **div** الي عملته وبعدين خدت ال **div** دا بال **textNode** الي جواه عملته هو كمان **appendChild** ليه داخل ال **body** عشان اعرضه في الصفحة

```
ElzeroWebSchool/Ajax
ElzeroWebSchool/Bootstrap3DesignOne
ElzeroWebSchool/Bootstrap3DesignTwo
ElzeroWebSchool/Bootstrap4DesignOne
ElzeroWebSchool/Bootstrap_5_Design_01_Bondi
ElzeroWebSchool/Branching
```

طبعا فيه بيانات كتير جدا تحت انا عرضت جزء منها بس

كل اللي فات دا كان شغل الفيديو انا عملت ال **loop** مع نفسي مرة بال **forEach** ومرة بال **for of** وعملت شوية **Styling** بسيط كدا ودا مثال

```
let myRequest = new XMLHttpRequest();
myRequest.open("GET", "https://api.github.com/users/elzerowebsschool/repos");
myRequest.send();
console.log(myRequest);
myRequest.onreadystatechange = function () {
  if (this.readyState === 4 && this.status === 200) {
    let jsData = JSON.parse(this.responseText);
    console.log(jsData);
    jsData.forEach((data) => {
      console.log(data.full_name);
      let div = document.createElement("div");
      let divText = document.createTextNode(data.name);
      div.appendChild(divText);
      document.body.appendChild(div);
      div.style.cssText =
        `padding: 10px; background: #009688;
        margin: 0px auto 10px; font-size: 20px;
        color: #fff; width: 500px;`;
    });
  }
};
```

Ajax

Bootstrap3DesignOne

Bootstrap3DesignTwo

Bootstrap4DesignOne

Bootstrap_5_Design_01_Bondi

Branching

ContactForm

content

CPlusPlus_Programming

articles.json

```
[
  {
    "id": 1,
    "name": "Amr",
    "title": "Front-End",
    "section": "Web Development",
    "content": "HTML, CSS, JS, React.js"
  },
  {
    "id": 2,
    "name": "Elzero",
    "title": "Full-Stack",
    "section": "Web Development",
    "content": "HTML, CSS, JS, React.js, PHP, Laravel"
  },
  {
    "id": 3,
    "name": "Osama",
    "title": "Back-End",
    "section": "Web Development",
    "content": "JS, Node.js"
  },
  {
    "id": 4,
    "name": "Ahmed",
    "title": "Android",
    "section": "Mobile Development",
    "content": "Kotlin"
  },
  {
    "id": 5,
    "name": "Ali",
    "title": "IOS",
    "section": "Mobile Development",
    "content": "Swift"
  }
]
```



```
// Assignment Two

let myReq = new XMLHttpRequest();
myReq.open("GET", "articles.json");
myReq.send();
console.log(myReq);
myReq.onreadystatechange = function () {
    if (this.readyState === 4 && this.status === 200) {
        console.log(this.responseText);
        console.log("Data Loaded");
    }
};
```

```
// Assignment Three

let myReq = new XMLHttpRequest();
myReq.open("GET", "articles.json");
myReq.send();
console.log(myReq);
myReq.onreadystatechange = function () {
    if (this.readyState === 4 && this.status === 200) {

        let mainData = JSON.parse(this.responseText);
        console.log(mainData);

        mainData.forEach((article) => {
            article["section"] = "all";
        });
        let updatedData = JSON.stringify(mainData);
        console.log(updatedData);
    }
};
```

```
// Assignment Four

let myReq = new XMLHttpRequest();
myReq.open("GET", "articles.json");
myReq.send();
console.log(myReq);
myReq.onreadystatechange = function () {
    if (this.readyState === 4 && this.status === 200) {
        let mainData = JSON.parse(this.responseText);
```


Name: Amr

Title: Front-End

Section: Web Development

Content: HTML, CSS, JS, React.js

Name: Elzero

Title: Full-Stack

Section: Web Development

Content: HTML, CSS, JS, React.js, PHP, Laravel

Name: Osama

Title: Back-End

Section: Web Development

Content: JS, Node.js

Name: Ahmed

Title: Android

Section: Mobile Development

Content: Kotlin

Name: Ali

Title: IOS

Section: Mobile Development

Content: Swift

لا تنسونا من دعواتكم

Never Forget That (Always Search)

By: Amr Elsayed

♥ عبارات تحفيزية ♥



بص لو وقع قدامك الملخص دا وتحديدا الجزء دا في يوم من الأيام .. افكر كلامي كويس وهو إنك مدام مستمر في التعلم، حتى لو بتمشي ببطء فصدقني انت بتتقدم اكر وبتتعلم اكر وخبرتك بتزيد اكر انت بس عشان باصص تحت رجلك .. عامل زي اللي قاعد باصص للساعة ومستغرب ازاى الوقت بيمر بالبطء دا مع إنك لو ركزت في مهارتك وتطورها هتكتشف ان فات سنة وبقيت أجمد بكثير من ما كنت عليه زمان

♥ اذكر الله

♥ الزموا ذكر الله ففيه العون على متاعب الحياة

سبحان الله وبحمده عدد خلقه ورضا نفسه وزنة عرشه ومداد كلماته

اللهم لك الحمد حمدًا طيبًا كثيرًا مباركًا فيه كما ينبغي لجلال وجهك ولعظيم سلطانك يا رب العالمين

اللهم صلّ وسلم وزد وبارك على نبينا محمد ﷺ

أَلَا بِذِكْرِ اللَّهِ تَطْمَئِنُّ الْقُلُوبُ

لا تنسونا من دعواتكم

Never Forget That (**Always Search**)

By: Amr Elsayed

Callback Hell Or Pyramid Of Doom Lesson #179

To Understand Ajax, Fetch, Promises

Pyramid Of Doom || Callback Hell

- What Is Callback
- Callback Hell Example

What Is Callback

- A Function That Is Passed Into Another One As An Argument To Be Executed Later
- Function To Handle Photos
- [1] Download Photo From URL
- [2] Resize Photo
- [3] Add Logo To The Photo
- [4] Show The Photo In Website

أتكلّمنا قبل كذا عن ال **Callback function** والتي هي بتكون **Function** بتقبل جواها **function** تانيه ك **Parameter** و ال **function** اللي هي **Parameter** دي أصلا في حد ذاتها ممكن يكون جواها **Function** تانيه ك **Parameter** ودا ينقلنا لمصطلح مقعد شوية اسمه **Callback hell** او **Pyramid Of Doom**

```
asyncFunc('something', function (err, data) {
  asyncFunc('something', function (err, data) {
    asyncFunc('something', function (err, data) {
      asyncFunc('something', function (err, data) {
        asyncFunc('something', function (err, data) {
          asyncFunc('something', function (err, data) {
            asyncFunc('something', function (err, data) {
              asyncFunc('something', function (err, data) {
                // do the final action
              });
            });
          });
        });
      });
    });
  });
});
```

واللي يعني دا بيكون **احد اشكالها** بعيدا عن ال **Code** المكتوب

علي سبيل المثال ممكن يكون عندي **Function** وظيفتها انها تنزل صورة من ال **URL** اللي هتاخده ال **function** دي بيكون جواها **callback function** تانيه وظيفتها ان لما الصورة تجيلها تعلمها **resize** وال **function** بتاعت ال **resize** دي بيكون جواها **callback function** تانية بعد ما الصورة يتعملها **resize** تقوم هي حاطة ال **Logo** ع الصورة دي وال **function** دي جواها **callback function** بتاخذ الصورة بعد ما يتحطّلها **Logo** وترفعها علي ال **website** كل دا بيؤدي لمصطلح ال **Callback Hell**

```
function makeItRed(e) {
  e.target.style.color = "red";
}

let p = document.querySelector(".test");
p.addEventListener("click", makeItRed);
```

عندي **function** بتخلي ال **target** اللي هدوس عليه يتحول لونه لأحمر
وعندي **function** تانيه مع **event** ال **click** بتقبل ميني أول **Function** اللي هي تغير اللون وتنفذها مع ال **click**

Testing Text

After Click ==>

Testing Text

نفس فكرة ال **Callback function hell** في المثال دا ويعتبر ابسط مثال

```
setTimeout(() => {
  console.log("Download Photo From URL");
  setTimeout(() => {
    console.log("Resize Photo");
    setTimeout(() => {
      console.log("Add Logo To The Photo");
      setTimeout(() => {
        console.log("Show The Photo In Website");
      }, 1000);
    }, 1000);
  }, 1000);
}, 1000);
```

Download Photo From URL

Resize Photo

Add Logo To The Photo

Show The Photo In Website

كل المشاكل بقا بتاعت ال **Callback hell** دا هتتحل لما نتعلم ال **promises** و ال **fetch**
احنا خدنا فكرة عن المشكلة قبل ما نتعرف علي حلها

Promise Intro And Syntax Lesson #180

Promise Intro And Syntax

- Promise In JavaScript Is Like Promise In Real Life
- Promise Is Something That Will Happen In The Future
- Promise Avoid Callback Hell
- Promise Is The Object That Represent The Status Of An Asynchronous Operation And Its Resulting Value
- Promise Status
 - Pending: Initial State
 - Fulfilled: Completed Successfully
 - Rejected: Failed

Story

- Once A Promise Has Been Called, It Will Start In A Pending State
- The Created Promise Will Eventually End In A Resolved State Or In A Rejected State
- Calling The Callback Functions (Passed To Then And Catch) Upon Finishing.
- Then
 - Takes 2 Optional Arguments [Callback For Success Or Failure]

انهاردا هنتكلم عن ال **Promise syntax** ومفهوم ال **promise** أولا ال **Promise** يعني وعد ال **promise** في ال **JS** عامل زي الوعد في الحقيقة بالظبط حاجة لسه هتحصل في المستقبل زي ما حد من صحابك يقولك اوعدي انك هتعمل 1 و 2 و 3 فانت تقوله اوعدك .. يعني حاجة لسه هعملها هو نفس الفكرة بالظبط بستخدم ال **Promise** عشان اتجنب مشكلة ال **callback hell**

ال **promise** مكون من 3 حالات

1- ال **Pending**: او الانتظار ودي الحالة اللي يحصل فيها **initial state** يعني ال **promise** لا بيكون اتحقق او متحققش بيكون لسه في البداية والانتظار **Pending <===**

2- ال **Fulfilled**: ودي الحالة اللي بيكون فيها الوعد اتحقق او العملية تمت بنجاح **Success <===**

3- ال **Rejected**: ودي الحالة اللي بيكون فيها الوعد متحققش او العملية فشلت **Failed <===**

أول ما تبدأ تنشئ ال **Promise** بتاعك بيكون في حالة ال **Pending** بعد ما تخلص ال **Promise** بيكون في حالة من اتنين وارد يحصل منهم واحدة يا أما حالة ال **Resolved** اللي هي فيها ال **Promise** بتاعك اتحقق او الحالة الثانية ال **Rejected** اللي فيها بيكون ال **Promise** متحققش وفشل

بعد كدا بيكون عندي **method** اسمها **then** بمرر ليها **Callback function** واللي منها بقا بطبع يا أما ال **Resolved Value** او ال **Rejected Value**

ال Syntax

```
const myPromise = new Promise((resolveFunction, rejectFunction) => {
  let connect = false;
  if (connect) {
    resolveFunction("Connection Established");
  } else {
    rejectFunction(Error("Connection Failed"));
  }
}).then(
  (resolveValue) => console.log(`Good ${resolveValue}`),
  (rejectValue) => console.log(`Bad ${rejectValue}`)
);
```

انا عندي متغير اسمه **myPromise** قيمته بتساوي **new Promise()**

ال **new Promise()** دا بيقبل جواه اتنين **function** واحدة لل **resolve** و واحدة لل **reject**

عملت متغير اسمه **Connect** قيمته **true** وحطيت القيمة دي في **IF & Else Statement**

في حالة ال **IF** و المتغير **connect** بالفعل **true** قولتله هات ال **function** بتاعت ال **resolve** دي وخليها ت **return** او ترجع الجملة دي **====> "Connection Established"**

اما في حالة ال **Else** لو المتغير **connect** بقا قيمته **false** يعني ال **Else** اللي هتتحق يعني حصل **reject** للبيانات فقولتله هات ال **function** بتاعت ال **reject** دي وخليها ت **return** او ترجع الجملة دي **====> "Connection Faliied"**

يعني ال **resolve** او ال **reject** مرتبط بالمتغير **connect** وقيمته وبناءً علي قيمته واحد من ال **function** اللي موجودة في ال **new Promise()** هو اللي هيتم

والله لو كانت ال **connect** قيمتها **true** في الحالة دي يبقي ال **resolve** هي اللي تمت وبقا فيه **Connection** و العملية بقت **fulfilled**

ولو كانت ال **connect** قيمتها **false** في الحالة دي يبقي ال **reject** هي اللي تمت و مبقاش في أي **Connection** و العملية بقت **Rejected**

بعد كدا عندي ال **then** اللي تبقبل مني **Callback function** ودي اللي بتيجي بعد ما اخلص ال **Promise** بتاعي سواء هو **resolved** او **rejected** في ال **then** انا هتعمل مع كل حالة من دول

طيب أي وظيفة ال **Callback function** اللي جوا ال **then** وظيفتها انها **تجيبلي النتيجة** بتاعت ال **Promise** لو **اتحقق** او **فشل** ولاحظ ان اول قيمة بتطلب مني في ال **then** هي ال **fulfilled** او ال **resolved value** يعني


```
}).then(
  (resolveValue) => console.log(`Good ${resolveValue}`),
  (rejectValue) => console.log(`Bad ${rejectValue}`)
);
```

فمثلا اول **callback function** دي بتاعت ال **resolveValue** هي أصلا في الأصل بتعود علي اول **function** في ال **promise** الي فوق لو اتحققت يبقى ال **resolveValue** دي هي الي هتطبع الكلام

لكن ثاني **callback function** دي بتاعت ال **rejectValue** الي هي برضو أصلا في الأصل بتعود علي ثاني **function** في ال **promise** الي فوق لو متحققش يبقى ال **rejectValue** دي هي الي هتطبع الكلام

بص انا عارف انك توهد سبكا او يمكن انا عشان توهد شوية فحاسب ان دا انطباع عام خليني اديك الزتونة زي ما بيقولو

ال **promise** الي هي فوق دي بتقبل مني اتنين **Function** حلو؟!
واحدة في حالة ال **resolve** او ال **fulfilled** --- و الثانية في حالة ال **reject** او ال **Rejected**
يبقي انا عندي كذا اتنين **Function** لكل حالة

ال **then** بقا بنتيجي هي كمان بإثنين **function** برضو واحدة في حالة ان ال **promise** بتاعك بقا **resolve** فتعوض عن حالة ال **resolve** الي هي اول **Function** في ال **promise**

والثانية في حالة ان ال **promise** بتاعك بقا **reject** فتعوض عن حالة ال **rejected** الي هي ثاني **Function** في ال **promise**

طبعا انت ممكن تخزين ال **functions** دي في متغير عادي وتكتب المتغير مكانهم

```
let resolver = (resolveValue) => console.log(`Good ${resolveValue}`);
let rejecter = (rejectValue) => console.log(`Bad ${rejectValue}`);

myPromise.then(resolver, rejecter);
```

Promise – Then, Catch And Finally Lesson #181

Promise Training

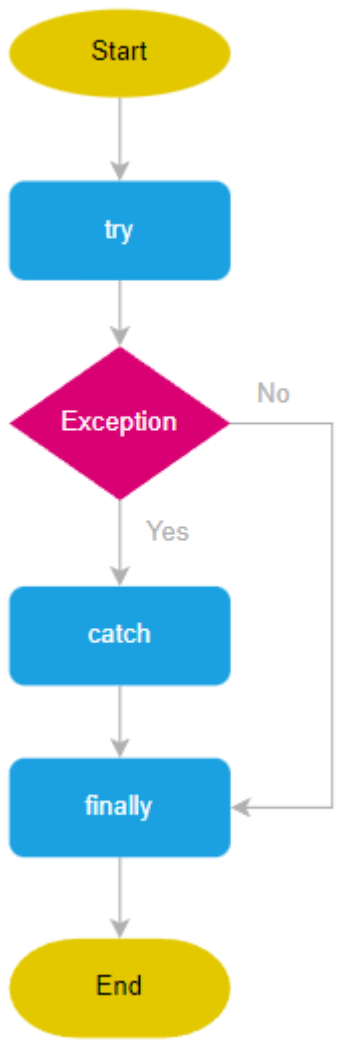
```
We Will Go To The Meeting, Promise Me That We Will Find The 4 Employees  
.then(We Will Choose Two People)  
.then(We Will Test Them Then Get One Of Them)  
.catch(No One Came)
```

Then => Promise Is Successfull Use The Resolved Data

Catch => Promise Is Failed, Catch The Error

Finally => Promise Successfull Or Failed Finally Do Something

هنتعرف انهاردا على مفاهيم مهمة في عالم ال **Promise** زي ال **catch** و ال **finally** وسبق واتعرفنا ع ال **then**



تخيل معايا القصة دي وهنشرحها بال **Code**

انا دلوقتي بتكلم مع واحد صاحبي وقولتله اوعديني اول ما نروح المكتب نلاقي **الأربع موظفين** اللي وعدتني انهم هيجوا

دلوقتي لو انا روجت ال **Meeting** ولقينا **الأربع اشخاص** يبقي الوعد اتحقق

بعد ما الوعد اتحقق ال **then** هتيجي عشان من **الاربعه** دول هنختار اثنين
بعد ما نختار الشخصين دول هنروح مكان تاني ونختار منهم **شخص واحد** بس

طيب فرضا اننا روحنا المكتب وملقناش **ولا شخص** موجود دا معناه أي؟!
معناه ان **الوعد متحققش** هنا بقا هيجي دور ال **Catch** مفيش ولا شخص
موجود اذًا **الوعد متحققش**

```
const myPromise = new Promise((resolveFunction, rejectFunction) => {  
  let employees = ["Osama", "Sayed", "Ahmed", "Mahmoud"];  
  if (employees.length === 4) {  
    resolveFunction(employees);  
  } else {  
    rejectFunction(Error("Number Of Employees Is Not 4"));  
  }  
});
```

عملت ال **promise** بتاعي وحطيت ال **employees** بتوعي بعددهم ال 4
بعدين قولتله **if** لو ال عدد الموظفين دا بيساوي 4 يبغي الوعد اتحقق وال **resolvedFunction** الخاصة بحالة
تحقيق الوعد هترجعلي الموظفين **resolveFunction(employees); <===** يعني ال **Array** بتاعت
الموظفين هترجعلي كلها

بعدا كدا ال **else** لو الوعد متحققش ال **rejectedFunction** الخاصة بحالة عدم تحقيق الوعد
هترجعلي ال **Error** ان عدد الموظفين مش بيساوي 4
rejectFunction(Error("Number Of Employees Is Not 4")); <===

```
myPromise.then((resolveValue) => {  
  resolveValue.length = 2;  
  console.log(resolveValue);  
});
```

جيت في ال **then** الأولي طبقت أول سيناريو عندي ... ال **resolveValue** اللي هي بتعود علي حالة تحقيق الوعد
الحالة اللي قولتله فيها رجعلي ال **Array** بتاعت الموظفين كاملة همسك بقا ال **resolveValue** دي
اخلي ال **length** بتاعها بيساوي 2 عشان اطبق اول سيناريو جالي 4 اشخاص خدت منهم 2

وال **console.log** دي هتطبعمهم **▶ (2) ['Osama', 'Sayed']**

```
myPromise.then((resolveValue) => {  
  resolveValue.length = 2;  
  return resolveValue;  
});
```

الكود دا هو هو نفسه اللي فات انا بس
شيلت ال **Console** وحطيت **return**
عشان ال **resolveValue** دي اللي بقت
قيمتها دلوقتي 2 موظفين ترجعلي واستخدم
معاه ال **then** ثاني عشان نطبق ثاني سيناريو

```
myPromise.then((resolveValue) => {  
  resolveValue.length = 1;  
  return resolveValue;  
});
```

جيت بقا في ال **then** التانيه خلّيت عدد الموظفين يكونو 1 بس مش دا السيناريو الي اتفقنا عليه في الحكاية فوق ... جميل

بعد كدا عملت ال **then** الأخير الي هي هتطبلي مسج فيها اسم الموظف الأخير الي تم اختياره

```
myPromise.then((resolveValue) => {  
  console.log(`The Chosen Employee Is ${resolveValue}`);  
});
```

The Chosen Employee Is Osama

يبقي ببساطة لحد الان انا بستخدم ال **then** عشان اهنّدل بيها الوعد لو اتحقق او متحققش يعني ببساطة كدا ال **then** بهنّدل بيها سواء ال **resolve** او **reject**

ركز أوي معايا بقا في الكلمتين الجايتين دول لو انا دلوقتي شيلت الموظفين من ال **Array** الي فوق دي هيحصل أي؟!

هيحصل ان ال **resolveFunction** هيتم تجاهلها الي هي حالة ال **true** في ال **if** الاول عني وهيتجه مباشرةً لـ **rejectFunction** الي هي حالة ال **false** في ال **else**

هنا بقا ال **then** الي انا حطيتها دي برضو هتهنّدل ال **reject** دا وتطبلي **error** مش اتفقنا فوق من تلت سطور بالظبط ان انا بستخدم ال **then** عشان اهنّدل بيها الوعد لو اتحقق او متحققش؟! لسه من سطرين تلاته قولت الكلام دا في الفقرة الي فوق علي طول

```

✖ ▶ Uncaught (in promise) Error: Number Of Employees Is Not 4    main.js:19
    at main.js:19:20
    at new Promise (<anonymous>)
    at main.js:14:19
✖ ▶ Uncaught (in promise) Error: Number Of Employees Is Not 4    main.js:19
    at main.js:19:20
    at new Promise (<anonymous>)
    at main.js:14:19
✖ ▶ Uncaught (in promise) Error: Number Of Employees Is Not 4    main.js:19
    at main.js:19:20
    at new Promise (<anonymous>)
    at main.js:14:19

```

اهم التالت حالات **then** الي كتبتهم اتحولوا من حالة ال **resolve** لحالة ال **reject** وطبعو ال **Error** وخلي بالك انا بكتب في حالة ال **reject** كلمة **Error** في ال **console** قبل الكلام عشان هي الي بتعرفني مكان ال **Error**

```

Error("Number Of Employees Is Not 4")

```

يعني دي لزمته تعرفني مكان ال **Error**

```

✖ ▶ Uncaught (in promise) Error: Number Of Employees Is Not 4    main.js:19
    at main.js:19:20
    at new Promise (<anonymous>)
    at main.js:14:19

```



ولو شيلتها شوف الي هيحصل

```

rejectFunction(("Number Of Employees Is Not 4"));

```

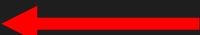
```

✖ ▶ Uncaught (in promise) Number Of Employees Is Not 4    127.0.0.1/:1

```

هيطلبلي رسالة قرعة زي ما انت شايف كدا من غير ما يعرفني مكان ال **Error**

هنا بقا يا معلم هيجي دور ال **catch** انا بس كل هعلمه اني هعمل **chain** او ربط لكل ال **then** التالته الي كتبتهم دول مع المتغير بتاعي **myPromise** بس وكمان هشيل كل الموظفين عشان اشتغل ع ال **rejectFunction** في حالة ال **else**

```
const myPromise = new Promise((resolveFunction, rejectFunction) => {  
  let employees = [];   
  if (employees.length === 4) {  
    resolveFunction(employees);  
  } else {  
    rejectFunction(Error("Number Of Employees Is Not 4"));  
  }  
});
```

```
myPromise  
  .then((resolveValue) => {  
    resolveValue.length = 2;  
    return resolveValue;  
  })  
  .then((resolveValue) => {  
    resolveValue.length = 1;  
    return resolveValue;  
  })  
  .then((resolveValue) => {  
    console.log(`The Chosen Employee Is ${resolveValue}`);  
  })  
  .catch((rejectedReason) => console.log(rejectedReason));
```

استخدمت ال **catch** وحطيت جواها **callback function** هيجل محل مين؟! هيجل محل محل ال **rejectFunction** الي فوق الي هي حالة ال **else** لما يحل محلها هيطبعلي الرسالة الي موجودة في ال **else** بقا

```
Error: Number Of Employees Is Not 4  
    at main.js:19:20  
    at new Promise (<anonymous>)  
    at main.js:14:19
```

هنا ال **then** بطلت تهندل ال **reject** وال **catch** هي بقا الي تولت المهمة دي

طيب أي بقا قصة ال **finally** دي ؟!

ال **finally** دي ملهاش علاقة ال **promise** بتاعي **resolved** ولا **rejected** هي ملهاش دعوة بالحوار دا انا بستخدمها مثلا عشان اطبع رسالة ... الرسالة دي هتتطبع في كل الحالات سواء الوعد اتحقق متحققش هي هتتطبعي رسالة وتشتغل

```
myPromise
  .then((resolveValue) => {
    resolveValue.length = 2;
    return resolveValue;
  })
  .then((resolveValue) => {
    resolveValue.length = 1;
    return resolveValue;
  })
  .then((resolveValue) => {
    console.log(`The Chosen Employee Is ${resolveValue}`);
  })
  .catch((rejectedReason) => console.log(rejectedReason))
  .finally(console.log("The Operation Is Done"));
```

The Operation Is Done

main.js:36

لا تنسونا من دعواتكم

Never Forget That (**Always Search**)

By: Amr Elsayed

♥ راقِ لي ♥



Ahmed Shawki

2d · 🌐

...

أكبر غلطة وقعت فيها في اول رحلتي وضيعت مني فرص كثير وضيعت من عمري كثير -وما زلت بقع فيها- بس الاول ما كنتش واخذ بالي منها هي :

الاستعجال

الاستعجال

الاستعجال

فنصحتي ليك ولنفسك قبلك ، ما تستعجلش ، ما تستعجلش ، ما تستعجلش واصبر علي نفسك واتقن تعليمك واتقن شغلك والشهر اللي هتتأخره دلوقتي في الاتقان هيوفر عليك سنة قدام ..

♥ اذكر الله

♥ الزموا ذكر الله ففيه العون على متاعب الحياة

سبحان الله وبحمده عدد خلقه ورضا نفسه وزنة عرشه ومداد كلماته

اللهم لك الحمد حمدًا طيبًا كثيرًا مباركًا فيه كما ينبغي لجلال وجهك ولعظيم سلطانك يا رب العالمين

اللهم صلّ وسلم وزد وبارك على نبينا محمد ﷺ

أَلَا بِذِكْرِ اللَّهِ تَطْمَئِنُّ الْقُلُوبُ

لا تنسونا من دعواتكم

Never Forget That (**Always Search**)

By: Amr Elsayed

Promise And XHR Lesson #182

Promise And XHR

```
const getData = (apiLink) => {  
  return new Promise((resolve, reject) => {  
    let myRequest = new XMLHttpRequest();  
    myRequest.onload = function () {  
      if (this.readyState === 4 && this.status === 200) {  
        resolve(JSON.parse(this.responseText));  
      } else {  
        reject(Error("No Data Found"));  
      }  
    }  
  });  
  
  myRequest.open("GET", apiLink);  
  myRequest.send();  
});
```

انا هنا عملت **Function** اسمها **getDate** مررت ليها **Parameter** اسمه **apiLink** ال **function** دي هتكون ديناميكية يعني انت هتشيل ال **Parameter** دا وتحط مكانه لينك ال **API** اللي هتيجيبه

داخل ال **Function** دي بقا عملت ال **Promise** بتاعي مررت فيه زي ما اتعودنا ال **resolve** في حالة ان فيه بيانات فعلا هترجعلي والوعد اتحقق ومررت كمان ال **reject** في حالة ان البيانات فيها مشكلة او الوعد متحققش

داخل ال **promise** عملت ال **AJAX Call** من خلال ال **XML** اللي خزنه في المتغير **myRequest** عملت **Event** ال **onload** علي ال **myRequest** دا وجواه عملت الشرط بتاعي انه حالة ال **readyState** بـ 4 وال **status** بـ 200

قولتله بقا جوا الشرط دا لو اتحقق يبقي نفذ الوعد جوا الشرط المتحقق دا وهاتلي البيانات او اعملها **Parse** وال **else** لو الشرط متحققش يبقي هاتي عدم تنفيذ الوعد هنا ورجعلي **Error** فيه مسج **No Data Found**

عملت بقا **open** وحطيت مكان ال **API** بتاعي ال **Parameter** بتاع ال **function**

```

getData("https://api.github.com/users/elzerowebsschool/repos")
  .then((result) => {
    result.length = 10;
    return result;
  })
  .then((result) => console.log(result[0].name))
  .catch((rej) => console.log(rej));

```

عملت **run** لد **function** الأساسية بتاعتي الي هي **getData** وعوضت مكان ال **Parameter** بتاعها بال **API Link**

وبعدين استخدمت عليها ال **then** ال **then** الأولي دي كلها علي بعضها كدا بتعوض عن حالة ال **resolve** الي في ال **promise** فوق يعني في **حالة تحقيق** الشرط بتاع ال **readyState** و ال **status** و كمان **الوعد اتحقق** جوا الشرط المتحقق دا هتعمل **Pares** للداتا ال **data** دي متخزنة في تنفيذ الوعد الي هو ال **resolve** و ال **resolve** دا بيروح في ال **then** الأولي لو اتحقق و اتنفذ يعني فهنا انا قولتله بقا هات ال **resolve** دا الي هو أصلا شاييل جواه ال **parse** بتاع ال **data** كلها الموجود في ال **API** قولتله قلل عددهم ل **10 repos** بس وبعدين رجعت ال **10 repos** دول لل **then** الي بعدها ال **then** الي بعدها خدت منها اول **repo** بس وعرضت اسمها

Ajax

main.js:30

بعد كدا جت ال **catch** بقا بال **function rej** الي هي في حالة **عدم تنفيذ الوعد** .. عد تنفيذ الوعد دا ممكن يكون ليه اكر من سبب زي لينك ال **API** مش سليم مثلا في الحالة دي اطبعلي ال **rej** دا في ال **Console** الي هو أصلا بيحل محل ال **reject** الي موجود في ال **promise** فوق ... ففي **حالة عدم تنفيذ** الوعد هيطبع ال **Error** الي موجود في ال **reject function**

```

Error: No Data Found                               main.js:31
    at myRequest.onload (main.js:16:16)

```

لا تنسونا من دعواتكم

Never Forget That (**Always Search**)

By: Amr Elsayed

Fetch API Lesson #183

Fetch API

- Return A Representation Of the Entire HTTP Response

باستخدام ال **fetch** هنستغني عن قصة ال **Promise** و ال **XML** اللي كنا بنعملها

```
fetch("https://api.github.com/users/elzerowebsschool/repos")
  .then((result) => {
    console.log(result);
    let myData = result.json();
    console.log(myData);
    return myData;
  })
  .then((full) => {
    full.length = 10;
    return full;
  })
  .then((ten) => {
    console.log(ten[0].name);
  })
```

ال **Fetch** بحط فيه لينك ال **API** اللي عايز اسحب منه ال **Data**

عملت كذا فعلا وبعدين كتبت ال **then**

```
fetch("https://api.github.com/users/elzerowebsschool/repos")
  .then((result) => {
    console.log(result);
  });
```

```
main.js:8
Response {type: 'cors', url: 'https://api.github.com/users/elze
▼ rowebsschool/repos', redirected: false, status: 200, ok: tru
e, ...} ⓘ
  body: (...)
  bodyUsed: false
  ▶ headers: Headers {}
  ok: true
  redirected: false
  status: 200
  statusText: ""
  type: "cors"
  url: "https://api.github.com/users/elzerowebsschool/repos"
  ▶ [[Prototype]]: Response
```

ال **fetch** رجعلي ال **Entire HTTP Response** من اللينك انا حطيتها جوا ال **Fetch**
طيب انا عايز ارجع مجموعة ال **Objects** الي فيها ال **Repos** بتاعتي أنا مش عايز ال **Response**
الي طبعته في ال **console** دا

```
fetch("https://api.github.com/users/elzerowebsschool/repos")
  .then((result) => {
    console.log(result);
    let myData = result.json() ←
    console.log(myData)
  });
```

```
main.js:10
▼ Promise {<pending>} ⓘ
  ► [[Prototype]]: Promise
  [[PromiseState]]: "fulfilled"
  ▼ [[PromiseResult]]: Array(30)
    ► 0: {id: 222275692, node_id: 'MDEwO1JlcG9zaXRvcnkyMjIyNzU2OTI='}
    ► 1: {id: 64010371, node_id: 'MDEwO1JlcG9zaXRvcnk2NDAxMDM3MQ='}
    ► 2: {id: 87401782, node_id: 'MDEwO1JlcG9zaXRvcnk5NzQwMTc4Mg='}
```

انا كدا عملت **Extract** للبيانات بتاعتي كلها الي في اللينك بتاع ال **API**

```
fetch("https://api.github.com/users/elzerowebsschool/repos")
  .then((result) => {
    let myData = result.json();
    return myData;
  })
  .then((full) => {
    full.length = 10;
    console.log(full);
    return full;
  });
```

عملت **return** لل **myData** دي عشان استخدمها في ال **then** الي بعد كدا

ال **then** الي بعد كدا سميتها اسم مختلف عادي جدا كدا كدا هي هتشتغل ع البيانات الي رجعلي

من اول **then** فمش شرط تبقي نفس الاسم وجواها قللت ال **length** بتاع ال **myData** الي رجعلي من أول
then وعملتها **return** عشان ال **then** الثالثة تستلمها وتشتغل عليها وطبعتها في ال **Console** عشان اورها لك
بس

```
main.js:13
▼ (10) [{...}, {...}, {...}, {...}, {...}, {...}, {...}, {...}, {...}, {...}] ⓘ
  ► 0: {id: 222275692, node_id: 'MDEwO1JlcG9zaXRvcnkyMjIyNzU2OTI='}
  ► 1: {id: 64010371, node_id: 'MDEwO1JlcG9zaXRvcnk2NDAxMDM3MQ='}
```

```
fetch("https://api.github.com/users/elzerowebsschool/repos")
  .then((result) => {
    let myData = result.json();
    return myData;
  })
  .then((full) => {
    full.length = 10;
    return full;
  })
  .then((ten) => {
    console.log(ten[0].name);
  });
```

جت ال **then** التالته وسميتها باسم مختلف هي كذا كذا مش هي فرق معاها الاسم .. الاسم دا ليك انت لان ال **then** التالته دي كذا كذا هتشتغل ع ال **data** الي جاية من ال **then** بتاعت ال **full** الي قبلها مسكتها بقا وخذت اول **item** بال **index[0]** من ال **10** الي رجعولي من ال **then** الي فوقها وجبت اسمه

Ajax

main.js:16

لا تنسونا من دعواتكم

Never Forget That (**Always Search**)

By: Amr Elsayed

Promise – All, All Settled And Race Lesson #184

```
Promise
- All
- All Settled
- Race
```

```
const myFirstPromise = new Promise((res, rej) => {
  setTimeout(() => {
    res("Iam The First Promise");
  }, 5000);
});

const mySecondPromise = new Promise((res, rej) => {
  setTimeout(() => {
    res("Iam The Second Promise");
  }, 1000);
});

const myThirdPromise = new Promise((res, rej) => {
  setTimeout(() => {
    res("Iam The Third Promise");
  }, 2000);
});
```

دلوقتي عندي **3 Promises** كل **promise** من دول بيعمل **Task** معين مختلف عن الثاني عندي أول حاجة .. حاجة اسمها **promise.all** وبتطلب مني **Array** فيها كل ال **promises** اللي عندي بترجعلي في الآخر **promise** لو كل ال **promises** اللي عندي حصلها **resolve** او **اتحققت** بيرجعلي ال **Resolved** منها كلها ولو فيه **promise** واحد منه حصله **Reject** يبقى ال **promise** اللي هيرجعلي هيحصله **reject** هو كمان

```
Promise.all([myFirstPromise, mySecondPromise, myThirdPromise]).then(
  (resolvedValues) => console.log(resolvedValues)
);
```

```
► (3) ['Iam The First Promise', 'Iam The Second Promise', 'Iam The Third Promise']
```

فزاى ما انت شايف كدا رجعلي ال **resolve** من كل ال **promises** اللي عندي اللي هي المسجات اللي في كل **promise** فوق

```
const myFirstPromise = new Promise((res, rej) => {
  setTimeout(() => {
    res("Iam The First Promise");
  }, 5000);
});

const mySecondPromise = new Promise((res, rej) => {
  setTimeout(() => {
    res("Iam The Second Promise");
  }, 1000);
});

const myThirdPromise = new Promise((res, rej) => {
  setTimeout(() => {
    rej("Iam The Third Promise");
  }, 2000);
});
```

```
Promise.all([myFirstPromise, mySecondPromise, myThirdPromise]).then(
  (resolvedValues) => console.log(resolvedValues),
  (rejectedValue) => console.log(`Rejected: ${rejectedValue}`)
);
```

Rejected: Iam The Third Promise

ال **promise.allSettled** بقا مختلفة شوية لان هنا هيرجعلي **array** بال **promises** كلها سواء فيها **resolved values** او **rejected values** يعني أي الكلام دا؟!
دلوقتي لنعتبر ان ال **promises** كلها **resolved** هيرجعلي **Array** فيها كل ال **promises** سواء فيها كلهم **resolved** او فيهم واحد **rejected**

main.js:32

```
▼ (3) [{...}, {...}, {...}] ⓘ
  ▶ 0: {status: 'fulfilled', value: 'Iam The First Promise'}
  ▶ 1: {status: 'fulfilled', value: 'Iam The Second Promise'}
  ▶ 2: {status: 'fulfilled', value: 'Iam The Third Promise'}
    length: 3
  ▶ [[Prototype]]: Array(0)
```


طيب لنفترض اني جيت عند تاني **promise** اللي هو **mySecondPromise** وخليته **rejected** برضو هيرجعلي نفس ال **array** اللي فوق ولكن في تاني **promise** هيقولي انه **rejected**

```
main.js:32
▼ (3) [{...}, {...}, {...}] ⓘ
  ▶ 0: {status: 'fulfilled', value: 'Iam The First Promise'}
  ▶ 1: {status: 'rejected', reason: 'Iam The Second Promise'}
  ▶ 2: {status: 'fulfilled', value: 'Iam The Third Promise'}
    length: 3
  ▶ [[Prototype]]: Array(0)
```

هنا مختلف بقا لانه بيرجع كل ال **promises** ويقولي مين اتحقق ومين متحققش لكن التاني ال **all** وهو ماشي في السكة لو لقي كل ال **promises** متحققة هيرجعهم ولو لقاهم فيهم واحد بس متحققش هو دا اللي هيرجعولي ويعمل **reject** لكل ال **promises** اللي عندي

ال **promise.race** يعني فيه سباق باين من اسمه طيب دا بيعمل أي؟! هو بيمشي علي كل ال **promises** اللي عندي وأول حاجة بيقابلها في سكتة سواء **resolve** او **reject** بيرجعهاالي هي الأول من خلال الوقت بتاعها

```
const myFirstPromise = new Promise((res, rej) => {
  setTimeout(() => {
    res("Iam The First Promise");
  }, 5000);
});

const mySecondPromise = new Promise((res, rej) => {
  setTimeout(() => {
    rej("Iam The Second Promise");
  }, 1000);
});

const myThirdPromise = new Promise((res, rej) => {
  setTimeout(() => {
    res("Iam The Third Promise");
  }, 2000);
});
```

Rejected: Iam The Second Promise

لاحظ توقيت كل **promise** فمثلا التاني هياخد ثانيه واحد بس يبقي دا اسرع واحد في التلاته .. حالته أي؟ **Reject** يبقي هو اللي هيجيلي اسرع وأول واحد بغض النظر طبعا عن حالته سواء كان **resolve** او **reject** ولو عدلت توقيت اول **promise** خليفته نص ثانيه هيبقي هو الأسرع وهو اللي هيجيلي الأول وبسرعة سواء كان **resolve** او **reject** في الحالة بتاعته

دا تدريب بسيط كدا عملته مع نفسي مرة بال `XMLHttpRequest()` ومرة عملته بال `Fetch` ودا مثال ال `Fetch`

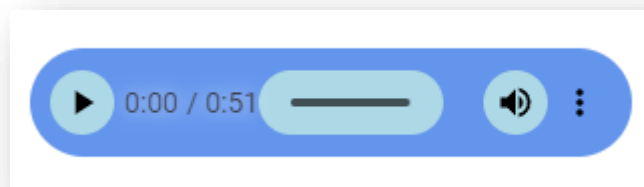
JS

```
fetch("http://api.alquran.cloud/v1/ayah/262/ar.alafasy")
  .then((resolve) => {
    console.log(resolve);
    let myData = resolve.json();
    console.log(myData);
    return myData;
  })
  .then((ayah) => {
    console.log(ayah.data.audio);
    let audio = new Audio(ayah.data.audio);
    audio.setAttribute("type", "audio/mp3");
    audio.setAttribute("controls", "");
    let div = document.createElement("div");
    let ayahAudio = audio;
    div.appendChild(ayahAudio);
    document.body.appendChild(div);
  })
  .catch((rej) => console.log(Error`${rej}: Your API Is Wrong`));
```

CSS

```
Audio {
  position: absolute;
  top: 50%;
  left: 50%;
  transform: translate(-50%, -50%);
}
audio::-webkit-media-controls-panel {
  background-color: cornflowerblue;
}
audio::-webkit-media-controls-play-button {
  background-color: lightblue;
  border-radius: 50%;
}
audio::-webkit-media-controls-current-time-display {
  color: #333;
}
audio::-webkit-media-controls-time-remaining-display {
  color: #333;
}
audio::-webkit-media-controls-timeline {
  border-radius: 40px;
  background-color: lightblue;
}
```

```
audio::-webkit-media-controls-volume-slider{
  background-color: lightblue;
  border-radius: 50px;
  padding-left: 10px;
  padding-right: 10px;
}
audio::-webkit-media-controls-mute-button{
  background-color: lightblue;
  border-radius: 50%;
}
```



انا عملت **fetch** ل **API** فيه اية الكرسي نفسها وطبقت عليه كل اللي اتعلمناه لكن الجزء دا هو اللي زيادة

```
let audio = new Audio(ayah.data.audio);
audio.setAttribute("type", "audio/mp3");
audio.setAttribute("controls", "");
let div = document.createElement("div");
let ayahAudio = audio;
div.appendChild(ayahAudio);
document.body.appendChild(div);
```

عملت متغير اسمه **audio** حزنه فيه ال **audio object** وجواه ال **audio** بتاع ال الآيه الكريمة من ال **api** نفسه

```
▶ data: {number: 262, audio: 'https://c
```

اما الجزء الخاص بتشغيل ال **Audio** لازم عشان ال **audio** يشتغل او يظهر في الصفحة يكون ليه **attribute** اسمه **controls** وكمان يتكتب ال **type** بتاع ال **Audio**

بعد كدا الباقي سهل بقا عملت **div** في الصفحة وعملت **Append** ليه الحاجات المعروفة دي

ابقي جرب [وفي اللينك دا هتلاقي API فيه القراءن كله بالكامل سواء text او Audio](#)

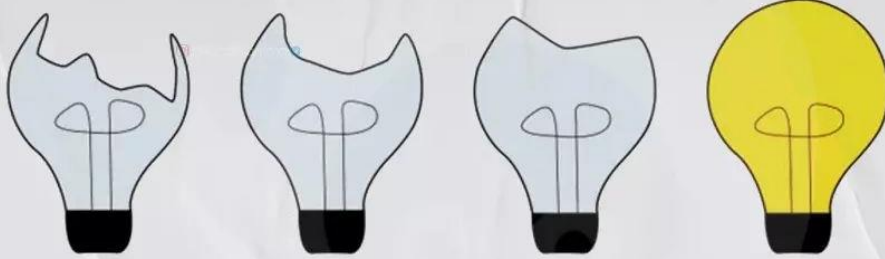
[لينك الفيديو فيه الكود دا وهو شغال](#)

لا تنسونا من دعواتكم

Never Forget That (**Always Search**)

By: Amr Elsayed

♥ عبارات تحفيزية ♥



الكثير من الافكار السيئة تسبق فكرة عظيمة

الأفكار السيئة .. التجارب السيئة او الصعبة كلها نتاج لخبرات ونتائج إيجابية في المستقبل وغلطات كثير عملتك وهتبدأ تتفادها في المستقبل فجرب كثير واغلط كثير ومتخافش

♥ اذكر الله

الزموا ذكر الله ففيه العون على متاعب الحياة ♥

أَلَا بِذِكْرِ اللَّهِ تَطْمَئِنُّ الْقُلُوبُ

اللهم لك الحمد حمدًا طيبًا كثيرًا مباركًا فيه كما ينبغي لجلال وجهك ولعظيم سلطانك يا رب العالمين

لا تنسونا من دعواتكم

Never Forget That (**Always Search**)

By: Amr Elsayed

Async And Trainings Lesson #185

Async

- Async Before Function Mean This Function Return A Promise
- Async And Await Help In Creating Asynchronous Promise Behavior With Cleaner Style

```
function getData() {
  return new Promise((res, rej) => {
    let users = [];
    if (users.length > 0) {
      res("Users Found");
    } else {
      rej("No Users Found");
    }
  });
}

getData().then(
  (resolvedValue) => console.log(resolvedValue),
  (rejectedValue) => console.log("Rejected " + rejectedValue)
);
```

في المثال دا عندي **function** اسمها **getData** بترجعلي **new Promise()** بيقبل من **res** و **rej** وعندي **array** اسمها **users**

عملت شرط **If** بقول فيه لو **array.length** اكبر من 0 يعني في **users** يبقي دا **true** ويبقي دا هو **res** بتاع **promise** فقولته رجعلي من **promise** دا **res** اللي هو فيه المسج **"User Found"** بعد كدا عندي **else** يعني مفيش ولا **user** يبقي دا **false** ويبقي دا هو **rej** بتاع **promise** فقولته رجعلي من **promise** دا **rej** اللي هو فيه المسج **"User Not Found"**

بعد كدا استخدمت **then** علي **function** اللي اسمها **getData** دي

عندي فيها **resolvedValue** بتحل محل **res** في **promise** لو الوعد اتحقق فلو الوعد اتحقق يبقي الشرط **true** يبقي المسج بتاعت **"User Found"** هيعوض مكان **resolvedValue** في **then** ويبدا يطبع في **Console**

و عندي كمان فيها **rejectedValue** بتحل محل **rej** في **promise** لو الوعد متحققش فلو الوعد متحققش يبقي الشرط راح لـ **else** يبقي المسج بتاعت **"User Not Found"** هتعوّض مكان **resolvedValue** في **then** ويبدا يطبع في **Console**

لو فيه users

```
let users = ["Osama"];
```

Users Found

لو مفيش Users

```
let users = [];
```

Rejected No Users Found

طيب دي طريقة تيجي نشوف طريقة تانيه ابسط شوية من دي واسهل !؟

```
function getData() {  
  let users = ["Osama"];  
  if (users.length > 0) {  
    return Promise.resolve("Users Found");  
  } else {  
    return Promise.reject("No Users Found");  
  }  
}  
  
getData().then(  
  (resolvedValue) => console.log(resolvedValue),  
  (rejectedValue) => console.log("Rejected " + rejectedValue)  
);
```

لاحظ فيه فرق بين الكود دا واللي فات انا في الكود دا جوا ال **Function** بتاعتي حذفت ال **new Promise()** بس في ال **if** بتاعتي في حالة ال **true** انا عملت **return** لل **promise.resolve** فيه نفس المسج اللي كانت في الكود اللي فات اللي هي **"Users Found"**

وفي ال **else** في حالة ال **false** عملت **return** لل **promise.reject** فيه نفس المسج اللي كانت في الكود اللي فات اللي هي **"Users Not Found"**

هتبقى هي هي نفس النتيجة بتاعت ال **code** الأول اللي فوق

طيب ما تسيبك من الطريقتين الي فاتو وتعالى نشوف **طريقة تالته ايسط واسهل** بمراحل وبتصنف كمان انها **Clean** يعني عندي **keyword** اسمها **async** بمجرد ما احطها قبل ال **function** يبقى انا بعمل نفس ال **behavior** بتاع ال **promise** بالظبط

```
async function getData() {  
  let users = ["Osama"];  
  if (users.length > 0) {  
    return "Users Found";  
  } else {  
    throw new Error("No Users Found");  
  }  
}  
  
console.log(getData());  
  
getData().then(  
  (resolvedValue) => console.log(resolvedValue),  
  (rejectedValue) => console.log("Rejected " + rejectedValue)  
);
```

هاجي قبل ال **Function** بتاعتي بس احط كلمة **async** وال **keyword** دي بتعمل عمل ال **promise** بالظبط
بص بقا ع الكود الأخير دا هتلاقيه مجرد **function** عادية زي الي خدناها قبل كدا

```
► Promise {<fulfilled>: 'Users Found'}  
Users Found
```

لا تنسونا من دعواتكم

Never Forget That (**Always Search**)

By: Amr Elsayed

Await And Trainings Lesson #186

Await

- Await Works Only Inside Async Functions
- Await Make JavaScript Wait For The Promise Result
- Await Is More Elegant Syntax Of Getting Promise Result

ال **Await** بتشتغل بس مع ال **Async function**

ال **Await** بتخلي لغة الجافاسكريبت تنتظر نتيجة ال **promise** قبل ما أي حاجة موجودة تانية تحصل

```
const myPromise = new Promise((resolve) => {
  setTimeout(() => {
    resolve("Iam The Good Promise");
  }, 3000);
});

async function readData() {
  console.log("Before Promise");
  myPromise.then((resolvedValue) => console.log(resolvedValue));
  console.log("After Promise");
}

readData();
```

انا عندي **promise** جواه **setTimeout** بيرجع مسج "**Iam The Good Promise**"

عملت **function** تانية اسمها **readData()** جواها
console.log("Before Promise")

وبعدين ال **then** بتاعت ال **promise**

بعدين **console.log("After Promise")**

فبشكل بديهي ومنطقي جدا الاتنين **console** هما اللي هيطلعوا الأول بعدين ال **setTimeout** بتاعت
ال **promise** هتبقى هي اخر حاجة

Before Promise

After Promise

Iam The Good Promise

لكن مع وجود الـ **await** بقا مع الـ **promise** بتاعي دا هيخلي لغة الجافاسكريبت تنتظر الـ **response** بتاع الـ **promise** الأول

```
const myPromise = new Promise((resolve) => {
  setTimeout(() => {
    resolve("Iam The Good Promise");
  }, 3000);
});

async function readData() {
  console.log("Before Promise");
  console.log(await myPromise);
  console.log("After Promise");
}

readData();
```

Before Promise

Iam The Good Promise

After Promise

الـ **console.log("After Promise")** مش تغلش غير بعد ما جالي الرد بتاع الـ **promise** الأول كل دا في حالة الـ **resolve** وفي حالة الـ **reject** برضو نفس الفكرة والله وهتعلم **Catch** للـ **Error**

```
const myPromise = new Promise((resolve, reject) => {
  setTimeout(() => {
    reject(Error("Iam The Bad Promise"));
  }, 3000);
});

async function readData() {
  console.log("Before Promise");
  console.log(await myPromise.catch((err) => err));
  console.log("After Promise");
}

readData();
```

Before Promise

Error: Iam The Bad Promise
at main.js:11:12

After Promise

Try, Catch, Finally With Fetch Lesson #187

Async & Await With Try, Catch, Finally

```
const myPromise = new Promise((resolve, reject) => {
  setTimeout(() => {
    // resolve("Iam The Good Promise");
    reject("Iam The Bad Promise");
  }, 3000);
});

async function readData() {
  console.log("Before Promise");
  try {
    console.log(await myPromise);
  } catch (reason) {
    console.log(`Reason: ${reason}`);
  } finally {
    console.log("After Promise");
  }
}

readData();
```

ال (`console.log("Before Promise")` حاجة مش مرتبطة باي حاجة فتهتبطع كدا كدا الأول

بعد كدا عندي ال `try` انا بقوله انا هجرب هل ال `promise` بتاعي هيتحقق ويحبلي ال `resolved data` ولا لا ؟!

```
try {
  console.log(await myPromise);
}
```

طيب لو فيه مشكلة ؟! يبقي يجي دور ال `catch` لو حصل `reject`

```
catch (reason) {
  console.log(`Reason: ${reason}`);
}
```

بعد كذا ال **finally** اللي ممكن احط فيها ال `console.log("After Promise")`

```
finally {  
    console.log("After Promise");  
}
```

تدريب على ال **fetch** مع ال **async** و ال **await**

```
async function fetchData() {  
    console.log("Before Fetch");  
    try {  
        let myData = await fetch("https://api.github.com/users/elzeroweb school/repos");  
        console.log(myData);  
    } catch (reason) {  
        console.log(`Reason: ${reason}`);  
    } finally {  
        console.log("After Fetch");  
    }  
}  
  
fetchData();
```

Before Fetch

[main.js:32](#)

[main.js:35](#)

```
▼ Response {type: 'cors', url: 'https://api.github.com/users/elzeroweb school/repos', redirected: false, status: 200, ok: true, ...} ⓘ  
  body: (...)  
  bodyUsed: false  
  ▶ headers: Headers {}  
  ok: true  
  redirected: false  
  status: 200  
  statusText: ""  
  type: "cors"  
  url: "https://api.github.com/users/elzeroweb school/repos"  
  ▶ [[Prototype]]: Response
```

After Fetch

[main.js:39](#)

ال **await** رجعتلي من ال **fetch** بتاعت ال **repos** ال **http response**

ولما اكتب ال `json` مع ال `fetch` بتاعي هيرحلي ال `promise` وحالته وكمان الداتا اللي فيه لو هو `resolved`

```
console.log(myData.json());
```

```
▼ Promise {<pending>} ⓘ  
  ► [[Prototype]]: Promise  
  [[PromiseState]]: "fulfilled"  
  ▼ [[PromiseResult]]: Array(30)  
    ► 0: {id: 222275692, node_id: 'MDEwO1JlcG9zaXRvcnkyMjIyNzU  
    ► 1: {id: 64010371, node_id: 'MDEwO1JlcG9zaXRvcnk2NDAxMDM3  
    ► 2: {id: 97401782, node_id: 'MDEwO1JlcG9zaXRvcnk5NzQwMTc4  
    ► 3: {id: 105766165, node_id: 'MDEwO1JlcG9zaXRvcnkxMDU3NjY  
    ► 4: {id: 465037937, node_id: 'MDEwO1JlcG9zaXRvcnkxMDU3NjY  
    ► 5: {id: 105766165, node_id: 'MDEwO1JlcG9zaXRvcnkxMDU3NjY  
    ► 6: {id: 105766165, node_id: 'MDEwO1JlcG9zaXRvcnkxMDU3NjY  
    ► 7: {id: 105766165, node_id: 'MDEwO1JlcG9zaXRvcnkxMDU3NjY  
    ► 8: {id: 105766165, node_id: 'MDEwO1JlcG9zaXRvcnkxMDU3NjY  
    ► 9: {id: 105766165, node_id: 'MDEwO1JlcG9zaXRvcnkxMDU3NjY  
    ► 10: {id: 105766165, node_id: 'MDEwO1JlcG9zaXRvcnkxMDU3NjY  
    ► 11: {id: 105766165, node_id: 'MDEwO1JlcG9zaXRvcnkxMDU3NjY  
    ► 12: {id: 105766165, node_id: 'MDEwO1JlcG9zaXRvcnkxMDU3NjY  
    ► 13: {id: 105766165, node_id: 'MDEwO1JlcG9zaXRvcnkxMDU3NjY  
    ► 14: {id: 105766165, node_id: 'MDEwO1JlcG9zaXRvcnkxMDU3NjY  
    ► 15: {id: 105766165, node_id: 'MDEwO1JlcG9zaXRvcnkxMDU3NjY  
    ► 16: {id: 105766165, node_id: 'MDEwO1JlcG9zaXRvcnkxMDU3NjY  
    ► 17: {id: 105766165, node_id: 'MDEwO1JlcG9zaXRvcnkxMDU3NjY  
    ► 18: {id: 105766165, node_id: 'MDEwO1JlcG9zaXRvcnkxMDU3NjY  
    ► 19: {id: 105766165, node_id: 'MDEwO1JlcG9zaXRvcnkxMDU3NjY  
    ► 20: {id: 105766165, node_id: 'MDEwO1JlcG9zaXRvcnkxMDU3NjY  
    ► 21: {id: 105766165, node_id: 'MDEwO1JlcG9zaXRvcnkxMDU3NjY  
    ► 22: {id: 105766165, node_id: 'MDEwO1JlcG9zaXRvcnkxMDU3NjY  
    ► 23: {id: 105766165, node_id: 'MDEwO1JlcG9zaXRvcnkxMDU3NjY  
    ► 24: {id: 105766165, node_id: 'MDEwO1JlcG9zaXRvcnkxMDU3NjY  
    ► 25: {id: 105766165, node_id: 'MDEwO1JlcG9zaXRvcnkxMDU3NjY  
    ► 26: {id: 105766165, node_id: 'MDEwO1JlcG9zaXRvcnkxMDU3NjY  
    ► 27: {id: 105766165, node_id: 'MDEwO1JlcG9zaXRvcnkxMDU3NjY  
    ► 28: {id: 105766165, node_id: 'MDEwO1JlcG9zaXRvcnkxMDU3NjY  
    ► 29: {id: 105766165, node_id: 'MDEwO1JlcG9zaXRvcnkxMDU3NjY
```

ولو كتبت قبلها `await` كمان

```
console.log(await myData.json());
```

```
main.js:36  
(30) [{...}, {...}, {...}, {...}, {...}, {...}, {...}, {...}, {...}, {...}, {...}, {...}, {...},  
  ► {...}, {...}, {...}, {...}, {...}, {...}, {...}, {...}, {...}, {...}, {...}, {...}, {...}, {...},  
    {...}, {...}, {...}]
```

هرجع ال `Data` مباشرة انا كدا استغنيت عن ال `then` تمام

ودي الميزة في استخدام ال `async` و ال `await` ان الموضوع أحسن و `Clean` اكتر من استخدامك للطريقة العادية واستخدام ال `then`

لا تنسونا من دعواتكم

Never Forget That (**Always Search**)

By: Amr Elsayed

♥ راقّت لي ♥



Mahmoud Adel • Following

Full-stack Web Developer - Instructor && Penetration Tester

2h • 🌐

بشكلًا ما انا فشلت مرتين في تكوين وبناء مشروع في ال بحلم بيه

- وانت فشلت بشكلًا ما في الكورس ال انت خدته
- وانت في فشلت بشكلًا ما في الترينج ال دخلته
- وهو فشل بشكلًا ما في انه يبني بيت او علاقة ناجحه
- وغيرنا فشل بشكلًا ما في مشروعه او الكامبين او الريبورت طلع نتيجته اسوأ من المتوقع
- وفشل فلان بشكلًا ما والعميل كان غير راضي تمامًا وصمم انه يفسخ العقد وطلع بخسارة

بس هنا النقطة الفاصلة , لم احي ابدأ شركتي ثاني هبدأ بخبرة فاشل بدرجتين وقبل ما تضحك لا انا عارف وقعت فين مرتين ولو وقعت التالته ف انا هبقي عارف وقعت فين 3 مرات

وانت هتبقي عارف ليه مستفدتش وفشلت في الكورس ده
وانتي هتبقي عارفه ليه فشلت في الترينج ده ف هتتجحي في ال بعده

وهنفضل كده , حتي لو فشلت في كل مره حاولت فيها , ف يكفي نصيحتك لفلان ال لسه
بيبدأ لم قتلته " متعملش كذا علشان نتيجته كذا " وفعلًا نجح وعدي بسبب نصيحتك

انت نجحت في انك تنصح نجحت في انك بنيت وعملت خبرة , انت دلوقتي تقدر تباع
نصايحك وتجاربك دي بفلوس ,

تجاربك وفشلك عمره ما كان ببلاش , ولازم كل مره تقع وتفشل فيها تبقي مبسوط , انت
اكتشفت خطوة سيئة جديدة وتجربة فاشله ممكن تحكي للناس ازاى يتجنبوها و انت نفسك
في الرحلة الجيه هتجنبها و هتبقي انجح من غيرك

وبمنتهي البساطة

لا يصل الناس إلى حديقة النجاح، دون أن يمروا بمحطات التعب والفشل واليأس، وصاحب
الإرادة القوية لا يُطيل الوقوف في هذه المحطات.

♥ اذكر الله ♥

الزموا ذكر الله ففيه العون على متاعب الحياة ♥

اللهم إني أستغفرك واتوب إليك، لا إله الا انت سبحانك إني كنت من الظالمين

سبحان الله وبحمده سبحان الله العظيم

لا إله الا الله وحده لا شريك له .. له الملك و له الحمد وهو على كل شيء قدير

The End And Advices Lesson #188

The End

- Other Information => Practice + Tutorials
- Problem Solving
- Search In Lessons
- Advanced Books

طبعاً خـلينا متفـقين وانا بقول الكلام دا لنفسي قبلكم ان مفيش **Course** في العالم هيقدر يغطي كل حاجة موجودة في أي لغة برمجة

المطلوب:

- في امثلة و حاجات هيتـم التطبيق عليها في **playlist** خاصة بيها **لينك ال Playlist** في ال **playlist** دي هيتـم التطبيق ع الحاجات اللي اتعلمناها ومخدناش عليها أمثلة عملية قوية وكمان الحاجات الجديدة اللي بتظهر في اللغة هتكون موجودة هنا
- حل مسائل كتير جدا جدا **Problem Solving** عشان تطور المنطق البرمجي فيه مواقع زي (1- **Edabit**)----- (2- **Codewars**) ----- (3- **Leetcode**) ----- (4- **JSchallenger**)
- بعد كل أسبوع كان فيه قائمة فيها شوية حاجات تبـحث عنها يفضـل انك فعلا تبـحث عنها حتي لو هتبـحث في كل يوم عن اسمين تلاته من اللي كانو مكتوبين تحت كل أسبوع
- قراءة الكتب المتقدمة هتفيدك وتساعدك تكمل معلوماتك

لا تنسونا من دعواتكم

Never Forget That (**Always Search**)

By: Amr Elsayed

بص بعد ما خلصنا اخر أسبوعين دول خليني الخصلك الدنيا فيهم سريعا كدا حسب فهمي الحالي!!

أولا انا كان عندي **XMLHttpRequest** والي من خلاله كنت بعمل **AJAX Call** عشان اتصل بـ **Data** من مكان معين واتعامل معاها كنت معتمد علي حدوث الاتصال دا بشكل سليم ان يكون الـ **status** بـ **200** والـ **readyStage** بـ **4** كنت بحط شرط اتأكد من خلاله ان الاتصال دا سليم ورجعلي بيانات

بعدين تطور الموضوع شوية وبقا فيه حاجة اسمها **Promise** يعني وعد لو الوعد اتحقق ترجعلي بيانات لو متحققش يبقى مفيش حاجة ترجعلي او يرجعلي **Error** بقا

كنت بعبر بقا عن رجوع البيانات دي بحاجة اسمها **Resolved Value** او بمعنى ثاني **fulfilled** وكنت بعبر عن عدم رجوع البيانات او فشل الوعد بحاجة اسمها **Rejected Value** او **Rejected**

طيب الـ **promise** دا عشان يتعمل بيقبل مني **2 Callback function** بسميهم بأي أسم احبه اول **Callback function** دي بخصصها لحالة الـ **Resolve** او الـ **fulfilled** ثاني **Callback function** بخصصها لحالة الـ **Reject**

طيب حلو الكلام دا دلوقتي انا بقا عايز اهندل الداتا الي هترجعلي دي لو الوعد اتحقق وبرضو اهندل الـ **Error** الي هيحصل لو الوعد متحققش

قال في حاجة اسمها **then** بتقبل مني برضو **2 Callback functions** واجباري اول **Function** دي بتحل محل الـ **resolved function** الي موجودة في الـ **promise** يعني ببساطة كدا الوعد اتحقق؟! **اتحقق** يبقى الـ **promise** حالته **fulfilled** يبقى الحالة دي تروح لـ **then** مكان أول **callback function** وتعرض البيانات

طيب الوعد بقا متحققش يبقى الـ **Promise** حالته **Rejected** يبقى الحالة دي تروح لـ **then** مكان ثاني **callback function** وتقولي انه فيه **Error** موجود الوعد متحققش

الـ **then** دا والله كان اصيل وابن ناس كان بيهندل الحالتين سواء **resolved** او **Rejected** بعدها قالولك بص في حاجة اسمها **Catch** افضل في انها تهندل الـ **rejected** دا اعتمد عليها قولنا ماشي

بعد كدا انتقلنا لـ **fetch** والي هو في حد ذاته بيرجع **promise** برضو ولكن الـ **Syntax** بتاع الطف واخف ومفيش بقا اول **2 Callback functions** دول خالص ولا حوار الـ **resolved** دا ولا الـ **rejected**

انت عندك الـ **fetch** بيرجع **promise** كدا كدا بس خلي بالك هو بيتعامل مع برضو مع الـ **then** عشان يهندل البيانات الي راجعه من الـ **fetch** الي حصل وكدا كدا الـ **catch** موجودة معنا برضو عشان تهندل الـ **Error**

بعد كدا انتقلنا لـ **async** والـ **await** وقالك ياعم دي حاجة كدا **Cleaner** ورايقة ع الاخر ومفهاش وجع الدماغ بتاع الـ **then** و الـ **promise** و الكلام دا .. انت هتخط **async** قبل الـ **function** وتستخدم الـ **await** معاها طيب أي فائدة الـ **await** الـ **await** بتعمل عمل الـ **then** ولكنها بتخلي أي حاجة جايه بعدها تستني لحد ما هي هي تخلص وترجعلي بالبيانات الي انا عايزها يعني الوعد بتاعي وهو بيتحقق ويرجع بيانات أي حاجة جاية بعده تستني لحد ما البيانات دي ترجع طبعا انت بتخط الـ **await** دا في **try** بتقوله يعني محاولة نجيب فيها البيانات وبعد الـ **try** بتخط الـ **catch** الي هينهدل الـ **Error** لو الـ **try** فشل في الرجوع ببيانات

data.json

```
[
  {
    "userId": 10,
    "title": "Article Title Number 1",
    "description": "Article Description Number 1"
  },
  {
    "userId": 5,
    "title": "Article Title Number 2",
    "description": "Article Description Number 2"
  },
  {
    "userId": 5,
    "title": "Article Title Number 3",
    "description": "Article Description Number 3"
  },
  {
    "userId": 5,
    "title": "Article Title Number 4",
    "description": "Article Description Number 4"
  },
  {
    "userId": 5,
    "title": "Article Title Number 5",
    "description": "Article Description Number 5"
  },
  {
    "userId": 5,
    "title": "Article Title Number 6",
    "description": "Article Description Number 6"
  },
  {
    "userId": 25,
    "title": "Article Title Number 7",
    "description": "Article Description Number 7"
  },
  {
    "userId": 25,
    "title": "Article Title Number 8",
    "description": "Article Description Number 8"
  },
  {
    "userId": 15,
    "title": "Article Title Number 9",
    "description": "Article Description Number 9"
  },
]
```

```

{
  "userId": 15,
  "title": "Article Title Number 10",
  "description": "Article Description Number 10"
}
]

```

main.js

```

function getData(fileJson) {
  return new Promise((resolve, reject) => {
    let myRequest = new XMLHttpRequest();
    console.log(myRequest);

    myRequest.onload = function () {
      if (this.readyState === 4 && this.status === 200) {
        resolve(JSON.parse(this.responseText));
      } else {
        reject(Error("Your Json File Is Wrong"));
      }
    };
    myRequest.open("GET", fileJson);
    myRequest.send();
  });
}

getData("data.json")
  .then((data) => {
    data.length = 5;
    console.log(data);
    return data;
  })
  .then((showData) => {
    showData.forEach((data) => {
      let containerDiv = document.createElement("div");

      let title = document.createElement("h3");
      let titleText = document.createTextNode(data.title);
      title.appendChild(titleText);

      let description = document.createElement("p");
      let descriptionText = document.createTextNode(data.description);
      description.appendChild(descriptionText);

      containerDiv.appendChild(title);
      containerDiv.appendChild(description);
      document.body.appendChild(containerDiv);
    });
  });
}

```


main.js

```

async function fetchData() {
  try {
    let myResponse = await fetch("data.json");
    console.log(myResponse);
    let allData = await myResponse.json();
    allData.length = 5;

    for (let data of allData) {
      let containerDiv = document.createElement("div");

      let title = document.createElement("h3");
      let titleText = document.createTextNode(data.title);
      title.appendChild(titleText);

      let description = document.createElement("p");
      let descriptionText = document.createTextNode(data.description);
      description.appendChild(descriptionText);

      containerDiv.appendChild(title);
      containerDiv.appendChild(description);
      document.body.appendChild(containerDiv);
    }
  } catch (err) {
    console.log(err);
  }
}
fetchData();

```

حل اخر

```

fetch("data.json")
  .then((result) => {
    return result.json();
  })
  .then((result) => {
    result.length = 5;
    console.log(result);
    return result;
  })
  .then((result) => {
    for (let i = 0; i < result.length; i++) {
      let containerDiv = document.createElement("div");

      let title = document.createElement("h3");
      let titleText = document.createTextNode(result[i].title);
      title.appendChild(titleText);
    }
  })

```

```
let description = document.createElement("p");
let descriptionText = document.createTextNode(result[i].description);
description.appendChild(descriptionText);

containerDiv.appendChild(title);
containerDiv.appendChild(description);
document.body.appendChild(containerDiv);
}
})
.catch((rej) => console.log(rej));
```

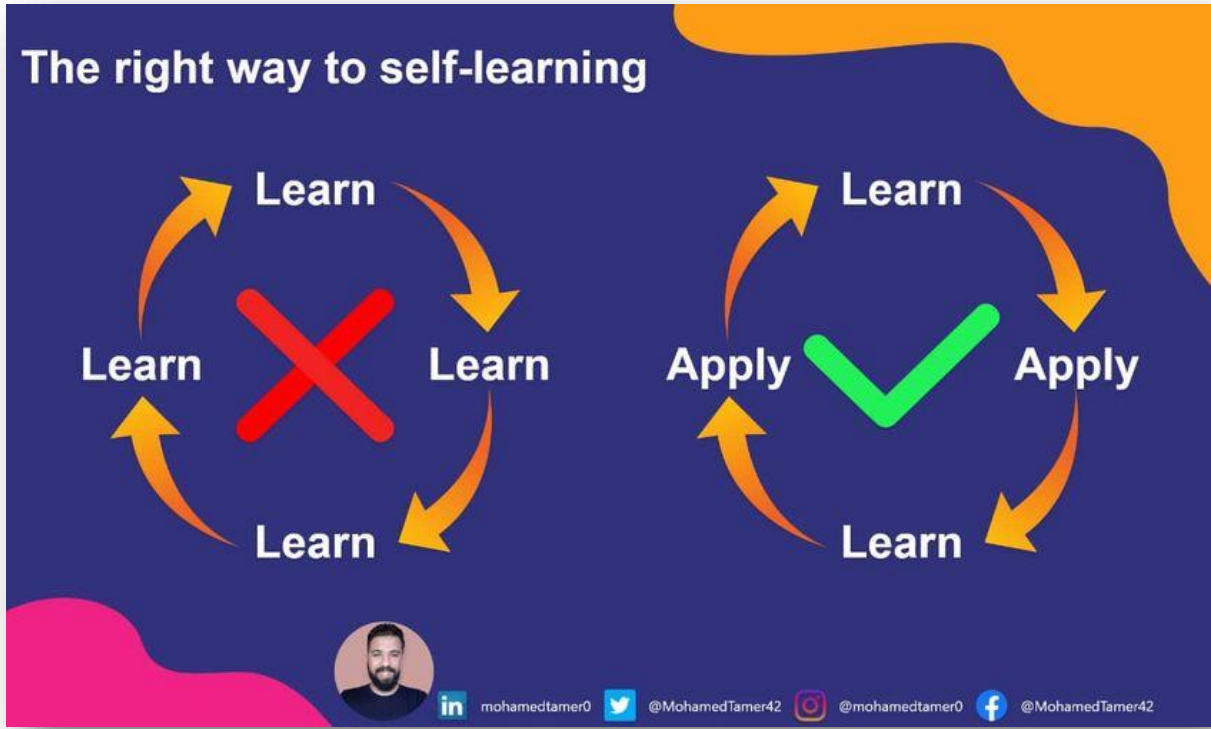
الحمد لله رب العالمين حمداً طيباً كثيراً مباركاً فيه
سبحانك اللهم وبحمدك أشهد أن لا إله إلا أنت أستغفرك وأتوب إليك.

لا تنسونا من دعواتكم

Never Forget That (**Always Search**)

By: Amr Elsayed

♥ راقِ لي ♥



التطبيق العملي أساسي ومهم جدا فلو خلصت ال Course علي خير لازم لازم تتطبق من نار ٨٨

♥ اذكر الله

الزموا ذكر الله ففيه العون على متاعب الحياة ♥

أَلَا بِذِكْرِ اللَّهِ تَطْمَئِنُّ الْقُلُوبُ

اللهم لك الحمد حمداً طيباً كثيراً مباركاً فيه كما ينبغي لجلال وجهك ولعظيم سلطانتك يا رب العالمين

اللهم اني أسألك علماً نافعاً ورزقاً طيباً وعملاً متقبلاً

اللَّهُمَّ إِنِّي أَسْأَلُكَ عِلْماً نَافِعاً، وَرِزْقاً طَيِّباً، وَعَمَلاً مُتَقَبَّلاً.

اللَّهُمَّ أَعِنِّي عَلَى ذِكْرِكَ، وَشُكْرِكَ، وَحُسْنِ عِبَادَتِكَ.

اللهم صلّ وسلم وزد وبارك على نبينا محمد ﷺ

لا تنسونا من دعواتكم

Never Forget That (Always Search)

By: Amr Elsayed

♥ فقرة إضافية حببت اضيفها ♥

♥ جائزة أفضل دروس في ال Playlist كلها .. استمتعت بيهم بجد

#052 – Loop For Advanced Examples ==> Week 7

#066 – Arrow Function Syntax ==> Week 9

#089 – Create and Append Elements ==> Week 12

#096 – CSS Styling And Stylesheet ==> Week 13

#100 – AddEventListener ==> Week 13

#112 – Local Storage Color App Practice ==> Week 15

#134 => #146 Regular Expression All Lessons ==> Week 18*

#153 – Class Inheritance ==> Week 19

#154 – Class Encapsulation ==> Week 19

#167 – Module Import And Export ==> Week 20

#168 – Named vs Default Export And Import All ==> Week 20

#171 – What Is API ==> Week 21

#173 – Asynchronous vs Synchronous Programming ==> Week 21

#174 – Call Stack And Web API ==> Week 21

#175 – Event Loop And Callback Queue ==> Week 21

#183 – Fetch API

#185 – Async And Trainings

#186 – Await And Trainings

#187 – Try, Catch, Finally With Fetch

♥ جائزة أفضل أسبوع وامتع أسبوع على الاطلاق راحت للأربع أسابيع دول

- الأسبوع 12 و 13 أسبوعين ال DOM

- أسبوع 18 أسبوع ال Regular Expression

- أسبوع 21 و 22 أسابيع ختام ال Course ال API وال JSON وال Fetch

جائزة اكثر أسبوع حاسس اني مفهمتش أوي وأكثر دروس مستمتعش بيها اوي اطلاقا

- أسبوع 15 أسبوع ال Local Storage عدا درس 112 كان ممتع بس حسيته معقد شوية 😊😞

- أول 5 دروس في أسبوع 17 من اول 123 لحد 127 الي هو أسبوع ال Set Data وال Weak Map

الختامة وشويه فضفضة

❖ في البداية وبفضل الله سبحانه وتعالى أتوجه بجزيل الشكر والامتنان والعرفان بالجميل [للبرمهندس أسامة الزيرو](#) اللي بفضل الله علينا ثم مجهوده مكنتش **ال Text** دا تم ربنا يجازيك كل خير ويكرمك ويوسع رزقك ويملا حياتك خير وبركه وسعادة ويرزقك الفردوس الأعلى انت وكل احبابك

[لينك القناة](#)

[لينك البلاي ليست الخاصة بال JavaScript](#)

[خطة الدراسة JavaScript Bootcamp 2021](#)

[جميع أكواد دروس ال JavaScript موجودة هنا](#)

[جميع تكليفات ال JavaScript موجودة هنا](#)

❖ استكمالا للمشوار وبعد تلخيص ال **HTML** و ال **CSS** و ال **SASS** والجزء الأول من ال **JS** وفقنا الله لتلخيص الجزء الثاني من ال **JS** .. ارجوا من الله عز وجل ان يكون ذا نفع للجميع .. و ارجوا من الله عز وجل ان يجعل هذا العمل خالصا لوجهه الكريم وان يجعله في ميزان حسناتنا جميعا

❖ بعذر عن وجود أي أخطاء املائية او أخطاء في أي كود فالكمال لله وحده واتمني لو لقيت أي خطأ سواء في الشرح او حل التكليفات تواصل معايا وتبلغني بيه

❖ في الجزء الثاني تم تكملة وتلخيص الدروس من اول 102 الي 188 بالتكليفات

❖ خصصت في الملخص دا تلت أجزاء .. الجزء الأول **لذكر الله** عشان متخليش حاجه ابدا تشغلك عن ذكر الله اللي قادر ينجيك ويفتحلك ابواب الرزق ويرزقك ويسهل عليك الفهم و جزئين **العبارات التحفيزية** و جمل وعبارات **راقت لي** اه هما يعتبروا سبب رئيسي في تكبير حجم الملخص، ولكن والله حبيت انهم يكسروا ملل المذاكرة والقراءة وربما يكون فيهم نفع او الهام لحد

❖ مش عايزك تتضايق لاي سبب لو **مفهمتش أي تكليف** او **مقدرتش تحله** عادي في تكليفات كتير خدت مني وقت كبير جدا وكانت أسهل من تكليفات تانية أصعب بكثير مخدمتش معايا غير دقائق واتحلت .. لذلك لو تعثر عليك فهم حاجة **حاول مرة و اثنين وتلاته**

❖ لأي حد هيوصله الكلام دا مش طالب منك غير طلب واحد وامانة عليك لازم تنفذه و هو إنك تفتكرني بدعوة طيبة لوجه الله

❖ ان شاء الله لو لينا عمر نبقى نعمل ملخص لل Framework اللي هنتعلمه مع كام Technology تانيه باذن الله

❖ نصيحتي الأخيرة ليك إنك تتطبق كتير جدا وتشوف حاجات فيها Logic ثقيل واصبر ومتستعجلش النتائج

❖ **Never Forget That Always Search**

[LinkedIn Account](#)

[GitHub Account](#)

[Codepen Account](#)

[Twitter Account](#)

[Mail Me](#)

[Facebook Account](#)

افتكروني بدعوة طيبة لوجه الله واشوفكم ان شاء الله مع حاجه أفضل وأفضل مستقبلا

شكرا لكم 

By: Amr Elsayed